Boring Linux

K19G

2026-10-30

Table of contents

Introduction **25**

Boring Linux **26**
Starting point . . . 26
How to learn . . . 26
Current writing progress . . . 26
Curriculum map . . . 28

The Linux Machine and the Administration Lab **30**

System Foundations **31**

What Linux Consists Of **32**
A map of the system . . . 32
The kernel . . . 32
Userspace . . . 34
Libraries and system calls . . . 34
The distribution . . . 34
The installation and its running state . . . 35
Follow one ordinary request . . . 35
Worked example 1: identify the operating system and kernel separately . . . 36
Worked example 2: separate a command from the kernel it queries . . . 37
Worked example 3: distinguish program, process and manager . . . 37
Worked example 4: inspect one system call boundary . . . 38
The same Linux label can describe different boundaries . . . 38
Break/fix: a package upgrade did not change the running kernel . . . 39
Practice: build a component evidence table . . . 39
Recall card . . . 40

Hardware, Firmware and Virtual Machines **41**
Map: a machine can be physical or presented by another system . . . 41
What firmware does before Linux is usable . . . 41
Platform firmware and device firmware are related but distinct . . . 43
What a virtual machine actually receives . . . 43
A disk image illustrates layered ownership . . . 44
Virtual CPUs are not physical-core reservations . . . 44
Virtualization, emulation and containers . . . 45
Worked example 1: establish what virtualization is detected . . . 45
Worked example 2: inspect the guest-visible machine . . . 46

Worked example 3: observe the firmware interface available here . . . 47
Worked example 4: understand the KVM boundary without creating a VM . . . 47
Keep configuration, execution and saved state separate . . . 48
Break/fix: the guest returns to the installer after installation . . . 48
Practice: draw the ownership boundary . . . 49
Recall card . . . 49

CPU, Memory, Storage and Network Resources 50
Map: four resources, one workload . . . 50
CPU: execution capacity and the need to wait . . . 50
Two logical CPUs do not make every operation twice as fast . . . 52
Worked example 1: inventory CPU topology . . . 52
Worked example 2: elapsed time is not CPU time . . . 53
Load average is a demand clue . . . 53
Memory: active state, cache and limits . . . 53
Used RAM can be useful cache . . . 54
Worked example 3: read a small set of memory facts . . . 54
Storage: a device, a filesystem and a path are different objects . . . 55
Worked example 4: connect a path to mounted storage . . . 55
File size and allocated storage can differ . . . 56
Capacity and I/O performance are separate . . . 56
Networking: an interface is one part of a service path . . . 57
Worked example 5: inventory interfaces and listening endpoints . . . 57
Worked example 6: look at counters without overinterpreting them . . . 58
Follow a request through all four resources . . . 58
Break/fix: “the VM has resources, but the service is slow” . . . 59
Practice: inventory and explain before tuning . . . 59
Recall card . . . 60

System Paths, Configuration and State 61
One tree, several sources . . . 61
Worked example: identify the backing filesystem . . . 63
Directories express responsibility . . . 63
Persistence is a property to verify . . . 64
Configuration has an owner and an activation path . . . 65
Worked experiment: a file changes before its consumer does . . . 66
Diagnosis: three failures that look like missing files . . . 67
The directory exists, but the data is wrong . . . 67
The data returns after reboot, but access fails . . . 67
The administrator can read it, but the service cannot . . . 67
Practice: classify before changing . . . 67
Recall card . . . 68

Processes, Services and Sessions 69
Program, process, service: three different objects . . . 69
Inspect a process without guessing from its name . . . 71
Worked experiment: create, observe and finish one process . . . 71

Services give processes an operational boundary . . . 72
Running now and starting later are independent questions . . . 73
Active does not always mean a process is still running . . . 73
A session is a user's access context . . . 73
Observe from the client back toward the process . . . 74
First failure investigation . . . 75
Practice: build an evidence table . . . 75
Recall card . . . 75

Distributions, Releases and Documentation **77**
A distribution is an integration and maintenance boundary . . . 77
Release models create different change schedules . . . 77
Worked example: separate distribution identity from kernel identity . . . 79
Packages, repositories and origins . . . 80
Installed version and candidate version . . . 80
Which package owns a file? . . . 81
Versions are evidence, not complete security verdicts . . . 81
Find documentation that belongs to the installed system . . . 82
Worked investigation: a guide does not match the machine . . . 83
Practice: write a documentation and support card . . . 83
Recall card . . . 84

Building and Observing the Lab **85**

Building the First Virtual Machine **86**
Define the lab before opening the installer . . . 86
Understand the two network phases . . . 87
Prepare and validate the host . . . 88
Inspect the setup network . . . 89
Obtain the installer and verify what you downloaded . . . 89
Create the virtual hardware . . . 90
Install the guest through its console . . . 90
Create the isolated practice network . . . 91
Establish a baseline you can actually use . . . 93
Prove recovery with a visible change . . . 95
A local clone is not an independent backup . . . 95
Diagnose the layer that failed . . . 95
Completion record and recall card . . . 96

Inventory, Evidence and First Troubleshooting **97**
Start with an operational question . . . 97
Build an inventory with purpose and scope . . . 99
Worked inventory: identity, time and boot . . . 99
Worked inventory: resources and storage . . . 100
Worked inventory: networking and listeners . . . 100
Worked inventory: service state and bounded logs . . . 101
Save a small evidence bundle . . . 101

Separate facts, interpretations and actions . . . 102
Worked failure: a service reads the wrong pathname . . . 103
Observe without repairing . . . 103
Make the smallest supported correction . . . 104
Clean up the experiment . . . 104
Write the change note before forgetting the reason . . . 105
A second investigation: connectivity changed as designed . . . 105
Practice: produce an evidence-led handoff . . . 106
Recall card . . . 106

Installing and Provisioning Linux **107**

Installation Foundations **108**

Installation Media, Authenticity and Integrity **109**
Choose the kind of artifact before its filename . . . 109
Architecture names describe compatibility . . . 110
Four separate claims about a download . . . 110
Collect a matched verification set . . . 112
Establish the signing key's identity . . . 112
Verify the manifest, then verify the image . . . 113
Worked experiment: matching bytes are not trusted bytes . . . 114
Carry the verification across the copy boundary . . . 115
Diagnose verification failures precisely . . . 115
Practice and recall . . . 116

Planning Firmware, Storage, Identity and Networking **117**
Write the purpose before choosing the layout . . . 117
Separate the host, virtual machine and guest identities . . . 117
Choose firmware before installing . . . 119
Plan capacity at more than one layer . . . 120
Choose a storage policy, not just a checkbox . . . 121
Reserved growth space can be intentional . . . 121
Encryption requires an access and recovery decision . . . 121
Separate filesystems express separate policies . . . 122
Plan account access before the first reboot . . . 122
Make networking assumptions explicit . . . 122
Decide what may change during installation . . . 123
Review the destructive boundary . . . 123
Completed plan for the guided lab . . . 124
Practice: explain a change before approving it . . . 124
Recall card . . . 125

Guided Installation and the Resulting Storage Layout **126**
Create the target and reconcile its identity . . . 126
Turn installer screens into recorded decisions . . . 127
Review the proposed storage map . . . 127

Finish identity and software selection . . . 128
Prove that the installed guest booted . . . 128
Draw the storage stack from observations . . . 129
Distinguish three kinds of free space . . . 130
Worked diagnosis: “the installer lost half my disk” . . . 131
Connect persistent mount intent to current mounts . . . 132
Observe swap without assuming its form . . . 132
Verify access and compare the two installations . . . 133
Diagnose installation failures at the correct stage . . . 134
Practice and completion criteria . . . 134
Recall card . . . 134

Manual Storage Choices, Encryption and Swap **136**
Start with mount responsibilities . . . 136
Why separate service data? . . . 137
Build the custom layout . . . 138
Verify boundaries rather than partition numbers . . . 138
Choose encryption by the recovery requirement . . . 139
Optional encrypted installation comparison . . . 140
Swap is part of the data-protection decision . . . 140
Diagnose a layout mismatch . . . 141
Practice and acceptance record . . . 141
Recall card . . . 141

First Boot and Device Readiness **143**

First Boot: Administrative Access, Updates and Service Inventory **144**
Prove access before changing software . . . 144
Establish an update preflight . . . 145
Refresh metadata and review the transaction . . . 145
An automatic update may already be running . . . 147
Inventory what the machine is running . . . 148
Reboot and repeat the acceptance checks . . . 149
Worked diagnosis: repository refresh fails . . . 149
Practice and acceptance record . . . 150
Recall card . . . 150

Hardware and Virtual-Device Discovery **151**
From presentation to usable interface . . . 151
Establish what the guest sees . . . 151
Connect an interface to its kernel and udev records . . . 153
Distinguish driver code from device firmware . . . 153
Lab: add a second virtual NIC and observe its lifecycle . . . 154
A worked diagnosis: the new NIC is not in `ip link` . . . 155
Common observations and next questions . . . 155
Acceptance and recall . . . 156

Provisioning and Recovery **157**

Image-Based and Unattended Installation **158**
Two ways to produce a machine . . . 158
Identity is not the same as a disk copy . . . 159
Prepare a local installer seed . . . 160
Review before allowing writes . . . 161
Install on a new, single-disk guest . . . 162
Detach sources and verify the result . . . 162
Diagnose by stage . . . 163
Practice and cleanup . . . 164
Recall card . . . 164

Reinstallation and Host Configuration Recovery **165**
Decide what recovery means . . . 165
Define a small recovery contract . . . 166
Identify what would be needed for a real host . . . 167
Prepare the demonstration on `bl02` . . . 167
Assemble and inspect a selected bundle . . . 168
Export the bundle to the host . . . 169
Install a replacement without touching the original . . . 169
Deliver and inspect the bundle on the replacement . . . 170
Apply selected files, then prove the result . . . 171
Worked diagnosis: restored configuration points to an old device . . . 171
Record, reset and retain . . . 172
Recall card . . . 172

Boot Shutdown and Recovery **173**

Firmware and Boot Selection **174**

Firmware, UEFI Variables, GPT and the EFI System Partition **175**
The first handoff . . . 175
Establish firmware and guest identity . . . 176
Read the firmware's boot options . . . 177
Where variables are exposed . . . 178
Identify the partition map . . . 178
Inspect the EFI System Partition . . . 179
Lab: build a boot-path evidence table . . . 180
Worked diagnosis: "the ESP is empty" . . . 180
Practice and recall . . . 181

Bootloader Selection: GRUB and systemd-boot **182**
Two decisions before Linux runs . . . 182
How an Ubuntu GRUB menu is produced . . . 183
Prepare a disposable GRUB test guest . . . 184
Change a menu timeout and roll it back . . . 184

What changes for systemd-boot? . . . 186
Diagnose the correct selection layer . . . 186
Practice and recall . . . 186

Kernel and Early Userspace **188**

Kernel Images, Command Lines and Matching Modules **189**
A versioned boot set . . . 189
Identify the running kernel and root . . . 189
Inventory files for the running release . . . 191
Follow a module without loading it . . . 192
Distinguish planned arguments from this boot . . . 193
Worked diagnosis: new kernel installed, old one running . . . 193
Practice and acceptance . . . 193
Recall card . . . 194

Initramfs and Reaching the Real Root **195**
From boot image to installed root . . . 195
Inspect the installed image and its inputs . . . 196
Construction and dependencies . . . 197
Optional lab: rebuild one image on a clone . . . 197
Diagnose an early-root failure . . . 198
Practice and recall . . . 199

Root Handoff, PID 1 and the Boot Transaction **200**
From temporary root to installed manager . . . 200
Observe PID 1 and this boot . . . 200
Requirement and ordering are different . . . 202
A transaction can succeed partially . . . 203
Practice and recall . . . 203

Startup and Shutdown **204**

Targets, Activation, Readiness and Boot Timing **205**
What a target tells you . . . 205
Activation and readiness are separate . . . 205
Read timing without assigning blame too quickly . . . 206
Worked example . . . 207
Practice and recall . . . 207

Shutdown, Reboot and Services That Delay Shutdown **208**
What a clean transition attempts . . . 208
Preflight the transition . . . 209
Reboot and compare boot instances . . . 209
Diagnose a slow or blocked shutdown . . . 210
Practice and recall . . . 210

Boot Trust and Recovery 211

Secure Boot, Unified Images and Measured Boot 212
Four related mechanisms . . . 212
Secure Boot and a chain of trust . . . 212
Inspect without changing trust . . . 213
What a UKI packages . . . 214
Measurements and policy . . . 214
Read-only availability check . . . 215
Worked diagnosis: update followed by an unlock prompt . . . 215
Practice and recall . . . 216

Rescue, Emergency and External-Media Recovery 217
Identify the failed boundary . . . 217
Know which root and manager you are using . . . 218
Prepare the controlled fault . . . 218
Observe the fault and choose recovery . . . 219
Practice and recall . . . 220

Offline Inspection, Chroot Repair and the Next Normal Boot 221
Boot a separate environment . . . 221
Mount the verified installed root . . . 222
Repair the specific file . . . 224
What a broader chroot repair would require . . . 224
Boot normally and verify . . . 225
Recall card . . . 225

Users Identity and Administrative Access 226

Local Identities and Account Lifecycle 227

UIDs, GIDs, Groups and Ownership 228
Names describe numeric identities . . . 228
Read ownership and permissions . . . 228
Primary and supplementary groups . . . 229
Worked diagnosis: group was added but access still fails . . . 230
Practice and recall . . . 230

Account Databases, NSS and Identity Lookup 232
Local records have different purposes . . . 232
A lookup is not the whole login . . . 233
Worked diagnosis: a lookup fails . . . 234
Practice and recall . . . 234

Account Creation, Locking, Expiration and Removal 235
Plan the identity and its scope . . . 235
Create and verify . . . 235

Lock, expire and recover deliberately . . . 236
Revoke membership and retire the test user . . . 237
Diagnose and recall . . . 238

Home Directories, Skeletons and Session Environments **239**
A record, a directory and an environment . . . 239
Skeletons are initial copies . . . 240
Lab: create and inspect a fresh home . . . 240
Diagnose a missing or inaccessible home . . . 241
Recall card . . . 242

Authentication and Privilege 243

Passwords, Credential Storage and Policy **244**
Passwords are not stored as login text . . . 244
Separate the controls . . . 244
Lab: create an account and observe its policy . . . 245
Diagnose and recover a failed password login . . . 246
Recall card . . . 246

PAM Stacks and Diagnosing Login Failures **247**
What PAM coordinates . . . 247
Inspect the relevant service . . . 248
Lab: observe a bounded failure and recovery . . . 248
Follow a failure through the boundaries . . . 249
Recall card . . . 250

Administrative Privilege and Bounded Delegation **251**
Identify the privilege provider and policy . . . 251
Create a dedicated test operator . . . 252
Delegate one diagnostic . . . 253
Test allow, deny and rollback . . . 253
Recall card . . . 254

Service Identities 255

Service Accounts and Runtime Credentials **256**
A process identity is not a human session . . . 256
Establish a bounded identity and state directory . . . 256
Run a transient workload and verify ownership . . . 258
Handle credentials as runtime data . . . 258
Retirement and diagnosis . . . 258
Recall card . . . 259

Sessions and Central Identity **260**

Sessions, logind and Authorization Beyond the Terminal **261**
User, process, session and seat . . . 261
How a polkit request is evaluated . . . 262
Inspect actions without exercising one . . . 263
Worked diagnosis: a request cannot be authorized . . . 263
Practice and recall . . . 264

Central Identity: LDAP, Kerberos and SSSD Foundations **265**
What central identity changes . . . 265
Kerberos in the minimum useful model . . . 267
Where SSSD fits . . . 267
Read-only local inventory . . . 268
Worked diagnosis: central lookup works but login fails . . . 268
Design exercise and acceptance . . . 269
Recall card . . . 269

Packages Software and Updates **270**

Package State and Transactions **271**

Package Metadata, File Ownership and Installed State **272**
From software to a running process . . . 272
Query the installed database . . . 272
Map an executable to its package . . . 274
Worked diagnosis: “the package owns the wrong version” . . . 275
Practice and recall . . . 275

dpkg, APT and Dependency Resolution **276**
What each tool knows . . . 276
Refresh and preview on the clone . . . 276
Install, verify and keep the test package . . . 278
Diagnose an incomplete transaction . . . 278
Recall card . . . 279

Sources Selection and Configuration **280**

Repositories, Signatures and Source Configuration **281**
The authenticated repository chain . . . 281
Read configured sources . . . 281
Connect a package candidate to sources . . . 282
Diagnose a failed refresh . . . 283
Practice and recall . . . 283

Version Selection, Holds, Pinning and Dependency Conflicts **284**
Installed, candidate and available . . . 284
Priorities and pins . . . 284
Lab: hold and release the disposable package . . . 285
Investigate a dependency conflict . . . 285
Recall card . . . 286

Configuration Files, Alternatives and Package Hooks **287**
A conffile is a package-managed configuration file . . . 287
Maintainer scripts and triggers . . . 288
Alternatives select an implementation . . . 288
Lab: finish and remove the test package . . . 289
Worked diagnosis: a service changed after upgrade . . . 289
Practice and recall . . . 290

Updates and Kernel Lifecycle 291

Security Updates and Maintenance Windows **292**
Decide what must change . . . 292
Observe policy and evidence . . . 292
Perform a controlled update on the clone . . . 293
Diagnose a missed update . . . 293
Practice and recall . . . 294

Kernel Updates, Reboots and Livepatch **295**
Inspect the running and installed kernels . . . 295
Plan the transition . . . 295
Livepatch changes the timing, not the lifecycle . . . 296
Worked diagnosis: package updated, vulnerability remains . . . 296
Practice and recall . . . 296

Other Formats and Software Supply 297

Snaps, Flatpak and Other Software Formats **298**
Identify the owner and update path . . . 298
Snap: revisions, confinement and refresh . . . 298
Flatpak: applications, runtimes and remotes . . . 299
Make one maintenance inventory . . . 299
Worked diagnosis and practice . . . 299

Source Builds, Local Packages and Reproducibility **301**
Record the supply chain . . . 301
Stage instead of installing into the host . . . 301
Lab: build and inspect a tiny local package . . . 302
Diagnose and practice . . . 302

Offline Repositories, Mirrors and Disconnected Installation **304**
Design the supply route . . . 304
Inventory a target without changing it . . . 304
Prepare and verify a transfer . . . 305
Diagnose and practice . . . 305

Transaction Recovery **307**

Interrupted Package Transactions and Recovery **308**
Establish the failure boundary . . . 308
Lab: recover a deliberately failed configuration . . . 308
Rollback has boundaries . . . 310
Practice and recall . . . 310

Services Systemd and Scheduled Work **311**

Service Supervision and Execution **312**

Service Supervision, Units and Managers **313**
Unit state is not application health . . . 313
Prepare a separate service lab . . . 313
Create a bounded example service . . . 314
Practice and recall . . . 315

Execution Environment, Identity and Exit Status **316**
Inspect what the manager will run . . . 316
Environment and privileges . . . 316
Read exit evidence . . . 317
Worked diagnosis and practice . . . 317

Readiness and Dependencies **318**

Startup Types, Readiness, Watchdogs and Restarts **319**
How a service reports startup . . . 319
Examine the lab service . . . 319
Restart policy and watchdogs . . . 320
Worked diagnosis and practice . . . 320

Requirements, Ordering and Failure Propagation **321**
Build the dependency model . . . 321
Lab: observe a failed prerequisite . . . 321
Practice and recall . . . 323

Running Multiple Kernels on One Bare-Metal Server **324**

Running Multiple Kernels on One Bare-Metal Server **325**

Running Multiple Kernels on One Bare-Metal Server **326**
Curriculum Architecture 326
Chapter Progression 327

Introduction: The Multikernel Problem **329**

Introduction: The Multikernel Problem **330**
Why You Want Multiple Kernels 330
What Multikernel Actually Means 331
The Landscape of Approaches 331
How to Use This Book 332

Chapter 1: Linux Kernel Fundamentals **333**

Chapter 1: Linux Kernel Fundamentals **334**
The Kernel as Resource Monopolist 334
Boot and Initialization Sequence 334
Hardware Discovery and Enumeration 335
CPU Topology and NUMA Architecture 335
Memory Management Fundamentals 336

Chapter 2: The Boot Process in Depth **337**

Chapter 2: The Boot Process in Depth **338**
Firmware and the EFI Environment 338
Bootloaders: GRUB, systemd-boot, and Others 338
Kernel Command-Line Parameters 339
Initramfs Construction and Role 339
The Handoff to Userspace 340

Chapter 3: Virtualization vs True Bare-Metal Multi-Kernel **341**

Chapter 3: Virtualization vs True Bare-Metal Multi-Kernel **342**
Type-1 Hypervisors: ESXi, Xen, KVM in Bare-Metal Mode 342
KVM Architecture Deep Dive 342
Linux Containers and Shared-Kernel Architectures 343
MicroVMs and Lightweight Virtualization 343
True Bare-Metal Partitioning Concepts 344

Chapter 4: kexec and Kernel-to-Kernel Transitions **345**

Chapter 4: kexec and Kernel-to-Kernel Transitions **346**
How kexec Works 346
kexec-tools Configuration and Usage 347
kdump for Crash Dumping 347
Limitations of kexec for Multi-Kernel Goals 347

Production Use Cases for kexec . . . 348

Chapter 5: Hardware-Assisted Multi-Kernel with KVM/QEMU 349

Chapter 5: Hardware-Assisted Multi-Kernel with KVM/QEMU 350
KVM Setup and Kernel Module Configuration . . . 350
Guest Kernel Installation and Configuration . . . 350
CPU Pinning and NUMA-Aware VM Placement . . . 351
Memory Ballooning vs Static Allocation . . . 351
Complete Working Deployment Example . . . 352

Chapter 6: Device Assignment and I/O Isolation 353

Chapter 6: Device Assignment and I/O Isolation 354
PCI Passthrough (VFIO) Fundamentals . . . 354
SR-IOV and Virtual Functions . . . 354
Network Interface Assignment . . . 355
Storage Device Partitioning . . . 355
Specialized Hardware: GPUs, FPGAs, Accelerators . . . 356

Chapter 7: Networking Between Kernel Instances 357

Chapter 7: Networking Between Kernel Instances 358
Internal Networking Architectures . . . 358
External Network Configuration . . . 358
Network Performance Optimization . . . 359
Inter-Kernel Communication Patterns . . . 359
Complete Networking Configuration Example . . . 359

Chapter 8: Resource Isolation Strategies 361

Chapter 8: Resource Isolation Strategies 362
CPU Isolation: isolcpus, nohz_full, rcu_nocbs . . . 362
Cpusets and Cgroup-Based CPU Management . . . 362
Memory Isolation and NUMA Policies . . . 363
Interrupt Routing and IRQ Affinity . . . 363
I/O Scheduling and Storage Contention . . . 364
What Can and Cannot Be Safely Shared . . . 364

Chapter 9: Security Analysis and Threat Modeling 365

Chapter 9: Security Analysis and Threat Modeling 366
Attack Surfaces in Multi-Kernel Architectures . . . 366
VM Escape Vulnerabilities and Recent CVEs . . . 366
Secure Boot and Kernel Trust . . . 367
DMA Protection and IOMMU Security . . . 367
Secrets and Credential Isolation . . . 367
Comparing Multi-Kernel vs VM Security Posture . . . 368

Chapter 10: Observability and Operations 369

Chapter 10: Observability and Operations **370**
Metrics and Monitoring Architecture . . . 370
Logging Strategy Across Kernel Instances . . . 371
Crash Diagnostics and kdump Integration . . . 371
Performance Profiling Methodology . . . 371
Health Checks and Alerting Design . . . 372

Chapter 11: Performance Engineering **373**

Chapter 11: Performance Engineering **374**
Boot Time Analysis . . . 374
CPU Overhead and Virtualization Tax . . . 374
Cache Effects and NUMA Locality . . . 375
I/O Performance: Virtio vs Passthrough . . . 375
Benchmarking Methodology for Multi-Kernel Systems . . . 375

Chapter 12: Production Operations and Lifecycle Management **376**

Chapter 12: Production Operations and Lifecycle Management **377**
Kernel Upgrade Strategies . . . 377
Configuration Management at Scale . . . 377
Disaster Recovery Architecture . . . 378
Maintenance Windows and Change Management . . . 378
Decommissioning and Migration Procedures . . . 378

Chapter 13: Troubleshooting Guide **379**

Chapter 13: Troubleshooting Guide **380**
Boot Failures and Recovery . . . 380
Symptom: Secondary Kernel Freezes on Early Boot . . . 380
Symptom: Host Panics During Kernel Boot with `isolcpus` . . . 380
Device Assignment and VFIO Issues . . . 380
Symptom: `Device is ineligible for IOMMU domain attach` . . . 380
Symptom: PCIe Device Fails Function-Level Reset (FLR) . . . 381
Networking Problems . . . 381
Symptom: Asymmetric Routing or Dropped Packets on Inter-Kernel Bridge . . . 381
Performance Degradation Diagnosis . . . 381
Symptom: Unexplained High Latency Jitter (P99 Spikes) . . . 381
Kernel Panics and Crashes . . . 382
Symptom: Host Freezes During Secondary Kernel Shutdown . . . 382
Resource Exhaustion . . . 382
Symptom: `Cannot allocate memory` When Allocating Hugepages . . . 382

Chapter 14: Architecture Comparisons and Decision Framework **383**

Chapter 14: Architecture Comparisons and Decision Framework **384**
Multi-Kernel vs Multiple Physical Servers . . . 384
KVM vs Xen vs ESXi for Multi-Kernel Workloads . . . 384
Containers vs VMs vs Multi-Kernel: When to Use What . . . 385

MicroVMs and Unikernels as Alternatives . . . 385
Hardware Partitioning Approaches . . . 385
A Decision Matrix for Architecture Selection . . . 386

Chapter 15: Complete Project Plan: From Requirements to Decommissioning 387

Chapter 15: Complete Project Plan: From Requirements to Decommissioning 388
Phase 1: Requirements Gathering and Hardware Assessment . . . 388
Phase 2: Architecture Selection and Lab Validation . . . 388
Phase 3: Kernel Preparation and Configuration . . . 389
Phase 4: Deployment and Verification . . . 389
Phase 5: Operations, Maintenance, and Decommissioning . . . 390

Conclusion 391

Conclusion 392
Key Architectural Takeaways . . . 392
The Future of Multikernel Systems . . . 392
Operational Synthesis . . . 393

Linux Performance Engineering & Server Benchmarking 394

Linux Performance Engineering & Server Benchmarking 395

Linux Performance Engineering & Server Benchmarking 396
Curriculum Breakdown . . . 397

Introduction: The Discipline of Performance Engineering 400

Introduction: The Discipline of Performance Engineering 401
Performance Is a System Property, Not a Configuration . . . 401
Latency, Throughput, Utilization, Saturation, Availability . . . 401
Core Dimensions Defined . . . 402
The Measurement Imperative . . . 402
Common Pitfalls and How to Avoid Them . . . 403

Chapter 1: The Performance Engineering Methodology 404

Chapter 1: The Performance Engineering Methodology 405
Characterizing the Workload . . . 405
Defining Success: SLOs, SLIs and Performance Targets . . . 405
Baseline Creation and Reference Measurements . . . 406
Experimental Design: Variables, Controls and Reproducibility . . . 406
Statistical Rigor: Variance, Outliers and Significance . . . 407
The Diagnosis Loop: From Symptom to Root Cause . . . 407

Chapter 2: Linux Kernel Architecture and Performance 409

Chapter 2: Linux Kernel Architecture and Performance **410**
Kernel Space and User Space: The Boundary Cost . . . 410
Fast Userspace Alternatives: vDSO . . . 411
System Calls: Entry, Exit and Overhead . . . 411
Processes, Threads and the Task Structure . . . 411
Context Switching: Mechanics and Cost . . . 412
The Two Costs of Context Switches . . . 412
Interrupts, Softirqs and Bottom Halves . . . 412
Synchronization Primitives: Locks, Spinlocks and Contention . . . 413

Chapter 3: CPU Architecture and Performance Counters **414**

Chapter 3: CPU Architecture and Performance Counters **415**
Modern CPU Microarchitecture: Pipelines and Parallelism . . . 415
Instructions Per Cycle (IPC) . . . 416
Cache Hierarchy: L1, L2, L3, TLB and Cache Misses . . . 416
Cache Lines and False Sharing . . . 416
Branch Prediction and Speculative Execution . . . 417
Cores, Hyperthreading and SMT . . . 417
CPU Frequency Scaling and Power States . . . 417
Performance Counters and Hardware Events . . . 418

Chapter 4: CPU Scheduling and Process Placement **419**

Chapter 4: CPU Scheduling and Process Placement **420**
The Completely Fair Scheduler: Internal Mechanics . . . 420
The Transition to EEVDF (Linux 6.6+) . . . 421
Virtual Runtime, Timeslices and Fairness . . . 421
Real-Time Scheduling: SCHED_FIFO, SCHED_RR, SCHED_DEADLINE . . . 421
CPU Affinity, Isolation and Task Migration . . . 422
Pinning Tasks with CPU Affinity . . . 422
Full Core Isolation (`isolcpus` & `nohz_full`) . . . 422
Load Balancing Across NUMA Nodes . . . 423

Chapter 5: Memory Management: Virtual Memory and Pages **424**

Chapter 5: Memory Management: Virtual Memory and Pages **425**
Virtual Memory: Address Spaces and Page Tables . . . 425
The Translation Lookaside Buffer (TLB) . . . 425
Page Faults: Minor and Major Faults . . . 426
Physical Memory: Allocation and Management . . . 426
Swapping: Mechanics, Costs and Modern Behavior . . . 426
Modern Swap Best Practice: `zswap` and `zram` . . . 427
Huge Pages and Transparent Huge Pages . . . 427
Transparent Huge Pages (THP) vs Explicit HugeTLB . . . 427
Memory Accounting: RSS, PSS and Kernel Metrics . . . 427

Chapter 6: NUMA Architecture and Memory Placement **429**

Chapter 6: NUMA Architecture and Memory Placement **430**
What NUMA Means for Performance . . . 430
NUMA Nodes, Local vs Remote Memory Access . . . 431
NUMA-Aware Allocation and Policies . . . 431
Diagnosing NUMA Imbalance . . . 431
Profiling Cache-to-Cache Traffic with `perf c2c` . . . 432
Optimizing for NUMA: Placement and Binding . . . 432
Binding with `numactl` . . . 432
Automatic NUMA Balancing (`kernel.numa_balancing`) . . . 432
NUMA in Virtualized and Cloud Environments . . . 432

Chapter 7: The Page Cache and Memory-Mapped Files **434**

Chapter 7: The Page Cache and Memory-Mapped Files **435**
Page Cache: Architecture and Behavior . . . 435
Read-Ahead and Write-Behind . . . 436
Adaptive Read-Ahead . . . 436
Memory-Mapped Files and mmap . . . 436
`mmap` vs `read()` / `write()` Trade-offs . . . 436
Cache Pressure and Eviction Policies . . . 437
Dirty Page Management and Writeback . . . 437
Critical Writeback Knobs . . . 437

Chapter 8: I/O Subsystems: From Block Layer to Disk **438**

Chapter 8: I/O Subsystems: From Block Layer to Disk **439**
The I/O Path: From open() to the Disk . . . 439
The Block Layer and Request Queues (`blk-mq`) . . . 440
I/O Schedulers: noop, deadline, cfq, mq-deadline, kyber, bfq . . . 440
Block Devices and Queueing Models . . . 441
I/O Merge, Splitting and Bio Structures . . . 441
I/O Completion and Interrupt Handling . . . 441
Hybrid Polling Mode for Extreme Latency . . . 442

Chapter 9: Storage Technologies and Performance Characteristics **443**

Chapter 9: Storage Technologies and Performance Characteristics **444**
HDDs: Seek Latency, Rotational Latency and Throughput . . . 444
SSDs: NAND Characteristics, Wear Leveling and GC . . . 444
NVMe: Protocol Advantages and Queue Architecture . . . 445
RAID and Hardware Controllers . . . 445
Hardware Controller Cache Modes . . . 446
Cloud Block Storage and Shared Storage . . . 446
Choosing Storage for the Workload . . . 446

Chapter 10: Filesystems and Mount Options **448**

Chapter 10: Filesystems and Mount Options **449**
ext4: Journaling, Allocation and Performance . . . 449

XFS: Design Principles and Scalability . . . 449
btrfs and Copy-on-Write Semantics . . . 450
Filesystem Journaling and Write Performance . . . 450
The Cost of `fsync()` . . . 450
Mount Options: Performance Implications . . . 451
Filesystem-Level Caching and Coherency . . . 451

Chapter 11: The Networking Stack and Performance 452

Chapter 11: The Networking Stack and Performance 453
The Networking Data Path . . . 453
Socket Buffering and Memory . . . 454
NIC Interrupts and Softirq Processing . . . 454
RSS, RPS, XPS: Distributing Network Load . . . 454
TCP Congestion Control and Buffer Tuning . . . 455
UDP and Raw Sockets Performance . . . 455

Chapter 12: Network Performance Tuning in Practice 457

Chapter 12: Network Performance Tuning in Practice 458
Core TCP Parameters and Their Effects . . . 458
Socket Buffer Sizing Strategy . . . 458
Practical Sizing Calculation . . . 459
Congestion Control Algorithm Selection . . . 459
Multiqueue Configuration and RSS Tuning . . . 459
Pinning IRQs with `irqbalance` or Static Scripts . . . 460
Validating Network Performance Changes . . . 460

Chapter 13: System Call Tracing and Process Analysis 461

Chapter 13: System Call Tracing and Process Analysis 462
strace and ltrace: Mechanics and Use Cases . . . 462
Safe Production Use of `strace` . . . 462
perf: Architecture and Profiling Modes . . . 463
ftrace: Kernel Tracing Infrastructure . . . 463
Tracepoints and Dynamic Probes . . . 463
SystemTap: Overview and Comparison . . . 464
Building a Tracing Strategy . . . 464

Chapter 14: eBPF and BPF-Based Observability 465

Chapter 14: eBPF and BPF-Based Observability 466
eBPF Architecture and Safety Model . . . 466
BPF Map Types and Communication . . . 467
bpftrace: High-Level Tracing . . . 467
BCC Tools: Key Utilities . . . 468
Writing Custom BPF Programs (CO-RE and libbpf) . . . 468
eBPF Overhead and Production Safety . . . 469

Chapter 15: Performance Analysis Tools and Flame Graphs **470**

Chapter 15: Performance Analysis Tools and Flame Graphs **471**
Flame Graphs: Construction and Interpretation . . . 471
Reading Rules . . . 471
perf Record and Report . . . 472
Unwinding Call Stacks: Frame Pointers vs DWARF . . . 472
Generating the Interactive SVG . . . 472
Hotspot Analysis and Call Path Tracing . . . 472
On-CPU vs Off-CPU Profiling . . . 472
Combining Tools: Integrated Workflows . . . 473
Sampling vs Tracing: Trade-offs . . . 473
Automated Analysis Pipelines . . . 474

Chapter 16: System Monitoring and Observability **475**

Chapter 16: System Monitoring and Observability **476**
Classic Tools: sar, vmstat, iostat, mpstat, pidstat . . . 476
`vmstat 1` (System Overview) . . . 476
`iostat -xz 1` (Extended Storage Metrics) . . . 476
`mpstat -P ALL 1` (Per-Core Breakdown) . . . 477
`pidstat 1` (Per-Process Attribution) . . . 477
Modern Monitoring: top, htop, numastat . . . 477
Network Monitoring: ss, ethtool, nstat . . . 477
Pressure Stall Information (PSI) and Modern Metrics . . . 478
Building Production Dashboards and Alerting . . . 478

Chapter 17: Benchmarking: Design and Execution **480**

Chapter 17: Benchmarking: Design and Execution **481**
What Makes a Valid Benchmark . . . 481
Designing Representative Workloads . . . 481
Test Environment Isolation and Control . . . 482
Warm-Up, Steady State and Cooldown . . . 482
Repetition, Reproducibility and Statistical Analysis . . . 483
Benchmarking Mistakes: Gaming, Noise and Bias . . . 483

Chapter 18: Benchmarking Tools and Suites **485**

Chapter 18: Benchmarking Tools and Suites **486**
fio: Storage Benchmarking Deep Dive . . . 486
Standard NVMe Random Read/Write Job File . . . 486
stress-ng: Comprehensive System Stress Testing . . . 487
sysbench: CPU, Memory, File I/O, Database Tests . . . 487
iperf3: Network Throughput and Latency . . . 488
lmbench: Latency and Bandwidth Microbenchmarks . . . 488
Application-Level: wrk, ApacheBench, curl . . . 488
`wrk` (High-Concurrency HTTP Benchmarking) . . . 488
`curl` (Detailed Connection Timing) . . . 489

Phoronix Test Suite and Comparative Testing 489

Chapter 19: CPU and Memory Tuning for Production 490

Chapter 19: CPU and Memory Tuning for Production 491
CPU Governor Selection and Configuration 491
IRQ Affinity and NUMA-Optimal Placement 491
CPU Isolation for Critical Workloads 492
Huge Pages: Configuring and Validating 492
Allocating 1GB Static Huge Pages at Boot 492
Runtime Allocation of 2MB Huge Pages 493
Memory Sysctl Parameters 493
Validating CPU and Memory Tuning 493

Chapter 20: I/O and Filesystem Tuning for Production 495

Chapter 20: I/O and Filesystem Tuning for Production 496
I/O Scheduler Selection by Workload 496
Filesystem Mount Options for Performance 496
Recommended XFS Mount Options 497
Recommended ext4 Mount Options 497
Writeback and Dirty Page Tuning 497
Device-Level Parameters: Queue Depth and Read-Ahead 498
SSD and NVMe-Specific Tuning 498
Validating I/O Performance Improvements 498

Chapter 21: cgroups, systemd and Resource Management 500

Chapter 21: cgroups, systemd and Resource Management 501
cgroups v1 vs v2: Architecture Differences 501
CPU Controllers and Throttling 502
The CPU Throttling Latency Trap 502
Memory Controllers: Limits and Pressure 502
I/O Controllers: Throttling and Prioritization 503
systemd Resource Controls and Service Isolation 503
ulimits and Resource Limits 503

Chapter 22: Containers, Virtualization and Noisy Neighbors 505

Chapter 22: Containers, Virtualization and Noisy Neighbors 506
Container Performance: Isolation and Overhead 506
cgroups and Namespaces in Containers 507
Virtualization Overhead: Hypervisors and Emulation 507
Detecting and Mitigating Noisy Neighbors 507
Hardware-Level Mitigation: Intel RDT and AMD MBA 508
Cloud vs Bare Metal: Performance Implications 508
Best Practices for Containerized Workloads 508

Chapter 23: Databases and High-Performance Applications 509

Chapter 23: Databases and High-Performance Applications **510**
How Databases Use Linux: I/O and Memory Patterns . . . 510
Shared Memory and IPC for Databases . . . 510
Locking and Contention in Database Workloads . . . 511
NUMA and Database Placement . . . 511
Application Runtimes: JVM, Go, Python Considerations . . . 511
Tail Latency Optimization . . . 512

Chapter 24: Distributed Systems and Cluster Performance **513**

Chapter 24: Distributed Systems and Cluster Performance **514**
Distributed Latency: Network and System Contributions . . . 514
Clock Synchronization and Time Skew . . . 515
Network Partitions and Performance . . . 515
Consensus Protocols and Performance Costs . . . 515
Cluster Resource Management . . . 516
Observability at Scale . . . 516

Chapter 25: Production Case Studies **517**

Chapter 25: Production Case Studies **518**
Case Study: CPU Saturation from Interrupt Storms . . . 518
The Incident . . . 518
Diagnostic Investigation . . . 518
Root Cause . . . 518
Mitigation & Resolution . . . 519
Case Study: Memory Pressure and Swapping Crisis . . . 519
The Incident . . . 519
Diagnostic Investigation . . . 519
Root Cause . . . 519
Mitigation & Resolution . . . 519
Case Study: NUMA Imbalance Causing 5x Latency . . . 520
The Incident . . . 520
Diagnostic Investigation . . . 520
Root Cause . . . 520
Mitigation & Resolution . . . 520
Case Study: I/O Bottleneck on Shared Storage . . . 520
The Incident . . . 520
Diagnostic Investigation . . . 521
Root Cause . . . 521
Mitigation & Resolution . . . 521
Case Study: Noisy Neighbor in Multi-Tenant Cloud . . . 521
The Incident . . . 521
Diagnostic Investigation . . . 521
Root Cause . . . 522
Mitigation & Resolution . . . 522

Chapter 26: Capacity Planning and Performance Regression **523**

Chapter 26: Capacity Planning and Performance Regression **524**
Capacity Modeling and Forecasting . . . 524
Little's Law . . . 524
Queueing Theory and the Hockey-Stick Effect (M/M/m Models) . . . 525
Baseline Management and Versioning . . . 525
Regression Detection: Automated and Manual . . . 525
Load Testing and Stress Testing Strategies . . . 526
Performance Budgets and Guardrails . . . 526
Continuous Performance Validation . . . 526

Chapter 27: Advanced Topics and Emerging Technologies **527**

Chapter 27: Advanced Topics and Emerging Technologies **528**
Real-Time Kernels and PREEMPT_RT . . . 528
Kernel Bypass: DPDK, XDP and AF_XDP . . . 528
x86-64 vs ARM64: Performance Differences . . . 529
RDMA and Low-Latency Networking . . . 530
eBPF in Production: Advanced Patterns . . . 530
Future Directions: Kernel and Hardware Trends . . . 530

Conclusion: Putting It All Together **531**

Conclusion: Putting It All Together **532**
The Engineer's Mindset for Performance . . . 532
When Not to Optimize . . . 533
Amdahl's Law . . . 533
The Costs of Premature and Over-Optimization . . . 533
Building a Performance Culture . . . 533

Introduction

Boring Linux

A detailed, progressive guide to understanding and operating Linux systems, from the first administered machine to advanced systems engineering.

The book begins with the machine, installation and boot. It develops single-host administration, storage, Linux networking and network services, security, application services, virtual machines and containers. Later parts bring those facilities together through observability, performance, automation, fleets, availability, recovery and kernel-level investigation.

Starting point

You should be comfortable opening a terminal, editing a text file and running a simple command. No prior systems-administration knowledge is assumed. New systems concepts are explained when they appear, and each practical task includes the command details needed to complete it.

Worked examples use Ubuntu 26.04 LTS as the reference environment. General Linux mechanisms are distinguished from distribution-specific packaging, policy and defaults. Other distribution behavior is explained where it changes the administrative task.

How to learn

Follow the parts in order on the first pass. Each subject connects its mechanism to configuration, observation, operation, diagnosis and recovery. Diagrams show relationships; worked examples explain input, output and the limits of an observation. Labs provide results you can verify, and recall cards support later reference.

The opening inspection examples can be studied before building the lab VM. Repeat them on that VM after completing the lab-building chapter. Configuration and recovery experiments belong in the designated disposable lab.

Current writing progress

The syllabus has been rebuilt and writing has restarted. Part 01 currently contains eight complete chapters:

1. What Linux Consists Of
2. Hardware, Firmware and Virtual Machines
3. CPU, Memory, Storage and Network Resources

4. System Paths, Configuration and State
5. Processes, Services and Sessions
6. Distributions, Releases and Documentation
7. Building the First Virtual Machine
8. Inventory, Evidence and First Troubleshooting

Part 02, Installing and Provisioning Linux, now contains eight chapters:

1. Installation Media, Authenticity and Integrity
2. Planning Firmware, Storage, Identity and Networking
3. Guided Installation and the Resulting Storage Layout
4. Manual Storage Choices, Encryption and Swap
5. First Boot: Administrative Access, Updates and Service Inventory
6. Hardware and Virtual-Device Discovery
7. Image-Based and Unattended Installation
8. Reinstallation and Host Configuration Recovery

Part 03, Boot, Shutdown and Recovery, begins with:

1. Firmware, UEFI Variables, GPT and the EFI System Partition
2. Bootloader Selection: GRUB and systemd-boot
3. Kernel Images, Command Lines and Matching Modules
4. Initramfs and Reaching the Real Root
5. Root Handoff, PID 1 and the Boot Transaction
6. Targets, Activation, Readiness and Boot Timing
7. Shutdown, Reboot and Services That Delay Shutdown
8. Secure Boot, Unified Images and Measured Boot
9. Rescue, Emergency and External-Media Recovery
10. Offline Inspection, Chroot Repair and the Next Normal Boot

Part 04, Users, Identity and Administrative Access, contains:

1. UIDs, GIDs, Groups and Ownership
2. Account Databases, NSS and Identity Lookup
3. Account Creation, Locking, Expiration and Removal
4. Home Directories, Skeletons and Session Environments
5. Passwords, Credential Storage and Policy
6. PAM Stacks and Diagnosing Login Failures
7. Administrative Privilege and Bounded Delegation
8. Service Accounts and Runtime Credentials
9. Sessions, logind and Authorization Beyond the Terminal
10. Central Identity: LDAP, Kerberos and SSSD Foundations

Part 05, Packages, Software and Updates, is complete:

1. Package Metadata, File Ownership and Installed State
2. dpkg, APT and Dependency Resolution

3. Repositories, Signatures and Source Configuration
4. Version Selection, Holds, Pinning and Dependency Conflicts
5. Configuration Files, Alternatives and Package Hooks
6. Security Updates and Maintenance Windows
7. Kernel Updates, Reboots and Livepatch
8. Snaps, Flatpak and Other Software Formats
9. Source Builds, Local Packages and Reproducibility
10. Offline Repositories, Mirrors and Disconnected Installation
11. Interrupted Package Transactions and Recovery

Part 06, Services, systemd and Scheduled Work, begins with:

1. Service Supervision, Units and Managers
2. Execution Environment, Identity and Exit Status
3. Startup Types, Readiness, Watchdogs and Restarts
4. Requirements, Ordering and Failure Propagation

The following map is the planned curriculum. Later parts are added to the chapter tree as they are written.

Curriculum map

Part	Subject
01	The Linux Machine and the Administration Lab
02	Installing and Provisioning Linux
03	Boot, Shutdown and Recovery
04	Users, Identity and Administrative Access
05	Packages, Software and Updates
06	Services, systemd and Scheduled Work
07	Processes, Memory and Resource Administration
08	Configuration, Devices, Time and Local Logs
09	Disks, Partitions and Filesystems
10	Volumes, RAID, Encryption and Storage Health
11	Backup, Restore and Data Lifecycle
12	Linux Host Networking
13	DNS, Address Assignment and Time Services
14	Remote Access, Certificates and Trust Services
15	Linux Gateways, Firewalls and Virtual Networks
16	File, Storage and Messaging Services
17	Linux Access Control and Isolation
18	Hardening, Secrets, Audit and Security Response
19	Web, Application and Data Services

Part	Subject
20	Virtualization and Virtual Machine Operations
21	Containers and System Containers
22	Kubernetes and Linux Cluster Workloads
23	Troubleshooting and Observability
24	Performance and Capacity Engineering
25	Provisioning and Configuration Automation
26	Fleet Maintenance and Software Delivery
27	High Availability and Shared Infrastructure
28	Reliability, Incidents and Disaster Recovery
29	Advanced Linux Architecture and Kernel Internals
30	Tracing, eBPF and Failure Analysis
31	Specialist Linux Systems
32	Integrated Systems Engineering Projects
33	Running Multiple Kernels on One Bare-Metal Server
34	Linux Performance Engineering & Server Benchmarking

Specialist branches include multikernel bare-metal architectures, performance engineering and production benchmarking, desktop/session infrastructure, hardware and accelerators, cloud hosts, edge systems and disconnected environments. Each identifies its prerequisites so the main progression remains usable without specialized equipment.

The Linux Machine and the Administration Lab

System Foundations

What Linux Consists Of

You log in to a Linux machine, run a command, and receive an answer. Several distinct components cooperated to make that happen: firmware started the machine, a kernel manages its resources, a shell interpreted your command, a program ran, and libraries helped that program request operating-system services. The files and policies around those components came from a distribution and from whoever configured this particular installation.

Learning administration begins by separating those responsibilities. Otherwise, every failure looks like "Linux is broken," and every attempted fix becomes a guess about which component to restart, reinstall or replace.

By the end of this chapter, you should be able to distinguish a kernel from a distribution, a program from a process, and an installed system from the system currently running. You will also build a small evidence table about a machine without changing its configuration.

A map of the system

Think of an installed Linux system as three cooperating things:

1. **A mechanism for running and protecting work:** the kernel, its supported interfaces and its device support.
2. **A collection of software:** libraries, administrative tools, service managers and applications.
3. **A particular configured installation:** its identities, packages, settings, data and runtime state.

The kernel project's introduction explicitly distinguishes the kernel from a complete distribution. That distinction becomes practical whenever the kernel version, operating-system release and application version tell different stories.

The kernel

The **kernel** is the central privileged software managing the machine's resources. It schedules runnable work on processors, also called CPUs, manages memory, implements filesystem and networking facilities, and coordinates access to devices through drivers.

A **driver** is software that lets an operating system control a device or implement a device-facing interface. Some Linux drivers are built into the kernel image; others can be loaded as modules. You do not need to compile drivers to begin administration. You do need to recognize that a visible device, a loaded driver and a working application are different milestones.

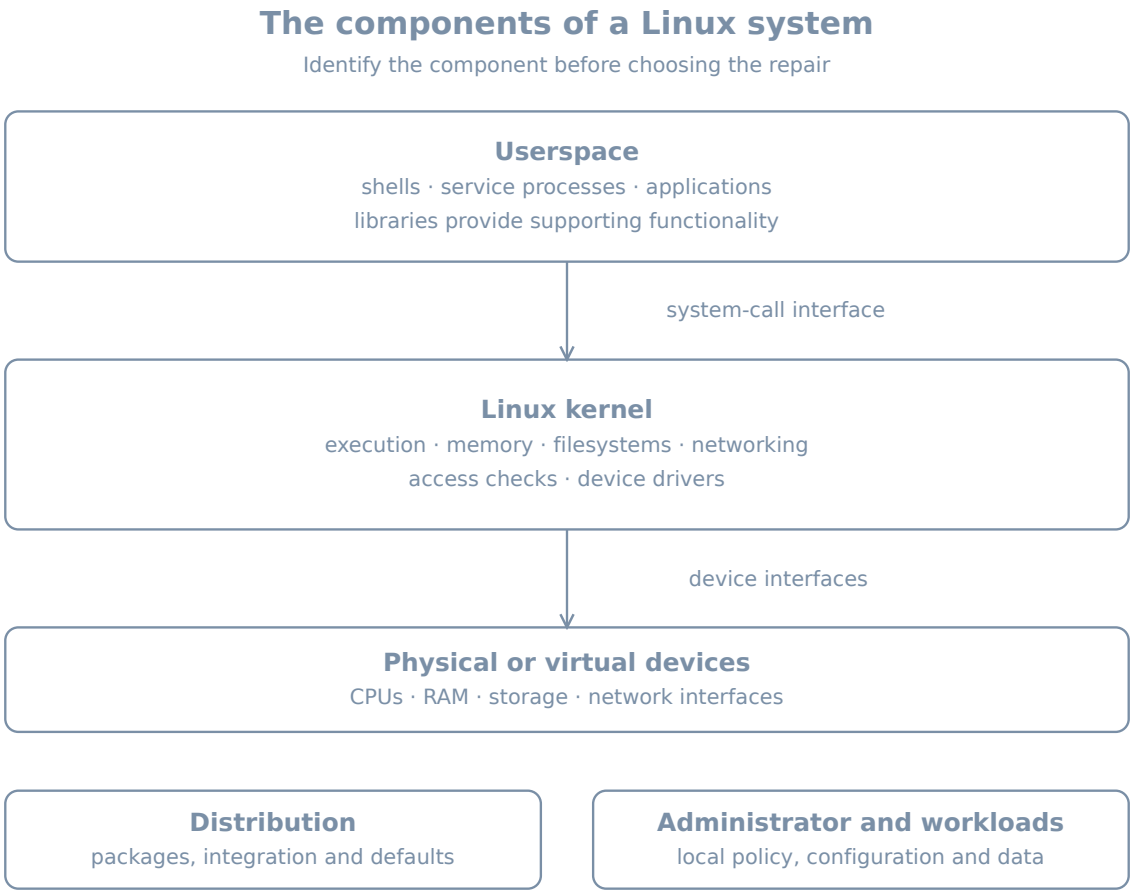


Figure 1: An installed Linux system combines userspace programs, a kernel and the machine's devices.

The kernel is not your shell, graphical desktop, web server or package manager. Those programs can request its services, but each also has its own configuration and failure modes. A web server can have a syntax error while the kernel continues scheduling processes normally.

Userspace

Userspace is the environment in which ordinary application and system-service code runs. It includes shells, editors, databases, login programs and many administrative daemons. A **daemon** is a program intended to provide background service; a **service** is an operational role that can involve one or several processes.

Root-owned programs are still userspace programs. Being a privileged user does not turn a process into part of the kernel. Privilege changes which operations it may request, subject to the host's other policies.

The separation provides a useful diagnostic question: did an application reject its own configuration, did a library fail to load, or did a kernel operation fail? All can prevent a command from completing, but they call for different investigations.

Libraries and system calls

A **library** supplies reusable functionality to programs. A program might use a library to format text, look up a name, handle encryption or interact with the operating system. Libraries can be shared by many installed programs or incorporated into a program when it is built.

A **system call** is an interface through which userspace asks the kernel to perform an operation such as opening a file or creating a socket, a communication endpoint. Programs commonly reach it through library wrappers. A library function is not necessarily one system call: it might do all its work in userspace, make several calls, or use a different call on different architectures. The Linux system-call reference describes this boundary.

For administration, the important idea is the contract. A program asks for an operation; the kernel applies the relevant mechanism and policy, then returns a result. A failed request is evidence. Reinstalling the requesting program does not necessarily change why the request was denied.

The distribution

A **distribution** assembles a usable operating system from the kernel and other software. Its maintainers choose package versions, build options, update policies, installation tools, integration and defaults.

Ubuntu, Debian and Fedora can all run Linux while differing in package management and security policy. Two Ubuntu machines can also differ because they were installed from different images, upgraded along different paths or configured for different roles.

The distribution supplies a starting point. It does not make every installation identical. When a runbook says "the default configuration," ask which release, image and installation path the author actually observed.

The installation and its running state

An **installed system** is software and persistent state arranged on storage. A **running system** is the current execution of a kernel and its processes, with active mounts, interfaces and other live state.

Installing a new kernel package does not switch the running kernel to that version immediately. Editing a service configuration file does not guarantee the running service has read it. These are examples of a general distinction:

Persistent state	Related runtime state
Kernel and boot files on disk	Kernel selected for this boot
Service unit and configuration files	Loaded manager configuration and running processes
Network configuration files	Addresses and routes active now
Account database entries	Identity and group information held by existing sessions
Filesystem table	Filesystems actually mounted

Administration means understanding both sides and the operation that brings them into agreement.

Follow one ordinary request

Suppose a shell runs:

```
cat /etc/os-release
```

The pathname names a file containing operating-system identification data. In an ordinary interactive session, the shell locates `cat` and arranges to run it. The resulting process opens the file, reads bytes and writes output to its standard output, which is connected to the terminal in this example.

That explanation crosses several ownership boundaries:

Component	Responsibility in this example
Shell	Interpret the command and arrange program execution
`cat` program	Implement the requested read-and-copy behavior
Loader and libraries, where applicable	Prepare the executable and provide supporting functions
Kernel	Perform the requested file and output operations under the applicable policy
Filesystem and backing storage	Supply the file's content, possibly already cached in memory

Component	Responsibility in this example
Terminal environment	Present the output to the user

Not every read reaches a disk. The content can already be cached. Not every program needs the same shared libraries. The table identifies responsibilities without assuming one low-level implementation for every executable.

If the file is absent, the executable is missing, or the output destination has failed, the same command line can fail for different reasons. Learn to preserve the actual error rather than translating all failures into "command does not work."

Worked example 1: identify the operating system and kernel separately

The following inspection commands can be run in an existing Linux environment. If you do not yet have a lab host, study the examples now and repeat them after the VM-building chapter. No installation is required to understand the model.

Input:

```
cat /etc/os-release
uname -s
uname -r
uname -m
```

Illustrative output, with unrelated release fields omitted:

```
NAME="Ubuntu"
VERSION_ID="26.04"
ID=ubuntu

Linux
<running-kernel-release>
x86_64
```

Read the answers as four different facts. The release file describes the operating-system userspace in the filesystem view you are inspecting. `uname -s` identifies the kernel name. `uname -r` identifies the running kernel release. `uname -m` reports a machine architecture identifier for this execution environment.

The os-release specification defines the release fields. Do not edit them to "upgrade" a system. Changing its label changes neither its packages nor the running kernel.

An application package can have yet another version. There is no expectation that an Ubuntu release number, Linux kernel release and web-server version should match numerically.

What this does not prove: which kernel will boot next, whether the installation is fully updated, or whether this filesystem view is the physical host's root. Record those as unanswered questions.

Worked example 2: separate a command from the kernel it queries

Input in a Bash session:

```
command -v uname
uname --version
cat /proc/sys/kernel/osrelease
uname -r
```

Possible observations:

```
/usr/bin/uname
<the installed implementation's version banner>
<running-kernel-release>
<running-kernel-release>
```

Why: `uname` is itself an installed userspace program. Its `--version` banner describes its implementation; its `-r` result describes the running kernel. An implementation without `--version` can reject that option while still providing the kernel query. Use the local manual rather than treating one vendor's options as universal.

The `/proc/sys/kernel/osrelease` interface supplies a second route to the kernel release. `/proc` is a kernel-provided filesystem view, not an ordinary directory of files copied onto a disk. Reading it is observation; writing to other writable kernel interfaces can change runtime behavior. Here we use only the documented read operation.

On a Debian-family installation, you can additionally ask who owns a known installed pathname:

```
dpkg-query -S /usr/bin/uname
```

This queries the local package database. The recorded path and package can differ with the installation's layout or implementation. A "no path found" result needs investigation; it is not proof that the kernel has disappeared. File ownership, runtime kernel identity and package availability answer different questions.

Worked example 3: distinguish program, process and manager

A **program** is executable code and associated files. A **process** is an instance executing with a process ID, credentials, memory and open resources. Starting the same program twice usually produces distinct processes.

In Bash, input:

```
printf 'This shell PID: %s\n' "$$"
ps -p "$$" -o pid=,ppid=,comm=
ps -p 1 -o pid=,comm=,args=
```

Illustrative output:

```
This shell PID: 1842
1842 1799 bash
1 systemd /sbin/init
```

Your shell is one process. PID 1 identifies the initial process in the PID namespace visible to you. On the reference system's ordinary host environment, it is normally the system manager. In a container, PID 1 can instead be the container's application or a small init program.

This is why finding a `systemctl` executable does not prove a systemd manager is running in the environment. Likewise, finding a server executable on disk does not prove the server has a running process or an active listening socket.

You will learn namespace details later. For now, record the boundary: a process listing describes the process view available to that command.

Worked example 4: inspect one system call boundary

This is an optional observation exercise if `strace` is already available. It does not require writing a program or installing packages now. The strace manual documents tracing a child command and selecting system calls.

```
strace -e trace=openat,read,write,close cat /etc/os-release
```

A successful trace might include lines with this shape:

```
openat(AT_FDCWD, "/etc/os-release", O_RDONLY) = 3
read(3, "NAME=...", 131072) = <positive-byte-count>
write(1, "NAME=...", <byte-count>) = <byte-count>
close(3) = 0
```

These lines are schematic, not a byte-for-byte transcript. You can also see executable startup, libraries and locale files. The numeric file descriptor `3` is a handle chosen in this process, not a permanent identifier for the release file. Descriptor `1` is standard output by convention.

Now predict the result for a pathname you know does not exist. The open should fail, and there can be no successful read of that nonexistent file through the failed open. A trace is useful when it tests a prediction; a long trace alone is not an explanation.

Some restricted environments do not permit tracing even your own child process. Preserve that error as an observation about the environment. Do not alter host security policy merely to complete this optional first exercise.

The same Linux label can describe different boundaries

You may encounter a physical installation, a full VM, a container, a live image or an offline filesystem exposed through a repair environment. Each can contain familiar Linux paths.

Ask two questions together:

- **Which kernel is answering?** Kernel queries describe the running execution environment.
- **Whose files am I reading?** A mounted installation or isolated filesystem view can supply userspace files from a different environment.

Consider an offline installation accessed from rescue media. Its release file belongs to the installed userspace. The rescue environment's running kernel still answers kernel queries. There is no inconsistency to fix simply because those versions differ.

This habit prevents a common repair mistake: building an installed artifact for the recovery kernel because a command returned the recovery environment's release. The full recovery procedure comes later, but the reasoning starts here.

Break/fix: a package upgrade did not change the running kernel

Symptom. An administrator installs an updated kernel package, then reports that the update failed because `uname -r` still shows the previous release.

Hypotheses. The package could have failed to install; it could be installed but not selected by a boot entry; or no new boot has happened yet. These are different conditions.

Diagnosis. Record the running release, inspect the installed package result and identify which boot artifacts and entries exist. At this point in the book, the important observation is that package installation changes stored files, while `uname` queries current execution. Do not delete old kernels to force the two observations to agree.

Resolution. If installation and boot configuration are correct, the new kernel takes effect through a planned boot into that entry. Check its release afterward. If the host already rebooted into the old release, investigate selection and fallback rather than repeating the same install blindly. Later boot and update chapters provide the exact workflow.

Success criterion. You can explain both the installed state and the running state, including any intentional difference.

Practice: build a component evidence table

Make the following table in your notes using the read-only examples above. If an interface is absent or a command is restricted, record that exact result.

Question	Evidence source	Your observation	Remaining uncertainty
Which userspace release is visible?	`/etc/os-release`		Is this the host, a guest or another filesystem view?
Which kernel is running?	`uname -r`		Which kernel will boot next?

Question	Evidence source	Your observation	Remaining uncertainty
What architecture is reported?	`uname -m`		Which hardware/virtualization layer exposes it?
What program is PID 1 here?	`ps -p 1`		What is outside this process view?
Where is the query command?	`command -v uname`		Which package or installation method supplied it?

Then classify these statements:

1. "The package manager installed a new executable." This is a claim about persistent software state.
2. "A process is still running the previous executable." This is a claim about runtime state.
3. "The command exists, so its corresponding service must be healthy." This conclusion is unsupported.
4. "The release file and kernel release have different numbering." This can be normal; they identify different components.

The exercise is complete when you can explain the component behind each answer and one thing the answer does not establish. No configuration cleanup is needed because the inspection did not change system settings.

Recall card

- Linux's kernel manages resources and supplies interfaces to userspace.
- Shells, service managers, libraries and applications are separate software components.
- A distribution integrates software, packages, policy and update practices.
- A program on disk and a running process are different objects.
- Installed files and current runtime state can intentionally differ.
- `os-release`, kernel queries and package queries identify different things.
- Record which environment supplied an answer before interpreting it.
- Diagnose the responsible component, then verify the specific claim you made.

Hardware, Firmware and Virtual Machines

A Linux server can be a physical computer in a rack, a VM on your laptop, a cloud instance, or a guest nested inside another VM. Its administrative tools may show a CPU, memory, disks and interfaces in every case. The crucial difference is who supplies those resources and which layer you can actually control.

Before building the lab, learn to name these layers. This will explain why changing a VM's boot order can fix one startup problem, why installing a guest driver cannot create missing host storage, and why a four-vCPU guest does not necessarily own four physical CPU cores.

By the end of the chapter, you should be able to describe a machine from firmware through operating system, distinguish a VM from a container, and collect evidence about your execution environment without treating detection as proof of every underlying detail.

Map: a machine can be physical or presented by another system

Hardware is the physical equipment: processors, memory, controllers, storage media, network interfaces and supporting components. **Firmware** is software associated with hardware or platform startup. A **virtual machine** is a presented machine model capable of running its own guest operating system.

The **host** provides the VM's execution environment. The **guest** runs inside the VM. A **hypervisor** implements or coordinates virtualization so guest execution and resources can be managed. These words identify relationships; they do not imply that every virtualization implementation is organized as the same software stack.

In our reference example, Linux's Kernel-based Virtual Machine (KVM) supplies virtualization facilities, QEMU supplies a machine/device model, and management software can coordinate VM definitions and lifecycle. The guest runs its own kernel. The QEMU system introduction explains its machine model and virtualization accelerators.

What firmware does before Linux is usable

On a typical general-purpose machine, platform firmware initializes enough of the machine to select and start a boot path. The Unified Extensible Firmware Interface (UEFI) is used on many current systems; older PC environments can use a legacy Basic Input/Output System (BIOS) path. Other architectures and devices can have different startup arrangements.

For the first mental model, separate these jobs:

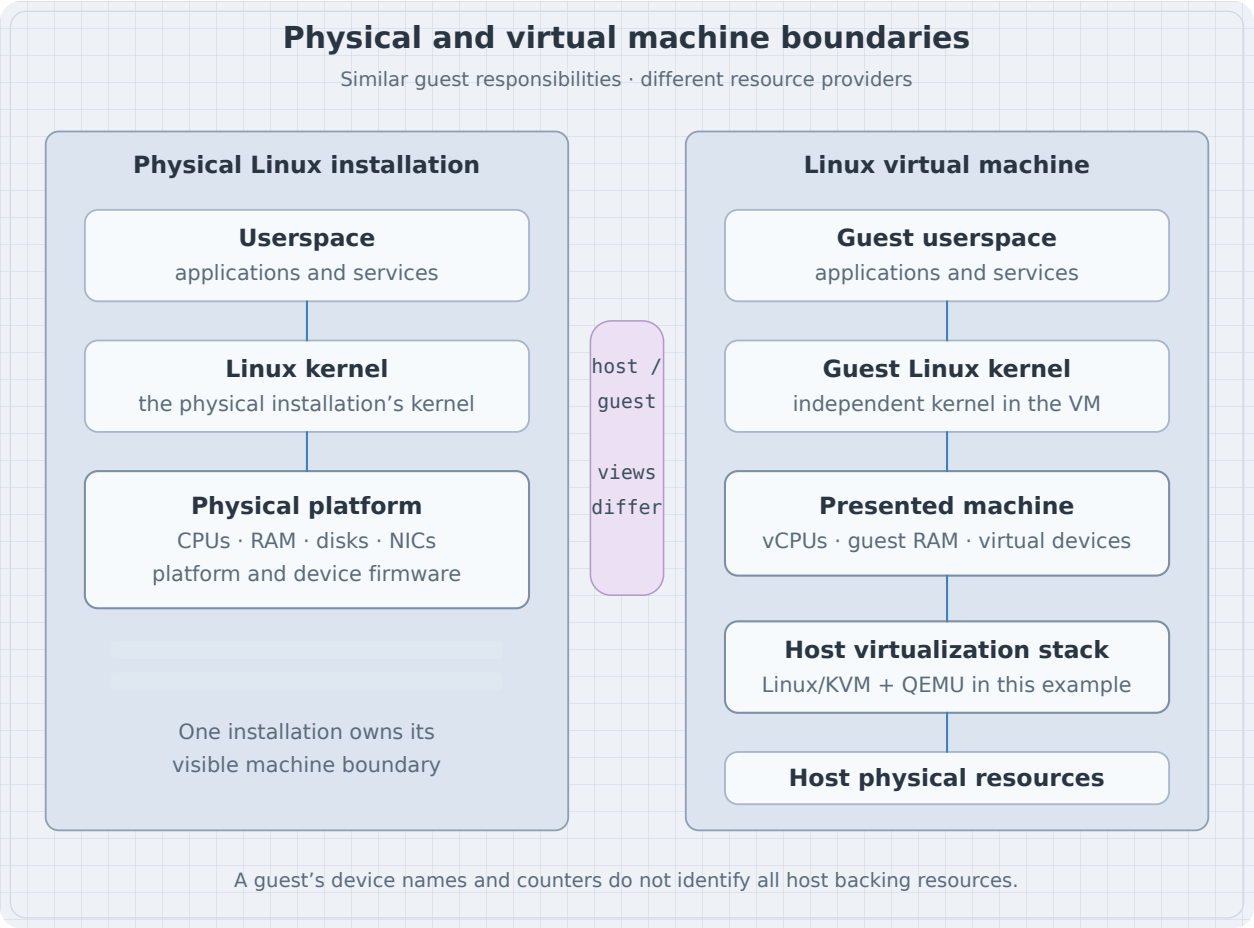


Figure 1: A physical Linux installation and a Linux VM have similar internal responsibilities but different resource providers.

Stage	Responsibility	An example failure
Platform startup	Initialize the platform sufficiently to proceed	No usable console or memory initialization failure
Boot selection	Choose media or an executable boot entry	Wrong disk or installation image selected
Loader/kernel launch	Make the intended kernel and startup inputs available	Referenced kernel file is missing
Kernel initialization	Discover devices and establish core OS facilities	Required storage support is unavailable
Userspace startup	Reach the installed root and start its services	A required filesystem or service fails

Some arrangements combine stages. A firmware entry can launch a kernel-containing artifact directly. The table is a model of responsibilities, not a promise that every machine executes five separate programs.

An administrator can therefore ask a more precise question than "will it boot?": **which stage produced the last reliable evidence?** A boot menu proves something different from a successful login.

Platform firmware and device firmware are related but distinct

A network card, SSD or other device can also have firmware of its own. Updating a platform's firmware does not necessarily update every device. Updating the Linux kernel is another separate operation.

Record which component an update targets, its version, compatibility requirements, activation procedure and recovery route. For a VM, there can also be virtual platform firmware files supplied by the host. Updating a package in the guest does not automatically update those host-provided files.

Detailed firmware updates and Secure Boot policy come later. At this stage, identify ownership before planning a change.

What a virtual machine actually receives

A guest is presented with a set of resources and devices. These need not map one-to-one to physical parts.

Guest-visible resource	Possible host-side implementation	Operational consequence
Virtual CPU, or vCPU	A schedulable guest execution context	The host can delay its execution while serving other work

Guest-visible resource	Possible host-side implementation	Operational consequence
Guest physical memory	Memory backed by host resources under virtualization policy	Guest and host memory pressure need separate evidence
Virtual disk	Image file, logical volume, physical device or remote storage	Free space and failure can exist at several layers
Virtual NIC	A virtual device attached to a host networking path	Guest link state alone cannot prove external connectivity
Console	A virtual display or serial channel	It can remain available when guest networking fails
Firmware and boot variables	Virtual firmware and persistent per-guest state	Firmware mode and saved settings are part of the guest definition

The guest's view is real and useful: its kernel really schedules processes and mounts filesystems. It is simply not the complete view of the infrastructure underneath it.

A disk image illustrates layered ownership

Imagine that guest `lab-01` sees `/dev/vda`, and that its host uses an image file called `lab-01.qcow2` as backing storage.

Inside the guest, an administrator might investigate partitions and filesystems on `/dev/vda`. On the host, another administrator might investigate the image, its backing chain and the filesystem storing it. The guest pathname does not name the host image file. The host file's size does not directly describe the guest filesystem's available space.

If the host filesystem becomes full, guest writes can fail even when the guest reports free filesystem space. If the guest filesystem becomes full, adding space to the host filesystem does not automatically enlarge the guest's virtual disk, partition and filesystem.

Write the complete path when troubleshooting: **application → guest filesystem → guest block device → virtual disk backend → host storage**. Each boundary has its own state and evidence.

Virtual CPUs are not physical-core reservations

Giving a guest four vCPUs makes four guest CPU contexts available. It does not, by itself, reserve four physical cores exclusively for it. Host scheduling, placement, limits and competing workloads affect the resources delivered.

The guest may also see a synthetic CPU topology. Treat `lscpu` inside the guest as information about the presented environment; physical topology requires evidence from the host. Guest-visible instructions and features can be selected for compatibility or migration needs.

The advanced virtualization part will develop allocation and tuning policy. For now, avoid explaining poor performance solely by the number written in the VM settings dialog.

Virtualization, emulation and containers

These terms solve different problems:

Environment	Kernel used by the workload	What is presented or isolated?
Physical installation	The kernel booted on the physical system	Physical platform resources
Full VM	A guest kernel	A machine environment with guest CPUs, memory and devices
Container	The containing system's kernel	Selected filesystem, process, network and resource views
Software machine emulation	A guest kernel	Machine behavior implemented in software, possibly for another CPU architecture

The rows can be combined. A container can run inside a VM. A VM can run on a host that is itself a VM. A virtual machine manager can use hardware-assisted execution or software CPU emulation where supported.

For the first lab, a full VM is valuable because you can practice the guest's own kernel startup, bootloader, storage and recovery without changing your personal system. A container alone does not provide an independent guest kernel boot sequence.

This explains why a container image's release file can name one distribution while `uname -r` reports the containing kernel's release. Neither observation needs to be falsified to make the labels agree.

Worked example 1: establish what virtualization is detected

If these tools are available in your current Linux environment, input:

```
systemd-detect-virt
systemd-detect-virt --vm
systemd-detect-virt --container
```

Possible outputs from separate commands in one environment:

```
kvm
kvm
none
```

The first query asks for a detected virtualization environment. The next two constrain detection to a VM or container. A detection command returns nonzero when it does not detect the requested type; that is a meaningful result, not necessarily a broken command. Do not run a simple collection script with unconditional exit-on-error and lose the rest of the inventory on this result.

The detection manual documents the identifiers and return behavior. Detection is bounded by available signals. A `none` result does not establish exclusive physical access, and a nested environment can have layers the current query does not fully describe.

Record the command's exact result and its execution location: "inside lab-01," "on its hypervisor," or "inside this container." Without that location, a correct observation can still lead to the wrong conclusion.

Worked example 2: inspect the guest-visible machine

Input:

```
uname -m
lscpu
lsblk -o NAME,TYPE,SIZE,MODEL,MOUNTPOINTS
ip -brief link show
```

Illustrative findings on a small VM:

```
Architecture:           x86_64
CPU(s):                 2
Hypervisor vendor:      KVM

NAME   TYPE SIZE MODEL        MOUNTPOINTS
vda    disk  24G Virtio Block
 vda1 part   1G              /boot/efi
 vda2 part  23G              /

lo      UNKNOWN ...
enp1s0  UP      ...
```

The output combines selected fields from several commands. It is not a required installation layout or a promise that a tool will print these exact model strings. Your VM may use SCSI-style virtual disks, different network names and a separate `/boot`.

Why: each tool interrogates a different presented resource. Together they establish what the guest can see, not the physical machine's total capacity. Use the lscpu documentation to distinguish presented CPU topology from host hardware.

Do not add or remove devices during this inventory. Device lifecycle operations belong to later controlled labs.

Worked example 3: observe the firmware interface available here

Input:

```
if test -d /sys/firmware/efi; then
    printf 'An EFI firmware interface is visible here.\n'
else
    printf 'No EFI firmware interface is visible here.\n'
fi
```

Why: the result is an observation about the current kernel's exported view. On an ordinary Linux host, the interface is useful evidence about EFI startup. In a container, restricted environment or specialized boot setup, an absent directory is insufficient to describe every underlying firmware layer.

Suppose you are inside a VM with UEFI firmware. Its interface belongs to the guest's virtual platform. It says nothing by itself about whether the physical host booted with UEFI, or which physical firmware version the host uses.

The safe conclusion is narrow: **this execution environment exposes this interface**. Detailed boot verification later combines it with selected entries, artifacts and command-line evidence.

Worked example 4: understand the KVM boundary without creating a VM

On a Linux host intended to provide KVM-based virtualization, input:

```
if test -e /dev/kvm; then
    ls -l /dev/kvm
else
    printf '/dev/kvm is not present in this device view.\n'
fi
```

The device node is an interface through which a virtual machine monitor can use KVM. The KVM API documentation distinguishes operations for the subsystem, VMs, virtual CPUs and devices.

Presence alone does not prove that your user can open it, that the needed capabilities are available, or that the proposed VM configuration will work. Absence inside a guest does not mean that guest is malfunctioning: nested virtualization may not be exposed or required at all.

Do not change device permissions or firmware settings for this observation exercise. During lab construction, the chosen host's prerequisites and authorized access are checked deliberately.

Keep configuration, execution and saved state separate

A VM definition can describe its next boot while a running VM uses settings established when it started. Some virtualization tools support live changes; others require shutdown. Know whether an edit changed persistent configuration, active state or both.

The libvirt domain format is one example of a definition that describes CPUs, memory, firmware and devices. The existence of a field in that format is not proof that a running guest already uses the edited value.

Also distinguish three useful but different recovery objects:

- **An installation image** supplies software for installing or booting a recovery environment.
- **A VM checkpoint or snapshot** records selected VM state under a particular storage/management implementation.
- **An independent backup** provides recoverable material outside the failure dependency being protected against.

A checkpoint that depends on the original image and host storage can be convenient for a lab experiment. It does not become independent merely because you gave it a reassuring name. Later backup chapters make independence and restore verification explicit.

Break/fix: the guest returns to the installer after installation

Symptom. A VM's installation completes. After restart, the installation screen appears again instead of the installed system.

First hypothesis. The VM may still be booting its attached installation image. That is a boot-selection question, not yet evidence that the installation failed.

Diagnosis path. Use the VM console. Check the selected boot device and whether installation media remains attached. Confirm the intended virtual disk exists and that the firmware mode is the one used for installation. A disk installed for one firmware arrangement may not boot under an arbitrarily changed arrangement.

Fix. Select the installed disk or detach the installer as appropriate for the VM manager. Preserve the firmware configuration unless evidence requires a change. Do not reinstall or repartition merely because the installer appeared.

Verification. Boot the installed system, identify its mounted root and confirm the expected installed files. Keep the console available. Only then proceed to ordinary guest administration.

Alternative result. If selecting the disk produces a loader error, the investigation has advanced to a different boundary. Record that new error; it changes the next question.

Practice: draw the ownership boundary

Choose a Linux environment you can inspect or use the illustrative VM above. Draw five boxes and fill them with facts or explicit unknowns:

1. The machine on which your current commands execute.
2. The kernel answering those commands.
3. Its firmware interface and presented devices.
4. The provider of its CPU, memory, disk and networking.
5. The console and recovery route available if networking stops.

For a VM, annotate at least one disk path twice: once as the guest sees it and once as the host or management system identifies its backing resource. If you lack host access, write "host backing unknown"; do not infer a host pathname from `/dev/vda`.

Then answer:

Statement	Supported conclusion
"The guest has two vCPUs."	It has two presented execution contexts, with allocation details still to establish
"The guest's disk has free space."	This guest filesystem has reported capacity; host backing capacity remains a separate question
"Virtualization detection printed `none`."	This query did not identify the requested virtualization type
"The guest has a console."	There may be an access route independent of guest networking; test the actual management path

The exercise succeeds when you can locate an observation and a proposed change on the correct side of the host/guest boundary. No VM changes are required in this chapter.

Recall card

- Firmware, kernel startup and userspace startup are separate responsibilities.
- A VM has its own guest kernel and presented hardware resources.
- Host and guest inventories describe different administrative boundaries.
- Guest vCPUs and disks do not imply exclusive physical resources.
- Containers share their containing kernel; a full VM can practice independent boot.
- Detection results are evidence with limits, especially in nested environments.
- Configuration, active execution and saved/checkpoint state can differ.
- Keep a console and an independently understood recovery route for experiments.

CPU, Memory, Storage and Network Resources

A service can be slow while its CPU is mostly idle. A machine can have little completely unused memory and still have room for new work. A filesystem can be full while the physical disk is healthy. A network interface can be up while no client can reach the service.

These are ordinary situations once you stop treating a machine as one number called "performance." A running workload uses several resources, waits on dependencies, and moves through queues. The first task is to identify which resource a measurement describes and which part of the workload is actually waiting.

This chapter builds that working model. You will inventory CPU, memory, storage and interfaces; interpret several basic measurements; and follow a request across the system. Detailed tuning comes later. Here the goal is to explain observations before changing settings.

Map: four resources, one workload

Use this first model:

Resource	What it supplies	Typical question
CPU	Execution of instructions	Can the work get execution time when it needs it?
RAM	Fast working storage for active state	Can required data remain available without excessive reclaim or waiting?
Persistent storage	Durable space and read/write service	Is there capacity, and can the stack complete I/O in time?
Networking	Communication between endpoints	Can the intended path carry useful traffic with acceptable delay and loss?

Capacity, demand and delivered service are different quantities. A 24-GiB disk describes a size. Twenty milliseconds describes a delay. A thousand operations per second describes a rate. None is a substitute for the others.

CPU: execution capacity and the need to wait

A **central processing unit (CPU)** executes instructions. A **CPU core** is a hardware execution resource. A **logical CPU** is an execution context exposed to the operating system. A core can

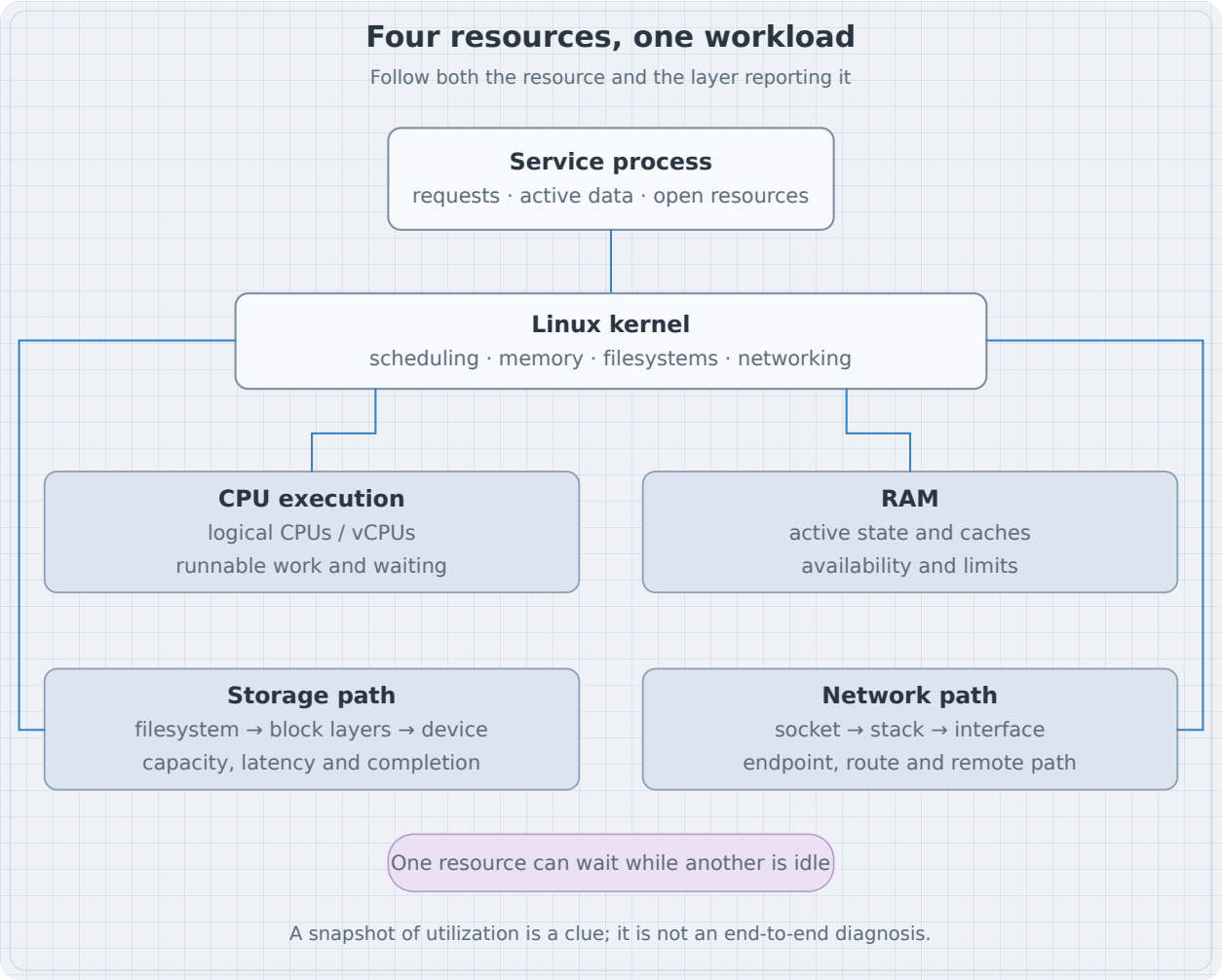


Figure 1: CPU, RAM, storage and network devices cooperate under the Linux kernel.

expose more than one logical CPU through simultaneous multithreading; a VM can expose virtual CPUs supplied by its host.

A **thread** is an execution stream that the scheduler can arrange to run. A multithreaded process can have several such streams. A runnable thread needs a turn on an eligible CPU. A thread waiting for a timer, disk result or remote reply may not be runnable at all.

The **scheduler** is the kernel mechanism that selects runnable work for execution. You do not need its implementation details yet. You do need to distinguish **running**, **ready to run**, and **waiting for something else**.

Two logical CPUs do not make every operation twice as fast

Suppose one request needs 20 milliseconds of CPU work and cannot execute that work in parallel. An otherwise idle second CPU does not halve that request's CPU time. The extra context may allow another independent request to execute concurrently, subject to actual hardware and allocation.

For a simple capacity estimate, 100 requests per second at 20 milliseconds of CPU work each require:

```
100 requests/s × 0.020 CPU-seconds/request = 2 CPU-seconds/s
```

That already consumes the nominal two-CPU execution budget of a two-vCPU guest, before overhead and other processes. It is a starting model, not a benchmark result. Shared host capacity, affinity, quotas, cache behavior and simultaneous multithreading can change delivered work.

This is why adding a service without considering its workload can overload a host whose installation requirements looked small.

Worked example 1: inventory CPU topology

Input:

```
lscpu
lscpu -e=CPU,ONLINE,SOCKET,CORE,NODE
```

Illustrative table:

```
CPU ONLINE SOCKET CORE NODE
  0    yes      0    0    0
  1    yes      0    1    0
```

Here, two logical CPUs are online in the presented topology. A **socket** is a processor-package grouping in this view; a **NUMA node** groups memory/CPU locality information. On a VM these can be virtual arrangements. On another architecture some columns can be absent or unavailable. The lscpu manual describes the inventory sources and output options.

Record the environment with the table. Do not multiply a count twice because both a summary and a per-CPU table display the same resources. Also distinguish "online" from "available to this particular workload": affinity, resource policy and host scheduling can impose additional limits.

Worked example 2: elapsed time is not CPU time

In Bash, input:

```
time sleep 1
```

One possible result:

```
real    0m1.002s
user    0m0.001s
sys     0m0.001s
```

Why: approximately one second passed, but the command spent very little time executing. It was mostly waiting for its timer. `user` accounts for execution in userspace; `sys` accounts for kernel execution charged to the command. Exact timing varies.

A slow network request or a blocked disk read can exhibit the same broad pattern: long elapsed time with little CPU execution. The correct question is then "what was it waiting for?" rather than "why is the CPU not busy?"

Load average is a demand clue

Input:

```
cat /proc/loadavg
```

The first three values describe load averages over different time windows. Linux includes runnable work and tasks in uninterruptible sleep in this accounting; it is not simply a percentage of CPU time. The load-average interface documents the fields.

A rising load with idle CPU can lead you toward waiting tasks and I/O. A low average does not prove a latency-sensitive request never waited. Time windows and sampling can hide short events. Treat this as a lead to investigate, not a verdict about the bottleneck.

Memory: active state, cache and limits

Random-access memory (RAM) holds active program state, kernel data and caches. It is usually volatile: a power loss removes its contents. Persistent storage supplies durability through a different path.

A process normally accesses a **virtual address space**, an arrangement of addresses mapped and protected by the kernel and processor. The amount of address space described for a process is not

automatically the amount of physical RAM it occupies. Some regions may not have been touched; some mappings refer to files; some pages are shared.

The kernel's memory concepts overview introduces virtual memory, protection and controlled sharing. Our practical task is to ask which memory is actively needed, what can be reclaimed, and which limits govern the workload.

Used RAM can be useful cache

Linux can retain file data in RAM so later reads avoid slower storage. This **page cache** uses memory, but some cached pages can be reclaimed for other work. Dirty pages, which contain changes not yet written back as required, have different constraints from clean cache pages.

Therefore, "almost no completely unused RAM" does not by itself mean the machine is out of memory. Conversely, a large cache is not a guarantee that a particular workload will fit: active demand, writeback and configured limits matter.

Swap supplies backing storage for some memory that can be moved out of RAM. It is not equivalent to additional RAM with the same access cost. A little swap use alone does not prove current thrashing. Sustained pressure and the workload's waiting behavior provide stronger evidence.

Worked example 3: read a small set of memory facts

Input:

```
free -h
awk '/^(MemTotal|MemFree|MemAvailable|Cached|SwapTotal|SwapFree):/ {print}' \
    /proc/meminfo
```

Illustrative selected fields:

```
MemTotal:        4096000 kB
MemFree:          180000 kB
MemAvailable:    2400000 kB
Cached:          2100000 kB
SwapTotal:       1048576 kB
SwapFree:        1048576 kB
```

Why: this machine has little entirely free RAM but a much larger estimate of memory available for new work. The estimate accounts for more than simply adding two fields. Do not independently sum overlapping accounting categories and present the sum as additional capacity.

`MemTotal` is usable memory reported by the kernel, so it need not match the exact number entered in a VM configuration or printed on a RAM module. The meminfo reference defines these fields, and free presents a summary of them.

In a container or other constrained workload, host-style totals may not express the limit governing that process. A **cgroup** is a kernel grouping used to account for and control resources; a workload can hit its group limit while the whole host still has available memory. The cgroup v2

documentation provides the underlying hierarchy and controller model. We will configure it in the resource-administration part.

What this snapshot cannot show: how long pressure lasted, which allocation failed, whether swap traffic is currently intense, or whether a service's own internal limit was reached. Keep the scope of the measurement with its value.

Storage: a device, a filesystem and a path are different objects

A **block device** exposes addressable storage blocks. A **partition** describes a region of a partitioned device. A **filesystem** organizes data and metadata. A **mount** attaches a filesystem view at a location in the directory tree.

The relationships can be simple:

```
virtual disk → partition → ext4 filesystem → mounted at /
```

Or layered:

```
physical storage → encryption mapping → logical volume → filesystem → /srv/data
```

Those are examples, not universal ordering rules. An installation may use RAID, several devices, datasets, network storage or a different arrangement of layers. Always inspect the actual stack.

A **pathname** such as `/srv/data/report.txt` names an object through the current filesystem view. It does not tell you which physical disk supplies the data. A mount can change which filesystem serves that same pathname.

Worked example 4: connect a path to mounted storage

Input:

```
lsblk -o NAME,TYPE,SIZE,FSTYPE,MOUNTPOINTS
findmnt --target / -o TARGET,SOURCE,FSTYPE,OPTIONS
df -h /
df -i /
```

An illustrative device tree:

```
NAME   TYPE SIZE FSTYPE MOUNTPOINTS
vda    disk  24G
 vda1 part   1G vfat   /boot/efi
 vda2 part  23G ext4   /
```

`lsblk` describes visible block devices and related filesystem information. `findmnt` identifies the mounted filesystem serving a path. `df` asks about that filesystem's space or inode accounting. An

inode holds a filesystem object's metadata; inode exhaustion can prevent new files even when byte capacity remains.

The lsblk manual explains why explicit output columns and many-to-many storage relationships matter. The findmnt manual distinguishes asking about a path from asking whether the path itself is a mountpoint.

If this is a container, `/` might be an overlay filesystem with backing storage outside your current view. Its source need not appear as a simple partition in `lsblk`. Record the visible relationship without inventing an unseen physical device.

File size and allocated storage can differ

A **sparse file** can represent a logical range without allocating blocks for every byte in that range. Compressed filesystems, snapshots and shared extents introduce other differences between logical size and physical allocation.

Optional experiment in an ordinary writable lab filesystem:

```
workdir=$(mktemp -d)
printf 'Experiment directory: %s\n' "$workdir"
truncate -s 64M "$workdir/sparse.bin"
ls -lh "$workdir/sparse.bin"
du -h "$workdir/sparse.bin"
```

On a filesystem supporting the expected sparse-file behavior, the listed logical size is 64 MiB while the allocated space reported by `du` can be much smaller. This does not mean that storing 64 MiB of actual new data is free. Allocation occurs as data is written, subject to the filesystem's behavior. The du documentation distinguishes allocated storage from apparent size.

Cleanup only the file created for this experiment:

```
rm -- "$workdir/sparse.bin"
rmdir -- "$workdir"
```

No partitioning, formatting or privileged disk operation is needed. Avoid carrying a pathname from this example into unrelated cleanup commands.

Capacity and I/O performance are separate

A device can have ample space but poor response time. A fast device can still be full. Workloads also differ: many small random reads stress a different path from a large sequential write.

Useful dimensions include:

- **Latency:** time for an operation or request to complete.
- **Throughput:** amount of useful data or work completed per unit time.
- **IOPS:** storage operations completed per second, meaningful alongside operation size and workload.

- **Queue depth:** operations outstanding at a particular point in the stack.

A queue can absorb short bursts. Sustained arrival faster than completion causes waiting and eventually pressure or failure. More queueing is not automatically more performance, particularly when the user cares about response time.

Networking: an interface is one part of a service path

A Linux **network interface** is an object through which traffic can enter or leave a network stack. It can correspond to a physical network interface controller (NIC), virtual NIC, bridge, tunnel or another software interface. The **loopback** interface supports communication within the local network stack.

An IP address belongs to an interface or networking context under configuration policy. A **route** selects a next path for a destination. A **socket** is an operating-system communication endpoint used by a program. A server listening on a socket still depends on routing, filtering and the remote client path.

Later chapters will configure these facilities and their services. For now, keep four questions distinct:

1. Is the interface present and usable at the link layer?
2. Is addressing and routing suitable for the intended destination?
3. Is the service listening at the address and port the client uses?
4. Does the end-to-end application operation succeed?

Worked example 5: inventory interfaces and listening endpoints

Input:

```
ip -brief link show
ip -brief address show
ip route show
ss -lnt
```

Illustrative observations:

```
lo      UNKNOWN ...
enp1s0  UP      ...

LISTEN 0 128 127.0.0.1:8080 0.0.0.0:*
```

Why: the interface exists and appears up, while the example TCP listener is bound to loopback. A remote client cannot reach that loopback-only listener by using the server's LAN address. That is an endpoint configuration issue to investigate before blaming a physical cable.

`ss -lnt` lists listening TCP sockets with numeric addresses. Other transport types need the corresponding query. See the socket inspection manual. The output reflects the network namespace in which the command runs; another container or VM can have a different view.

Worked example 6: look at counters without overinterpreting them

First choose an interface name from your inventory. For an interface named `enp1s0`:

```
ip -s link show dev enp1s0
```

This is an example name; replace it if the host uses another. The counters describe traffic and selected errors or drops at this reporting interface. The kernel's interface-statistics documentation explains the available reporting paths.

One cumulative counter cannot establish the current rate. If received bytes increase by 10,000,000 over 10 seconds, the observed average is:

```
10,000,000 bytes ÷ 10 seconds = 1,000,000 bytes/s = 8,000,000 bits/s
```

Use consistent units. A nominal 1-Gbit/s link corresponds to 125 million bytes per second before protocol and implementation overhead. It is not 1 gigabyte per second, and it does not guarantee that rate for an application.

Counter interpretation also needs timing, reset/wrap awareness and the relevant interface. Traffic can pass through several virtual interfaces; summing every counter along the same path can count the same work repeatedly.

Follow a request through all four resources

Imagine a small catalog service returning a file-backed record:

1. Traffic reaches the host's network stack and the appropriate socket.
2. The service becomes runnable and receives CPU time.
3. It parses the request and accesses in-memory state.
4. It needs record data. A cached result may be available in RAM; otherwise the path can require storage I/O.
5. While waiting for necessary I/O, the thread need not consume CPU execution time.
6. The service becomes runnable again, constructs its response and submits output.
7. Networking carries the result toward the client.

The exact implementation might be asynchronous or use several threads, but the resource questions remain. End-to-end delay can include several waits. A snapshot showing idle CPU cannot identify which wait occurred.

For an invented request, suppose total elapsed time is 100 milliseconds, with 8 milliseconds of CPU execution. There are approximately 92 milliseconds left to explain in that accounting model. Possible contributors include scheduling delay, storage, locks, a connection pool or a remote dependency. The missing time is a question, not evidence for whichever subsystem you happen to know best.

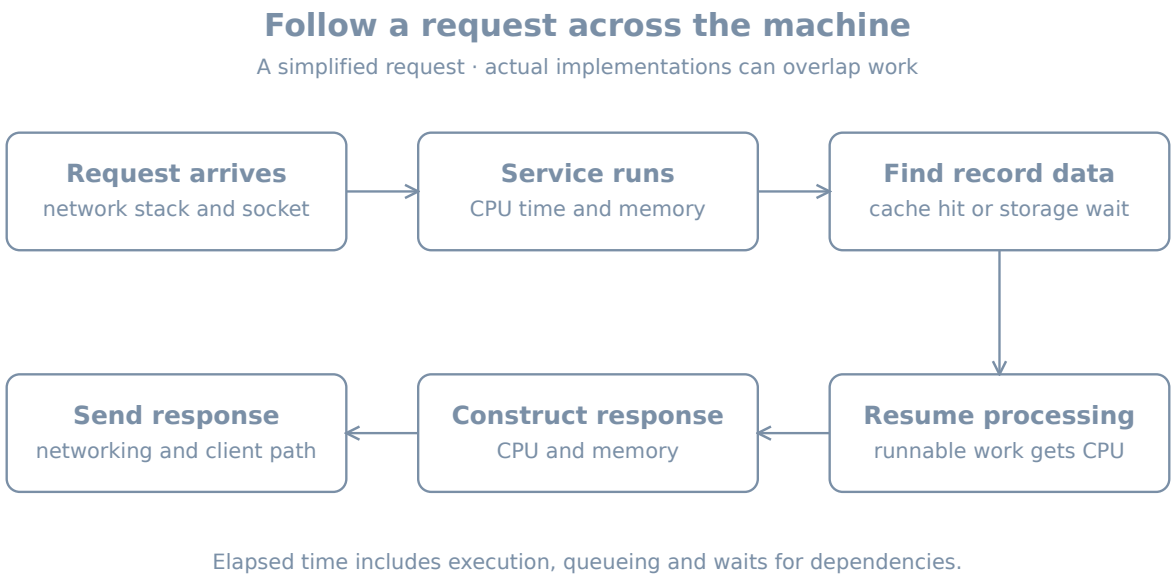


Figure 2: A request can alternate between CPU execution and waiting on other resources.

Break/fix: "the VM has resources, but the service is slow"

Symptom. A two-vCPU VM has free disk capacity and some available memory. Requests still slow down badly during a scheduled backup.

Initial claim to reject. "There is unused CPU, so the guest must need a kernel tuning change." The evidence does not support that conclusion.

Investigation. Compare a baseline period and the backup period. Record request latency and rate, CPU execution, memory pressure, storage completion behavior, interface traffic and the backup schedule. Keep guest and host evidence separate. The backup may compete for guest I/O, host storage, network capacity or another shared dependency.

Bounded action. If evidence identifies backup contention, change its schedule or resource policy in the lab and repeat the same workload. If the cause is elsewhere, the prediction should fail and the hypothesis should change. Do not disable backups permanently as a performance fix.

Success criterion. The service meets its measured objective and the backup still completes within its required window. A lower utilization percentage alone is not the goal.

Practice: inventory and explain before tuning

Build a small worksheet from the examples:

Resource	Inventory	Current observation	What remains unknown?
CPU	Online logical CPUs and topology	Load and a small timing observation	Workload eligibility, quotas and host contention
Memory	Usable RAM and swap	Available estimate and current state	Demand over time and workload-specific limits
Storage	Device → filesystem → mount	Byte/inode capacity	Latency, backing layers and failure behavior
Network	Interfaces, addresses and routes	Listener bindings and counters	Remote path and application success

Do not change configuration to make your worksheet match the illustrative output. Different values and device names are expected. If you are reading before building the VM, complete the worksheet from the examples and mark the values as illustrative, then repeat on the lab host later.

Finally, explain these cases in a sentence each:

1. **A command took one second but used almost no CPU.** It can spend elapsed time waiting.
2. **Free memory is small but available memory is much larger.** Some current uses can potentially be reclaimed; inspect pressure and limits before declaring exhaustion.
3. **`lsblk` shows 24 GiB but a filesystem is smaller.** Devices, partitions and filesystem capacity are different layers.
4. **The interface is up but a service listens only on loopback.** Link availability does not establish the desired remote endpoint.
5. **A guest has free space but its host storage is full.** The write path contains several independent capacity boundaries.

The exercise is complete when each conclusion identifies both its evidence and its limits. Clean up the optional sparse-file experiment if you ran it; all other commands in this chapter inspect state.

Recall card

- Separate capacity, utilization, pressure, latency and throughput.
- CPU execution, runnable demand and waiting are different states.
- Virtual CPU counts and topology describe the presented environment.
- Available memory, caches and workload limits need separate interpretation.
- A device, filesystem, mount and pathname describe different storage objects.
- Interface state, routes, listeners and client success are separate networking observations.
- A request crosses resources and can spend most of its time waiting.
- Measure a baseline, form a testable hypothesis, and verify an operational result.

System Paths, Configuration and State

A service works until the machine reboots. An administrator recreates a directory, but the service still cannot find its database. A configuration file contains the intended value, but requests use the old setting. These incidents share a missing distinction: a pathname, stored data, and the state a running program uses are different things.

This chapter gives you a map for finding the right object before changing it. You will distinguish directories from mounts, persistent data from temporary state, and configuration on disk from effective configuration. The examples assume the terminal skills used in the preceding chapters. Inspection commands do not require administrator privileges; the small writing experiment uses only a new directory inside your home directory.

One tree, several sources

An **absolute pathname** begins at `/`, the root of the pathname tree. A **relative pathname** is interpreted from a process's current working directory. `/etc/hosts` and `etc/hosts` can therefore identify different objects. The account called `root`, its home directory `/root`, and the root directory `/` are three separate concepts.

A directory associates names with filesystem objects. A **filesystem** supplies the rules and structures used to represent files, directories and their metadata. Some filesystems store data on disks. Others expose information maintained by the kernel or keep their contents in memory.

A **mount** attaches a filesystem, or another view of existing filesystem content, at a location in the visible tree. That location is the **mount point**. Walking from `/` into `/home` does not tell you whether you crossed into a separate filesystem. The directory names alone cannot answer that question.

Imagine a machine with one filesystem for `/` and another mounted at `/srv/archive`. Before the second filesystem is mounted, `/srv/archive` is an ordinary directory in the root filesystem. Once mounted, looking inside that path shows the mounted filesystem's contents. Existing files in the underlying directory are hidden by the mount; mounting does not automatically copy or delete them.

This explains a common storage incident. A backup job writes into `/srv/archive` while its intended storage is absent. The files consume space on `/`. When the expected storage is mounted later, those files seem to vanish. The correct investigation starts with the mount, not with a claim that the backup application deleted data. Later storage chapters develop guarded mounts and recovery procedures; do not unmount a working service's storage just to explore this behavior.

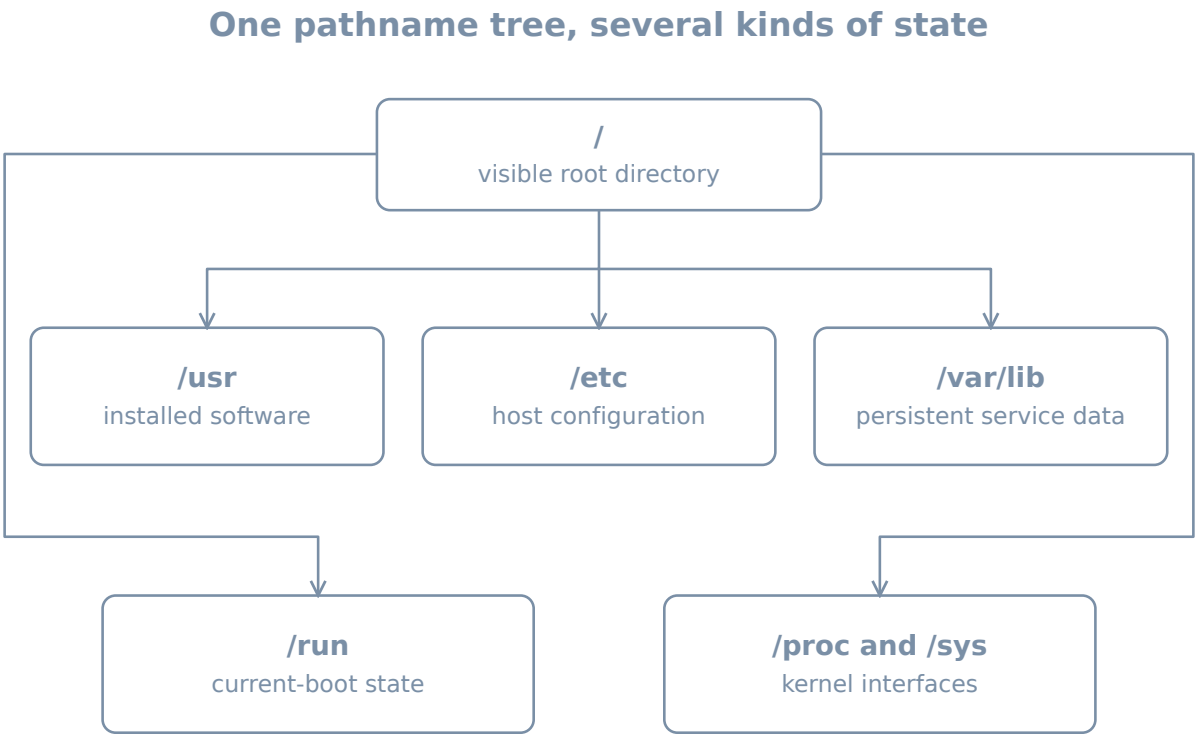


Figure 1: A pathname tree joins storage, volatile state and kernel interfaces.

Worked example: identify the backing filesystem

Run:

```
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
findmnt --target /etc --output TARGET,SOURCE,FSTYPE
findmnt --target /run --output TARGET,SOURCE,FSTYPE
findmnt --target /proc --output TARGET,SOURCE,FSTYPE
```

Illustrative results for the last two commands:

```
TARGET SOURCE FSTYPE
/run   tmpfs  tmpfs

TARGET SOURCE FSTYPE
/proc  proc   proc
```

`--target` locates the filesystem containing the requested path. It also works when that path is not itself a mount point. A result with `TARGET` equal to `/` for `/etc` means that `/etc` is on the root mount in this view. It does not mean `/etc` is missing. The findmnt manual documents the distinction between a containing target and an exact mount point.

`tmpfs` is memory-backed storage that can use swap; it is not a promise that every byte stays in physical RAM. `proc` exposes process and kernel information. Reading `/proc/meminfo` retrieves a kernel-generated view rather than a report saved by someone into a disk file.

These results belong to the environment where you ran the command. A container can see a different mount arrangement from its host. A service can also run with a restricted filesystem view. For now, record whether you inspected the VM, its host, or a container; later isolation chapters explain why those views differ.

Directories express responsibility

The following map is a starting guide to conventional Linux layouts. An installed application may use additional paths, and distribution policy can refine them.

Path	Usual responsibility	Question to ask
`/usr`	Installed system programs, libraries and shared resources	Which package or image supplies this?
`/etc`	Host configuration	Who owns the intended setting?
`/var/lib`	Persistent application and service state	How is this backed up and restored?
`/var/log`	Logs stored as files	What is the retention policy?
`/var/cache`	Data intended to be reconstructible	Can this application actually regenerate it?

Path	Usual responsibility	Question to ask
`/run`	State used during the current boot	Which process recreates it?
`/tmp`	Temporary working files	When can cleanup remove them?
`/home`	Ordinary users' home directories	Is this local, mounted or managed elsewhere?
`/dev`	Device interfaces	What kernel device does this represent?
`/proc`, `/sys`	Kernel-maintained interfaces	Am I reading information or changing a kernel setting?
`/boot`	Boot-related artifacts	Which boot tooling owns these files?

The Linux filesystem hierarchy specification describes these roles in more detail. Use the table to choose an investigation, not as permission to remove an unfamiliar directory.

For example, a database beneath `/var/lib` is not disposable merely because the path begins with `var`. "Variable" describes data that changes, not data you can discard. Similarly, a cache may be reconstructible in principle but expensive or impossible to rebuild while its upstream source is unavailable.

On merged-`/usr` installations, names such as `/bin` can be symbolic links into `/usr`. A **symbolic link** is a filesystem object that points to another pathname. Inspect the machine instead of assuming two visible names imply two independent copies:

```
ls -ld /bin /sbin /lib
readlink -f /bin
```

An illustrative result for the second command is `/usr/bin`. That is pathname resolution, not evidence that a program was moved while you were looking. Package ownership, links and compatibility names matter when interpreting installation instructions.

Persistence is a property to verify

Persistent state is intended to survive the relevant lifecycle boundary, often a reboot. **Runtime state** describes a live instance: processes, open connections, current locks and similar working information. Some runtime information is represented by files, but the presence of a file does not make it durable.

Consider a fictional service called `catalog`:

Object	Example	Intended lifecycle
Program	`/usr/bin/catalogd`	Replaced by software installation or upgrade

Object	Example	Intended lifecycle
Configuration	`/etc/catalog/catalog.conf`	Retained until an authorized configuration change
Business data	`/var/lib/catalog/items.db`	Preserved across process restarts and host reboots
Runtime socket	`/run/catalog/control.sock`	Created for a live service instance
Working cache	`/var/cache/catalog/`	Reconstructed according to application behavior

These are explanatory paths, not a package to install. A socket is an endpoint through which processes can communicate; its pathname is not the database it serves.

Reboot is only one boundary. A file can survive a guest reboot yet disappear when that VM is deleted. A file in a container's writable layer can survive a process restart but disappear when the container is replaced. A VM snapshot can return storage to an earlier state without providing an independent backup. Always complete the sentence: "This survives **which event**?"

Do not use `/tmp` or `/run` for the only copy of important data. Cleanup schedules and mount choices vary. Conversely, do not assume everything under `/var/log` is recorded forever: retention, rotation, storage limits and logging configuration determine what remains available.

Persistence also does not guarantee durability at every instant. A successful application write can still be buffered before reaching persistent storage. Databases coordinate writes and recovery using their own mechanisms. Backup and storage chapters will separate application consistency, filesystem behavior and device durability; at this stage, recognize that copying a live database file is not automatically a valid backup.

Configuration has an owner and an activation path

A **configuration source** expresses intended behavior. **Effective configuration** is what the running component actually uses. Between them may sit include files, defaults, generators, environment variables, command-line arguments and activation steps.

Use four questions whenever you find a promising file:

1. Is it an input owned by an administrator, or an output generated by another tool?
2. What other sources can override or supplement it?
3. How does the application validate the combined configuration?
4. When does the running application read it?

Suppose `catalog.conf` specifies `listen_port=8080`, but the running process still listens on its old port. Several explanations remain possible: the process has not reread the file, it uses a different file, an override takes precedence, or the edit failed validation. Repeatedly editing the same line does not distinguish these hypotheses.

The useful evidence chain is:

```
intended setting → authoritative input → validation
                 → activation → observation of live behavior
```

A **reload** asks a component to reread supported settings without a full stop/start. A **restart** replaces its running instance. Neither is a generic guarantee that every intended change becomes effective. Some settings require restart, and some applications cannot reload at all. Read that application's instructions before choosing an operation.

Generated files deserve special attention. If a network tool writes a runtime resolver file, manually editing that output may appear to work and then be overwritten. The durable fix belongs in the tool's input configuration. The ownership question prevents a cycle of repeatedly "repairing" the generated result.

Worked experiment: a file changes before its consumer does

This experiment models an application that reads a setting once. It does not edit system configuration. Run the commands in one Bash terminal:

```
state_lab_dir=$(mktemp -d "$HOME/boring-state.XXXXXX")
printf 'blue\n' > "$state_lab_dir/setting"
read -r loaded_setting < "$state_lab_dir/setting"
printf 'green\n' > "$state_lab_dir/setting"
printf 'Loaded value: %s\n' "$loaded_setting"
printf 'File value: '
cat "$state_lab_dir/setting"
```

Expected result:

```
Loaded value: blue
File value: green
```

`mktemp -d` creates a uniquely named directory; the variable holds its actual path. `read` copies one line into the shell's memory. Replacing the file does not change that already loaded variable. This deliberately small model reproduces the distinction between stored configuration and a running consumer's state without touching a service.

Now simulate rereading:

```
read -r loaded_setting < "$state_lab_dir/setting"
printf 'Loaded after reread: %s\n' "$loaded_setting"
```

The result should be `green`. This is not an implementation of a service reload protocol. It illustrates why an activation step exists at all.

Clean up only the file and directory created above:

```
rm -- "$state_lab_dir/setting"
rmdir -- "$state_lab_dir"
unset state_lab_dir loaded_setting
```

If you closed the terminal, do not guess the variable's old value. Locate your named experiment directory and inspect it before removing anything.

Diagnosis: three failures that look like missing files

The directory exists, but the data is wrong

A path can exist on the wrong filesystem. Inspect its backing mount and compare it with the expected storage before creating more data there. Preserve observations of the path, mount source and service error. Creating an empty replacement database can make recovery harder by mixing new state with the incident.

The data returns after reboot, but access fails

The file may be persistent while the runtime directory or socket through which a client reaches it has not been recreated. Investigate the service that owns that runtime object. Copying yesterday's socket pathname from a backup does not recreate a listening process.

The administrator can read it, but the service cannot

The service may run under a different identity or see a restricted filesystem tree. "It exists in my shell" establishes only one observer's view. Capture the service identity and its error before changing permissions. Making a file readable by everyone can expose data while leaving the real view or path problem unresolved.

Practice: classify before changing

On your lab VM, select `/etc/hosts`, `/var/lib`, `/run`, `/proc/meminfo` and your home directory. For each, record the intended role, the backing filesystem reported by `findmnt`, the likely owner, and the lifecycle event that could remove its contents. You do not need to read other users' files or list service secrets.

Then answer these scenarios:

1. A service stores its only customer records in `/run`. What failure should you expect after reboot?
2. A mounted archive is absent and a job writes to the mount-point directory. Which filesystem receives the writes?
3. A file contains the new value but the running application behaves as before. What evidence is missing?

Success means you can explain the answers in terms of lifecycle, mount attachment and activation. The first design loses its intended persistence; the second uses the containing filesystem; the third needs the actual configuration sources and live consumer state checked.

Recall card

- A pathname identifies an object through a visible tree; it does not identify the backing storage by itself.
- A mount can hide a directory's previous contents without deleting them.
- Classify data by responsibility and lifecycle before editing, deleting or backing it up.
- Persistent across reboot does not mean protected from replacement, corruption or loss.
- A configuration change is complete only after validation, activation and a behavior check.
- Identify who generates a file before treating it as the authoritative input.

Processes, Services and Sessions

A machine can contain an installed server package without running its server. It can run a server process without making that server reachable. It can accept connections while still returning an application error. Administration becomes much clearer when these are treated as separate observations.

This chapter connects the files from the previous chapter to the work a machine is doing now. You will observe a process, relate it to its parent and identity, inspect a service manager, and distinguish a login session from a long-lived service. The reference environment is a systemd-based Linux VM. The process exercise works in an ordinary Bash terminal; manager and login commands require the corresponding facilities to be running.

Program, process, service: three different objects

A **program** is executable code. Its file can exist while no instance of it is running. A **process** is a running instance with an identity, execution state, memory and references to resources such as open files. Multiple processes can execute the same program. A program can also create workers, so one application's activity need not fit into one process.

A **process identifier**, or **PID**, identifies a process within a PID namespace. A namespace is a mechanism that gives a group of processes a particular view of a resource; for this chapter, the practical consequence is that a container's process numbering may differ from the host's. A **parent PID**, or **PPID**, identifies the parent in that view. Parentage helps explain how work was started, but it is not a complete inventory of responsibility.

A **service** is an operational function managed across a lifecycle: start, readiness, ongoing work, failure and stop. A service manager can start the program, track its processes, collect exit results and apply a restart policy. Some services perform a short task and exit. Others keep several worker processes running.

The **system manager** is the system-wide manager. On the reference VM, systemd runs as PID 1. A **user manager** can separately manage services for an individual user. The fact that both are controlled with `systemctl` does not make them the same manager: `systemctl --user` selects the user manager instead of the system manager.

A container or a minimal recovery environment may have another program at PID 1. If `systemctl` reports that systemd is not running as the system manager, first identify where you are. Installing more service files does not change the identity of the already running PID 1.

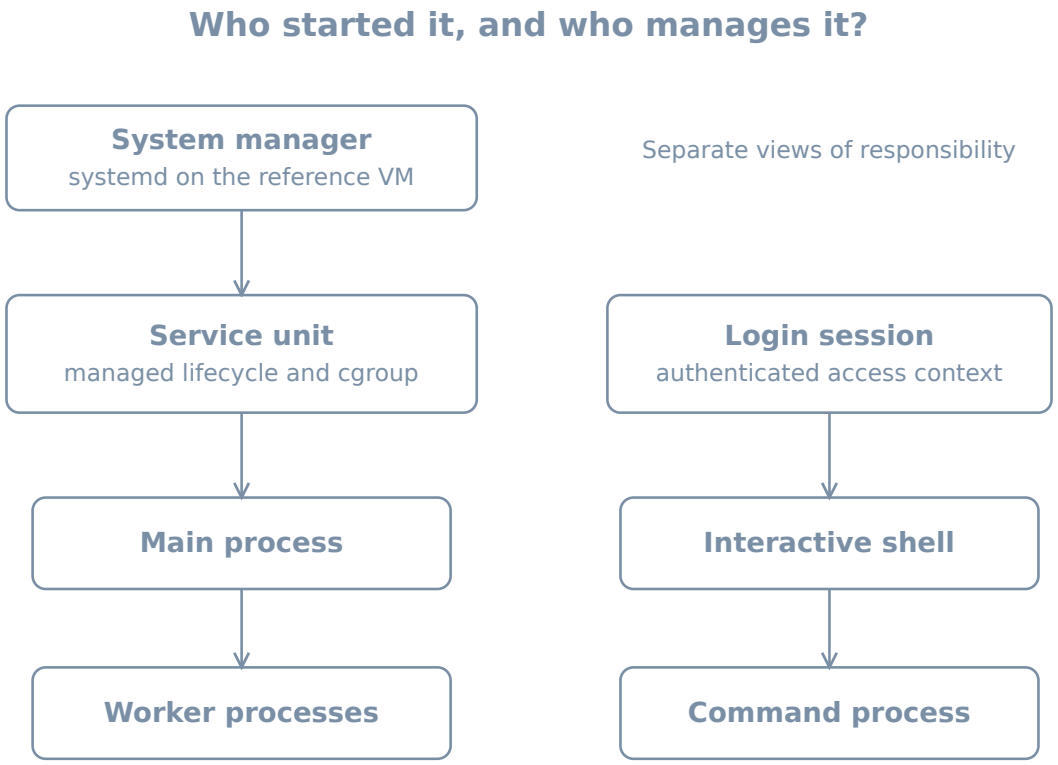


Figure 1: Process ancestry, service ownership and login sessions answer different questions.

Inspect a process without guessing from its name

Start with a small, explicit view:

```
ps -p 1 -o pid,ppid,user,stat,comm
ps -p "$$" -o pid,ppid,user,stat,etime,args
```

In Bash, `$$` expands to the shell's PID. `ps` selects that process and displays the requested columns. `ETIME` is elapsed time since process start. `ARGS` presents command arguments, while `COMM` is a shorter command-name view. Neither name alone proves which package supplied the executable or what the process is doing.

An illustrative shell row is:

```
    PID    PPID USER     STAT     ELAPSED COMMAND
   2410    2409 learner  Ss         08:14 -bash
```

Treat PIDs and times in examples as placeholders for observations, never as values to paste into a later termination command. PIDs are reused after processes exit. A number written in an old incident note might identify unrelated work now.

The first letter of `STAT` describes a scheduling or lifecycle state. Common letters are `R` for running or runnable, `S` for interruptible sleep, `D` for uninterruptible sleep, `T` for stopped and `Z` for a zombie. Additional letters describe other properties; the lowercase `s` above indicates a session leader. A zombie has exited but still has exit information awaiting collection by its parent. It is not doing useful computation that can be stopped again. The ps manual defines the fields and state codes.

Most healthy idle processes spend time sleeping. A sleeping process is not necessarily broken; it may be waiting for a timer or request. Conversely, a runnable process is not proof that useful work is completing. Combine state with elapsed behavior and the task the process is meant to perform.

Command arguments can contain sensitive information. Do not paste an unrestricted process listing into a public bug report without reviewing it. For a first inventory, select the processes and columns that answer your question.

Worked experiment: create, observe and finish one process

Run this in one interactive Bash terminal. It creates only a short-lived timer process:

```
sleep 300 &
observed_pid=$!
printf 'Child PID: %s\n' "$observed_pid"
ps -p "$observed_pid" -o pid,ppid,user,stat,etime,args
jobs -l
```

`&` asks the shell to run the command in the background. `$!` is the PID of its most recently started background job. `jobs` shows work tracked by this shell, not every process on the host.

A typical `ps` row looks like:

```
  PID    PPID USER     STAT     ELAPSED COMMAND
 2581    2410 learner  S          00:03 sleep 300
```

The parent is the shell observed earlier. The process is sleeping because its job is to wait for a timer. It is still a real process with an owner and lifecycle.

Finish the experiment promptly, while this terminal and job are still present:

```
kill -TERM "$observed_pid"
wait "$observed_pid"
printf 'Wait status: %s\n' "$?"
unset observed_pid
```

`TERM` requests termination. In this example, Bash commonly reports status `143`, meaning termination by signal 15. If the timer had already finished, the result differs; do not interpret a missing process as a failure to create it. The shell's `wait` collects the child's completion result. This exercise is for an interactive shell without a strict `set -e` policy, which could stop a script at a nonzero `wait` result.

Do not repeat the `kill` command later using a copied PID. Also do not generalize the example into killing any process whose name looks similar. For a managed service, operate on the service through its manager; a policy may otherwise start a replacement process immediately.

A **signal** is a notification delivered to a process. Some signals can be handled by the application; `SIGKILL` cannot. Immediate forced termination gives an application no opportunity to finish its own shutdown work. The right response to a stuck service begins with evidence and its documented lifecycle, not an automatic escalation to force.

Services give processes an operational boundary

Systemd represents managed objects using **units**. A `.service` unit describes a service, but units also describe sockets, timers, mounts and other objects. This is why a list of all units is not just a list of daemons. The systemd.unit reference describes their types and common configuration.

Inspect the manager without changing its state:

```
systemctl list-units --type=service --all --no-pager
systemctl list-unit-files --type=service --no-pager
systemctl --failed --no-pager
```

The first view concerns service units loaded in the manager. The second concerns installed unit files and their enablement state. A machine can have many installed definitions that are not currently running. `--failed` narrows attention to recorded failed units; it is not a complete application-health test.

On the reference VM, inspect the journal service:

```
systemctl status systemd-journald.service --no-pager
systemctl show systemd-journald.service \
  -p LoadState -p ActiveState -p SubState -p MainPID -p FragmentPath
```

`status` is a human-oriented summary. `show` selects named properties. `FragmentPath` identifies the unit's main definition file; it is not necessarily the whole configuration because additional settings may exist. A denied or partial log view can reflect your account's access rights.

For these inspection commands and the loaded-versus-installed distinction, consult the systemctl manual. This chapter uses long-established operations; the installed manual remains the reference for your specific version.

Running now and starting later are independent questions

Active state concerns the current lifecycle. **Enablement** concerns installation of activation relationships, commonly used to arrange startup through a target. A target groups related units or marks a system state. An enabled service is not automatically healthy, and a disabled service is not automatically stopped.

A service might have been started manually or activated by a dependency. A socket can trigger service activation when traffic arrives. A timer can activate work on a schedule. Units marked `static` generally lack installation instructions for direct enablement but can still be activated by other units.

Keep two columns in your notes: "what it is doing now" and "how it is intended to be activated." Conflating them leads to repairs that work until reboot, or changes that affect future boots while leaving today's incident untouched.

Active does not always mean a process is still running

An `active (running)` service usually has continuing work. An `active (exited)` service can represent a completed setup action whose unit remains active. For example, a one-shot task may establish state and then exit; `RemainAfterExit=yes` allows its unit to retain an active state after successful completion. There may then be no main process to inspect. Service type and configuration explain the state, as described in systemd.service.

The **main PID** is useful but not the entire workload. A service can have workers. Systemd uses **control groups**, or **cgroups**, to group processes for management and resource accounting. Parent ancestry and cgroup membership are different relationships: one explains creation, the other helps express management membership. Resource-control chapters develop those mechanisms later.

A session is a user's access context

A login session represents an authenticated interaction such as a local console login, graphical login or remote login. It can contain several processes. The login session is not identical to the shell, and a shell is not required to be an interactive login shell.

On a VM with systemd-logind, use:

```
loginctl list-sessions
loginctl session-status
```

The first command lists sessions known to the login manager. With no ID, the second asks about the caller's session. Some environments, including terminals launched outside a registered login session, have no suitable session to report. That is a limitation of the context, not evidence that your shell has no process. See the loginctl manual.

A shell job, a login session and a user service have different owners of their lifecycle. Closing a terminal can affect jobs through hangup handling and session policy. A user service can have another lifecycle, with details depending on the user manager and system policy. Merely adding `&` is not a reliable way to create an unattended server that must survive logout and reboot.

Suppose an administrator starts a development server from an SSH terminal. It responds during the demonstration but disappears after logout. Before looking for a network fault, ask who was responsible for maintaining that process. Was it a shell job, a user service or a system service? The permanent design needs an explicit lifecycle, logging and restart behavior. Later service chapters build that design rather than relying on a terminal remaining connected.

Observe from the client back toward the process

For a fictional inventory service, the useful chain is:

```
package present → unit definition present → process started
                → application ready → endpoint reachable
                → intended operation succeeds
```

Each arrow requires evidence. The chain is a diagnostic model, not a guarantee that every service has a network endpoint. A scheduled backup job, for example, is verified by its completion result and a usable restore rather than by a listening port.

Consider three reports:

Report	What it establishes	Next question
"The package is installed."	Software files were installed	What starts the intended function?
"The service is active."	The manager reports an active unit	What does active mean for this unit type?
"A TCP connection succeeds."	A network endpoint accepted a connection	Can an authorized client complete the real operation?

A process can remain alive while its database is unavailable. A network path can work while authentication fails. A manager's restart policy can replace a crashing process often enough that one snapshot catches it running. Repeated observations, logs and a meaningful client operation make these cases distinguishable.

First failure investigation

When a service does not behave as expected, write the symptom as an operation: "the local client cannot obtain the inventory list," rather than "Linux is broken." Then identify the service and record the time and environment.

For the journal service used above, a bounded log inspection is:

```
journalctl -b -u systemd-journald.service -n 30 --no-pager
```

`-b` selects the current boot, `-u` selects a unit and `-n 30` limits the tail. Missing records may reflect access, retention or the absence of messages. Do not treat an empty log view as proof of successful operation.

For another service, substitute its verified unit name. Capture the manager state, error and recent relevant records before restarting. A restart can restore service while discarding evidence about the original failure. After an authorized repair, repeat the client operation that originally failed; a new PID alone is insufficient.

Do not stop the journal service, login manager or remote-access service as a beginner exercise. The timer experiment already provides a process lifecycle you can observe safely. Later disposable-VM labs introduce deliberately broken services with defined recovery steps.

Practice: build an evidence table

Record your shell PID, its parent, the owner and one observed state. Run the timer exercise and explain why its state is consistent with its purpose. Then select one service already present on your VM and record its active state, enablement state, main PID if any, and the useful operation it provides.

Answer these questions:

1. If a PID changes after a service failure, does that prove the application recovered?
2. Can a disabled unit still be active?
3. Can a healthy service have no continuing process?
4. Does `jobs` show work started by another user's shell?

The answers are no, yes, yes and no. Explain the reason for each: process identity differs from behavior; activation has multiple paths; completed one-shot work can retain active state; job control belongs to a particular shell.

Recall card

- Program files, process instances and managed services are different objects.
- A PID is temporary evidence, not a permanent identity.
- Process ancestry, service membership and login sessions describe different relationships.
- Check both current activity and intended activation.

- Interpret service state using the unit's purpose and lifecycle.
- Verify the useful operation after a repair, not just the existence of a process.

Distributions, Releases and Documentation

Two machines both run Linux, yet a guide's package name exists on only one of them. A configuration option appears in an upstream manual but fails validation on the installed server. A security scanner flags an old-looking version even though the distribution reports that its package contains the fix.

These are problems of identity and responsibility. The kernel is only one part of the installed system. To choose instructions or plan a change, identify the distribution release, package source, installed build and party responsible for maintaining it.

This chapter teaches that identification process. Its package examples use the Ubuntu/Debian APT and dpkg toolchain. They inspect local metadata and do not install software or add repositories. You can read them before creating the reference VM; use that VM for the matching commands rather than assuming your host has the same package tools.

A distribution is an integration and maintenance boundary

A distribution combines the kernel, libraries, system tools and applications into an installable system. Its maintainers decide how software is built, which defaults are supplied, how dependencies fit together, how updates are delivered and which releases remain supported.

The **upstream project** develops a component such as a web server. The **distribution package** integrates a particular build into an operating-system release. The **installed system** adds local configuration and workload data. A cloud image or appliance vendor may add another layer. Each layer can make decisions that affect your machine.

Suppose an upstream guide places a server's example configuration in `/usr/local/etc`, while your package reads `/etc/example`. Neither path should be chosen by familiarity alone. Inspect the installed package and service. Your responsibility is to find the configuration actually used by this instance.

The same distinction applies to service names, optional modules and startup defaults. A distribution may split software into several packages or ship a separate integration unit. Knowing the upstream product name is useful; it is not a complete installation inventory.

Release models create different change schedules

A **release** is a named, coordinated version of a distribution. A **fixed-release** model groups substantial changes into releases, with maintenance updates during each release's supported life. A **rolling-release** model delivers an evolving collection of packages without requiring the same kind of major-version transition.

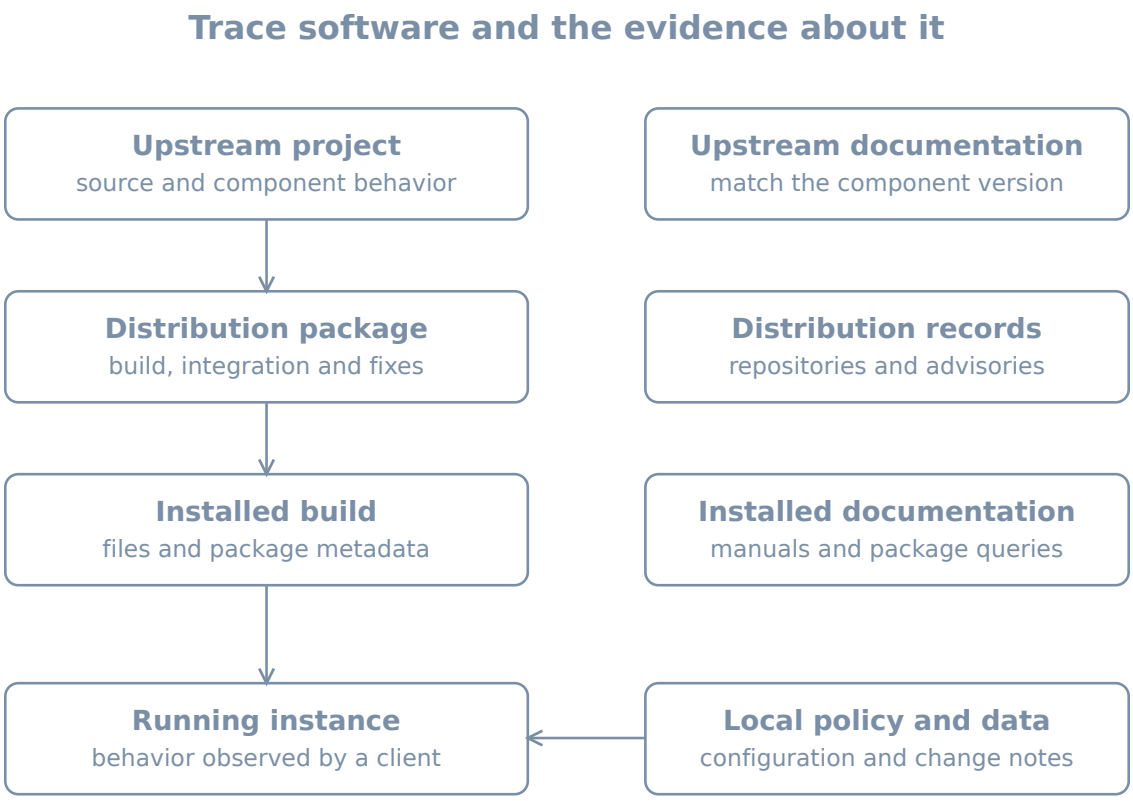


Figure 1: Software passes through several owners before it becomes a running service.

Neither model eliminates operational work. Fixed releases require planned release upgrades and continued maintenance between them. Rolling systems require regular attention to package transitions and compatibility. Both need backups, recovery access and testing appropriate to the workload.

Long-term support, or **LTS**, describes a maintenance commitment for a particular product and scope. It does not mean every program installed on the machine receives identical coverage. A separately installed vendor application, container image or manually built tool can have its own lifecycle.

Ubuntu's official release-cycle page distinguishes LTS, interim releases and support offerings. The book uses Ubuntu 26.04 LTS as its reference release. For deployment decisions, check the current coverage of the exact release and packages rather than copying a date from a tutorial. Record the check date alongside the result, since an operational plan must remain meaningful after time passes.

A practical support record includes:

Field	Why it matters
Distribution and release	Identifies the base maintenance stream
Architecture	Determines available builds and installation media
Package and source	Identifies who supplied the software
Coverage and conditions	Establishes which updates are actually available
Maintenance deadline	Gives a date by which action must be planned
Application compatibility	Identifies what needs testing before an upgrade
Recovery route	Explains how service will be restored if a change fails

"Supported operating system" is a useful starting statement, but the record must become specific enough to guide action. Conversely, an unsupported component does not become supported merely because it runs on an LTS host.

Worked example: separate distribution identity from kernel identity

On the machine being investigated, run:

```
cat /etc/os-release
uname -r
uname -m
```

`/etc/os-release` describes the operating-system identity. `uname -r` reports the running kernel release, and `uname -m` reports the machine architecture as exposed to the running environment.

Fields you may see include `ID`, `VERSION_ID`, `PRETTY_NAME` and `ID_LIKE`. Use `ID` and `VERSION_ID` to record the distribution and version; `PRETTY_NAME` is intended for display. `ID_LIKE` can suggest relationships but is not a guarantee that instructions for another distribution are compatible. The

os-release specification defines these fields and identifies `/etc/os-release` as the preferred location, with `/usr/lib/os-release` as fallback.

Do not edit release-identification fields to make an installer accept the host. That changes the reported identity without providing the software compatibility the installer expected.

A container offers an instructive case: its distribution identity can describe the container's userspace, while `uname` reports the kernel it shares with the host. A VM normally has its own guest kernel. Two identity commands can therefore report different layers without either being wrong.

Likewise, installing a kernel package does not by itself change the kernel already running. An update inventory and a runtime inventory answer different questions. Later maintenance chapters explain activation and reboot planning; here, preserve both observations rather than forcing them into one "Linux version" field.

Packages, repositories and origins

A **package** is a managed software artifact with metadata such as name, version, architecture and dependencies. A **repository** publishes packages and metadata for tools that select and retrieve them. A repository is a software supply relationship, not merely a convenient download address.

On Ubuntu and Debian, dpkg tracks installed Debian packages, while APT uses available-package metadata and dependency rules to plan operations. Ubuntu's package-management documentation explains the responsibilities and workflows. Other distributions use different tools; the questions about installed versions and origin remain the same.

A mirror provides a delivery location for repository content. Its proximity does not by itself establish trust. Package-management trust involves the authenticated metadata, configured authorities and repository policy. A third-party source can introduce packages and dependency choices outside the base distribution's maintenance process. Adding one is an administrative decision that needs an owner and removal plan.

This chapter does not ask you to add a repository. Before considering one, ask: What software is missing? Who signs and maintains this source? Which distribution releases and architectures are covered? Which installed packages could it replace? How will updates and retirement work? A copied command containing a signing key and a repository URL does not answer those questions.

Installed version and candidate version

On the reference VM, inspect the APT package itself:

```
dpkg-query -W -f='${binary:Package}\t${Version}\t${Architecture}\n' apt
apt-cache policy apt
```

The first command asks the installed-package database for selected fields. Single quotes keep the shell from expanding the field expressions before `dpkg-query` reads them. The dpkg-query manual defines the query format.

`apt-cache policy` distinguishes the installed version from the candidate selected using current local metadata and policy. Its version table helps identify the available origins and priorities. The candidate is not necessarily a newer version and is not necessarily the highest version published anywhere. Repository selection and policy constrain what this host considers.

The apt-cache manual also explains an important limitation: metadata is cached locally and can be stale. These inspection commands do not refresh it. If the cache is old or absent, label that limitation in the inventory rather than concluding that no updates exist.

For a beginner's observation sheet, record the installed value, candidate value and listed source. If they differ, explain the difference before proposing an upgrade. If the candidate is absent, investigate metadata and source configuration; do not immediately substitute a package from an unrelated release.

Which package owns a file?

A pathname can be supplied by a package, a local installation, an application at runtime or a configuration generator. On the reference VM, inspect a known packaged file:

```
dpkg-query -S /usr/bin/apt
```

An expected result is a package ownership record naming `apt`. This establishes the package database's ownership claim for that path. It does not verify that the current file's contents are unchanged, and it does not identify every possible manually installed program on the machine.

This distinction is useful when a guide says to overwrite an executable or edit a vendor-supplied file. First identify ownership. A local replacement may be overwritten by an upgrade, while a package-managed file may have a supported override mechanism that survives updates. Later chapters cover verification and configuration-file handling in detail.

Versions are evidence, not complete security verdicts

A distribution can apply a selected correction to an older software series without adopting all features of a newer upstream release. This is called **backporting**. Consequently, comparing only the upstream version prefix can misclassify a distribution package. Debian's security FAQ explains why its security maintenance uses backported fixes and why version-only checks can be misleading.

The opposite shortcut is also wrong: do not assume that an old-looking package is patched merely because backports exist. Identify the advisory, distribution release and exact package revision. Then compare the installed package with the advisory's affected and fixed builds.

For an Ubuntu-managed component, Ubuntu Security Notices are an authoritative place to begin that lookup. A component installed from another vendor needs that vendor's advisory path. Container images require attention to the packages inside the image as well as the host.

A useful security investigation record is:

```
Reported issue:
Affected component and source:
Distribution release and architecture:
Installed package revision:
Applicable vendor advisory:
Fixed revision or other required action:
Runtime activation needed:
Verification result and time:
```

This record separates discovery, installation and activation. Even after a corrected library is installed, a long-running process may still use previously mapped code until it is restarted. The appropriate action depends on the component; checking only the package database can leave the runtime question unanswered.

Find documentation that belongs to the installed system

Use several kinds of evidence together:

Source	Best use	Limitation
Installed manual and `--help`	Syntax and features shipped with installed tools	Manuals may be absent or incomplete
Distribution documentation	Packaging, integration and supported workflows	Check release applicability
Upstream manual for the matching version	Component behavior and configuration semantics	Distribution defaults or patches can differ
Release notes and advisories	Changes, known issues and required actions	Match the affected product and build
Local change records	What administrators intentionally changed	Records can be incomplete or stale

For example, `man 1 ps` asks for the user-command manual named `ps`. Manual section numbers distinguish categories: section 1 commonly describes commands, section 5 file formats, and section 8 administration tools. `man -k` searches manual descriptions, provided a suitable local index is available. The man manual documents lookup and section selection.

On the reference VM, try:

```
man 1 ps
man 5 os-release
man 8 apt-cache
```

Press `q` to leave the usual manual pager. If `man` or the pages are absent on a minimal image, use the distribution's versioned manual site and record that you used an external source. Missing local documentation is not a reason to run an unrelated installer from a search result.

Upstream pages marked “latest” can describe options added after your installed release. Before adopting an unfamiliar option, inspect the installed version, its local manual and the option’s version notes. A failed option does not necessarily indicate a damaged installation; it may be a feature mismatch.

Worked investigation: a guide does not match the machine

Suppose a guide instructs you to enable a new configuration directive, but the service’s validator reports “unknown directive.” A productive investigation proceeds as follows:

1. Identify the actual executable and service instance that produced the error.
2. Record its package, installed revision and source.
3. Check the installed manual for the directive and its valid location.
4. Check the guide’s target version and distribution assumptions.
5. Decide whether an older supported equivalent meets the requirement, or whether a planned upgrade is necessary.

Do not remove validation just to make startup proceed. Do not install a foreign release’s library to satisfy one command without evaluating the rest of the system. The service may depend on an integrated collection of libraries and plugins. A narrow documentation mismatch can become a broad recovery incident if treated as a package-download problem.

This is also where local records matter. If the executable came from `/usr/local` and the unit was written locally, distribution instructions may be only part of the answer. Ownership must be traced to the actual installation path.

Practice: write a documentation and support card

For your lab VM, record distribution identity, running kernel, architecture, the installed APT package version and its candidate. Add the source of your metadata and whether its freshness is known. Then locate one installed manual, one relevant distribution page and one upstream reference for a component you inspected.

Write one sentence explaining which source answers each question:

- Where does this distribution put the component’s configuration?
- Does this installed version implement a particular option?
- Which package revision fixes a reported vulnerability?
- What local change was made before the failure?

Finally, choose a maintenance deadline from the official lifecycle information applicable to the reference installation and record its scope and your access date. Do not silently include third-party software under that deadline.

Success means another reader could identify your environment and find the same class of documentation without guessing from a screenshot or a marketing name. Keep machine-specific values out of the book itself; they belong in your lab notes.

Recall card

- Distribution identity, running kernel and installed package versions describe different layers.
- Support belongs to a defined product, release, source and scope.
- Repository configuration establishes a software supply relationship.
- Installed and candidate versions are different observations; cached metadata has an age.
- Backported fixes require package-specific advisory checks, not version-prefix guesses.
- Match documentation to the installed component, then verify local behavior.

Building and Observing the Lab

Building the First Virtual Machine

The first six chapters gave you a way to describe a Linux machine. Now build a machine you can restart, disconnect and recover without risking the system you use every day. The result is more than a successful installation: it is a named lab with a known console, a controlled network and a demonstrated return to a baseline.

This chapter uses a local **x86-64 Linux host**, hardware-assisted virtualization, KVM/QEMU, libvirt and the Virtual Machine Manager graphical application. Host preparation commands are for an Ubuntu host; the guest is Ubuntu Server 26.04 LTS. If the host uses another distribution, install its supported KVM/libvirt and virt-manager packages, then join the walkthrough at the host validation step. Do not run Ubuntu package commands on an unrelated host distribution.

An ARM host requires a matching guest architecture, firmware and virtualization setup. Cross-architecture emulation is a different performance and compatibility exercise. A managed work computer also needs permission to install a hypervisor and create virtual networks. A remote server without a graphical session needs a remote console workflow; that is outside this first local-host path.

Define the lab before opening the installer

Use the following identifiers consistently. The hypervisor's VM name and the guest's hostname are separate settings, even when both initially say `bl01`.

Item	First-lab choice
VM name	`bl01`
Guest hostname	`bl01`
Guest login	`learner`, with a unique lab password
CPU	2 vCPUs
Guest memory	4096 MiB
System disk	One new 32-GiB qcow2 disk
Firmware	UEFI, selected before installation
Initial network	A libvirt NAT network for installation and updates
Practice network	`blpractice`, isolated, with host address `192.168.231.1/24`
Console	Virtual Machine Manager's graphical display
Baseline	Powered-off full clone `bl01-baseline`
Recovery rehearsal	Another clone, `bl01-recovered`, started alone

These are a teaching budget, not a minimum specification for every Linux workload. Leave several GiB of RAM and adequate CPU capacity for the host. Budget roughly 100 GiB of free host storage for the guest, two full-size copies, installation media and headroom. Sparse images initially consume less than their virtual capacity; that is not a guarantee they will stay small. If the host cannot accommodate the budget, reduce the scope or use another lab host before running out of space during a copy.

A **qcow2 image** is a QEMU disk-image format with metadata and optional features such as backing files. Our first disk is a standalone image with no backing file. Do not attach a physical disk, personal home directory, shared clipboard secrets, USB security key or host filesystem share to this lab. They are unnecessary dependencies for the opening exercises.

Understand the two network phases

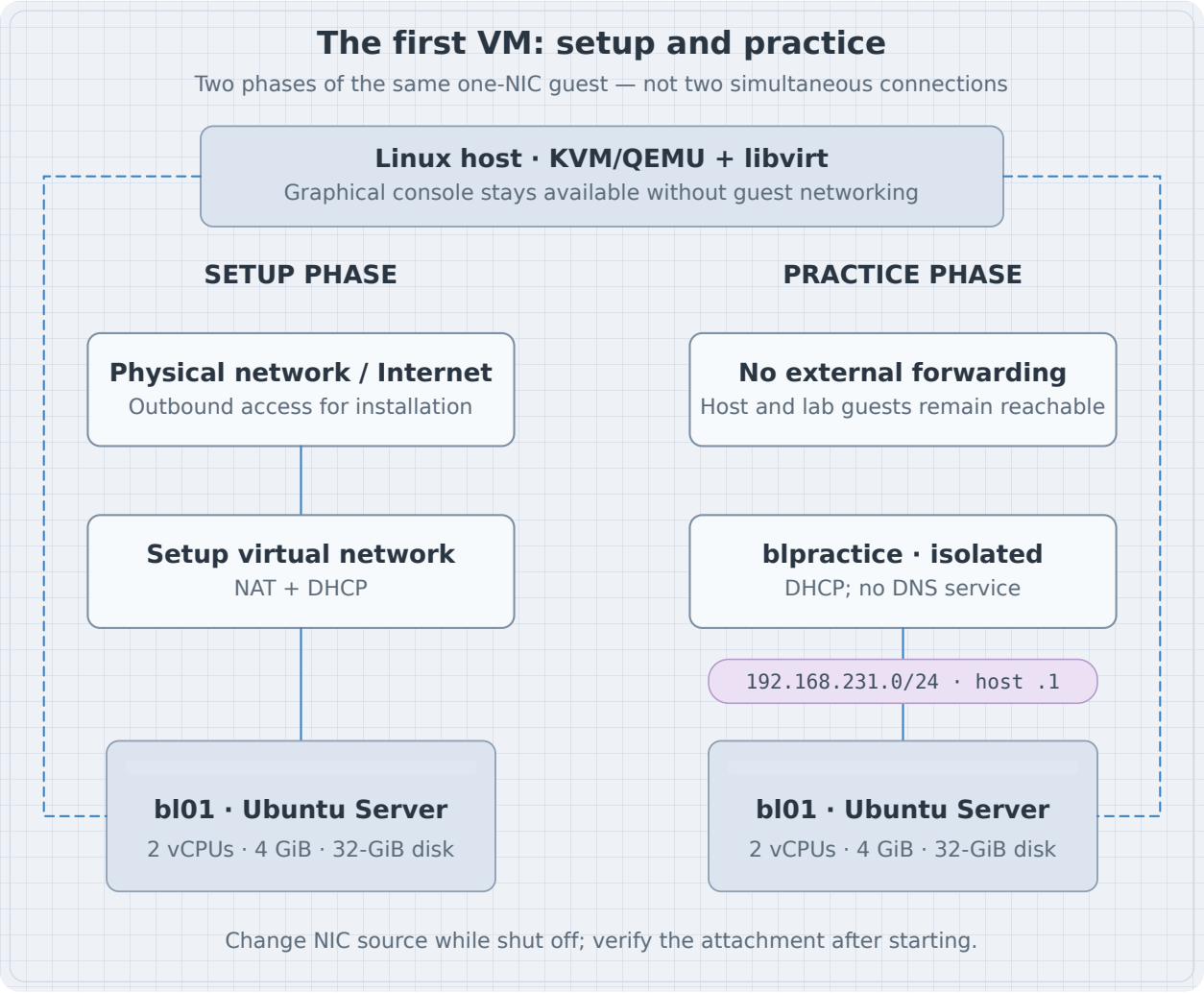


Figure 1: The same lab guest uses NAT during setup and an isolated network during practice.

Network address translation, or **NAT**, allows guest traffic to use a host-side outbound path. It is convenient for downloading packages. It is not complete isolation: the guest may reach services

on the host, the local network and the Internet according to routing and policy. Do not run intentionally vulnerable network services during this setup phase.

An **isolated libvirt network** lacks an external forwarding configuration. It can still connect the host and guests on that virtual network. It is therefore suitable for controlled beginner practice, not a sandbox for hostile code. Avoid attaching a second NIC that silently provides another route out. Do not bridge the practice network to the physical LAN or configure the host to route it elsewhere.

A **subnet** describes a group of IP addresses sharing a network prefix. Here `/24` corresponds to the mask `255.255.255.0`. Our chosen prefix must not conflict with the host's LAN, VPN or existing virtual networks. **DHCP** automatically supplies addresses and related network information; the first guest does not need a manually assigned static address.

Prepare and validate the host

On an Ubuntu host, install the supported tools:

```
sudo apt update
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients \
  virt-manager virt-install ovmf
```

`ovmf` supplies virtual UEFI firmware on this x86-64 path. The management tools coordinate the guest definition, storage and networking. The Ubuntu libvirt guide explains the base host setup and authorization. These commands change the host, not the guest.

Validate before allocating a disk:

```
virt-host-validate qemu
sudo virsh --connect qemu:///system list --all
sudo virsh --connect qemu:///system net-list --all
```

`qemu:///system` explicitly selects the system libvirt connection. A per-user connection is a different inventory; checking one and creating the VM in the other produces confusing "missing VM" reports. The `sudo` prefix makes the scope of host management explicit. Virtual Machine Manager can use the desktop's authorization prompt for the same system connection.

Read validation failures individually. Lack of usable hardware virtualization is relevant to this lab; an unavailable optional passthrough facility may not be. If `/dev/kvm` is unavailable, check host capability and firmware settings. If the host is itself a VM, its provider must expose nested virtualization. Installing more guest packages will not supply a missing host capability.

Open Virtual Machine Manager from the desktop. Confirm that its connection is **QEMU/KVM, system**, and that it connects successfully. Do not run the graphical application through `sudo` merely to bypass a connection problem; use the intended desktop authorization or the distribution's documented group setup. VM-management privileges grant considerable control over the host.

Inspect the setup network

Many libvirt installations define a network named `default`. Its presence, activation and subnet are observations, not assumptions:

```
sudo virsh --connect qemu:///system net-info default
sudo virsh --connect qemu:///system net-dumpxml default
```

Check that its XML contains a NAT forwarding mode and a subnet that does not conflict with the host's networks. If that existing network is suitable but inactive, start it:

```
sudo virsh --connect qemu:///system net-start default
```

If it is absent, create a NAT virtual network through the connection's **Virtual Networks** settings in Virtual Machine Manager. Give it a distinct name such as `blsetup`, select an unused IPv4 subnet, enable DHCP and select NAT forwarding. Start it and record its actual name and subnet. Use that name wherever the walkthrough asks for the setup network. Do not replace another workload's network to make a screenshot match.

This book uses the graphical creation flow for that optional setup network because existing host networking varies. The isolated network below has an exact definition you can inspect and reproduce.

Obtain the installer and verify what you downloaded

Use the official Ubuntu Server download page to select the **26.04 LTS** server image for **amd64**, meaning the x86-64 architecture. A newer point release within the 26.04 series is appropriate. Record the actual filename and download date; do not substitute an interim release just because it appears first on a page.

Download the matching `SHA256SUMS` and `SHA256SUMS.gpg` files from the same release directory. A checksum detects a mismatch; a verified signature connects the checksum list to a trusted signing key. Keep the ISO and those files in a dedicated download directory.

On the host, obtain the Ubuntu image-signing key according to the current official verification instructions, and compare its **full fingerprint** with Ubuntu's published fingerprint through a trusted channel. A key's display name alone is insufficient. Then, from the directory containing the downloaded files:

```
gpg --verify SHA256SUMS.gpg SHA256SUMS
sha256sum --ignore-missing -c SHA256SUMS
```

Proceed only when the signature is valid for the expected signing key and the exact ISO you downloaded reports `OK`. Do not hide failures by filtering the output to successful lines. `--ignore-missing` permits other images named in the checksum list to be absent; it does not excuse a mismatch for your image. The Ubuntu verification procedure supplies the key-distribution and fingerprint details. Part 02 develops this trust chain further.

The installer ISO is bootable virtual optical media. You do not need to write it onto the host's physical disk or a USB drive for this lab.

Create the virtual hardware

In Virtual Machine Manager, select **Create a new virtual machine** on the system connection. Choose local installation media and browse to the verified ISO. Confirm a suitable Ubuntu operating-system profile. If automatic detection fails, select an appropriate Ubuntu Linux profile rather than changing the guest architecture.

Set 4096 MiB memory, 2 CPUs and one **new** 32-GiB disk. Name the VM `bl01`. Select the inspected NAT setup network. Before finishing, choose the option to customize the configuration before installation.

Review the hardware details:

1. Select UEFI firmware before the first installation. Record the selected variant. Do not enable TPM-backed disk encryption for this initial lab; it introduces additional state that the simple clone exercise does not cover.
2. Confirm the disk is a new qcow2 image in the intended libvirt storage pool. Record its host pathname and virtual capacity. Use the VirtIO disk bus where available.
3. Confirm there is exactly one NIC, using the selected setup network and a VirtIO model where available.
4. Confirm a graphical display and video device are present. Open that display to interact with firmware and the installer.
5. Confirm that the bootable ISO is attached as optical media. Check the boot order if the installer does not start.

Labels vary slightly between application versions. The invariant is the resulting hardware: one guest disk, one NIC, a known firmware mode and a usable console. A **storage pool** is libvirt's managed location for storage volumes; a pool name is not a guest filesystem mount point.

If libvirt cannot read an ISO inside a private host home directory, use its storage browser to place the verified file in a libvirt-accessible media location. Do not make your whole home directory world-readable. The error concerns host-side file access, not an installer driver.

Install the guest through its console

Start installation from the graphical display. This is a deliberately simple first installation; Part 02 revisits partitioning, encryption, authentication choices and reproducible provisioning.

Select the keyboard layout you actually use, Ubuntu Server, automatic networking on the one NIC, no proxy unless your environment requires one, and a reachable distribution mirror. For storage, choose guided use of the **new 32-GiB virtual disk**. Leave optional disk encryption off for this disposable introductory guest. Read the final storage summary and confirm its target before accepting it.

Create hostname `bl01` and account `learner`. Use a unique password and record it in your password manager. Skip optional software bundles and optional remote-access installation for now. Complete installation, reboot when prompted and detach the ISO when the installer requests it. Ubuntu's basic installation tutorial describes the installer flow.

If the VM returns to the installer, inspect optical-media attachment and boot order. Do not immediately reinstall over the disk. If you see a login prompt, log in through the graphical console and establish which environment you reached:

```
hostnamectl
cat /etc/os-release
uname -r
findmnt --target / --output TARGET,SOURCE,FSTYPE
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

The root source should be installed storage rather than the live installer environment. Device and logical-volume names depend on the selected storage layout. A firmware check is:

```
test -d /sys/firmware/efi && printf 'Guest booted with UEFI\n'
```

No output means the test did not find that directory; investigate the firmware mode rather than claiming UEFI was used merely because the host has it.

While still on the setup network, update the fresh guest:

```
sudo apt update
sudo apt upgrade
sudo reboot
```

Read the package plan before accepting it. A failed metadata refresh must be resolved before treating the package inventory as current. After reboot, log in through the graphical console again. This demonstrates that your access route survives guest shutdown and startup.

Create the isolated practice network

On the host, inspect existing addresses, routes and virtual networks:

```
ip -brief address
ip route show table all
sudo virsh --connect qemu:///system net-list --all
```

Check VPN and other active network prefixes too. If `192.168.231.0/24` overlaps an existing route, choose another unused private subnet and substitute it consistently below and in your notes. Do not remove an existing route to make room.

Save the following as `blpractice.xml` in a host working directory:

```
<network>
  <name>blpractice</name>
  <bridge name='virbr-blp' stp='on' delay='0'/>
  <dns enable='no'/>
  <ip address='192.168.231.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.231.100' end='192.168.231.199'/>
    </dhcp>
  </ip>
</network>
```

There is no `<forward>` element. The bridge connects the host-side address and attached guests; DHCP supplies addresses in the stated range. DNS service is disabled for this first isolated network so it is not used as a host-provided external resolver. The libvirt network-format reference explains these elements and isolated-network behavior.

Confirm that both the network name and bridge name are unused. Then define and start this new network:

```
sudo virsh --connect qemu:///system net-define blpractice.xml
sudo virsh --connect qemu:///system net-start blpractice
sudo virsh --connect qemu:///system net-autostart blpractice
sudo virsh --connect qemu:///system net-dumpxml blpractice
```

Definition makes the network persistent; starting activates it now; autostart requests activation with the host's libvirt network service. If definition or activation fails, read the error before continuing. Do not redefine another network or rename its bridge as an improvised repair.

In the guest, shut down cleanly with `sudo poweroff`. Wait until Virtual Machine Manager shows it shut off. In the guest's NIC hardware settings, change the network source from the setup network to `blpractice`. Retain a single NIC. Apply the change, start `bl01`, and log in through the same graphical console.

Inspect the resulting guest view:

```
ip -brief address
ip route
```

The NIC should acquire a `192.168.231.100`–`192.168.231.199` address if you kept the example range. Record the actual address and interface name. Do not hard-code an interface name from someone else's screenshot. External package downloads are no longer an expected operation in this phase. A displayed default route alone would not prove that external forwarding works; inspect the host network definition as well as the guest route.

On the host, confirm attachment with:

```
sudo virsh --connect qemu:///system domiflist bl01
sudo virsh --connect qemu:///system net-dhcp-leases blpractice
```

The source must be `blpractice`. An absent lease might reflect timing or a guest DHCP problem; use the console to inspect the guest. Failure of one ping is not proof of isolation, because packet filtering can block a probe for unrelated reasons. Isolation evidence here is the one-NIC attachment, no forwarding configuration, no extra host bridge or routing setup, and the observed guest configuration.

Establish a baseline you can actually use

A **checkpoint** is a known state you intend to return to. A **snapshot** is one implementation mechanism, often dependent on other images and the same host storage. For this first lab, use a full powered-off clone rather than relying on a snapshot feature whose support depends on disk, firmware and manager versions.

In the guest, create a harmless baseline marker:

```
printf 'baseline-v1\n' > "$HOME/lab-baseline.txt"
sudo poweroff
```

On the host, confirm the VM is shut off, not paused or saved with suspended memory:

```
sudo virsh --connect qemu:///system domstate bl01
sudo virsh --connect qemu:///system domblklist bl01 --details
```

The following clone recipe assumes the single writable disk created above, no shared writable disks, no backing chain and no TPM state. The destination filenames and VM names must be new. Check free space before copying, then run:

```
sudo virt-clone --connect qemu:///system \
  --original bl01 --name bl01-baseline \
  --file /var/lib/libvirt/images/bl01-baseline.qcow2
```

The clone receives a distinct libvirt identity and network MAC address. For a UEFI guest, verify that its definition also references a distinct NVRAM file, which holds persistent firmware variables:

```
sudo virsh --connect qemu:///system dumpxml bl01
sudo virsh --connect qemu:///system dumpxml bl01-baseline
```

Compare disk source paths and the `<nvram>` entries. Keep the baseline shut off, with no autostart enabled. The virt-clone manual documents disk-copy behavior and generated NVRAM paths. Do not use `--preserve-data` or `--skip-copy`: those options do not create the independent disk copy intended here.

The guest's own hostname, machine ID, credentials and application identity are copied too. The clone is a recovery artifact, not yet a prepared second server. Keep only one copy running during this exercise. Later provisioning chapters handle identity changes for concurrently deployed clones.

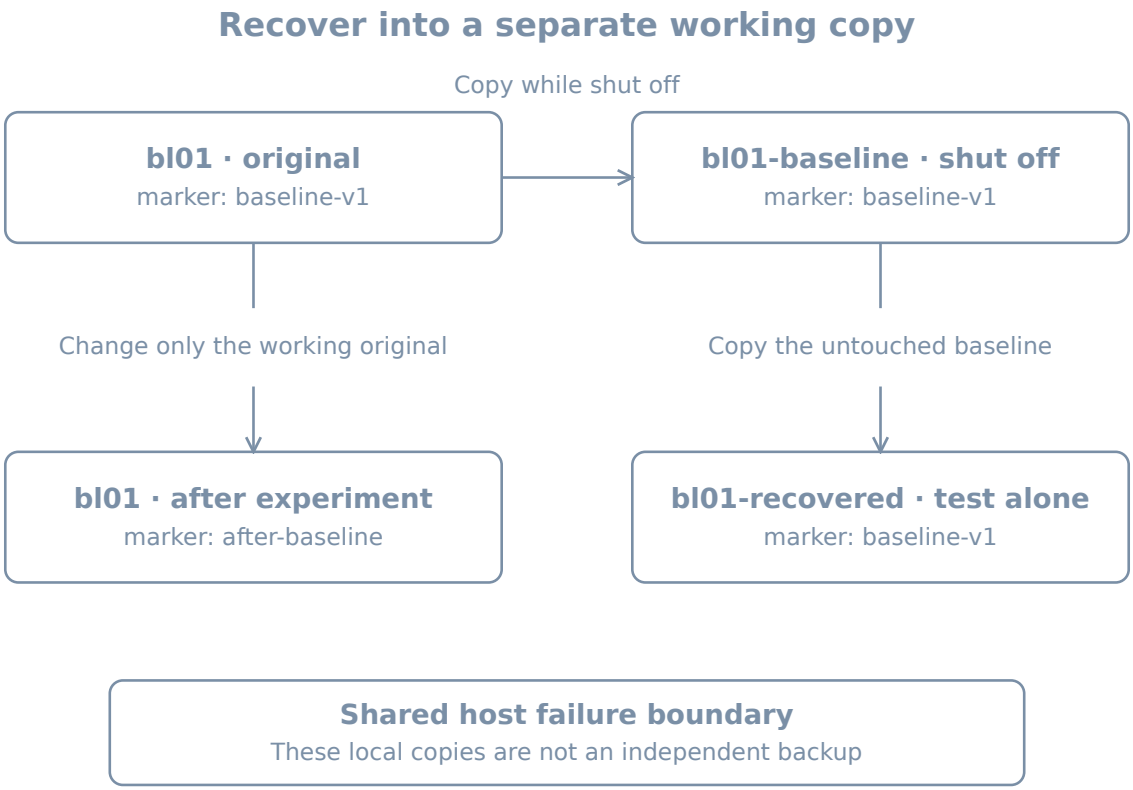


Figure 2: The untouched baseline supplies a separate recovery copy.

Prove recovery with a visible change

Start the original `bl01`. In its guest console, replace the marker:

```
printf 'after-baseline\n' > "$HOME/lab-baseline.txt"
cat "$HOME/lab-baseline.txt"
sudo poweroff
```

Leave both original and baseline shut off. **On the host**, create a working recovery copy from the untouched baseline:

```
sudo virt-clone --connect qemu:///system \
  --original bl01-baseline --name bl01-recovered \
  --file /var/lib/libvirt/images/bl01-recovered.qcow2
```

Inspect its disk, NVRAM and NIC source as before. Start only `bl01-recovered` and open its graphical console. The guest hostname can still be `bl01`; the console window's VM identity tells you which copy you are testing. Read the marker:

```
cat "$HOME/lab-baseline.txt"
```

Success is `baseline-v1`, a successful login, and an expected isolated-network configuration. This proves that the baseline's earlier filesystem state can boot as a separate recovery copy. It does not prove that every future workload, multi-disk database or external service can be recovered by the same procedure.

Shut down the recovery copy after the test and resume the original `bl01` for normal work. Keep the baseline unchanged. The recovery copy can be retained shut off or removed later through the manager after verifying its unique disk path. Do not select shared media or the original VM's disk for deletion.

A local clone is not an independent backup

All three copies can disappear if their host storage fails. A backup plan needs independent storage, an inventory of required state, retention and a demonstrated restore. Firmware variables, VM configuration, disk dependencies, encryption material and application consistency can all matter. Never copy or modify a live image with an offline image utility; QEMU's image-tool documentation explains that restriction.

The baseline is valuable for returning from an experiment. It is not protection against losing the entire lab host. Keep your lab notes outside the guest so they remain available when a guest cannot boot.

Diagnose the layer that failed

Symptom	First evidence	Likely investigation boundary
VM cannot start	Host manager error and available resources	Host virtualization or storage
Firmware says no boot device	Attached ISO/disk and boot order	Presented hardware and firmware
Installer cannot reach a mirror	NIC source, guest address and route	Setup-network path
Installed VM reopens installer	Optical media and boot selection	Boot configuration
Console works but package downloads fail in practice	NIC source is `blpractice`	Expected isolated-network behavior
Clone does not contain the marker	Exact source VM, disk paths and shutdown sequence	Baseline selection or copying

Do not solve a console issue by exposing SSH to the physical LAN. Recover the management path you designed first. Do not solve a full host filesystem by deleting an unfamiliar qcow2 image; it may be another VM's only disk or a backing dependency.

Completion record and recall card

Before moving on, write down the VM and guest names, firmware mode, CPU/RAM allocation, image path, NIC source, guest address, installer filename and verification result. Record how to open the console while networking is unavailable. Record the baseline creation time and recovery-test result.

You are ready for the next chapter when you can shut down and start the guest, log in without SSH, explain its current network boundary, and show the earlier marker in the separate recovery copy.

- Host commands manage virtual hardware; guest commands administer the installed Linux system.
- NAT is a setup convenience, not complete isolation.
- A console independent of guest networking is part of the recovery design.
- A powered-off clone has a limited, explicit consistency boundary.
- Cloning hardware identities does not automatically regenerate identities inside the guest.
- A baseline becomes useful evidence only after a recovery rehearsal.

Inventory, Evidence and First Troubleshooting

A useful inventory explains what a machine is, what it is meant to do and how to reach it when something fails. A useful incident record explains what was observed, what was inferred and what changed. Neither requires collecting every possible command output.

Use the disposable `bl01` VM from the preceding chapter. Keep its graphical console available, and keep your baseline clone shut off. The inspection examples run inside the guest unless marked **host**. The worked failure creates one temporary user service and one private directory; it does not break login, networking or system services.

By the end of the chapter you will have a baseline inventory, a change note, an evidence-led repair and an explicit record of what your checks do not prove.

Start with an operational question

"The server is broken" does not identify a test. "After today's change, the console user cannot read the report that was available yesterday" gives an operation, observer and time boundary. A good symptom statement includes:

- The intended operation and the result that would count as success.
- Who or what attempted it, from which environment.
- The actual result, including the exact error when available.
- The first known failure time and last known success.
- The scope: one user, one service, one VM or several machines.

Avoid turning a hypothesis into the symptom. "The firewall is blocking the application" already assumes a cause. "The client times out when connecting to the configured endpoint" leaves routing, filtering, listener and application behavior open for investigation.

A **hypothesis** is a proposed explanation that evidence can support or contradict. A **discriminating check** is one whose possible outcomes separate competing explanations. A command is useful because of the question it answers, not because it appears in a long troubleshooting checklist.

For example, if a service reports that it cannot open `/srv/report/current.txt`, checking the configured pathname and mount can separate "wrong path" from "missing storage." Rebooting may change both the process and mount state without explaining either.

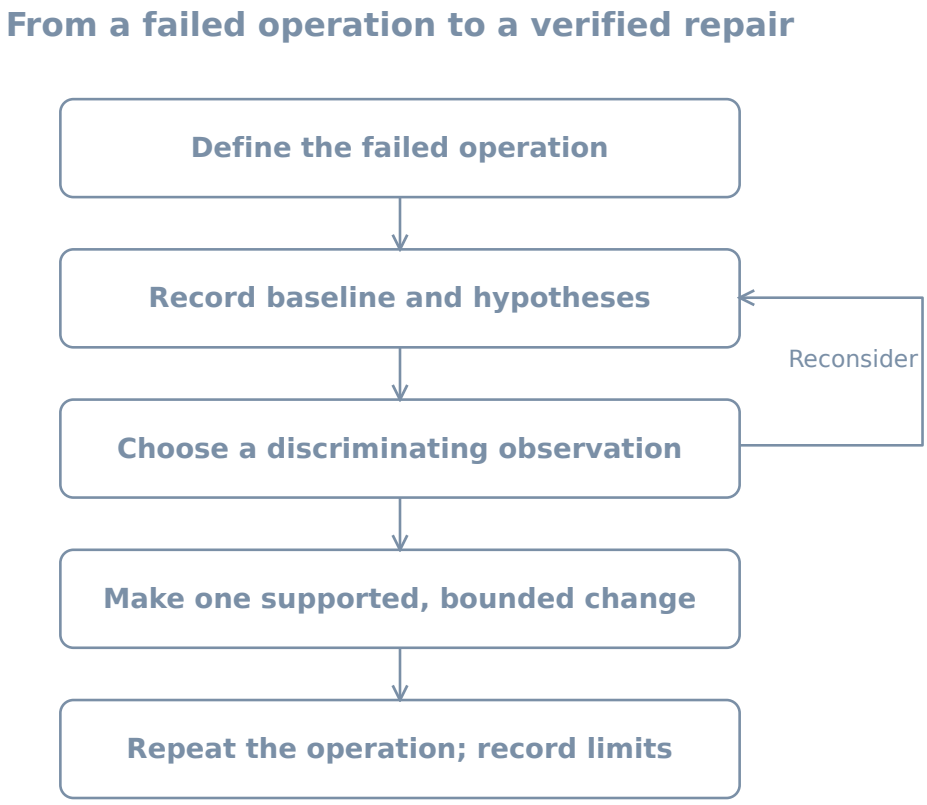


Figure 1: An investigation moves from a precise symptom through evidence to a tested change.

Build an inventory with purpose and scope

Create a lab record outside the guest, alongside the baseline and console instructions from the VM chapter. Record the following categories. Some values come from the host manager; others come from the guest. Label that boundary explicitly.

Category	Useful fields	Why it belongs
Purpose	Lab role, owner, expected operations	Defines what healthy means
Host-side identity	VM name, connection, disk paths, firmware	Locates the managed machine and its dependencies
Guest identity	Hostname, distribution, kernel, architecture	Matches instructions and runtime evidence
Resources	vCPUs, RAM, disks and filesystem capacity	Establishes expected limits
Network	NIC source, address, route, resolver context	Explains intended reachability
Access	Console route, login method, privilege boundary	Makes recovery possible
Service state	Expected services and known failures	Distinguishes baseline from regression
Recovery	Baseline date, scope, test result	States what can actually be restored
Maintenance	Update/source status and outstanding actions	Prevents a stale baseline from appearing current

Do not record passwords, private keys or authentication tokens in this document. A recovery record can identify the password-manager entry without copying its contents.

The inventory is not frozen truth. Include collection time and the environment from which each observation was made. An IP address can change after DHCP renewal; a kernel version can change after reboot; a disk image can move while the guest filesystem remains the same.

Worked inventory: identity, time and boot

In the guest, run:

```
date --iso-8601=seconds
hostnamectl
cat /etc/os-release
uname -r
uname -m
uptime
cat /proc/sys/kernel/random/boot_id
```

The **boot ID** identifies this kernel boot and helps distinguish records separated by a reboot. A PID alone cannot do that. The wall-clock time needs context too: a disconnected VM can have a different clock from the host. Record the offset or uncertainty rather than silently aligning events by an inaccurate timestamp.

For an event timeline within one boot, monotonic timestamps can be useful because they describe elapsed time from a boot-related origin rather than calendar time. For events spanning machines, synchronized wall clocks and timezone information matter. Later time-service chapters explain synchronization; for now, do not claim second-by-second ordering across unsynchronized machines.

Worked inventory: resources and storage

```
lscpu
free -h
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
df -h /
df -i /
```

`lsblk` describes block devices and their relationships; `findmnt` locates mounts. Their explicit columns make the intended observation clearer than relying on changing default displays. The lsblk manual describes its device information and column selection.

`df -h /` asks about filesystem space for `/`; `df -i /` asks about inode usage where the filesystem reports it. An **inode** represents filesystem metadata for an object. On filesystems with a finite inode supply, many small files can exhaust object capacity while data blocks remain available. The df manual distinguishes these reports.

Illustrative interpretation:

```
Filesystem     Size  Used Avail Use% Mounted on
/dev/vda2       30G  5.0G   23G  18% /
```

This describes a guest filesystem at one instant. It does not show the host filesystem's free space or prove that a database can write under its own account. Filesystem reservation, quotas, permissions and a read-only mount can produce different application results. Record this evidence as "root filesystem reported available blocks," not "storage is healthy."

On the **host**, separately record the free capacity of the filesystem containing the VM images and baseline copies. A guest's free-space report cannot detect every host-side capacity problem.

Worked inventory: networking and listeners

```
ip -brief address
ip route
ss -lnt
```

A **listener** is an endpoint prepared to accept incoming communication. `ss -lnt` selects listening TCP sockets, using numeric addresses and ports. It does not cover UDP services, outbound-only applications or all communication mechanisms. The ss manual describes these filters.

An address of `127.0.0.1` belongs to IPv4 loopback, usable inside the same network namespace. A listener bound only there is not automatically reachable through the guest's external NIC. An unspecified address such as `0.0.0.0` has a wider local binding scope, but does not prove that a route or firewall permits a client connection.

On our isolated practice network, failure to fetch Internet packages is expected. Do not switch to the NAT network simply because a generic checklist says an external ping must work. Compare the observation with the documented role and phase of the lab.

If you need to understand which route Linux would select for an address, `ip route get ADDRESS` performs a route lookup; it is not a packet-delivery test. Replace `ADDRESS` with the actual destination under investigation. The ip-route manual explains this distinction.

Worked inventory: service state and bounded logs

```
systemctl --failed --no-pager
systemctl list-units --type=service --state=running --no-pager
journalctl --list-boots --no-pager
journalctl -b -p warning -n 30 --no-pager
```

A failed-unit list is a starting clue. A healthy application can have dependencies not represented there, and a failed optional unit might not explain the reported operation. Similarly, selecting warning-priority records can omit the informational event that caused the incident.

The journal commands show retained boots and a bounded view of current-boot warnings. Retention and your account's access rights limit what is available. The journalctl reference documents boot, priority, unit and time filters. If a query reports insufficient access, record that limitation before deciding whether a privileged read is warranted.

Do not dump all logs into a public issue. Logs can contain usernames, addresses, request details and credentials accidentally emitted by software. Retain original evidence privately, and create a separately reviewed excerpt when sharing.

Save a small evidence bundle

For this exercise, create a private directory in the guest and collect a few explicitly selected reports. Run the commands in one Bash terminal:

```
umask 077
evidence_dir=$(mktemp -d "$HOME/bl-evidence.XXXXXX")
date --iso-8601=seconds > "$evidence_dir/collected-at.txt"
cat /proc/sys/kernel/random/boot_id > "$evidence_dir/boot-id.txt"
cat /etc/os-release > "$evidence_dir/os-release.txt"
```

```
uname -r > "$evidence_dir/kernel.txt"
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS \
  > "$evidence_dir/root-mount.txt" 2>&1
ip -brief address > "$evidence_dir/addresses.txt" 2>&1
systemctl --failed --no-pager > "$evidence_dir/failed-units.txt" 2>&1
printf 'Evidence directory: %s\n' "$evidence_dir"
```

`umask 077` restricts permissions for newly created files and remains in effect for this shell. `mktemp -d` avoids overwriting an older evidence set. `2>&1` saves error output with ordinary output, so an unavailable command or denied query does not look like an empty successful report.

Read the saved files. The existence of a file is not proof that the command succeeded. Keep the directory path in your notes, and copy reviewed evidence to storage outside the VM when it matters for recovery. An evidence bundle only inside the guest can be lost when you revert that guest.

This small bundle is not a backup. It does not contain the VM disk, service databases or every configuration input. Its purpose is to preserve the context for a bounded investigation.

Separate facts, interpretations and actions

Use a table like this while investigating:

Kind	Example
Observation	Unit log says `/home/learner/.../expected.txt` cannot be opened
Interpretation	The configured input may not exist at that path
Alternative	The parent directory may be inaccessible to the service user
Discriminating check	Inspect the exact configured path and parent directory as the same user
Planned change	Place the intended file at that path, if the evidence confirms a naming mismatch
Verification	Repeat the same unit invocation and inspect its result

An observation can still be incomplete or stale. Its value comes from a recorded source, time and scope. An interpretation should be revisable when new evidence appears.

Change one relevant variable at a time when practical. If you change the pathname, account, permissions and startup command together, a successful retry will not tell you which change mattered. In a real outage, restoring service may take priority over a perfect experiment, but record that uncertainty rather than inventing a definitive root cause afterward.

Worked failure: a service reads the wrong pathname

This experiment uses a **transient user service**: a unit created through the manager for this session of work rather than installed as a persistent system configuration file. It runs under your ordinary account. No `sudo` is needed.

Check that the user manager is reachable:

```
systemctl --user list-units --type=service --no-pager
```

If it reports that it cannot connect, use a normal local login in the reference VM's console. Do not change system authentication or the login manager just to run the exercise. A shell in an unrelated container or recovery environment may not have a user manager.

Use the unit name `bl-reading-probe.service` only if it is not already in use. An old attempt should be inspected and cleaned up before reusing the name. Create the input directory and the deliberately differently named file:

```
probe_dir=$(mktemp -d "$HOME/bl-reading-probe.XXXXXX")
printf 'inventory-ready\n' > "$probe_dir/actual.txt"
printf 'Experiment directory: %s\n' "$probe_dir"
systemd-run --user --unit=bl-reading-probe \
  --property=Type=oneshot --property=RemainAfterExit=yes \
  /usr/bin/cat "$probe_dir/expected.txt"
```

Run interactively; do not wrap this expected failure in a script that exits immediately on any nonzero result. `Type=oneshot` makes this a finite task. `RemainAfterExit=yes` keeps the successful unit active after its process exits, making the final state easy to inspect. Do not add `--wait`: combining waiting for deactivation with a unit intentionally left active is inappropriate here. The systemd-run manual describes transient-unit creation and its execution options.

The command requests a file that you did not create. It should fail. Record the symptom precisely: "the user unit cannot read its configured input." Do not call this a disk failure or a permission failure yet.

Observe without repairing

```
systemctl --user status bl-reading-probe.service --no-pager
systemctl --user show bl-reading-probe.service \
  -p ActiveState -p SubState -p Result -p ExecMainStatus
systemctl --user cat bl-reading-probe.service
journalctl --user -b -u bl-reading-probe.service -n 20 --no-pager
ls -ld "$probe_dir"
ls -l "$probe_dir"
```

The unit's executed command should contain the exact `expected.txt` pathname. The directory listing should contain `actual.txt` instead. The log normally reports a missing file; wording depends

on locale and software version. A nonzero status from a status-query command can reflect the failed unit being observed, rather than a broken query tool.

There are now several mutually supporting observations: the requested name, the actual name, and the failure at that path. Permissions remain worth understanding, but broad permission changes would not create the missing name. The isolated network is irrelevant because this command reads a local file.

Make the smallest supported correction

Move the intended input to the requested name, then start the same unit again:

```
mv -- "$probe_dir/actual.txt" "$probe_dir/expected.txt"
systemctl --user start bl-reading-probe.service
systemctl --user show bl-reading-probe.service \
  -p ActiveState -p SubState -p Result -p ExecMainStatus
journalctl --user -b -u bl-reading-probe.service -n 20 --no-pager
```

Expected properties include `ActiveState=active`, `SubState=exited`, `Result=success` and `ExecMainStatus=0`. The journal should include `inventory-ready` from the successful read. It may still contain the earlier failure; logs preserve history rather than becoming uniformly successful after a repair. Compare the timestamps and invocation order.

This is a direct example of an active service without a continuing main process. The useful operation was the file read, and its evidence is successful completion plus the expected content. Do not diagnose the absence of a running `cat` process as a new failure.

Clean up the experiment

Save the relevant output in your notes first. Then stop the transient unit and remove only the file and directory you created:

```
systemctl --user stop bl-reading-probe.service
rm -- "$probe_dir/expected.txt"
rmdir -- "$probe_dir"
unset probe_dir
```

A stopped transient unit can be unloaded by the manager. A later status query may report that it is not found; that is expected after cleanup. If an interrupted attempt remains failed, inspect it and use `systemctl --user reset-failed bl-reading-probe.service` before creating a new attempt. Do not reset all failed units and erase unrelated failure state.

The journal can retain the experiment's records after cleanup. Removing the input directory is not a request to erase that historical evidence.

Write the change note before forgetting the reason

A compact note for this experiment might read:

```
Purpose: restore the user probe's expected file read.
Environment: bl01 guest, local console, learner user manager.
Baseline: probe input directory contains actual.txt only.
Symptom: bl-reading-probe.service exits unsuccessfully.
Evidence: ExecStart requests expected.txt; listing shows actual.txt.
Hypothesis: the intended input has the wrong pathname.
Change: rename actual.txt to expected.txt in the experiment directory.
Rollback: rename it back while the probe is stopped.
Verification: same unit starts successfully; result=success;
              journal contains inventory-ready.
Cleanup: stop transient unit; remove experiment input and directory.
Limit: proves this local read, not general storage or service health.
```

Add actual timestamps and the boot ID to your real note. For a production change, also record the authorized scope, affected users, maintenance constraints and recovery decision. The shape is the same: purpose, evidence, bounded action, verification and limits.

A second investigation: connectivity changed as designed

After moving `bl01` from NAT to `blpractice`, an update attempt cannot contact its repositories. List three hypotheses before changing anything:

1. The guest has no usable NIC or address.
2. The guest is deliberately attached to a network without an external path.
3. The package configuration refers to an unavailable repository.

A discriminating sequence is to inspect the guest address, check the host-side NIC source, and compare it with the lab's documented phase. If the guest has a practice address and the source is `blpractice`, external download failure agrees with the intended design. Replacing repository URLs would change an unrelated component.

If a later exercise needs updates, plan a brief return to the setup network while the guest is shut down, perform the maintenance, and return to the isolated network. Record both attachment changes and verify the final state. Do not silently leave an extra NAT NIC attached after the task.

This example matters because successful troubleshooting sometimes concludes that the system is behaving correctly and the expectation must change. Every failure message is evidence about an attempted operation; not every failed operation indicates a defect.

Practice: produce an evidence-led handoff

Give another learner a short handoff containing the VM inventory, the last known healthy state, the probe failure's evidence, the repair and verification, and the recovery route. They should be able to identify the correct VM without guessing from its hostname alone.

Answer these questions in the handoff:

1. What did the successful probe prove, and what did it leave untested?
2. Which evidence would be lost if you reverted the guest before copying your notes out?
3. Why would changing permissions on the input directory not have fixed the original naming mismatch?
4. Why is an external ping failure insufficient proof that a VM is isolated?
5. How would you distinguish an old failure in the journal from the latest invocation's result?

Success means you can trace each conclusion to a specific observation. The probe proved one read under one account and manager context; it did not validate an entire storage stack. Guest-local notes can vanish on rollback. Missing names and permissions are different causes. Network probes have several failure modes. Invocation timing and current result properties separate historical errors from a completed repair.

Recall card

- Define the failed operation before selecting commands.
- Record time, boot and observer context with evidence.
- Distinguish observations from explanations and proposed actions.
- Choose checks that separate plausible causes.
- Preserve evidence before a restart or rollback changes it.
- Verify the original operation and relevant side effects after a bounded change.
- A useful handoff includes what remains uncertain.

You now have a working lab and an investigative method. Part 02 returns to installation choices in depth: media trust, firmware and storage planning, manual layouts, first-boot verification and reproducible provisioning.

Installing and Provisioning Linux

Installation Foundations

Installation Media, Authenticity and Integrity

An installer starts with unusually broad authority. It can replace partitions, create administrator accounts and supply the first software the machine trusts. Before giving it that authority, establish which artifact you have and why you trust it.

Part 01 used a verified Ubuntu Server ISO to establish a lab. This chapter develops the verification process rather than treating it as a download-page ritual. You will select an image for a particular machine, distinguish integrity from authenticity, verify a signed checksum manifest, and keep a record that identifies the bytes actually used.

Run the download and verification steps on the **Linux host**, before attaching media to a new guest. The worked example uses Ubuntu Server 26.04 LTS for an x86-64 VM. No physical disk needs to be written. Preserve the existing `bl01` lab and its baseline; later chapters create a separate installation target named `bl02`.

Choose the kind of artifact before its filename

An **installation image** supplies an environment that installs an operating system onto target storage. A **preinstalled disk image** supplies storage that already contains an operating system, usually expecting further initialization. A **container image** supplies a userspace filesystem and metadata for a container runtime; it does not by itself provide the independent guest kernel and firmware path used in our VM lab.

Artifact	Typical use	Question before downloading
Server installer ISO	Interactive or automated installation onto new storage	Does it support this architecture and boot environment?
Desktop live/installer ISO	Try or install a graphical workstation	Is its package and interface selection appropriate?
Cloud disk image	Provision an instance from a prepared disk	What initializes credentials, networking and instance identity?
Network-boot artifacts	Start an installer through network infrastructure	Where will the remaining installer and package data come from?
Container image	Start isolated userspace on a host kernel	Which runtime and image trust policy apply?

An ISO is a disk-image format commonly used for virtual optical media. The suffix alone does not establish its operating system, architecture or trustworthiness. Likewise, a `.qcow2` file identifies a storage format, not a guarantee of a bootable, authenticated or correctly configured guest.

The official Ubuntu release directory distinguishes server and desktop installation media and publishes checksum files. The separate Ubuntu Cloud Images site distributes images tailored for cloud and virtualization environments. Downloading a cloud image and then expecting the interactive server installer is a mismatch of workflow, not a failed bootloader.

Architecture names describe compatibility

For this lab, the host and presented guest machine are x86-64. Ubuntu names this architecture `amd64`; the name also applies to compatible Intel CPUs. It does not mean that only AMD-branded processors work. `arm64` describes a different instruction-set family.

A VM manager can present different machine types and device models, but hardware-assisted execution normally depends on a compatible host/guest architecture. Software emulation can bridge some architecture differences at another performance cost. Select the intended architecture deliberately rather than assuming that the largest or newest image is suitable.

Also distinguish the release series from the image build. Two artifacts can both belong to Ubuntu 26.04 LTS and contain different package revisions or installer updates. Record the exact filename and digest, not just "Ubuntu LTS." A URL that always points to the newest build is convenient for acquisition but insufficient as a permanent identifier.

Four separate claims about a download

Integrity asks whether the bytes match an expected value. A cryptographic hash such as SHA-256 computes a fixed-size digest from the complete file. A changed byte will, for practical verification purposes, produce a different digest.

Authenticity asks whether the expected value or artifact came from the authority you intended to trust. A signature can authenticate a checksum manifest to a signing key. You still have to establish that the key belongs to the intended publisher.

Suitability asks whether the authentic artifact is appropriate for this machine and task. A valid signature on an ARM image does not make it the right image for our x86-64 installation.

Freshness asks whether the artifact and its maintenance state are acceptable now. A correctly signed older image remains an older image. Its signature does not install later security fixes or promise that its release is still supported.

These checks are complementary. HTTPS protects communication with the site you reached, subject to its certificate trust. A signed manifest can remain verifiable after copying through a mirror or offline storage. Neither eliminates the need to know which publisher and artifact you intended to use.

A checksum copied from the same untrusted page as a malicious image can match perfectly. The attacker could replace both. The hash calculation would be correct, but the trust decision would be unsupported.

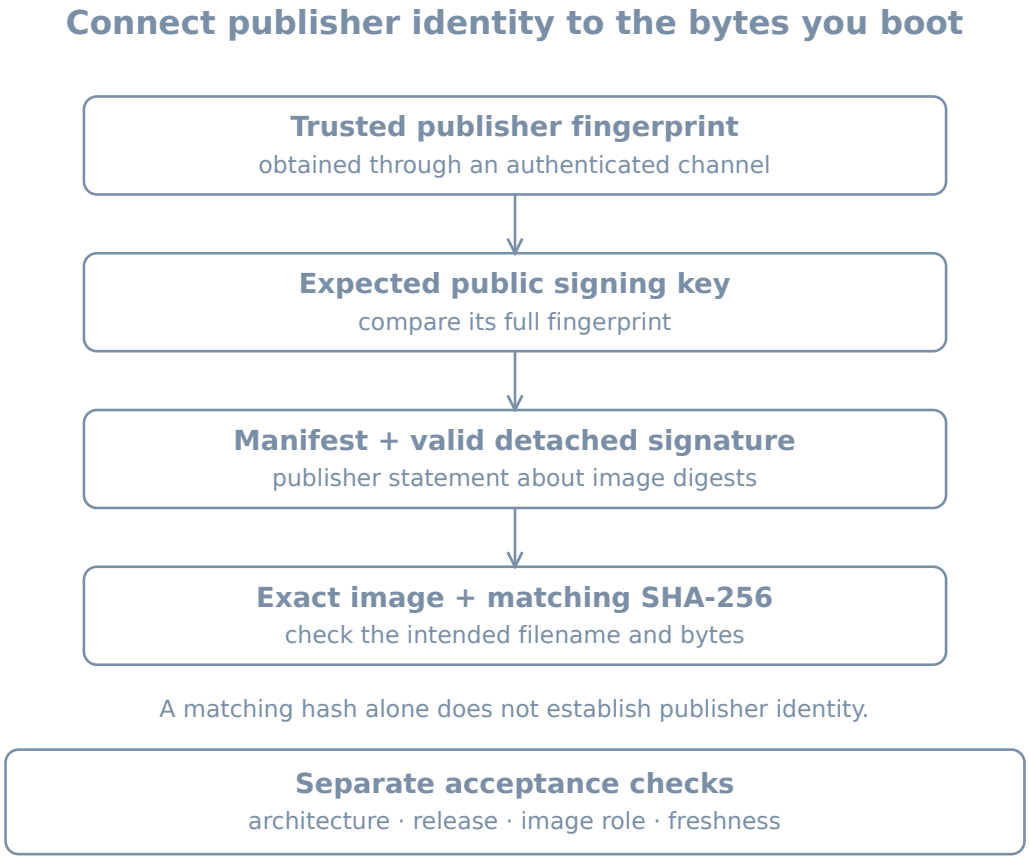


Figure 1: Verification joins trusted publisher identity to a manifest and then to the selected image.

Collect a matched verification set

Create a new host directory for one release/build verification. Keeping each set separate prevents a checksum file from one release being confused with another:

```
umask 077
media_dir=$(mktemp -d "$HOME/bl-install-media.XXXXXX")
printf 'Media directory: %s\n' "$media_dir"
```

Download the selected server ISO, `SHA256SUMS`, and `SHA256SUMS.gpg` into this directory from the same official release listing. Use the browser's save location or move the three completed downloads there. Avoid using an incomplete `.part` download or an automatically renamed duplicate without checking which file you selected.

`SHA256SUMS` is the manifest: a list of filenames and expected hashes. `SHA256SUMS.gpg` is a detached signature over that manifest. "Detached" means the signature is stored separately from the signed content.

Enter the directory and inspect the manifest:

```
cd "$media_dir"
ls -lh
cat SHA256SUMS
```

Identify the exact server ISO entry for the intended architecture. A different filename may be legitimate after a point release, but it must correspond to your downloaded image. Do not edit the manifest to force a match; that would change the signed object.

If a moving release directory changes during your download, the ISO and manifest may legitimately belong to different publication states. A mismatch remains a reason to stop. Acquire a consistent set from the publisher instead of assuming the signature or checksum tool is being overly strict.

Establish the signing key's identity

A public signing key can be distributed widely. Its availability on a keyserver does not by itself prove whose key it is. A **fingerprint** is a compact identifier derived from the key material. Compare the full expected fingerprint using the publisher's authenticated documentation or another trusted organizational distribution channel.

Ubuntu's download-verification instructions publish its image-signing key information. The documented 2012 image-signing key has this fingerprint:

```
8439 38DF 228D 22F7 B374 2BC0 D94A A3F0 EFE2 1092
```

Verify that Ubuntu still identifies the relevant key for your selected image when you perform the exercise. A future key rotation should lead you to verify the new publisher guidance, not accept an arbitrary key just because it makes an error disappear.

Use a dedicated verification key directory so this experiment is separate from your normal keyring:

```
verify_key_dir=$(mktemp -d "$media_dir/keys.XXXXXX")
expected_fingerprint=843938DF228D22F7B3742BC0D94AA3F0EFE21092
gpg --homedir "$verify_key_dir" \
  --keyserver hkps://keyserver.ubuntu.com \
  --recv-keys "$expected_fingerprint"
gpg --homedir "$verify_key_dir" \
  --fingerprint "$expected_fingerprint"
```

Compare the displayed full fingerprint with the trusted publisher record. `hkps` requests a TLS-protected keyserver connection. The requested fingerprint narrows the key retrieval; a matching user-ID name or email address alone is not enough. The GnuPG manual documents key retrieval, fingerprints and verification operations.

These commands import public verification material. They do not create a private signing key or require you to certify the publisher's key as universally trusted. Keep the scope of the decision narrow: this is the key authorized to verify the selected distribution's manifest.

If key retrieval fails because the host cannot reach the keyserver, diagnose that acquisition failure. An approved offline copy of the public key can be imported and checked against the same trusted fingerprint. Do not disable artifact verification to compensate for a network problem.

Verify the manifest, then verify the image

First check the detached signature against the manifest:

```
gpg --homedir "$verify_key_dir" \
  --verify SHA256SUMS.gpg SHA256SUMS
```

Read the complete result. You need successful cryptographic verification using the expected publisher key. A message about an unknown local trust level can occur when a newly imported key has not been certified in your personal trust database. That is a different question from a bad signature. Resolve key ownership through the fingerprint check; do not treat every warning as equivalent or blindly suppress warnings.

A valid signature binds the manifest bytes to the signing key. It does not yet say that your ISO matches an entry. Perform that second check:

```
sha256sum --check --ignore-missing SHA256SUMS
```

Require the **exact downloaded ISO filename** to report `OK`, and check that the command reports no failed verification for it. `--ignore-missing` permits other files listed in the release manifest to be absent locally. It does not prove that the file you meant to test was present. A successful check of a desktop ISO in the same directory does not validate a server ISO that was never downloaded.

The GNU Coreutils manual documents checksum-checking behavior. For this exercise, use a directory containing only the intended artifact set and read the output without filtering it. In particular, do not pipe the result to a search for `OK` and mistake the search command's success for the verifier's complete result.

A completed record should now connect these facts:

```
Trusted publisher fingerprint
    → valid signature on this manifest
    → matching digest for this exact ISO
    → intended release, architecture and installation role
```

If any link is missing, label the artifact unverified and do not attach it as trusted installation media.

Worked experiment: matching bytes are not trusted bytes

This small exercise uses ordinary text, not an installer. It demonstrates why a checksum must come from a trusted source. Run it in one Bash terminal:

```
hash_demo_dir=$(mktemp -d "$HOME/bl-hash-demo.XXXXXX")
printf 'approved-content\n' > "$hash_demo_dir/sample.txt"
(
  cd "$hash_demo_dir" || exit
  sha256sum sample.txt > checksums.txt
  sha256sum --check checksums.txt
)
```

The result should include `sample.txt: OK`. Now alter the data but retain the old manifest:

```
printf 'changed-content\n' > "$hash_demo_dir/sample.txt"
(
  cd "$hash_demo_dir" || exit
  sha256sum --check checksums.txt
)
```

Expect `FAILED` and a nonzero result. Run this interactively; a shell configured to exit on any nonzero command could stop at this expected failure. The mismatch means the data no longer agrees with the saved digest. It does not identify whether the cause was malicious, accidental or an intentional edit.

Now replace the manifest too:

```
(
  cd "$hash_demo_dir" || exit
  sha256sum sample.txt > checksums.txt
  sha256sum --check checksums.txt
)
```

The changed content reports `OK`. Nothing about the hash algorithm was defeated: you simply changed the expected value. A publisher signature prevents you from silently treating a modified manifest as the publisher's original statement, provided the verification key itself is trusted.

Remove only this exercise's files:

```
rm -- "$hash_demo_dir/sample.txt" "$hash_demo_dir/checksums.txt"
rmdir -- "$hash_demo_dir"
unset hash_demo_dir
```

Keep the real installer verification set and its record for the installation lab. They serve a different purpose from these temporary files.

Carry the verification across the copy boundary

The verified file must be the file the VM actually boots. If you copy the ISO to a libvirt-accessible media directory, compare its SHA-256 digest with the verified source or verify the destination against the authenticated manifest. Record the destination path in the VM's installation plan.

A recognizable basename is not sufficient: two directories can contain different files with the same name. The VM manager's optical-media path establishes which one is attached. If an administrator downloads a replacement into that path later, your old record no longer proves the attached bytes are unchanged.

For this VM exercise, attach the ISO as read-only virtual optical media. Writing it onto a physical USB device is a separate operation with a destructive target-selection step. There is no reason to use a raw disk-writing command on the host's physical drives to create this guest.

Store these fields outside the guest:

Field	Purpose
Official source and retrieval time	Identifies the publication context
Exact filename, size and SHA-256	Identifies the verified bytes
Manifest and detached signature	Preserves the publisher's signed statement
Expected signing-key fingerprint and its trusted source	Records the trust decision
Verification outcome	Records both signature and image checks
Attached-media path	Connects verification to the installer actually booted
Release/architecture and intended role	Records suitability

This record can reproduce an acquisition decision, but it is not a claim that the resulting host is permanently secure. Package updates, installer refreshes, later configuration and workload software all add new state.

Diagnose verification failures precisely

Observation	What it establishes	Next action
Public key unavailable	Verification cannot yet identify a usable key	Acquire the intended key and check its fingerprint
Bad signature	Manifest/signature pair does not verify	Stop; reacquire a matching official set
Valid signature from an unexpected key	A signature exists, but publisher authorization is unresolved	Check the publisher's current signing policy
ISO hash mismatch	Downloaded bytes differ from the authenticated manifest entry	Check build pairing and reacquire the image
No exact ISO entry checked	The intended artifact has not been validated	Check filenames, directory and manifest coverage
Valid image for the wrong architecture	Authenticity does not solve suitability	Obtain the correct artifact

Retain diagnostic output before deleting a failed download if you need to investigate the cause. Repeated mismatches after clean downloads warrant examination of storage, transfer paths and publication state; they do not justify bypassing the check.

Practice and recall

Verify the server image you will use for `bl02`, then have another learner trace your record from the trusted fingerprint to the optical-media path. They should not need to infer which of several downloads was used. Complete the text-file experiment and explain why replacing both file and unsigned manifest produces a successful checksum result.

You are ready to proceed when the signature, exact ISO digest, architecture, release series and attached path are all accounted for.

- Integrity compares bytes with an expected value; authenticity establishes who supplied that value.
- A keyserver distributes keys; it does not replace the publisher-identity check.
- Verify the manifest before using its digest as trusted evidence.
- Require the exact intended artifact to be checked.
- A valid signature does not establish suitability, freshness or a secure final configuration.
- Preserve enough evidence to identify the bytes actually booted.

Planning Firmware, Storage, Identity and Networking

An installation plan turns "install Linux" into a set of decisions that can be reviewed before storage is changed. The installer can create a working system with defaults, but it cannot know which data you must preserve, who will recover an encrypted machine, or whether a DHCP address is acceptable for its future service.

This chapter plans a second lab VM, `bl02`, while preserving `bl01`. The two machines will be comparable without needing to be identical. Their differences will become evidence about installation choices rather than accidents you discover later.

The outcome is a short installation record with a clear target, resource budget, access path, network phase, storage policy and acceptance checks. Planning does not require predicting every future workload. It does require identifying which assumptions are temporary and what will trigger a review.

Write the purpose before choosing the layout

For this pass, use this purpose statement:

> Install a second disposable Ubuntu Server VM, inspect the guided LVM storage layout, and compare its installed state with the original lab VM. Preserve the original VM and all baseline copies. Use graphical console access and isolated networking after setup.

This purpose rules out several unnecessary choices. There is no production database to migrate, no need for shared host files, and no requirement for remote login during installation. It also defines a useful comparison: the distribution, virtual resources and network phase can be held similar while the storage plan is examined deliberately.

Provisioning is the broader process of making a usable machine ready for its role. Operating-system installation is one part. Identity, credentials, networking, updates, services, monitoring and recovery also belong to provisioning. A successful installer does not automatically satisfy all of those responsibilities.

Separate the host, virtual machine and guest identities

The host is the physical Linux system that runs libvirt. The VM definition describes presented hardware. The installed guest has its own operating-system identity. Keep all three in the plan:

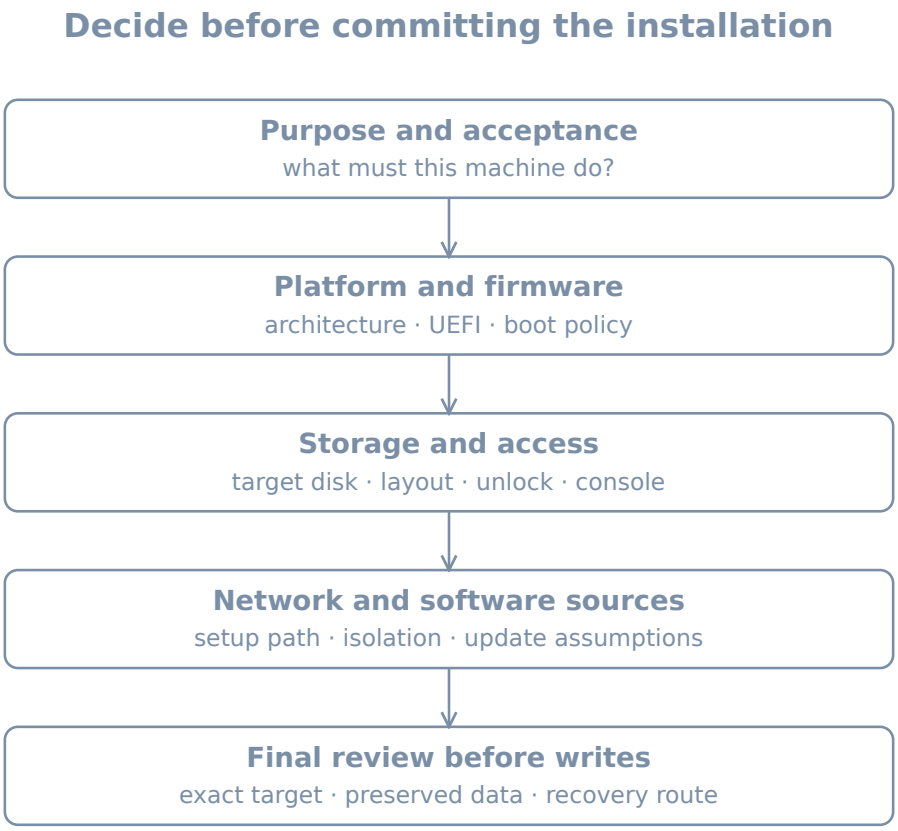


Figure 1: An installation plan passes through decisions that must remain connected.

Layer	Example identifier	What it identifies
Host	Your lab workstation's actual name	The resource and management provider
Libvirt connection	`qemu:///system`	The VM inventory and management scope
VM definition	`bl02`	The new presented machine
Guest hostname	`bl02`	The installed operating system's chosen name
Guest user	`learner`	An account inside the new guest
System disk	New 32-GiB image, recorded host path	The storage that may be overwritten

A familiar hostname in a console prompt is not enough to identify a destructive target. A clone can retain the original guest hostname. Check the VM manager's selected guest and the attached disk inventory before proceeding.

For our comparison, create `bl02` from new installation media and a new disk. Do not clone `bl01` as a substitute for performing the installation: that would reproduce its existing layout and identity rather than show the installer's decisions.

Choose firmware before installing

UEFI is the selected virtual firmware interface for this lab. The installed system will need a compatible boot path, including an **EFI System Partition**, or **ESP**, for firmware-readable boot artifacts. The partition table and the firmware mode are related choices, but they are not the same object.

A **partition table** divides a block device into named ranges of sectors. **GPT**, the GUID Partition Table, is the partition-table format used for this guided UEFI example. GPT does not mean that the guest definitely booted through UEFI; confirm the actual mode from the installer and installed guest.

An ESP is typically formatted using a FAT filesystem and mounted at `/boot/efi` in the installed system. A separate `/boot` filesystem, if the installer creates one, serves a different role. It can hold kernels and initial boot files without being the firmware-readable ESP. The next part examines the complete boot chain; here, distinguish these locations so you can review the installer's proposed layout.

Secure Boot adds signature-based policy to the boot path. It is not the same as checking the ISO signature, and it is not disk encryption. These mechanisms address different boundaries:

Mechanism	Main question
Installer-image verification	Are these the installation bytes authorized by the publisher?
Secure Boot policy	Are boot components accepted by the platform's configured trust policy?

Mechanism	Main question
Disk encryption	Is stored data protected when the required unlock material is unavailable?
Login authentication	Is this user allowed to access the running system?

For `bl02`, select UEFI before the first boot, retain the manager's supported Ubuntu-compatible firmware settings, and record whether Secure Boot is enabled. Do not switch between legacy BIOS and UEFI mid-install to work around an unexplained error. Such a switch can change which boot artifacts and entries are required.

Also record the guest's persistent firmware-variable storage. A UEFI VM can have a host-side NVRAM file as well as a disk image. A recovery plan that saves only the disk should explain how firmware state will be reconstructed or restored.

Plan capacity at more than one layer

Use the same basic teaching allocation as `bl01`: two vCPUs, 4096 MiB of RAM and a 32-GiB virtual system disk. These numbers make comparison straightforward. They are not sizing advice for a production application.

The new VM consumes additional host resources. Keep `bl01-baseline` and recovery copies shut off, and avoid running more guests than the host can support comfortably. If you run `bl01` and `bl02` together, their nominal RAM allocation alone totals 8 GiB, before host and virtualization overhead.

Storage requires a separate budget. A 32-GiB virtual disk is a limit presented to the guest. A sparse host image may consume less space now and more later. A future baseline copy adds another allocation. Host free space must cover the anticipated combined growth rather than just today's image-file size.

Inside the guest, the full disk is also not identical to usable root-filesystem space. Space can belong to partition metadata, the ESP, `/boot`, volume-manager metadata, unallocated volume-group capacity and filesystem overhead. The next chapter measures those layers instead of expecting `df /` to equal the VM manager's disk capacity.

A practical capacity note might say:

```
Guest: 2 vCPUs, 4096 MiB RAM, one 32-GiB system disk.
Host: enough free space for the new disk and one future full copy.
Concurrency: run only the guests needed for the current exercise.
Growth review: before adding data services or storage experiments.
```

Use your measured host free-space value in the real record. Do not copy a fictional number into a capacity decision.

Choose a storage policy, not just a checkbox

For the guided installation exercise, use the entire **new guest disk**, enable LVM if offered, and leave encryption off. This is a controlled disposable lab choice. It keeps the first inspection focused on the relationship between disk, partitions, volumes and filesystems.

Logical Volume Manager, or **LVM**, groups storage into allocatable volumes. Its introductory vocabulary is:

- A **physical volume** is a block device prepared for LVM use, often a partition.
- A **volume group** is a pool built from one or more physical volumes.
- A **logical volume** is an allocated block device drawn from that pool.
- A filesystem can be created on a logical volume and mounted at a path such as `/`.

These layers let storage allocation change without treating each filesystem as permanently tied to a fixed disk partition. They also introduce additional state to inspect. LVM by itself does not provide independent backup or guarantee tolerance of a disk failure.

The Subiquity storage guide describes guided use of a disk and the storage objects the installer can create. "Use an entire disk" authorizes replacing that disk's previous layout; it does not mean every byte must become part of the root filesystem.

Reserved growth space can be intentional

The installer may leave free capacity in the volume group. That capacity can support later allocations or extension. It is different from free blocks inside an existing filesystem. A root filesystem can be nearly full while the volume group still has unallocated capacity; using that capacity requires an explicit allocation and filesystem-growth operation.

For the first guided pass, retain the proposed allocation after reviewing it, and record what was left free. Do not enlarge everything simply to make two displayed numbers agree. The goal is to understand the layout before changing it.

Encryption requires an access and recovery decision

On a real portable or sensitive system, leaving storage unencrypted can expose data if the device is lost. Encryption policy belongs in the installation decision because introducing it later may require migration or reinstallation.

An encrypted installation also needs an unlock method. Ask who supplies it after an unattended reboot, whether a console is available, where recovery material is stored, and what happens when relevant hardware or firmware changes. Hardware-backed unlock and a typed passphrase have different operational dependencies.

Do not put an encryption passphrase into the installation worksheet. Record the responsible person and the protected location of the recovery material. A backup of encrypted bytes is unusable without the necessary keys; a running unlocked machine still needs access controls. The next installation chapter on manual choices will treat encryption and swap directly, and later storage/security parts deepen those mechanisms.

Separate filesystems express separate policies

A separate filesystem for application data can make capacity limits, mount options and recovery procedures clearer. It also creates another mount whose absence or failure can affect boot and services. A separate `/home` is useful in some workstation workflows but is not a universal requirement for servers.

For `bl02`, do not invent separate filesystems for every directory. Use the guided layout as a measured baseline. Later exercises add a spare disk and explicit data mounts with a clear reason. Keep that future spare disk detached during this installation, reducing the number of possible destructive targets.

Plan account access before the first reboot

Create the guest account `learner` with a unique lab password. The same username on `bl01` and `bl02` does not imply that the accounts share identity state or credentials. A local numeric user ID also has meaning within its account database; matching numbers across machines do not automatically establish a shared identity service.

For this pass, the graphical console is the primary access path. Do not enable remote login merely because an installer offers it. Remote access later requires its own authentication and exposure decisions.

After installation, acceptance will include logging in through that console and checking the account's administrative authorization. A privilege check is more useful than assuming the first account must be configured exactly as expected. Preserve recovery access until the intended login path has been tested after reboot.

Choose a hostname that reflects the lab role and is not easily confused with the host workstation. Record the difference between a hostname and a DNS record: setting `bl02` inside the guest does not automatically make every other machine resolve that name.

Make networking assumptions explicit

During installation, attach `bl02` to the already inspected setup NAT network from Part 01. Use one NIC and DHCP for this initial pass. Later, while shut down, move that NIC to `blpractice` and verify the change. Do not attach both networks as a shortcut.

The plan should distinguish four functions:

Function	What it supplies	Evidence to collect
Address configuration	An address and prefix on the guest interface	Guest interface report
Routing	A next-hop/path choice for nonlocal traffic	Route table and intended destination
Name resolution	Names mapped to usable addresses	Resolver configuration and a relevant lookup

Function	What it supplies	Evidence to collect
Software source access	Actual access to installer/package services	Installer mirror test or package operation

Success in one row does not prove success in the next. A DHCP lease does not prove that a mirror is reachable. A mirror failure can come from DNS, routing, proxy requirements, clock problems or the remote service.

Record whether the environment requires a proxy. A proxy is an intermediary for selected network requests, not another name for the default gateway. Do not enter a router address into the proxy field unless that router actually runs the required proxy service. If the network requires authentication or inspection that the lab cannot satisfy, resolve the host/network prerequisites before committing an installation dependent on remote sources.

For an offline installation, record exactly which installation and package artifacts are available locally and which setup actions must wait. Do not promise that a full update will complete while the VM is deliberately isolated from its repositories.

Decide what may change during installation

An installer can offer to refresh itself. Package repositories also change over time. Two installations from the same ISO can therefore produce different final versions if one refreshes the installer or retrieves newer packages.

For this lab, accept supported installer updates when connectivity allows, and record the choice. Then record installed versions after boot. Reproducibility here means you can explain and repeat the essential design choices, not that two disks must be byte-for-byte identical.

The distinction becomes important in automation. Repeating "use the latest image and all updates" repeats a policy but may not reproduce the same artifacts. Exact reconstruction needs retained artifacts or controlled repositories, together with configuration and secrets management. Later provisioning chapters introduce those controls after you understand an interactive installation.

Review the destructive boundary

Before accepting storage changes, compare the installer's proposed target with the host-side record. The authorized destructive scope is one new disk attached only to `bl02`.

Use at least two independent identifiers where available: the guest-visible size and device/model/serial information, and the host manager's disk source associated with the selected VM. If the installer unexpectedly lists several writable disks, stop and inspect attachments. Do not infer that the smallest one is safe or that `/dev/vda` always names the intended target.

The installer may show device names such as `/dev/vda`, `/dev/sda` or `/dev/nvme0n1` according to the presented controller. These names describe an observed enumeration. They are not labels saying "this disk is disposable."

A review record should explicitly say:

```
May replace: the new bl02 system disk recorded in this plan.
Must preserve: bl01, baseline/recovery copies, host disks and shared data.
Unexpected disk or size: stop and reconcile the VM definition.
Recovery if this install fails: recreate only bl02's disposable target;
                                keep verification records and notes.
```

This is not a request to make every administrative action bureaucratic. It is a way to catch an expensive class of mistake before the irreversible write begins.

Completed plan for the guided lab

Fill in the measured fields rather than leaving them as assumptions:

Decision	Planned value or required record
Purpose	Inspect a second installation and its guided LVM layout
VM/guest name	`bl02` / `bl02`
Artifact	Verified 26.04 LTS amd64 server ISO; exact filename and digest
Firmware	UEFI; actual Secure Boot setting recorded
Resources	2 vCPUs, 4096 MiB, new 32-GiB disk
Storage	Guided whole-disk LVM; encryption off for this lab
Disk target	Actual host image path and guest-visible identifiers
Access	Graphical console; local `learner` account
Setup network	Actual inspected NAT network name
Practice network	`blpractice`, one NIC, no external forwarding
Software choices	Standard server selection; no optional bundles
Expected changes	Installer refresh decision and package retrieval recorded
Acceptance	Installed boot, login, storage map, privilege check, network-phase check
Recovery	Preserve `bl01`; recreate only the disposable `bl02` target if necessary

Practice: explain a change before approving it

Review three hypothetical changes to the plan:

1. The guest needs to restart unattended while its owner is away, and encrypted root storage is required. What unlock and recovery decisions are missing?

2. The VM manager shows a 32-GiB disk, but the installer lists two writable disks of that size. What must be reconciled before continuing?
3. The installation must be repeatable in six months without Internet access. Which artifact and repository assumptions need to change?

A good answer names the dependency rather than simply selecting another checkbox. The first needs an approved unattended-unlock design and recovery route. The second needs exact attachment/target identification. The third needs retained media, required packages and initialization inputs, plus a way to validate their provenance and applicability later.

Recall card

- Define the machine's role and acceptance checks before selecting defaults.
- Keep host, VM and guest identities separate.
- Firmware mode, partition layout, Secure Boot and encryption solve different problems.
- Plan capacity at host, virtual-disk, volume and filesystem layers.
- Record access and recovery requirements before committing storage changes.
- The exact destructive target must be identifiable, not merely familiar-looking.
- Repeating a policy and reproducing exact artifacts are different goals.

Guided Installation and the Resulting Storage Layout

The installer says it will use the entire disk. After the first boot, the root filesystem appears much smaller than that disk. Before changing anything, trace where the capacity went. Partitions, volume-manager allocations and filesystems answer different questions about the same storage.

This chapter installs `bl02` using the plan from the previous chapter, then follows the resulting path from the host's disk image to the guest's mounted root. You will inspect the boot partitions, LVM objects, filesystem mounts and swap state, and compare the result with `bl01`.

Use the verified Ubuntu Server 26.04 LTS amd64 ISO, a new 32-GiB VM disk, UEFI, two vCPUs, 4096 MiB RAM and the setup NAT network. Keep the original lab and its baseline untouched. The installer changes `bl02`'s new disk; the post-install commands in this chapter inspect the resulting layout rather than resize or repartition it.

Create the target and reconcile its identity

In Virtual Machine Manager's system QEMU/KVM connection, create a **new** VM from the verified local ISO. Set the planned resources, a new qcow2 disk, the name `bl02` and the inspected setup network. Choose to review hardware before beginning installation.

Confirm UEFI, the selected Secure Boot setting, one writable disk and one NIC. Check that no original lab disk, host filesystem share or spare experiment disk was accidentally attached. Record the new disk's actual host path and the ISO path. A VM name alone is not proof that the storage is new.

On the host, you can inspect the definition through libvirt:

```
sudo virsh --connect qemu:///system domblklist bl02 --details
sudo virsh --connect qemu:///system domiflist bl02
```

An optical device is separate from the writable system disk. If the VM manager is still at a pre-creation wizard stage, complete creation first and inspect before accepting destructive storage changes in the installer. The important boundary is the disk-write confirmation, not whether the GUI has already opened a display.

Open the graphical console and boot the installer. If the console displays an existing login prompt before you have installed anything, investigate the selected VM and disk instead of assuming the image completed an unattended installation.

Turn installer screens into recorded decisions

The Subiquity installer walkthrough describes the supported screen sequence. Labels and optional screens can vary with the image and installer refresh. Follow the intended decisions below and record the actual result rather than relying on an exact screenshot.

Select the appropriate language and keyboard layout. If an installer refresh is offered, make the decision recorded in your plan and note whether it occurred. Select the standard server installation for this pass. Verify DHCP on the one setup NIC, enter a proxy only if required, and let the installer test the chosen mirror.

Do not treat a failed mirror test as permission to make unrelated storage changes first. Resolve required source access or explicitly choose a supported offline path with its limitations recorded. A progress indicator does not replace the final operation result.

Review the proposed storage map

Choose guided use of the entire **new 32-GiB disk**, select LVM if offered, and leave encryption off for this introductory comparison. Before accepting the storage summary, identify:

- The exact disk whose partitions will be replaced.
- The ESP and its intended mount point.
- Any separate `/boot` filesystem.
- The partition or device that will become an LVM physical volume.
- The volume group and root logical volume.
- The filesystem mounted at `/` and any capacity left unallocated.

The terms were introduced in the planning chapter; the table below connects them to review questions.

Proposed object	Review question
Disk/partition table	Is this the disposable disk recorded for `bl02`?
ESP	Is there a firmware-readable boot partition for the selected UEFI path?
`/boot`	Is this separate from root, and what size is proposed?
LVM physical volume	Which partition supplies the pool?
Volume group	How much capacity will it contain?
Root logical volume	How much of the pool is allocated to root?
Root filesystem	What type and mount point will be created?

Save a transcription or screenshot of the summary outside the guest. This becomes evidence to compare with the installed state. Do not publish an installer screen containing passwords or other secrets.

Only accept the destructive confirmation when the map matches the plan. Once confirmed, installation can begin writing in the background while later questions are answered. Closing the console

is not an undo operation. The detailed storage guide explains guided disk selection and supported storage objects.

Finish identity and software selection

Set hostname `bl02` and create the planned `learner` account with its unique password. Keep the graphical console as the access method for this pass. Skip optional SSH installation and optional software bundles so the initial comparison has a small, deliberate scope.

Read the installation result and any failed steps. If installation completes, reboot when offered and detach the ISO when requested. Keep the virtual system disk attached. The intended next boot is from installed storage, not another run through the installer.

If a later screen fails after storage was committed, assume the disk may be partially installed. Preserve the relevant error and plan before deciding whether to recreate this disposable target. Do not describe a partially completed installer as though no change occurred.

Prove that the installed guest booted

Log in through the `bl02` graphical console. Record identity and the root mount:

```
hostnamectl
cat /etc/os-release
uname -r
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
```

The observed hostname should match the plan. The root mount should be the installed filesystem rather than the installer's temporary live environment. A hostname alone is insufficient because it can be duplicated; combine it with the manager's selected VM and the storage evidence.

Check the firmware context:

```
if test -d /sys/firmware/efi; then
  printf 'This guest boot used UEFI\n'
else
  printf 'No EFI firmware interface found; reconcile the boot mode\n'
fi
```

If available, inspect the guest firmware boot entries without changing them:

```
sudo efibootmgr -v
```

`efibootmgr` reports UEFI boot-manager information when the required firmware-variable interface is available. Its manual also documents mutation options; none are needed here. Record `BootCurrent`, the boot order and relevant entry paths if reported. An entry in the list is evidence of configuration, not a guarantee that every fallback path has been tested.

If the utility is absent, record that fact and continue with the available firmware and filesystem observations. Do not modify boot order or delete entries just to make output resemble an example.

Draw the storage stack from observations

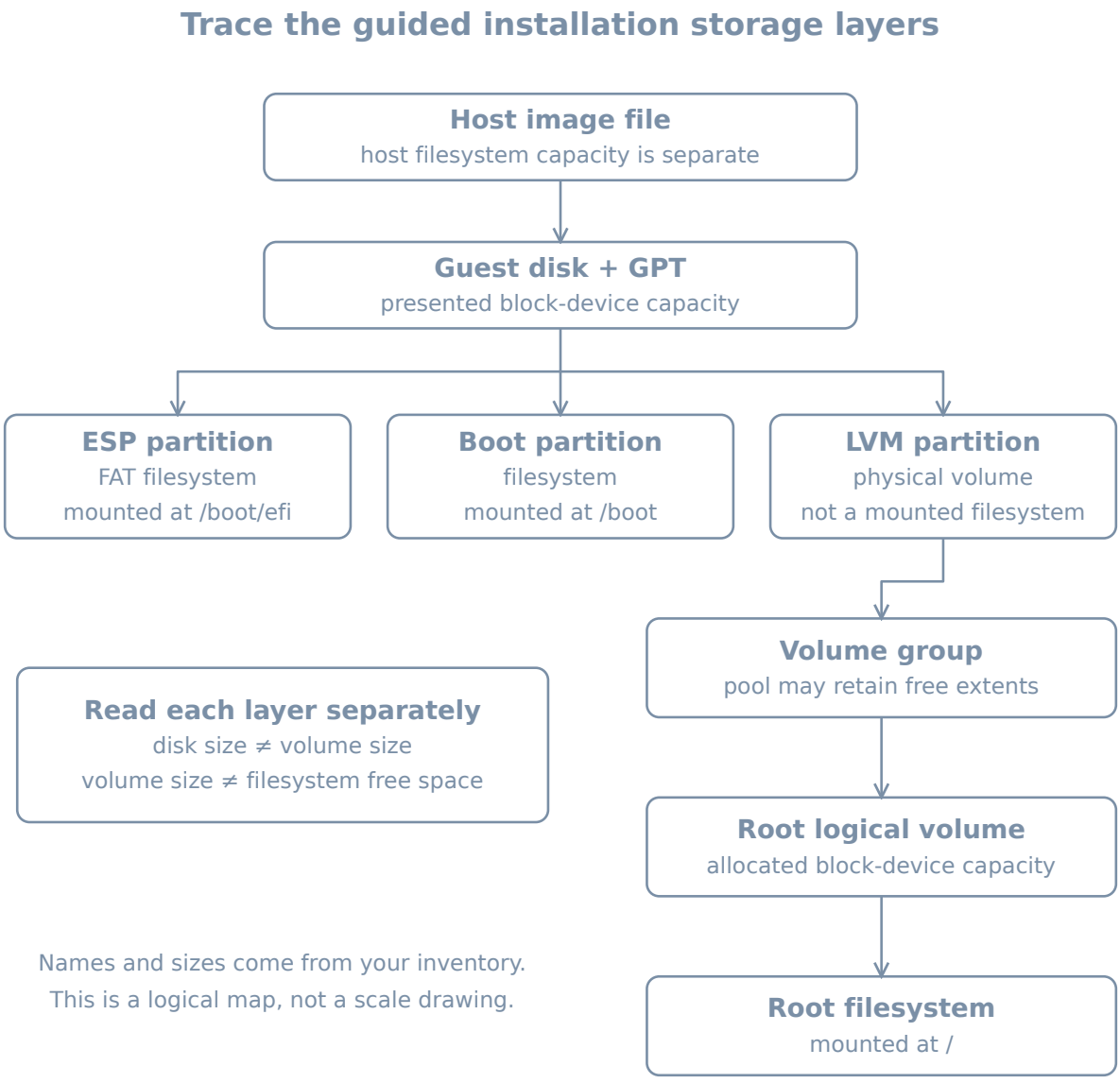


Figure 1: A guided UEFI installation can place root on LVM while keeping boot files on separate partitions.

Start with the guest block-device tree:

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

The lsblk manual explains these columns. A deliberately simplified, illustrative result is:

```
NAME                      SIZE TYPE FSTYPE      MOUNTPOINTS
vda                        32G disk
 vda1                       1G part vfat        /boot/efi
 vda2                       2G part ext4        /boot
 vda3                      29G part LVM2_member
   ubuntu--vg-ubuntu--lv 14.5G lvm  ext4        /
```

This is a teaching example, not a promise of the installer's exact defaults. Names, sizes, partition numbers and filesystem choices can differ. Rounded sizes also hide partition alignment and metadata overhead. Use your own output in the installation record.

Read the tree from the outside inward:

1. `vda` is the disk presented to the guest.
2. `vda1`, `vda2` and `vda3` are partitions on that disk.
3. `vda1` contains the FAT-family ESP filesystem mounted at `/boot/efi`.
4. `vda2` contains a separate filesystem mounted at `/boot`.
5. `vda3` is used by LVM rather than directly mounted as a filesystem.
6. The LVM logical volume provides the block device on which the root filesystem lives.

The doubled hyphens in a device-mapper name can be an escaping representation of hyphens in LVM names. Do not decode a complex name by guesswork when the LVM tools can report the volume-group and logical-volume names explicitly.

If your output shows root directly on a partition rather than on an LVM logical volume, compare it with the storage summary you accepted. That may be a different valid layout, but it does not match this chapter's intended LVM experiment. Record the deviation; do not run LVM creation commands over a mounted root in an attempt to retrofit the example.

Distinguish three kinds of free space

Inside this one VM, "free space" can mean at least three different things:

Location	Meaning	Typical observer
Outside partitions	Disk sectors not assigned to a partition	Partition-layout inspection
Inside a volume group, outside logical volumes	LVM extents available for allocation	`vgs`, `pvs`
Inside a filesystem	Filesystem capacity available for file data/metadata	`df`

There is also free space on the **host filesystem** containing the qcow2 image. It belongs to a different administrative layer.

An **extent** is an allocation unit used by LVM. The volume group can have free extents even while the root filesystem has no free blocks left. The Subiquity storage-layout reference explains that its LVM layout can reserve capacity for later use rather than allocating all of it to root immediately.

Inspect the three LVM views:

```
sudo pvs -o pv_name,vg_name,pv_size,pv_free
sudo vgs -o vg_name,vg_size,vg_free
sudo lvs -o lv_name,vg_name,lv_size,lv_path
```

`pvs` reports physical volumes, `vgs` volume groups, and `lvs` logical volumes. The respective pvs, vgs and lvs manuals define their report fields. These are inventory commands; they do not ask LVM to resize anything.

Suppose your measured volume group is roughly 29 GiB and its root logical volume is roughly 14.5 GiB. A report of roughly 14.5 GiB unallocated in the group would account for the difference. Those extents are not automatically usable as files under `/`; allocation and filesystem growth would be separate changes.

Now inspect the mounted filesystem:

```
df -h /
df -i /
```

The filesystem's displayed capacity can be smaller than the logical volume because the filesystem uses some space for its own structures and may reserve capacity. Unit conventions and rounding also affect displayed values. Do not add figures from different layers as if they were independent disks.

Worked diagnosis: "the installer lost half my disk"

The report is that a 32-GiB VM has a root filesystem around half that size. Competing explanations include a deliberately smaller root logical volume, another partition consuming capacity, an unexpectedly smaller virtual disk, or an inspection of the wrong mount.

A discriminating sequence is:

1. Confirm the VM manager's selected guest and virtual capacity.
2. Check `lsblk` for the guest disk and partition tree.
3. Use `findmnt --target /` to identify the actual root source.
4. Use `vgs` and `lvs` to compare group capacity with logical-volume allocations.
5. Use `df /` only after identifying the filesystem being measured.

If the difference is free volume-group capacity, the explanation is reserved allocation, not lost storage. Whether to extend root depends on workload requirements and the intended use of that reserve. This chapter stops at a supported explanation; later storage work performs growth with prerequisites and verification.

Connect persistent mount intent to current mounts

A filesystem has an on-disk identity as well as a current device pathname. A **UUID** is an identifier commonly used to refer to a filesystem independently of a changing enumeration name. A partition can have its own **PARTUUID**, which is a different identifier.

Inspect both types and the installed mount configuration:

```
lsblk -o NAME,FSTYPE,UUID,PARTUUID,MOUNTPOINTS
cat /etc/fstab
findmnt --target /boot --output TARGET,SOURCE,FSTYPE,OPTIONS
findmnt --target /boot/efi --output TARGET,SOURCE,FSTYPE,OPTIONS
```

`/etc/fstab` describes persistent mount and swap configuration. A typical filesystem entry includes a source, mount point, type and options, followed by fields used by maintenance tools. The fstab manual defines those fields. Do not paste UUIDs from an example into your installed configuration: they belong to specific objects.

Compare intent with observation. An fstab entry shows a requested persistent configuration; `findmnt` shows the current mount view. The findmnt manual distinguishes finding the filesystem containing a target path from querying an exact mount point.

This distinction matters for `/boot`. If it is not a separate mount, `findmnt --target /boot` can report the root filesystem that contains that directory. That is not an error by itself. It is evidence about the chosen layout. For our illustrative guided layout, expect separate mount results and reconcile any difference with the installer summary.

Do not edit fstab during this inspection. A wrong persistent mount reference can become a boot problem after the next restart, even while the current mount still works. The boot and storage parts provide deliberate exercises for those changes.

Observe swap without assuming its form

Swap supplies a location where selected memory pages can be stored outside ordinary RAM management. It may be backed by a partition, a file or another supported arrangement. An installed swap file is not necessarily visible as a separate block-device row in `lsblk`.

Inspect active swap:

```
swapon --show
free -h
```

The reporting form of `swapon` lists active areas; it does not activate a new one. Its manual distinguishes reporting from configuration operations. Record what is actually active, including an empty result if no swap is enabled.

An fstab entry and an active swap report answer different questions, just as they do for mounts. Also separate swap capacity from application memory requirements: swap does not make a small

VM equivalent to one with enough RAM for its working set. Later memory and storage chapters examine reclaim behavior, encryption and performance implications.

Verify access and compare the two installations

Before calling the installation complete, confirm that the planned account can log in after a reboot and that administrative authorization is as intended:

```
id
sudo -l
```

`sudo -l` reports allowed commands according to the account's policy and may request authentication. Do not publish its output if it exposes sensitive local policy. An unexpected restriction or broad grant belongs in the acceptance record, not in a silent workaround using another account.

Make a comparison table using observations from each guest:

Property	`bl01`	`bl02`	Explanation of difference
Exact installer/build record	Your value	Your value	Media or installer refresh
Running kernel	Your value	Your value	Installed/activated updates
Firmware mode	Your value	Your value	Selected virtual firmware
Root source and filesystem	Your value	Your value	Storage choice
Volume-group free capacity	Your value or not applicable	Your value	Allocation policy
`/boot` and ESP	Your values	Your values	Boot layout
Active swap	Your value	Your value	Installer/configuration choice
Console login and privilege check	Observed result	Observed result	Account policy
Current NIC source	Host observation	Host observation	Lab phase

Do not fill the table from memory of the settings you clicked. Use the installed evidence. Intended configuration and resulting configuration are separate columns in a good installation record.

After completing any required setup-network maintenance, shut down `bl02`, move its one NIC to `blpractice`, and restart it through the console. Verify its address, route and host-side NIC source as in Part 01. The loss of external repository access is expected in the isolated phase.

If `bl01` and `bl02` run together, they are separate installations with distinct guest identities. This differs from starting two unprepared copies of the same disk image. Keep the old baseline and recovery clones shut off.

Diagnose installation failures at the correct stage

Symptom	Evidence to collect before retrying
Installer never starts	Verified attached media, guest architecture, firmware selection and console message
Storage target differs from the plan	Host disk attachments, guest disk sizes and installer summary
Mirror or package step fails	Exact source, network phase, address/route and installer error
Installation completes but no bootable disk is found	Firmware mode, optical-media state, boot entries and installed disk attachment
Login succeeds but root layout differs	Accepted storage summary, block tree, LVM reports and current mounts
Root seems smaller than the virtual disk	Partition, volume-group, logical-volume and filesystem capacities

Keep a failure record before recreating the target. For this disposable VM, reinstalling may be reasonable, but it should test a changed hypothesis rather than repeat the same unknown failure. A failed install on a real machine with existing data would require a different recovery plan; do not transfer the lab's permission to recreate a disk into another environment.

Practice and completion criteria

Draw your own storage map from the host image path to the guest root mount. Label the ESP, any separate `/boot`, physical volume, volume group, root logical volume, filesystem and free allocation at the relevant layers. Explain any difference from the illustration.

Then answer:

1. Why can whole-disk guided installation leave free volume-group capacity?
2. Why does a root-filesystem free-space report not establish free space on the host?
3. What does an fstab entry establish that a current mount report does not, and vice versa?
4. What evidence proves you booted the installed guest rather than the live installer?

Completion means a successful installed boot and login, a storage map derived from commands, a reconciled comparison with the installer summary, and a documented final network phase. Keep the resulting record outside the guest before creating a new baseline.

Recall card

- The storage confirmation is a destructive boundary; review the exact target first.
- Trace capacity through disk, partition, volume and filesystem layers.
- Whole-disk selection does not imply whole-disk allocation to root.
- UUIDs, device names and mount paths describe different identities and relationships.

- Persistent configuration and current activation require separate observations.
- A successful installation is a verified result against a plan, not only a completed progress bar.

Manual Storage Choices, Encryption and Swap

A storage layout is a set of promises: where the machine will find boot files, how much space each workload can consume, what an offline reader can recover, and what the administrator must supply after a restart. The installer can create those objects, but it cannot decide whether the promises suit your workload.

This chapter designs a custom installation on a new disposable guest, `bl02-manual`. Keep `bl01` and the guided `bl02` installation intact. Use the same verified Ubuntu Server 26.04 amd64 media, UEFI settings, two vCPUs, 4096 MiB RAM and setup NAT network as before, with a **new 40-GiB system disk**. Only this guest needs to run during the exercise. Allow host disk capacity for the additional image to grow; a sparse image's small initial file size is not its eventual requirement.

The main lab uses plain partitions so each allocation is visible. Encryption is a separate optional installation experiment, with its own recovery conditions. Do not attempt to convert the existing guided root filesystem in place.

Start with mount responsibilities

A partition describes a region of a disk. A filesystem gives that region a structure for files. A mount attaches a filesystem to a path in the running system. These three decisions are related but different: naming a directory `/srv` does not create a partition, reserve capacity, or encrypt its contents.

For this exercise, `/srv` will hold a small demonstration file representing future service data. No real service is installed yet. Use this teaching layout:

Object	Allocation	Purpose
EFI System Partition	Installer-created; record actual size	UEFI boot files; mounted at `/boot/efi`
Root partition	24 GiB, ext4, mounted at `/`	Operating system, home directory and logs
Service-data partition	8 GiB, ext4, mounted at `/srv`	A separately measurable data filesystem
Swap partition	2 GiB, swap use	A small disk-backed swap area for this lab
Unallocated space	Remainder	Visible reserve for a future storage exercise

The sizes are a lab budget, not a production sizing formula. Account for the ESP, partition-table metadata and alignment before expecting the remainder to equal a particular number. Check how the installer interprets its size units and record the displayed result.

This unencrypted design keeps `/boot` as a directory on the root filesystem. The ESP is still a separate filesystem. A separate `/boot` is useful or necessary for some other boot/storage arrangements; it is not synonymous with the ESP.

Why separate service data?

A separate filesystem makes its capacity boundary easy to see. If `/srv` fills, it does not consume the free blocks in the root filesystem. However, application logs or temporary files elsewhere can still fill root. Separation also introduces another mount that must succeed and another allocation that can be badly sized.

Consider two imagined workloads:

- A small learning VM with little persistent data benefits from a simple layout and few failure points.
- A server storing growing application data benefits from an explicit data budget, but needs monitoring and a growth/restore plan for that budget.

Neither layout creates redundancy. Both partitions in this lab depend on the same virtual disk and host storage. Deleting that image loses both filesystems.

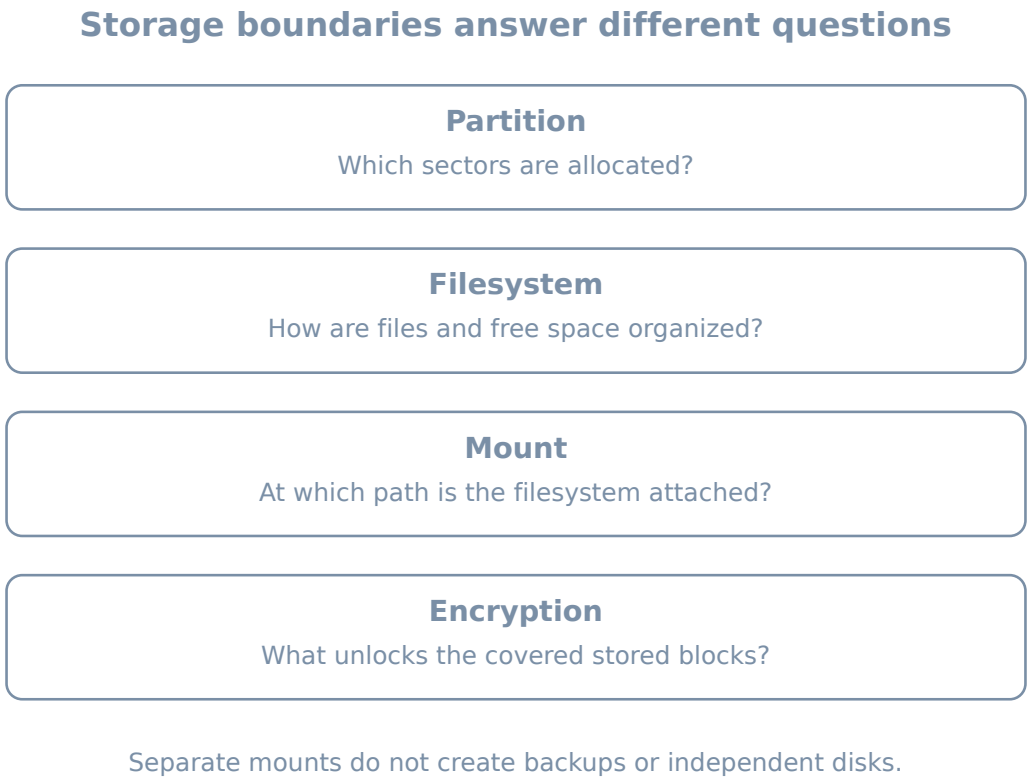


Figure 1: Storage boundaries have different jobs.

Build the custom layout

In Virtual Machine Manager, create `bl02-manual` with the specified resources and fresh disk. Before accepting any installer storage changes, inspect the guest definition on the host:

```
sudo virsh --connect qemu:///system domblklist bl02-manual --details
```

Confirm that its writable image is different from both existing guests. Keep only the new disk and installation ISO attached. Record the image path outside the guest, together with the intended layout.

Boot the installer in UEFI mode. At storage selection choose **Custom storage layout**. On the new disk select the boot-device action, inspect the ESP the installer creates, then add the root and `/srv` GPT partitions with the sizes, filesystems and mounts above. Add a 2-GiB partition with swap use. Leave the remainder unallocated. Canonical documents these controls and the automatic ESP creation in its Subiquity storage guide.

Screen wording can vary with the installer build or refresh. If an action differs, inspect its summary before proceeding. Do not substitute shell formatting commands against a guessed `/dev/vda` to force agreement with the lesson.

Before the final disk-write confirmation, reconcile all four intended objects. There must be exactly one root mount, the intended ESP mount, one `/srv` filesystem, and the selected swap area. Check that the remaining capacity is genuinely unallocated rather than silently absorbed into the last partition. Save the summary outside the guest.

Complete installation with hostname `bl02-manual`, the planned `learner` account and no optional SSH server or software bundles. Detach the ISO at the requested stage and boot from the new disk.

Verify boundaries rather than partition numbers

In the installed guest:

```
hostnamectl
lsblk -o NAME,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
findmnt --mountpoint / --output TARGET,SOURCE,FSTYPE,OPTIONS
findmnt --mountpoint /srv --output TARGET,SOURCE,FSTYPE,OPTIONS
findmnt --mountpoint /boot/efi --output TARGET,SOURCE,FSTYPE,OPTIONS
findmnt --target /boot --output TARGET,SOURCE,FSTYPE,OPTIONS
swapon --show
cat /etc/fstab
```

`--mountpoint` asks for an exact mount. `--target` asks which mounted filesystem contains a path. Thus this design should return a distinct `/srv` mount, while `/boot` should resolve to the root mount. The findmnt manual explains the distinction. A missing `/srv` result needs investigation even if the directory exists.

Use your observed device names and UUIDs; partition numbering is not an acceptance criterion. Record the actual swap arrangement too. If the installer also configured a swap file, explain that deviation and its capacity rather than silently claiming the partition is the only active swap.

Compare capacity at the filesystem layer:

```
df -h / /srv
df -i / /srv
```

Expect two different filesystem sources. The formatted capacity will differ from the requested partition size. `df -i` counts inodes, the filesystem records needed to represent files; running out of these can prevent creation even with free data blocks.

Create a small acceptance marker only after confirming the exact `/srv` mount:

```
printf 'bl02-manual storage acceptance\n' | sudo tee /srv/installation-check.txt
sudo cat /srv/installation-check.txt
```

The shell pipe sends the text to `tee`, which writes it using administrative privilege. There is no need to make all of `/srv` writable by every user.

Reboot through the console, log in again, repeat the exact mount check and read the marker. Persistence of the text proves little by itself: a file accidentally written beneath an absent mount can also survive a reboot. The file and its filesystem source together are the evidence.

Choose encryption by the recovery requirement

Block-device encryption transforms stored blocks so that unlocking material is required to interpret their contents. Linux installations commonly use LUKS, a format that manages encrypted-volume metadata and key slots, together with the kernel's device-mapper encryption facility. An unlocked mapping presents a usable block device to the layers above it.

For an installation using encrypted LVM, a possible data path is:

```
partition → LUKS container → unlocked mapping → LVM → filesystem → mount
```

Observe the actual order rather than assuming every encrypted system uses this one. Encryption and volume management answer different questions: access to stored blocks versus allocation of capacity.

A passphrase prompt creates an operational dependency. Who can supply it after a power failure? Can that person reach the virtual console? Where is the recovery material stored if the VM or its host is unavailable? Record the answers before installing an encrypted root.

Encryption protects covered data while locked. It does not prevent an authorized process on the running, unlocked system from reading that data. Boot partitions may remain outside that coverage. A host administrator who controls the running VM also occupies a different trust position from someone holding only a powered-off disk image.

Optional encrypted installation comparison

Use another disposable guest, `bl02-crypt`, with a fresh 40-GiB disk and the same remaining hardware budget. Shut down the other guests if host RAM is limited. In guided storage select LVM and its passphrase encryption option. This is a separate comparison, not a requirement to reproduce the manual partition scheme inside encryption. The installer storage documentation describes this guided choice.

Store the unique unlock passphrase in a trusted password manager outside the guest. Do not put it in screenshots, command history or the installation record. Record only where authorized operators retrieve it.

After installation, perform a full shutdown and start through the VM manager. A saved-memory resume does not test an unlock from cold boot. Observe the unlock prompt, supply the passphrase and verify console login. Then inspect:

```
lsblk -o NAME,TYPE,FSTYPE,MOUNTPOINTS
sudo cat /etc/crypttab
findmnt --target / --output TARGET,SOURCE,FSTYPE
swapon --show
```

`/etc/crypttab` describes encrypted devices to establish at boot; its fields include the mapping name, source device, key specification and options. It is not a listing of mounted filesystems. See the crypttab manual. Treat the configuration as sensitive inventory and do not publish any referenced key material.

Draw the root device's ancestry from the observed tree. Identify the unencrypted boot objects separately. This exercise proves the tested passphrase and boot path work; it does not prove recovery from damaged encryption metadata. Keep this guest disposable until a separate backup and recovery design exists.

Swap is part of the data-protection decision

Swap can contain information from process memory. An encrypted root does not establish that every swap location is encrypted. Trace each active area: a swap file inside a covered filesystem has a different storage path from an independent unencrypted swap partition.

For the main manual lab, the swap partition is deliberately unencrypted and the guest holds no sensitive workload. For the optional encrypted guest, record whether swap is present and where its backing storage sits relative to the encryption layer. Do not assume a swapfile will appear as its own block-device row.

The 2-GiB allocation is simply a bounded lab choice. Production swap sizing depends on workload, memory pressure, storage latency and recovery policy. Hibernation adds requirements for saving and resuming memory state; it is not part of this server VM exercise. No universal multiplier of RAM substitutes for those decisions.

Diagnose a layout mismatch

Symptom	Distinguishing evidence	Appropriate next action
`/srv` exists but exact mount lookup is empty	Compare fstab, UUIDs and current mounts	Preserve evidence; do not start writing service data there
`/srv` and `/` show the same source	Exact mount query versus containing-filesystem query	Reconcile the installed layout with the recorded summary
Swap report is empty	Compare active state with installer summary and fstab	Record missing activation; avoid inventing a device path
Encrypted guest never reaches login unattended	Console shows an unlock prompt	Supply the planned credential; evaluate unattended-boot needs separately
Unlock fails after restart	Keyboard layout, selected guest and stored credential	Check identity and input; formatting is not a password-recovery action
Partition budget differs	Installer units, ESP allocation, rounding and remaining space	Explain the arithmetic before resizing anything

An accidentally omitted `/srv` mount can hide a further trap. Files written into the underlying root directory become hidden when another filesystem is later mounted there. They are not automatically transferred into the mounted filesystem. Capture the current state and plan recovery before adding or removing a mount over existing data.

For an installation mistake on this empty teaching guest, recreating its verified disposable disk and reinstalling is a valid reset. First preserve the evidence explaining the mistake. On a populated server, preserving and restoring data comes before repartitioning.

Practice and acceptance record

1. Install the manual guest and record the accepted storage summary.
2. Explain which objects share the same failure domain despite having separate mounts.
3. Prove `/srv` is a distinct filesystem before and after reboot, and confirm the marker's contents.
4. Record active swap, root free space and the unallocated reserve at their respective layers.
5. If you completed the encryption extension, demonstrate a cold-boot unlock and identify what remains outside encryption.

Keep the manual guest powered off as comparison evidence, or retire it using the VM manager after saving the record. Before deleting any image, verify its path is the one assigned to this guest. Continue the next chapter on the guided `bl02` installation.

Recall card

- A path, a mount, a filesystem and a partition are different objects.
- Separate capacity is useful only when its size and mount lifecycle fit the workload.

- Encryption adds an unlock and recovery dependency; it does not create a backup.
- Swap protection follows its backing storage, not the hostname's security label.
- Accept the installed layout using observed sources and a reboot, not installer intent alone.

First Boot and Device Readiness

First Boot: Administrative Access, Updates and Service Inventory

An installation can reach a login prompt while still lacking a working update path, carrying a failed service, or exposing an unintended listener. First-boot administration turns "it starts" into a small set of demonstrated operating conditions.

Return to the guided `bl02` guest from Installation Foundations. Keep console access, the recorded disk identity and the setup NAT network. This chapter updates that disposable guest and reboots it. Keep `bl01` and its baseline unchanged. The manual-storage comparison guest can remain powered off.

The result will be an acceptance record: working administrative access, a reviewed package transaction, a verified reboot and an explained service/listener inventory. Detailed account policy and package management come later; this chapter supplies the steps needed to establish the initial host.

Prove access before changing software

Open the VM console and log in as `learner`. Confirm identity:

```
hostnamectl
cat /etc/os-release
id
sudo -l
sudo id -u
```

The hostname should be `bl02`. `id` describes the current account and groups; `sudo -l` reports its permitted administrative commands. The final command should print `0` if policy allows it, demonstrating that this invocation ran with root's numeric user ID. Group membership by itself is not a complete policy test.

If authentication or authorization fails, stop the update workflow. Check the selected guest, username and keyboard layout first. Preserve the exact error. Use a separately established administrator or the documented lab recovery route if available. Do not weaken policy globally to obtain a successful-looking test. On this fresh disposable guest, a documented reinstall is preferable to improvising an untracked privilege bypass.

Create a private notes directory in the guest:

```
umask 077
mkdir -p "$HOME/first-boot-record"
date -Is > "$HOME/first-boot-record/started.txt"
uname -r > "$HOME/first-boot-record/kernel-before.txt"
cat /proc/sys/kernel/random/boot_id > "$HOME/first-boot-record/boot-id-before.txt"
```

`umask 077` restricts permissions on newly created files and directories in this shell. It does not repair permissions on pre-existing files. The boot ID identifies this boot instance; it is different from the installed machine's persistent identity. Copy the final notes outside the guest using the lab's established evidence procedure. A record only on the failed disk is a poor recovery aid.

Establish an update preflight

Check the resources and dependencies the package transaction needs:

```
df -h / /boot /boot/efi
df -i /
ip -brief address
ip route
timedatectl status
```

The filesystem checks identify available capacity and inode pressure. The network commands show interface addresses and routes, not proof that a repository is reachable. `timedatectl` reports the clock and available synchronization status. An obviously wrong clock can affect validation and confuse the evidence timeline; establish correct time before interpreting expiry errors.

Inspect configured APT sources without changing them:

```
ls -l /etc/apt/sources.list.d
sudo cat /etc/apt/sources.list.d/ubuntu.sources
```

If this file is absent, inspect the actual source files listed in that directory and `/etc/apt/sources.list` if present. Newer installations commonly use deb822 `.sources` files; imported configurations can use `.list` entries. Record the configured release suites and repository locations. A suite is a named collection of packages, such as a release's main or update collection. Do not paste an example from another Ubuntu release into this guest.

The Ubuntu package-management guide describes source configuration and package administration. Review local source content before sharing it: private mirror URLs or proxy configuration may disclose internal information.

Refresh metadata and review the transaction

Run these commands one at a time and read each result:

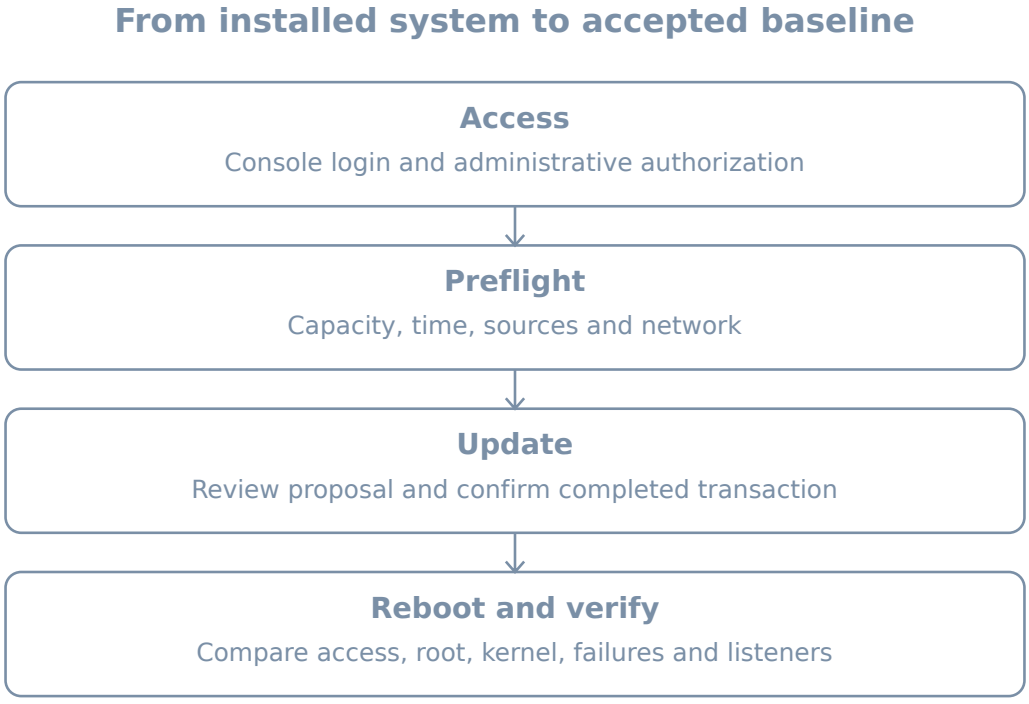


Figure 1: A first-boot acceptance pass includes verification after restart.

```
sudo apt update
apt list --upgradable
sudo apt --simulate upgrade
```

`apt update` refreshes repository metadata; it does not install upgrades. If it reports failed downloads or validation errors, do not treat a partial refresh as a clean result. Keep the error and investigate before proceeding.

The simulation previews proposed package changes without applying them. Review package names, download size, disk-space impact and any packages left unchanged. A simulation is a plan against the current state, not a reservation: another package job can change that state before the real transaction.

When the plan is understood, run:

```
sudo apt upgrade
```

Read the actual confirmation before accepting. This command can install new dependencies but does not remove installed packages to satisfy an upgrade. `full-upgrade` has different removal behavior; do not switch to it simply to make a pending count disappear. These distinctions are documented in Ubuntu's apt manual.

Allow the command to complete. Package hooks can update boot artifacts or restart services. If a configuration-file question appears, inspect the proposed difference and record your decision; there is no universal answer that fits both a fresh file and a deliberately customized one.

Review the final result and package history:

```
sudo tail -n 80 /var/log/apt/history.log
sudo tail -n 80 /var/log/apt/term.log
sudo dpkg --audit
```

History records package operations; the terminal log supplies transaction detail. The dpkg manual defines the audit of package-database consistency and partial installation state. A clean audit normally produces no output, but that alone does not prove every application works. Keep both the transaction result and the post-reboot checks.

An automatic update may already be running

A fresh host can have background package activity. Inspect it before assuming a lock is stale:

```
systemctl list-timers --all --no-pager
systemctl status apt-daily.service apt-daily-upgrade.service --no-pager
sudo journalctl -b -u apt-daily.service -u apt-daily-upgrade.service --no-pager
```

A scheduled timer, an active job and a completed update are different observations. A lock error should lead to identifying the current package process and waiting for legitimate work to complete. Deleting a lock file or killing an updater can turn harmless serialization into damaged state. If work appears stuck, preserve its status and logs before choosing recovery.

Inspect the installed automatic-update configuration separately:

```
apt-config dump | grep -E 'APT::Periodic|Unattended-Upgrade'
```

No matching lines is an observation, not a reason to invent defaults. Canonical's automatic-updates guide describes the settings and logging. Record whether unattended installation and automatic reboot are configured; an enabled update timer does not alone establish either policy. This first-boot pass leaves policy changes for the dedicated package chapter.

Inventory what the machine is running

Collect four views before reboot:

```
systemctl --failed --no-pager
systemctl list-units --type=service --state=running --no-pager
systemctl list-unit-files --type=service --no-pager
sudo ss -lntup
```

These views answer different questions:

View	Question
Failed units	What has entered a failed state?
Running services	Which service units are currently running?
Service unit files	What service definitions and enablement states exist?
Listening sockets	Which TCP listeners and UDP endpoints are bound?

Enabled does not mean running. An inactive service may be started on demand; a successful one-shot job can finish without remaining running. The systemctl manual defines these states and listings. Do not enable every inactive unit or clear failure records merely to make the inventory look healthy.

In `ss`, `-l` selects listening sockets, `-n` keeps numeric addresses and ports, `-t` and `-u` select TCP and UDP, and `-p` requests process details. Root privilege helps reveal ownership. A loopback address such as `127.0.0.1` limits that binding to local access; a wildcard address can accept traffic on more interfaces, subject to routing and policy. A socket listing is not a remote reachability test. The ss manual documents the filters and process reporting.

For this console-only installation, an SSH listener is an unexpected finding to explain. Check package/service choices and any socket activation rather than assuming a port number alone identifies the application. NAT does not make every guest listener irrelevant to other guests or the host.

For each unexpected item, record its name, purpose, owner and evidence before deciding whether to change it. This avoids disabling a service that provides the network or other dependency used by the current investigation.

Reboot and repeat the acceptance checks

Inspect Ubuntu's reboot-request marker if present:

```
if test -f /run/reboot-required; then
  cat /run/reboot-required
  if test -f /run/reboot-required.pkgs; then
    cat /run/reboot-required.pkgs
  fi
fi
uname -r
```

An installed kernel package and the running kernel are different states. The marker is a useful package-maintenance signal, not a complete certification that no restart is needed when absent. For this lab, perform one planned reboot after the successful update transaction regardless, with the console open:

```
sudo systemctl reboot
```

Wait for normal startup and log in again. Then inspect:

```
hostnamectl
sudo id -u
uname -r
cat /proc/sys/kernel/random/boot_id
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
systemctl --failed --no-pager
sudo ss -lntup
```

Compare the boot ID with the saved value: it should have changed. Compare the kernel with `kernel-before.txt`: it may remain the same if this transaction installed no different kernel. Do not invent an expected version number. Root must still resolve to the intended installed storage, administrative access must work, and any new failures or listeners need explanation.

Worked diagnosis: repository refresh fails

Suppose the console works but `apt update` reports a name-resolution failure. The evidence points to an update dependency before any package upgrade has started.

1. Preserve the exact repository hostname and error from the output.
2. Inspect the guest NIC and default route using the preflight commands.
3. Run `getent hosts` followed by that **actual hostname**, without the URL scheme or path. This asks the configured host-lookup system for its addresses.
4. Check the VM manager's network attachment against the recorded setup NAT network.
5. If the guest was deliberately moved to the isolated network, restore the planned setup attachment and repeat the checks.

A successful lookup only narrows the problem: proxies, transport access, repository availability and signature validation can still fail. A signature error with successful network access calls for checking time, source identity and trust configuration, not disabling authentication. After correcting the identified cause, repeat `apt update` and require a clean result before reviewing an upgrade.

For a failed reboot, keep the console message and identify whether firmware, the bootloader, early startup or the normal service manager is visible. Avoid repeated forced resets. Use the known baseline to restore this disposable lab if necessary, recording that updates applied after that baseline will need to be repeated. A checkpoint restores captured state; it is not equivalent to a package-by-package downgrade.

Practice and acceptance record

Complete a table with observed results:

Acceptance item	Evidence to retain
Correct guest and administrator	Hostname, selected VM, successful policy test
Update prerequisites	Capacity, time, network attachment and sources
Package change	Reviewed proposal, completion result and history
Normal restart	New boot ID and observed console login
Intended root	Mounted source and filesystem
Services and endpoints	Explained failed units and listeners
Recovery	Location of baseline and notes outside the guest

An unresolved failure belongs in an open-issues entry, not in an accepted baseline. Once checks pass, shut down normally and use the checkpoint method established in the lab chapter to label this state `bl02-first-boot-accepted`. Retain the earlier baseline until the new one has been verified. Keep credentials out of the label and notes.

As a bounded exercise, disconnect the setup NIC in the VM manager while retaining console access. Observe the changed address/route state and failed repository access, then restore the same attachment and verify a clean metadata refresh. Do not start an upgrade while connectivity is intentionally broken. The success criterion is a documented fault, its discriminating evidence and demonstrated restoration.

Recall card

- Test administrative access before making changes that depend on it.
- Metadata refresh, package installation and activation after reboot are separate steps.
- Review update errors and lock ownership before attempting repair.
- Running, enabled, listening and remotely reachable are different claims.
- Accept the baseline only after a normal reboot and repeated checks.

Hardware and Virtual-Device Discovery

A network interface is missing after installation. Is the virtual NIC absent from the guest, present but unsupported, waiting for firmware, or working without the expected network configuration? Those are different failures. Start with the boundary at which the evidence stops, then investigate that boundary.

This chapter follows a device from its presentation to Linux through driver binding and the interface visible to an administrator. The main lab adds and removes a **second virtual NIC** on the disposable `bl02` guest, using its graphical console for access. It does not disconnect the original NIC, change the system disk or install third-party drivers. You need the first-boot acceptance record, the isolated `blpractice` network from the lab setup, and enough host memory to run `bl02` alone. The optional firmware investigation uses evidence from whatever hardware is available; the virtual NIC is not expected to require a downloadable firmware file.

From presentation to usable interface

A physical bus exposes devices and identifying information to the kernel. In a VM, the hypervisor presents virtual hardware according to the guest definition. A virtual NIC may be backed by a host network, but the guest sees the virtual device model, not necessarily the physical NIC in the host. A **driver** is kernel code that knows how to operate a device. Driver binding associates a discovered device with a suitable driver; the driver probes and initializes it. The kernel driver-model documentation explains matching and binding.

After successful initialization, the kernel can expose an interface such as a network device or a block device. `/sys` gives a view of kernel objects and their relationships. Device events can reach `systemd-udevd`, which applies userspace rules and stores properties. Some devices also have nodes under `/dev`; network interfaces normally do not need a `/dev` node. The udevadm manual describes querying the udev database.

There is no guarantee that each step succeeds. A device can appear on a bus with no bound driver, bind while failing a later function, or provide a working interface whose network configuration is wrong. A readable device node does not prove its application-level use is correct.

Establish what the guest sees

Start `bl02` with its original setup NIC and open its console. Confirm the guest and running kernel:

```
hostnamectl
uname -r
```

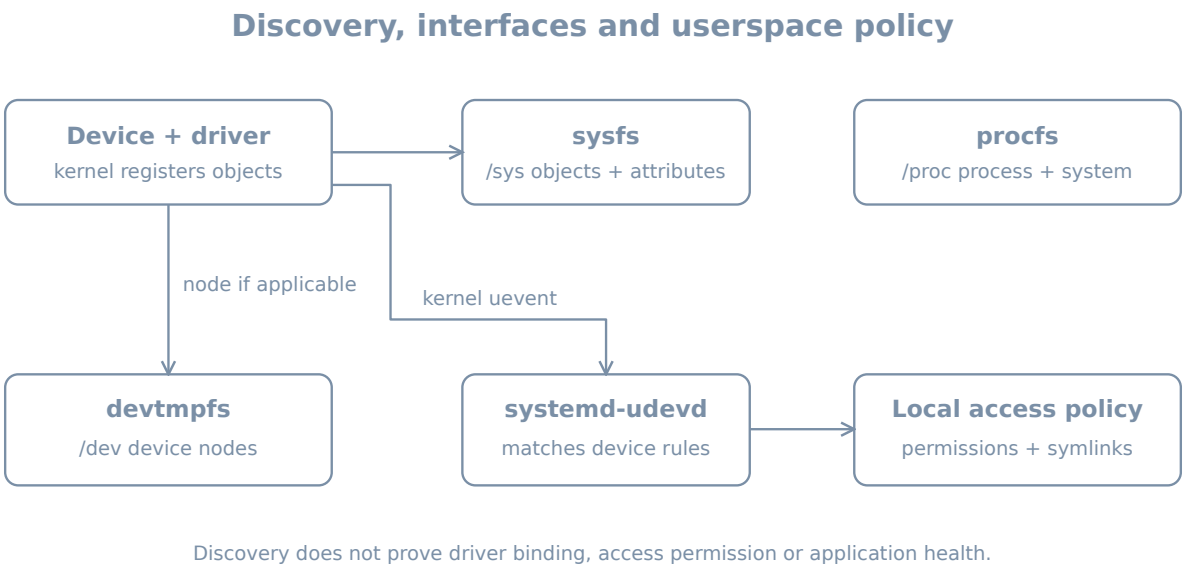


Figure 1: Device discovery produces several complementary views.

```
ip -brief link
ip -brief address
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

Record the original interface name and hardware address from `ip -brief link`. The address here is the link-layer MAC address, not the IP address. Record the root disk and current mounts from `lsblk`, but do not assume a particular `/dev/vdX` name or change any disk.

For PCI devices, use these commands if `lspci` is installed:

```
lspci -nn
lspci -nnk
```

`-nn` shows numeric vendor and device identifiers as well as names where available; `-k` adds driver information. The Ubuntu lspci manual defines the options. A missing utility means this view is unavailable, not that no PCI devices exist. If the lab has a working setup network, you can inspect the package proposal with `apt show pciutils` and `sudo apt --simulate install pciutils`, then install `pciutils` with `sudo apt install pciutils` after reviewing it. If the network is unavailable, continue with the views already present and record that limitation.

In `lspci -nnk`, **Kernel driver in use** is evidence of a bound driver for the listed PCI function. **Kernel modules** lists candidates, not a promise that each candidate is active. A built-in driver may be in use without appearing in `lsmod`; `lsmod` lists loaded modules, not all code built into the running kernel.

For USB devices, `lsusb`, when available, supplies another bus view. A USB device will not necessarily appear in the PCI listing. Conversely, a virtual NIC can appear behind a virtual transport and have additional objects in the kernel hierarchy. Select the bus relevant to the actual device instead of treating one command as an inventory of everything.

Connect an interface to its kernel and udev records

Substitute the actual original interface name for `INTERFACE` below. `lo` is the local loopback interface and is not the virtual NIC in this exercise. Use the selected name as one shell variable so the commands inspect the same object:

```
IFACE=INTERFACE
ip -details link show dev "$IFACE"
readlink -f "/sys/class/net/$IFACE/device"
udevadm info --query=property --path="/sys/class/net/$IFACE"
```

For example, replace `IFACE=INTERFACE` with `IFACE=enp1s0` only if that is the name actually observed. The link details show the interface's current state and MAC address. The sysfs link points towards its underlying device hierarchy; the resulting path depends on the driver and transport. `udevadm info` reports properties known to udev. An absent property or path is evidence to investigate, not proof that every device on the bus is missing.

If available, inspect the driver associated with the network interface:

```
ethtool -i "$IFACE"
```

`ethtool` may not be installed; record that and use the sysfs and PCI views. On a layered virtual device, the network interface's driver can differ from the driver shown for its parent PCI function. Follow the sysfs path instead of declaring a contradiction from two different layers. The reported driver version or firmware version can be empty or uninformative on virtual devices.

Check the kernel messages for the current boot:

```
sudo journalctl -k -b --no-pager | grep -Ei 'firmware|virtio|network|error|fail'
```

This filtered view is a starting point, not a complete diagnostic. It can return no lines or unrelated warnings. The pipeline's exit status can be nonzero when `grep` finds no matches. Look at the surrounding unfiltered journal for the actual device and time before attributing a line to your interface. Keep the complete relevant messages, including device identifiers, when saving evidence.

Distinguish driver code from device firmware

System firmware, such as UEFI, starts the machine and participates in boot. **Device firmware** can be code or data that a driver requests for a device after the kernel starts. The kernel firmware documentation describes these requests, which can include optional data as well as required firmware.

A missing-firmware message does not automatically mean that the missing file caused the symptom. Establish the exact filename, device, driver and whether the subsequent initialization failed. Some drivers request alternatives or optional files. Conversely, no matching log line does not prove that all firmware dependencies are satisfied.

On Ubuntu, investigate the installed files and package ownership before changing anything:

```
dpkg-query -W linux-firmware
apt-cache policy linux-firmware
```

If the package query reports that the package is absent, record that rather than assuming all devices need it. Package availability and versions depend on enabled sources and updates. For a filename actually reported in the log, inspect the path and ownership **after** identifying the file on the host. For example, with the observed absolute path substituted for `/actual/path/to/file`:

```
ls -l /actual/path/to/file
dpkg-query -S /actual/path/to/file
```

The owner query can identify an installed package providing an existing path; a missing path has no installed owner to report. Do not copy an arbitrary binary from a forum or change module policy just to suppress a message. Check the distribution's package, the hardware/vendor documentation and the running kernel's support for that exact device. Package installation is a change: review its source and proposed transaction, retain console access and plan the required restart if the documented remedy calls for one.

For a boot-critical storage device, missing support can prevent reaching the installed root at all. That is a different recovery context from a missing secondary network interface: preserve the boot console message, use known-good recovery media and avoid changing the only working boot path during this lab.

Lab: add a second virtual NIC and observe its lifecycle

The experiment tests **presentation and enumeration**, not Internet connectivity. It needs no packet exchange and does not require configuring the new interface. The original NIC remains attached as recorded; the second NIC connects only to `blpractice`, the host-local practice network. That network includes the host and any attached guests; it is not an Internet connection and does not protect against a compromised host.

1. Save the initial `ip -brief link` output. On the host run `sudo virsh --connect qemu:///system domiflist bl02` and save its output. Ensure the accepted `bl02` baseline is available and the console works.
2. Shut `bl02` down cleanly. In Virtual Machine Manager, add a **second** network interface using the `blpractice` virtual network. Select a model supported by your installation, such as virtio, if the manager offers it. Do not edit the original NIC, system disk, firmware or boot order. Apply the change while the guest is off.
3. On the host inspect the persistent guest definition and record both MAC addresses, models and network sources:

```
sudo virsh --connect qemu:///system dumpxml --inactive bl02
```

Find the two `<interface>` elements; identify the added one by its `blpractice` source and MAC address. Domain XML can contain host paths and other local details, so redact before sharing. Do not run both cloned guests at once merely to compare them: their guest identities were copied.

4. Start `bl02`, log in through the graphical console and inspect:

```
ip -brief link
ip -brief address
lspci -nnk
sudo journalctl -k -b --no-pager | grep -Ei 'virtio|network|firmware|fail'
```

Omit `lspci` if the utility is unavailable and note the missing view. Match the new guest interface to the recorded MAC address. Names and PCI addresses can vary. The interface can be administratively down and have no IP address; either is compatible with successful enumeration. If it has an address, record it, but do not infer that it has an Internet route.

5. Substitute the **new** interface name in the `IFACE` commands from the earlier section. Record its sysfs path, udev properties and, if available, `ethtool -i` driver. Compare the two interfaces by MAC address rather than by guessed names. If the new MAC is not visible, use the diagnosis table before making changes.
6. Shut down the guest, remove only the added NIC in Virtual Machine Manager, and apply the change. Inspect the inactive XML again to verify the original NIC and its original network source remain. Restart and confirm the original console login and NIC, and that the added MAC no longer appears in `ip -brief link`. Do not remove a disk or delete a network definition during cleanup.

The success record has a before/after MAC list, the second interface's observed driver or a stated observation limit, and a cleanup check. Rebooting after removal tests the persistent guest definition. If the original guest configuration has been altered unexpectedly, stop and reconcile it with the accepted baseline before proceeding.

A worked diagnosis: the new NIC is not in `ip link`

Suppose the manager shows an added NIC, but the guest lists only the original one and loopback. First compare the host's **inactive** definition with the configuration you intended and check that the VM was restarted after applying the change. A pending persistent change is not evidence that the currently running guest has received it; the live definition can be inspected with `sudo virsh --connect qemu:///system dumpxml bl02` while running.

If the NIC is present in the running definition, inspect `lspci -nnk`, the kernel journal and sysfs. A new PCI function without a driver suggests a different next investigation from no new bus object at all. If there is a bus object and a driver but no expected interface, inspect probe messages and the actual device hierarchy. If the interface appears with a different name, the recorded MAC resolves the identity without changing network settings.

A driver binding is still not an end-to-end network test. A bound NIC with no IP address may simply lack configuration or a reachable address server. Configuration, routing and network services have their own evidence and are developed later.

Common observations and next questions

Observation	What it supports	Next question
Device absent from guest bus inventory	No matching object in that particular view	Is it present in the running VM definition, and is this the correct bus?
PCI device listed without driver in use	Enumeration occurred; binding is not shown	Does the running kernel support the numeric ID, and what does the journal report?
Firmware request failed	A driver requested a file	Was the file required, and did initialization of this device fail?
Interface exists but no IP address	The network interface was registered	Is it up and configured, and is the intended network available?
Device appears only after restart	Persistent configuration differed from running state	Was the intended hardware change applied to the live guest?
Device works but a user cannot open it	Kernel registration alone is insufficient	Which node, permissions and process identity apply to this device?

Avoid testing by unbinding a root-disk driver, removing a production NIC, or repeatedly loading random modules. Record the observed boundary, form a specific hypothesis and choose a reversible test that does not cut off the recovery path.

Acceptance and recall

Save the host definition evidence and guest observations outside the VM, redacting MAC addresses, serial numbers or host paths before sharing as appropriate. The original NIC should be restored and usable, the added NIC removed, and the original root disk still mounted. Note any unresolved driver or firmware question rather than marking it fixed from a package name alone.

- A VM exposes its configured virtual hardware; host hardware inventory is a different view.
- Enumeration, driver binding, device initialization and working configuration are separate claims.
- `/sys`, udev, bus listings and the journal show different evidence about the same device.
- A firmware request must be tied to an actual device and failure before choosing a remedy.
- Use a MAC address and before/after observations to identify the experimental NIC and verify cleanup.

Provisioning and Recovery

Image-Based and Unattended Installation

A manual installation records decisions in a person's actions. A reproducible installation records them in inputs that can be reviewed, delivered to the right machine and checked against the resulting system. Automation makes repetition easier, but also repeats mistakes quickly: a configuration that selects the wrong disk can destroy the wrong system just as consistently.

This chapter compares a prepared image with an unattended installer. Then you will create a local configuration seed for the Ubuntu Server installer and apply it to a **new disposable** VM, `bl02-auto`. The example intentionally retains the installer's destructive-action confirmation. It automates the installation choices while keeping a human at the disk-write boundary.

You need the verified Ubuntu Server 26.04 amd64 installer ISO used earlier, working console access, a setup NAT network, host space for another 32-GiB virtual disk and a small seed ISO, and the ability to create a VM in Virtual Machine Manager. Use two vCPUs and 4096 MiB RAM, as for `bl02`. Keep `bl01`, `bl02` and their baseline disks untouched. Only the new guest needs to run during this exercise.

Two ways to produce a machine

An **installation image** contains an installer and a source from which an operating system can be assembled onto a target. An **autoinstall configuration** answers installer questions about that target. The installer still creates partitions, installs files and configures the system. The input file is not itself an installed disk.

A **prebuilt machine image**, in contrast, already contains an operating-system filesystem and boot configuration in a disk image. A provisioning system creates an instance from that image and applies instance-specific information such as hostname, credentials and network settings on first boot. The image must match the platform, architecture and boot method; merely copying the installer ISO does not make it a bootable installed disk.

Decision	Installer with configuration	Prebuilt machine image
Base	Installer media and package sources	Prepared filesystem and bootable disk
Target storage	Created during installation	A new disk or copy derived from the image
Host-specific inputs	Installer configuration and, optionally, later first boot	Instance metadata and first-boot configuration
Evidence	Installer result, logs and installed state	Image provenance, initialization result and instance state

Decision	Installer with configuration	Prebuilt machine image
Recovery concern	Reproduce inputs and preserve data before reinstalling	Keep a trusted base and separate persistent data/identity

An image can speed up deployment, but image age matters: an old image still requires a patch and acceptance process. An installation that uses a live repository may resolve different package versions on different dates even when its configuration is unchanged. Reproducibility requires recording media, configuration, sources and results, not just retaining one YAML file.

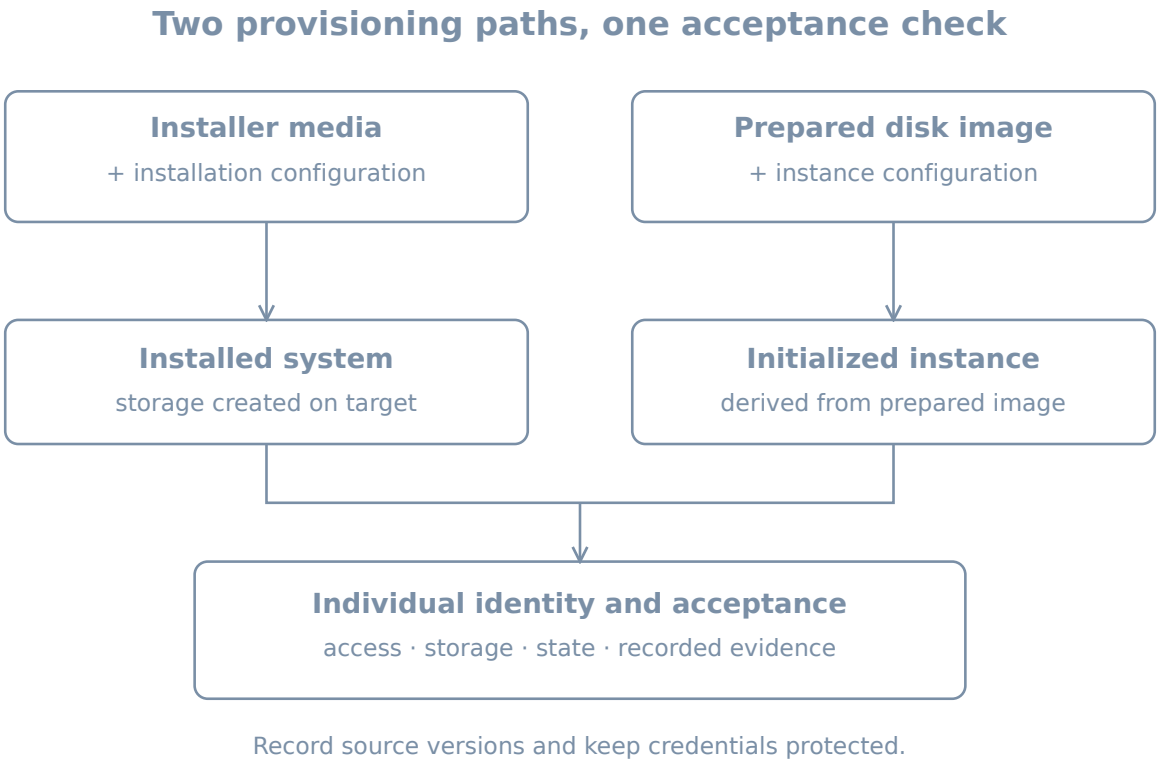


Figure 1: Provisioning starts with a common input and ends with an individually verified host.

Identity is not the same as a disk copy

The recovery clone in the first lab deliberately copied an existing machine, including its guest identity and credentials. It was kept powered off and used one copy at a time. A machine intended to run as an independent instance needs its own hostname, machine identity, credentials, network attachment and, when used, SSH host keys. A distinct VM name or MAC address does not by itself replace everything stored inside the guest disk.

Cloud images commonly use **cloud-init**, which reads an instance's configuration from a datasource and runs initialization stages. Its first-boot explanation describes how cached state and instance IDs

influence whether per-instance configuration runs. Copying an initialized disk and simply changing a name is not a complete image-preparation procedure. Do not run `cloud-init clean` on the existing lab hosts as a shortcut; it changes state and must be part of a deliberate image lifecycle.

An installer has its own context. Cloud-init can deliver data **to the live installer**, while Subiquity interprets the nested `autoinstall` configuration; cloud-init on the eventual installed system has its own first-boot role. Canonical's interaction guide explains the distinction. A successful installer-data read alone is not proof that every target-side initialization step succeeded.

Prepare a local installer seed

A **seed** is a small source of provisioning data. This lab uses a local NoCloud seed ISO, attached separately from the verified server installer ISO. It avoids depending on a configuration web server. The seed contains a password hash, so keep its files and disk image private; a hash is still sensitive because an attacker can attempt offline guesses.

On the **host**, in a private shell session, create a working directory and check for the required utilities:

```
umask 077
mkdir -p "$HOME/bl02-auto-seed"
cd "$HOME/bl02-auto-seed"
command -v cloud-localds
command -v openssl
```

If `cloud-localds` is unavailable on an Ubuntu host, inspect `apt show cloud-image-utils` and `sudo apt --simulate install cloud-image-utils`, then install it with `sudo apt install cloud-image-utils` after reviewing the proposal. On another host distribution use its package documentation or prepare the seed on a trusted Linux machine. `openssl` must also be available. Do not put the seed on a shared web server.

Create a **unique lab password** and keep it in your password manager. In the terminal run:

```
openssl passwd -6
```

Enter the password at the prompts; this generates a salted SHA-512 password hash without placing the plaintext on the command line. The terminal will display the hash. Do not use a production password, paste the hash into a public issue, or record it in shell history. Keep console scrollback private as well.

Create `user-data` in your editor with the following content. Replace the entire `REPLACE_WITH_HASH` string with the generated hash, keeping the single quotes around it. Do not include the plaintext password.

```
#cloud-config
autoinstall:
  version: 1
  identity:
```

```
    hostname: bl02-auto
    username: learner
    password: 'REPLACE_WITH_HASH'
  storage:
    layout:
      name: direct
  ssh:
    install-server: false
  shutdown: poweroff
```

The first line marks cloud-config data. `autoinstall` contains the installer's instructions; indentation is significant. The `identity` section establishes the initial account. The direct storage layout installs to the selected disk without the guided LVM layer used on `bl02`. It **replaces data on that disk**. The SSH server is not requested by this configuration; verify actual listeners later. The completion action asks the installer to power off so the media can be detached before the next boot. These settings are described in the autoinstall reference.

Create the required metadata file and the seed image:

```
printf 'instance-id: bl02-auto-installer-01\n' > meta-data
cloud-localds seed.iso user-data meta-data
ls -l user-data meta-data seed.iso
```

The `instance-id` labels this installer instance's seed. It is not a substitute for the installed system's machine ID or a universal identity-reset mechanism. If repeating the exercise on a new target, use a distinct seed instance ID and hostname. The NoCloud datasource reference describes the local data source, and the installer quick start demonstrates `cloud-localds` for installer data.

Run these commands in the private directory. `umask` affects files created in the current shell; if the directory or files existed already, inspect their permissions explicitly. Any user who can read `seed.iso` can extract its configuration. Restrict directory traversal and grant the libvirt/QEMU process only the access needed for this lab; the exact file permissions and access-control rules depend on the host. If the VM manager cannot open the file, use a suitable private libvirt storage location or its documented access controls rather than making the directory world-readable. Record the seed path, but not its contents or hash in shared notes.

Review before allowing writes

Check the exact `user-data` file in your editor for the header, indentation, hostname, username, quoted hash, direct layout and poweroff. Check there is no remaining placeholder. If you have a matching Subiquity source checkout and its validator dependencies, Canonical provides pre-validation instructions. A schema check cannot prove that the intended disk is attached or that runtime installation will succeed. Never execute an unfamiliar installer configuration or its commands without reviewing them.

This minimal configuration omits explicit network, mirror, locale and keyboard settings. Installer defaults and network access can affect the result. Record the installer image/build, network and

source observations; do not claim byte-identical reproduction across different runs. The chapter's scope is an introductory seed, not a fleet policy.

Install on a new, single-disk guest

In Virtual Machine Manager create `bl02-auto` with UEFI, the resource budget above, one **new 32-GiB writable system disk**, the verified server installer ISO and the setup NAT network. Choose to review hardware before installation. Add `seed.iso` as a second read-only CD-ROM. Both optical images are sources; neither is the writable target disk. If your manager cannot add a second optical device, follow Canonical's local-seed workflow using a supported virtual-machine interface and keep the same disk isolation conditions.

Before booting, inspect the selected VM on the host:

```
sudo virsh --connect qemu:///system domblklist bl02-auto --details
sudo virsh --connect qemu:///system domiflist bl02-auto
```

Confirm exactly one writable guest system disk, its new host path, both intended optical sources and the expected NIC source. Verify that no disk from `bl01`, `bl02`, a baseline or your host filesystem is attached. A new VM name is not evidence of a new disk. If the seed cannot be read because of host permissions, fix access to the intended file and recheck the definition; do not select a different disk as a workaround.

Boot the **server installer ISO** through the graphical console, leaving the seed attached. Do **not** add the `autoinstall` kernel parameter in this exercise. With the local configuration detected, the installer should ask for confirmation before modifying a disk; this behavior is described in the local-volume quick start. The parameter suppresses that confirmation, so it is deliberately omitted.

At the confirmation, stop and reconcile the actual target with the host-side single-disk record. The direct layout's default selects the largest eligible disk. With only one writable system disk this is bounded, but "largest" is unsafe as a general selection rule on a multi-disk machine. If the target is ambiguous, cancel and investigate; validation of the YAML does not validate the target. After confirming the disposable target, allow installation to continue and observe its outcome.

If the usual full interactive installation screens appear instead of an autoinstall confirmation, the installer may not have received the seed. Stop and investigate the seed attachment and format before making storage changes. If installation fails after confirmation, the disk may already be partially written. Preserve the relevant error and treat the target as changed.

Detach sources and verify the result

After the installer reports completion and powers off, remove both the installer ISO and seed ISO from this guest's virtual optical devices, leaving its system disk and NIC in place. Inspect `domblklist` again, then boot the installed disk. Log in at the console as `learner` with the lab password.

Collect the following evidence inside `bl02-auto`:

```
hostnamectl
cat /etc/os-release
uname -r
id
sudo -l
sudo id -u
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
systemctl --failed --no-pager
sudo ss -lntup
```

The hostname should be `bl02-auto`; administrative access should work according to the intended policy; root should be on the newly installed disk. Compare the actual partition and filesystem layout with the direct-layout intent and with `bl02`'s recorded LVM layout. Names and sizes are observations, not fixed strings to copy from this book. Investigate unexpected failures or listeners before accepting the host.

If `cloud-init` is installed on the target, inspect its own status and logs without assuming that they are the installer's logs:

```
cloud-init status --long
sudo journalctl -b -u cloud-init-local.service -u cloud-init.service -u cloud-config.service
```

Service names or available output can differ; record unavailable views. A status indicating completion does not prove all application requirements. The installer's own records are typically under `/var/log/installer/` on an installed Ubuntu system. Inspect filenames before reading them; generated installation records can contain credential hashes, network details or other sensitive inputs. Restrict access and redact before sharing.

Perform the first-boot update and reboot checks established earlier when this guest is going to be kept. Keep the acceptance record outside the VM. Detaching the seed prevents accidental reuse at the next boot; it does not securely erase the seed or any installer logs copied into the guest.

Diagnose by stage

Observation	Check	Recovery direction
Installer starts interactively	Both optical sources, seed contents, `#cloud-config` header and `autoinstall` indentation	Correct the seed, rebuild it and boot the disposable target again
Cannot attach seed	Host file path and QEMU access permissions	Grant appropriate access to the intended seed; do not expose it publicly
Configuration rejected	Installer error and schema validation	Fix the reported field; validate again and retain the failure record

Observation	Check	Recovery direction
No safe target is evident	Host disk list and installer's selected disk	Stop before confirmation; detach unrelated writable disks
Installer fails after disk confirmation	Installer console and logs	Treat target as partially modified; recreate only the verified disposable target
Login fails on installed guest	Selected guest, username, keyboard and password-hash substitution	Preserve evidence; use the lab recovery/reinstall plan rather than copying a shared password
Guest re-enters installer	Optical attachments and boot source	Shut down, detach installation media and boot the installed disk

When a seed was modified after `cloud-localds` ran, the already-created ISO still contains the earlier data. Rebuild it from the corrected files before booting again. If access to installer logs is needed during a failed run, use the installer console's reported log location and capture only the relevant, redacted evidence.

Practice and cleanup

Create an acceptance record containing the installer ISO identification, VM/disk and seed paths, seed instance ID, selected storage intent, observed root source, console login and `sudo` result, failed-unit/listener review, and media-detachment check. Keep the password and hash out of the record.

As a thought experiment, plan a second instance: list which values must differ and which can remain shared. A real second run needs a new disk, hostname, instance ID and unique credentials; it must also have separate guest identity and an explicit network role. Compare installed results as observations rather than assuming a matching input implies an identical system.

To finish, power off the guest if it will not be used. Retain or retire its dedicated disk only after checking the exact VM and host path. Delete the private seed files and ISO when they are no longer needed, according to your host's data-handling practice; ordinary deletion is not a guarantee of secure erasure. Do not remove the verified installer ISO or any other guest's disk. If preserving the guest, keep its account recovery method and separately protected record.

Recall card

- An installer plus configuration creates a target; a prebuilt machine image already contains a system.
- A provisioning seed is both a configuration input and a potential secret-bearing artifact.
- Installer data and installed-system first-boot data operate in different contexts.
- Review the target disk at the write boundary; a schema-valid file can still select the wrong device.
- Accept a host only after confirming its identity, access, storage and operating state.

Reinstallation and Host Configuration Recovery

A server's disk fails. Installing Linux again gets you an operating system, but does it get you the configuration, data, credentials and dependencies that made the old host useful? Recovery succeeds only when the intended behavior is restored and checked from the appropriate point of view.

This chapter rebuilds a small **demonstration workload** on a separate replacement VM. It exports two selected files from `bl02`, preserves a copy on the host, installs a fresh `bl02-recovery`, and verifies the restored files on that guest. No real application is installed: the files make the selection and verification process observable before later service and backup chapters add more complex consistency requirements.

You need the working `bl02` from earlier chapters, its graphical console and first-boot baseline, the verified Ubuntu Server installer ISO, and host space for a new 32-GiB guest disk and a small handoff file. The host must support libguestfs tools and the existing libvirt system connection. Both guests will be shut down while their disks are accessed from the host. Do not attach either guest's disk to the other, overwrite `bl02`, or use a personal workstation as the target.

The host-side copy in this exercise is a **handoff artifact** outside the guest. It still shares the physical host's failure domain and is not a complete disaster-recovery backup. The full backup and independent-restore design comes later.

Decide what recovery means

A repair attempts to return the existing installation to working order. A **reinstallation** creates an operating system again on a target disk. **Reprovisioning** builds a replacement from specified inputs and configuration. A restore brings selected prior state back from a saved copy. An incident can require more than one of these actions.

Before changing a failed machine, ask what must survive and what should be regenerated. A replacement guest used alongside the original has a distinct identity; a replacement that takes over the original's role needs an explicit cutover that prevents duplicate identities and conflicting access. This exercise stops before cutover and uses a new hostname.

Category	Example	Decision in this exercise
Reproducible base	Installer, packages and updates	Install a fresh base and verify it
Host-specific configuration	An explicitly chosen demonstration config	Copy, review and restore this one file
Workload data	A small demonstration data file	Restore and compare its content

Category	Example	Decision in this exercise
Machine identity	Hostname and `/etc/machine-id`	Keep the replacement's newly created identity
Secrets and trust	Passwords, SSH host keys, service certificates	Do not include them in this lab bundle
Hardware-dependent state	Filesystem UUIDs, network interface names, firmware variables	Observe the replacement and configure against its own values
External dependencies	DNS, upstream services, data stores	Record dependencies; this demonstration has none

A machine ID is confidential, should normally be unique, and is not a file to copy casually from one running host to another. The Ubuntu machine-id manual explains its role. SSH host keys and certificates also have lifecycles of their own: preserving an identity may be required for some migrations, while unplanned duplication or replacement can break trust. Treat that as a deliberate service-specific decision.

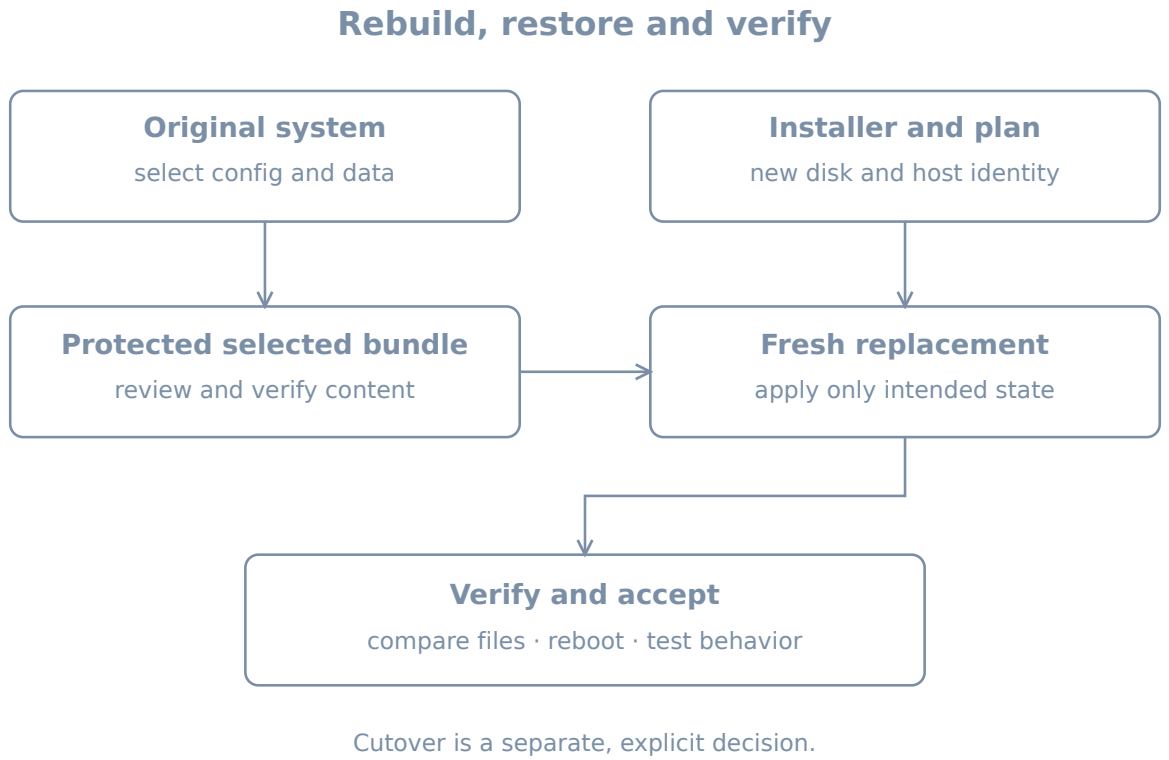


Figure 1: A replacement is accepted after selected state is restored and verified.

Define a small recovery contract

For this lab, success means:

1. The original `bl02` remains intact and shut down during offline export.
2. A fresh guest called `bl02-recovery` boots from its own disk and has working console and administrator access.
3. The selected configuration and data files are restored from the host-side bundle.
4. Both restored files match their source copies byte for byte and remain present after reboot.
5. The replacement retains its own hostname and disk identities; no automatic network cutover occurs.

The two demonstration files are not a real application's configuration format. They represent an operator-controlled setting and a small persistent data record. Their values contain no passwords or personal data.

Identify what would be needed for a real host

Before reinstalling a real service, inventory its packages and versions, configuration ownership, service units, data locations, mount sources, users and numeric ownership, certificates, keys, network addresses, external dependencies and restore order. Also record which software or configuration is generated and which authoritative system owns it.

A blanket copy of `/etc` can overwrite fresh network settings, account databases, filesystem references or machine identity with values from different hardware. Old `/etc/fstab` UUIDs can point nowhere or to the wrong device; old interface names can refer to a different adapter. A package list does not include application data, and a data directory copied while a database is writing may not be a consistent backup. Select and validate each component for its owner and recovery method.

Prepare the demonstration on `bl02`

Start the original guest and log in through its console. Confirm that this is the intended system:

```
hostnamectl
findmnt --target / --output TARGET,SOURCE,FSTYPE
sudo id -u
```

The hostname should be `bl02`, root should match its recorded installation, and the administrative check should return `0`. If any value is unexpected, stop and reconcile the selected VM and disk with your lab record.

Create the two **lab-only** files. Stop if either directory already exists: it may contain a previous experiment that you should preserve rather than overwrite.

```
sudo test ! -e /etc/bl-recovery-demo
sudo test ! -e /var/lib/bl-recovery-demo
```

Both commands must finish successfully with no output. If either fails, do not run the following creation commands; inspect and choose a fresh lab name instead. Otherwise:

```
sudo install -d -m 0755 /etc/bl-recovery-demo /var/lib/bl-recovery-demo
printf 'mode=observe\n' | sudo tee /etc/bl-recovery-demo/settings.conf
printf 'inventory-record=alpha\n' | sudo tee /var/lib/bl-recovery-demo/record.txt
```

Record the paths and their purpose in your change notes. In this demonstration both files are nonsecret and readable by the local learner. Real configuration and workload data can have stricter permissions and may require application-specific backup tools.

Assemble and inspect a selected bundle

As `learner` inside `bl02`, create a new private staging directory. This command fails if the directory already exists; choose a new name rather than mixing old and new contents.

```
umask 077
mkdir "$HOME/bl-recovery-export"
mkdir "$HOME/bl-recovery-export/config" "$HOME/bl-recovery-export/data"
cp /etc/bl-recovery-demo/settings.conf "$HOME/bl-recovery-export/config/"
cp /var/lib/bl-recovery-demo/record.txt "$HOME/bl-recovery-export/data/"
cd "$HOME/bl-recovery-export"
sha256sum config/settings.conf data/record.txt > SHA256SUMS
sha256sum -c SHA256SUMS
```

Both lines should report `OK`. The checksum records the bytes selected at export; it does not establish provenance or prove that a running application was consistent. This tiny demonstration has no writer and no application transaction. For a real workload, use the application's quiescing or backup method and test its restore.

Create an archive in the home directory, then inspect its members and hash:

```
cd "$HOME"
tar -cf bl-recovery-demo.tar bl-recovery-export
tar -tf bl-recovery-demo.tar
sha256sum bl-recovery-demo.tar
```

The archive should contain only the staging directory, its two subdirectories, the two files and `SHA256SUMS`. Record the archive hash in your private lab notes outside the VM. A checksum comparison detects unintended changes when the expected hash is protected separately; it is not a signature or a substitute for trusted handling.

Shut the guest down cleanly:

```
sudo poweroff
```

On the host, confirm that the **system** libvirt domain is actually shut off:

```
sudo virsh --connect qemu:///system domstate bl02
sudo virsh --connect qemu:///system domblklist bl02 --details
```

Proceed only when it reports `shut off` and the disk matches your record. Paused or saved-memory state is not a clean shutdown. Leave it off until the offline read is complete.

Export the bundle to the host

The following workflow uses `guestfish`, a libguestfs tool that can inspect a shut-down guest filesystem. On an Ubuntu host, its package is `guestfish`; inspect the proposed installation with `apt show guestfish` and `sudo apt --simulate install guestfish` before using `sudo apt install guestfish` if needed. Other distributions package the tooling differently. The guestfish manual documents `--ro`, automatic inspection (`-i`) and the libvirt connection option. If automatic inspection cannot open the installation, stop and use a separately documented export method; do not guess block-device paths or write to the original disk.

On the **host**, create a new private handoff directory. This intentionally fails if the name already exists, so old recovery material is not silently replaced:

```
sudo mkdir -m 0700 /var/tmp/bl02-recovery-handoff
sudo guestfish --ro -i -c qemu:///system -d bl02 \
  download /home/learner/bl-recovery-demo.tar \
  /var/tmp/bl02-recovery-handoff/bl-recovery-demo.tar
sudo sha256sum /var/tmp/bl02-recovery-handoff/bl-recovery-demo.tar
```

Compare the host hash with the separately recorded guest hash. Both must match before proceeding. The absolute source path assumes the `learner` home created in the installation lab; if your actual home differs, use the observed path. The host directory belongs to root and is a staging location, not encrypted storage. Real archives may contain sensitive data and need protected access, encrypted storage and a separate failure domain.

The host copy and the original guest disk are distinct files but reside on the same host in this lab. Loss of the host or its storage can destroy both. If a truly independent backup is required, transfer the verified artifact to separately managed storage and demonstrate retrieval; this exercise does not claim to have done so.

Install a replacement without touching the original

Create a new VM named `bl02-recovery` with a new 32-GiB disk, UEFI, two vCPUs, 4096 MiB RAM and the setup NAT network. Attach the verified Ubuntu Server installer ISO. Use the guided-install workflow from earlier, with hostname `bl02-recovery` and a new unique `learner` password. It is acceptable for this fresh installation to have a different storage layout from `bl02` because the two demonstration files will be restored by path, not by copying a filesystem image.

Before accepting disk writes in the installer, inspect the **host** definition:

```
sudo virsh --connect qemu:///system domblklist bl02-recovery --details
```

Confirm the system disk path differs from `bl02` and all baseline disks, and that no old writable disk is attached. After installation, detach the ISO and boot from the new disk. Verify hostname, root

mount, console login and `sudo id -u`. Update the new system using the first-boot procedure as appropriate; record its installed and running versions. Shut it down cleanly before the host writes the handoff file into it.

Deliver and inspect the bundle on the replacement

On the host confirm both guests are off and identify the replacement disk:

```
sudo virsh --connect qemu:///system domstate bl02
sudo virsh --connect qemu:///system domstate bl02-recovery
sudo virsh --connect qemu:///system domblklist bl02-recovery --details
```

Both states should be `shut off`, and the destination disk should be the new replacement disk. Do not proceed if a domain is running or if another tool is editing either disk. The virt-copy-in manual warns that writing into a live guest disk can corrupt it.

Copy the archive into the replacement's existing `/var/tmp` directory:

```
sudo virt-copy-in -c qemu:///system -d bl02-recovery \
  /var/tmp/bl02-recovery-handoff/bl-recovery-demo.tar /var/tmp
```

`virt-copy-in` is also supplied by the Ubuntu `guestfish` package. This is a write to the **replacement** disk; `-d bl02-recovery` is the destination identity, not the source. If the tool cannot inspect the target, stop rather than trying other domains. Leave `bl02` shut down, start `bl02-recovery`, and log in through its console.

Compare the replacement copy against the recorded archive hash:

```
sudo sha256sum /var/tmp/bl-recovery-demo.tar
sudo tar -tf /var/tmp/bl-recovery-demo.tar
```

Require the expected hash and the exact member list from the source before extracting. Only extract the archive you created and controlled in this lab. Archive extraction from untrusted sources requires additional path, link and provenance checks; do not apply an unfamiliar archive to system directories.

Create a fresh private restore workspace, extract there, then verify the contents:

```
umask 077
mkdir "$HOME/bl-recovery-restore"
sudo install -m 0600 -o "$(id -u)" -g "$(id -g)" \
  /var/tmp/bl-recovery-demo.tar "$HOME/bl-recovery-restore/bundle.tar"
tar -xf "$HOME/bl-recovery-restore/bundle.tar" -C "$HOME/bl-recovery-restore"
cd "$HOME/bl-recovery-restore/bl-recovery-export"
sha256sum -c SHA256SUMS
```

Both checks must say `OK`. A failure is a stop condition: preserve the failing copy and compare the source and host hashes before retrying. The archive and checksums can be changed together by an

attacker, so the independently recorded archive hash and trusted transfer still matter. The `install` step makes a private working copy owned by the logged-in user, so extraction does not require root. Ownership and access controls must be explicitly checked for real data.

Apply selected files, then prove the result

First ensure the destination paths do not already exist:

```
sudo test ! -e /etc/bl-recovery-demo
sudo test ! -e /var/lib/bl-recovery-demo
```

Only if both checks succeed, create the directories and install the reviewed files from the restore workspace:

```
sudo install -d -m 0755 /etc/bl-recovery-demo /var/lib/bl-recovery-demo
sudo install -m 0644 config/settings.conf /etc/bl-recovery-demo/settings.conf
sudo install -m 0644 data/record.txt /var/lib/bl-recovery-demo/record.txt
cmp config/settings.conf /etc/bl-recovery-demo/settings.conf
cmp data/record.txt /var/lib/bl-recovery-demo/record.txt
```

`cmp` prints nothing and exits successfully when each pair of files matches. The explicit `install` commands establish the destination mode; they do not recreate arbitrary ownership, ACLs, extended attributes or application state. Those are deliberate limitations of this demonstration, not a complete backup method.

Reboot `bl02-recovery`, log in again, and repeat the comparison from the restore workspace:

```
cd "$HOME/bl-recovery-restore/bl-recovery-export"
cmp config/settings.conf /etc/bl-recovery-demo/settings.conf
cmp data/record.txt /var/lib/bl-recovery-demo/record.txt
hostnamectl
findmnt --target / --output TARGET,SOURCE,FSTYPE
sudo id -u
```

Success is two silent, successful comparisons, the replacement hostname, the expected root filesystem and working administrative access. Check exit status for each `cmp` before running the next command if it is unclear; a later successful command does not erase an earlier failure. The payload's presence alone does not prove a real service consumes it. For an actual workload, add the service's own configuration validation, startup, client request, data integrity and access-control tests.

Worked diagnosis: restored configuration points to an old device

Suppose a recovered config refers to the old guest's filesystem UUID, but the replacement has a freshly created filesystem. The file can match the saved copy perfectly and still be wrong for the new machine.

1. Keep the old file in the restore workspace as evidence; do not apply it blindly to `/etc/fstab`.
2. Identify the intended role of the mount and use `lsblk -o NAME,FSTYPE,UUID,MOUNTPOINTS` and `findmnt` to observe the replacement's actual storage.
3. Determine whether the required data filesystem was restored or needs to be attached separately. A new root UUID alone is not proof that the old data exists.
4. Plan a configuration change using the replacement's verified object and a recovery route. Test it with the appropriate mount and service checks before accepting the change.

This separates **faithful file recovery** from **correct behavior on new hardware**. A copied UUID, interface name, IP address or certificate may need adaptation, reissuance or an explicit cutover depending on the service.

Record, reset and retain

Save an acceptance record outside the guests: original and replacement VM/disk identities, the selected file paths, the archive hash, extraction and `cmp` results, reboot check, and any unresolved differences. Keep machine IDs, passwords, host keys and sensitive configuration out of shared notes.

Leave `bl02` powered off until the offline-copy operations have finished. It can then be started for later work. The replacement is a separate teaching guest; shut it down when finished. Retain the host bundle only as long as needed and remove the temporary guest archive and workspaces using the exact recorded paths when they are no longer needed. Ordinary file deletion is not assured secure erasure. Do not delete either VM disk or any baseline as part of this cleanup.

The original's two demonstration files can remain as documented lab state, or be removed later after verifying their exact paths and that no later experiment depends on them. A real host retirement additionally needs service cutover, data retention, credential revocation and an independently verified recovery plan.

Recall card

- Reinstallation creates a base; restoring selected configuration and data completes only part of a service recovery.
- Preserve identity, regenerate it or migrate it only by an explicit service-specific plan.
- Work on shut-down guest disks for offline writes and verify the selected source and destination.
- Checksums compare bytes; they do not establish trusted provenance or application consistency.
- Prove recovery after reboot and test the behavior a user or dependent service actually needs.

Boot Shutdown and Recovery

Firmware and Boot Selection

Firmware, UEFI Variables, GPT and the EFI System Partition

A VM powers on and reports that no bootable device is available. The operating system may still be intact on disk: the firmware first needs to find and execute a suitable boot program. To diagnose this boundary, distinguish the firmware's boot selection, the disk's partition map, the EFI System Partition and the files stored on it.

This chapter follows one successful UEFI boot on the existing `bl02` guest. Its commands inspect the running system and the VM definition; they do not change firmware variables, partition tables or boot files. You need the console, the first-boot record and the known recovery baseline established earlier. Keep `bl01` and the other guests unchanged. Have the original verified installer ISO available as recovery media, but do not boot it or attach extra writable disks for this exercise.

The examples use an Ubuntu Server amd64 VM with UEFI and a GPT disk. Physical systems and other architectures can expose different firmware interfaces, device names and boot paths. Record what your own machine reports instead of substituting example identifiers.

The first handoff

When a machine starts, firmware initializes enough of the platform to select something to execute. In a VM, the hypervisor supplies virtual firmware, virtual devices and persistent firmware-variable storage. The UEFI boot manager can use configured boot options to locate an EFI application on a device. On this lab guest, an installed Ubuntu boot path normally starts from an EFI executable on disk, then continues through a bootloader and Linux startup.

A simplified sequence is:

```
power on → UEFI boot manager → EFI executable → bootloader → kernel
```

This is a map, not proof that every machine uses exactly the same executables. Booting from a network or removable medium introduces other paths. Linux also supports other firmware and boot arrangements; GPT alone does not prove the current boot used UEFI, and a UEFI interface does not prove that a particular disk uses GPT.

The UEFI boot-manager specification describes firmware boot options and their ordering. The later chapters follow the bootloader, kernel and early userspace; here the boundary ends at identifying the EFI application selected by firmware.

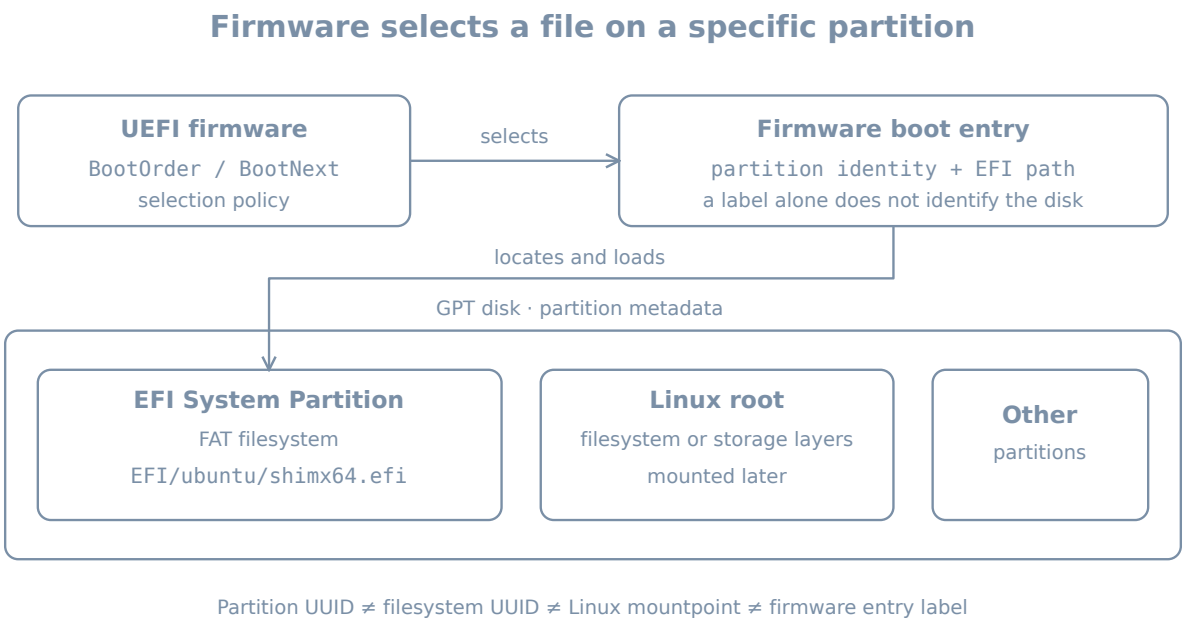


Figure 1: UEFI selects an EFI file through a boot option and a partition.

Establish firmware and guest identity

Open the `bl02` graphical console and log in. Check the selected VM in the manager as well as the guest hostname:

```
hostnamectl
cat /etc/os-release
uname -r
if test -d /sys/firmware/efi; then
  printf 'This running system exposes the EFI firmware interface\n'
else
  printf 'EFI firmware interface is not available in this boot\n'
fi
```

On this UEFI lab guest, expect the interface to be present. If it is absent, check that you are in the intended installed guest rather than a different VM or a live installer, then inspect the guest's firmware settings on the host. Do not switch firmware modes on the existing VM simply to force a match; the installed boot path and stored firmware variables depend on that choice.

The directory is evidence about the **current boot environment**. It does not by itself identify the exact firmware entry or prove that the next boot will use the same path.

On the host, inspect the configured virtual machine and its disks:

```
sudo virsh --connect qemu:///system domblklist bl02 --details
sudo virsh --connect qemu:///system dumpxml bl02
```

Find the system disk and the firmware loader and NVRAM-related settings in the XML, if present. The domain definition describes the configured virtual hardware; it is not a transcript of the firmware's last decision. A live and persistent definition can differ after hardware changes. If necessary, `sudo virsh --connect qemu:///system dumpxml --inactive bl02` shows the persistent definition. Protect the XML before sharing: it can contain host paths, MAC addresses and other local details.

UEFI variables for a guest may live in a separate virtual NVRAM file, while its ESP is inside the virtual disk. A VM disk copy and a firmware-variable copy therefore need to be considered separately in a recovery plan. Verify the actual XML and storage for your VM; do not assume two clones have the same or different NVRAM files from their names alone.

Read the firmware's boot options

If `efibootmgr` is available in the guest, request a verbose listing:

```
sudo efibootmgr -v
```

This form reads the boot variables; it does not alter boot order. If the tool is absent, record that and continue with disk inspection. On the setup network you may review `apt show efibootmgr` and `sudo apt --simulate install efibootmgr`, then install it with `sudo apt install efibootmgr` after reviewing the transaction. If access to EFI variables is unavailable, preserve the error and check the current firmware mode and permissions. An installed tool cannot manufacture firmware services for a non-UEFI boot.

A deliberately shortened, illustrative result might look like:

```
BootCurrent: 0003
BootOrder: 0003,0001
Boot0001* UEFI Virtual DVD ...
Boot0003* ubuntu HD(1,GPT,11111111-2222-3333-4444-555555555555,...)/File(\EFI\ubuntu\shimx64
```

Read this as follows:

Field	Meaning and limit
`Boot0003`	One firmware boot option numbered `0003`; its label is `ubuntu`
`*`	The option is marked active in this display
`BootOrder`	The ordered list of options normally considered by firmware
`BootCurrent`	The option reported for the current boot, when firmware provides it
`HD(...,GPT,...)`	A device-path component referring to a GPT partition; the GUID identifies that partition
`File(...)`	An EFI file path on the selected filesystem, using firmware-style backslashes

`BootNext`, when present, requests an option for the next boot once; it is not the same as the persistent `BootOrder`. The UEFI boot manager can also offer a boot menu or follow platform-specific recovery behavior. A listed option is configuration evidence, not proof that its device and file are currently usable. `BootCurrent` is stronger evidence about the selected entry for this boot, but does not prove what the bootloader did afterwards or guarantee the next attempt.

Labels and numbers are not portable identities. Another VM can have a `Boot0003` named `ubuntu` that points to a different disk. The sample GUID and path above are invented; never copy them into a repair command.

If the current option refers to removable media, network boot or an unexpected entry, reconcile it with the VM manager and console history before assuming the installed disk was selected. Do not use `efibootmgr` mutation options in this chapter. Changing or deleting firmware variables can affect future boots and belongs in a planned recovery exercise.

Where variables are exposed

Linux can expose UEFI variables through `efivarfs`. Inspect whether that filesystem is mounted:

```
findmnt --mountpoint /sys/firmware/efi/efivars \
  --output TARGET,SOURCE,FSTYPE,OPTIONS
```

An empty result means that exact mount was not found in the current mount namespace; it does not by itself explain why. The kernel efivarfs documentation explains that variables can be modified through this interface. Treat it as live firmware state. Do not delete, edit or clear files there to make a diagnostic command work.

Identify the partition map

In the guest, inspect the disk and its partitions:

```
lsblk -o NAME,PATH,TYPE,SIZE,PTTYPE,PARTTYPE,PARTUUID,FSTYPE,UUID,MOUNTPOINTS
```

If the output wraps on a narrow console, widen the display or run separate commands with fewer columns. The lsblk manual explains that this is a view assembled from system information and that relationships can be more complex than a single tree.

Read the columns at their own layers:

- `PTTYPE` identifies the partition-table type on a disk, such as `gpt`.
- `PARTTYPE` describes a GPT partition's **type**: a category, not the identity of this particular partition.
- `PARTUUID` identifies the individual partition in the partition table.
- `FSTYPE` and `UUID` identify a filesystem and its filesystem identifier, when present.
- `MOUNTPOINTS` reports current mount locations; it is not a firmware boot option.

The GPT type GUID for an EFI System Partition is `c12a7328-f81f-11d2-ba4b-00a0c93ec93b`. Several ESPs can share that type; their partition UUIDs should distinguish the individual partitions. Compare identifiers case-insensitively where tools display a different letter case. The GPT specification describes partition types, unique GUIDs and the primary and backup tables. A backup partition table helps detect or recover certain metadata problems; it is not a backup of the files inside the partitions.

A filesystem UUID and a partition UUID can both be called "UUID" in conversation. They are not interchangeable. For the GPT hard-drive device path shown in the example above, compare its partition GUID with `PARTUUID`, not with the FAT filesystem's `UUID`. `lsblk` may show additional layers such as LVM below another partition; do not expect the root filesystem to be a direct partition on every host.

Inspect the EFI System Partition

An **EFI System Partition** (ESP) is a partition intended to hold EFI boot-related files in a firmware-readable filesystem. The guided Ubuntu UEFI installation normally mounts one ESP at `/boot/efi`. That path is Linux's mount location after boot; firmware uses its device path and EFI file path independently of that Linux mountpoint. Canonical's storage guide describes the installer's UEFI boot-device setup.

First ask for an **exact** mount at the expected lab location:

```
findmnt --mountpoint /boot/efi --output TARGET,SOURCE,FSTYPE,OPTIONS
```

If the command produces a row, compare its source with the partition's `PARTUUID`, filesystem type and disk in the `lsblk` output. Do not assume the ESP is mounted just because the `/boot/efi` directory exists. If there is no exact mount, stop before listing files at that path: a directory on root can otherwise look like an empty ESP. Investigate the installed layout and persistent mount intent first:

```
cat /etc/fstab
findmnt --target /boot/efi --output TARGET,SOURCE,FSTYPE
```

The `--target` query identifies the filesystem containing a path and may therefore report `/` when `/boot/efi` is not mounted. An fstab entry represents mount intent, while `findmnt` reports current mounts. An ESP can also be mounted elsewhere on another system. Do not add a guessed fstab entry or format a partition to reproduce the example.

After confirming the exact ESP mount on the intended guest, list its EFI directory:

```
sudo ls -la /boot/efi/EFI
```

If the firmware entry refers to `\EFI\ubuntu\shimx64.efi`, and the matched ESP is the one mounted at `/boot/efi`, the corresponding Linux path in this example is `/boot/efi/EFI/ubuntu/shimx64.efi`. Inspect the actual directory and file named by **your** entry; filenames, capitalization and paths vary. A file with a plausible name is evidence of presence, not proof of signature validity or that it can boot the current system.

Some systems also place a removable-media fallback executable at an architecture-specific path such as `EFI/BOOT/BOOTX64.EFI` for x86-64. Its presence does not prove that the firmware will use it for a particular failure. Secure Boot and signed components are examined later; do not remove or replace EFI files during this inspection.

Lab: build a boot-path evidence table

Collect observations in a private record and reconcile them in this order:

Question	Evidence	Your observation
Which guest and disk?	VM manager, domain disk list and `hostnamectl`	Record name and host disk path
Which firmware context?	VM definition and `/sys/firmware/efi`	UEFI for this running boot, or an explained difference
Which boot option?	`efibootmgr -v`, if available	`BootCurrent`, relevant option and path
Which partition?	GPT device path and `lsblk PARTUUID`	Matching partition on the intended disk
Which mounted filesystem?	Exact `findmnt` result and `lsblk`	ESP source, filesystem type and mountpoint
Which file?	Listed path on the matched ESP	Actual EFI file, or an explained mismatch
What is the recovery route?	Existing baseline, console and installer record	Location and limitations of those resources

For a simple local-disk GPT entry, the device-path partition GUID should match the observed ESP's `PARTUUID`. If it does not, do not declare the boot broken immediately: the boot option may point to another ESP, or you may be examining a different boot entry or mount. Identify the actual objects first. A network or removable-media boot may not offer the same GPT comparison at all.

When a link is unavailable, mark the record **unresolved**. For example, the guest can run with an inaccessible variable interface; record the disk and mount evidence and the missing `efibootmgr` result separately. Do not invent a `BootCurrent` value.

Save the record outside the guest using the established lab evidence procedure. Redact device identifiers, host paths and firmware details before sharing outside your trusted environment. There is no need to publish a complete variable dump.

Worked diagnosis: "the ESP is empty"

Suppose `ls /boot/efi` appears empty even though the system booted successfully. Candidate explanations include an ESP that was not mounted after boot, an ESP mounted at another location, a different boot disk, or genuinely missing files.

1. Confirm the selected guest and current boot context.

2. Run the exact mount query. If it is absent, inspect the containing filesystem and `/etc/fstab` rather than assuming the directory is the ESP.
3. Inspect `lsblk` for candidate ESP partition types and their individual identifiers.
4. If firmware variables are available, compare the actual selected boot option's device path with the candidate partition UUIDs.
5. Once the correct ESP is mounted through a documented recovery procedure, inspect its actual EFI path and record the result.

This is an evidence sequence, not an instruction to mount or repair the ESP now. Do not mount or format a guessed device while the identity is unresolved. A successful past boot does not prove the relevant file still exists; the disk could have changed after firmware loaded it.

For a guest that cannot boot at all, preserve the console message, its VM definition and storage identity. Keep the existing disks untouched, use the verified recovery media and known baseline as the starting point, and determine whether the failure is firmware selection, loader, kernel, early root or later startup. Later chapters develop offline inspection and repair. A powered-off clone is a recovery aid with its own disk and NVRAM assumptions, not an independent backup against host loss.

Practice and recall

On `bl02`, complete the evidence table and explain how the partition GUID in an observed GPT boot path relates to `PARTUUID`, while the filesystem UUID describes a different object. Identify one conclusion that would remain unproven if `efibootmgr` were unavailable. Compare your record with the installation chapter's stored layout; explain any change before accepting the baseline.

- Firmware chooses an EFI boot option before the Linux root filesystem is mounted.
- Boot option number, partition type, partition UUID, filesystem UUID and mountpoint are different identifiers.
- `BootCurrent` describes a reported current-boot selection; `BootOrder` describes configured priority.
- A directory at `/boot/efi` is not evidence that an ESP is mounted there.
- Preserve boot evidence and a recovery route before changing firmware variables or disk metadata.

Bootloader Selection: GRUB and systemd-boot

Firmware has selected an EFI executable. Which operating-system entry does that executable select, and where does its configuration come from? Firmware boot options and bootloader menu entries are separate layers. Changing one does not automatically change the other.

This chapter identifies the loader used by the Ubuntu `bl02` installation, explains the configuration ownership of GRUB and systemd-boot, and changes a GRUB menu timeout on a disposable clone. You need the previous chapter's firmware and ESP observations, VM console access and the verified recovery baseline. The lab assumes the single-disk UEFI `bl02` setup without shared writable disks, a backing chain or TPM state. If yours differs, do not use the clone recipe until its storage and firmware state have their own safe copy procedure.

Two decisions before Linux runs

UEFI selects an EFI executable through a firmware boot option or another firmware boot path. That executable can be a bootloader, an intermediary such as shim, or another EFI application. A bootloader can then present its **own** menu and choose a kernel, initial RAM filesystem and command line. The first decision is made by firmware; the second is made by the selected boot software.

On a typical Ubuntu UEFI installation, firmware may load `shimx64.efi`, which participates in loading GRUB. The actual path depends on installation, architecture and security configuration. On other systems firmware may load systemd-boot directly. The presence of files for both loaders is not enough to establish which one was selected.

Inspect the previously recorded `BootCurrent` and its path with `sudo efibootmgr -v` if the tool and EFI variable interface are available. On systems supporting the relevant interface, `bootctl status` can report additional loader information:

```
sudo efibootmgr -v
bootctl status
```

A missing command, unavailable variable or incomplete `bootctl` report is an observation limit. In particular, `bootctl` provides more detailed selected-loader information when the loader implements its interface; a GRUB system need not report everything a systemd-boot system would. See the bootctl documentation. Compare the firmware entry, mounted ESP and actual console boot behavior; do not install a different loader to make the report match an example.

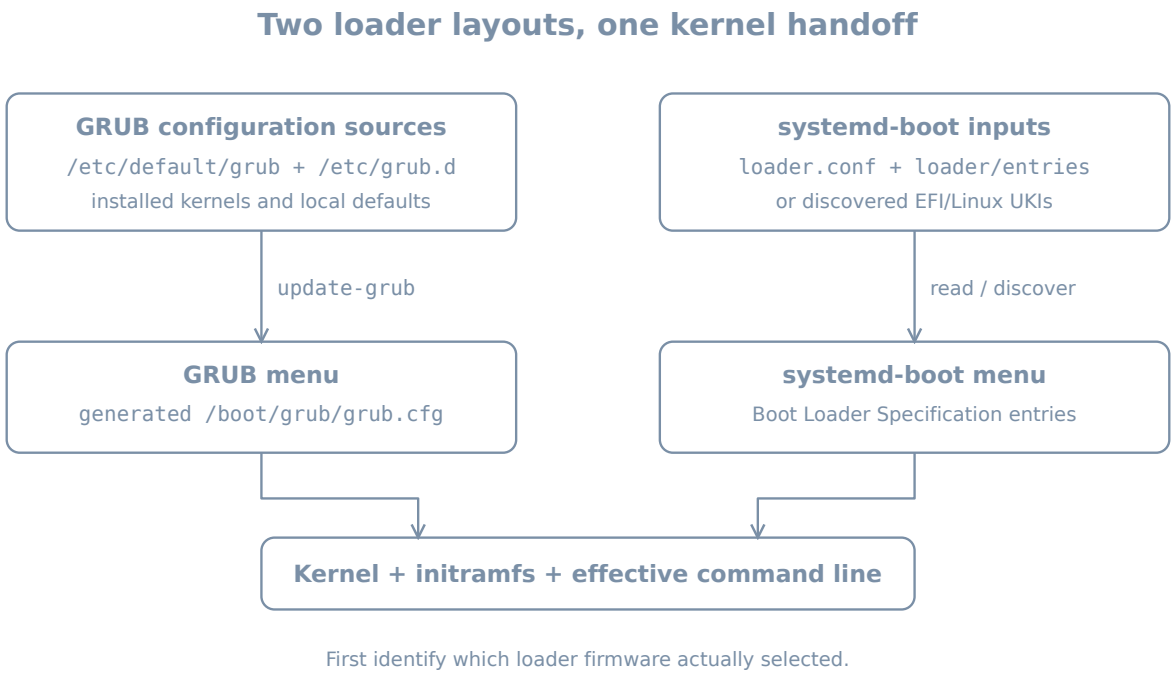


Figure 1: GRUB and systemd-boot obtain menu entries differently.

How an Ubuntu GRUB menu is produced

GRUB is a bootloader with its own configuration language and menu. On the Ubuntu installation used here, commonly inspected inputs include `/etc/default/grub`, optional local defaults under `/etc/default/grub.d/`, scripts under `/etc/grub.d/`, and the installed kernel artifacts they discover. The generated menu is usually `/boot/grub/grub.cfg`.

Inspect the actual host before assuming those paths or a particular value:

```
sudo cat /etc/default/grub
sudo ls -la /etc/default/grub.d /etc/grub.d
sudo ls -l /boot/grub/grub.cfg
sudo grep -E '^[[:space:]]*(menuentry|submenu) ' /boot/grub/grub.cfg
```

A missing optional directory can produce an error; inspect the paths that actually exist. The final command is a quick view, not a parser for all GRUB syntax or conditional behavior. Menu titles and submenus can change when installed kernels change. A displayed menu entry or generated configuration does not prove that its referenced files are readable at boot.

Ubuntu's `update-grub` manual identifies it as the wrapper that generates `/boot/grub/grub.cfg` using `grub-mkconfig`. Treat that file as generated output: a hand edit can be overwritten by the next generation. Inspect configuration inputs and package-owned scripts before modifying them. Files in `/etc/default/grub.d/` can override assignments from the main file, so reconcile their order and actual contents on your installation instead of assuming a single line controls the outcome.

In the GNU GRUB configuration reference, `GRUB_TIMEOUT_STYLE=menu` requests a visible menu,

while `GRUB_TIMEOUT` controls the wait in seconds under normal menu behavior. Generated code can include conditional paths, for example after a recorded boot failure. A value in the source file is intent; generated output and the next boot provide additional evidence.

Prepare a disposable GRUB test guest

The exercise changes local configuration and reboots. Use a **full powered-off clone**, not the original. In `bl02`, shut down cleanly. On the host:

```
sudo virsh --connect qemu:///system domstate bl02
sudo virsh --connect qemu:///system domblklist bl02 --details
```

Require `shut off` and reconcile the single system disk with the earlier record. Ensure there is enough host space for a full copy. The destination VM name and disk path in the following command must not already exist:

```
sudo virt-clone --connect qemu:///system \
  --original bl02 --name bl03-loader-lab \
  --file /var/lib/libvirt/images/bl03-loader-lab.qcow2
```

This follows the powered-off clone method established in Part 01. Inspect the original and clone with `domblklist --details` and `dumpxml`, using the same `qemu:///system` connection. Confirm distinct disk paths and distinct UEFI NVRAM files. If the definition has extra disks, shared state, a backing chain or TPM state, stop; those require a different copy and recovery plan. Keep `bl02` shut off throughout the test because the clone also contains the original guest's hostname, machine ID and credentials. In the manager, attach the clone's NIC to the isolated `blpractice` network while it is shut down. The graphical console supplies access and no package download is needed.

Start only `bl03-loader-lab`. The login hostname can still say `bl02`; use the console window's VM identity and the host-side disk record to distinguish the clone. Verify administrative access, root mount and the GRUB paths. If this is not a GRUB installation with `/etc/default/grub` and `/boot/grub/grub.cfg`, stop this GRUB-specific exercise. Do not install GRUB over the existing boot path merely to continue.

Change a menu timeout and roll it back

In the clone, first record the original configuration and inspect any local overrides:

```
sudo cat /etc/default/grub
sudo ls -la /etc/default/grub.d
sudo grep -nE '^[[:space:]]*GRUB_(TIMEOUT|TIMEOUT_STYLE)=' \
  /etc/default/grub /etc/default/grub.d/*.cfg
```

If the optional directory or matching files do not exist, the glob can produce an error. Record that and inspect any files that do exist. If local overrides or a management system own these settings, stop and plan with those inputs instead of relying on the simple edit below.

Create a private backup of the source file under a new name:

```
sudo test ! -e /etc/default/grub.bl03-original
sudo cp -a /etc/default/grub /etc/default/grub.bl03-original
```

Proceed with the copy only if the first command succeeds. Record that backup's path. Using your editor as administrator, edit `/etc/default/grub` so its effective assignments for these two variables are:

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=5
```

Change existing assignments rather than leaving conflicting duplicates. Do not change the default boot entry, kernel arguments, firmware variables or other settings. Review the diff and regenerate:

```
sudo diff -u /etc/default/grub.bl03-original /etc/default/grub
sudo update-grub
sudo grub-script-check /boot/grub/grub.cfg
```

`diff` normally exits with status 1 when it finds differences; here that is expected if only the intended lines changed. Stop if the diff shows unintended edits. Run the generation and syntax check only after reviewing it. Record the output and exit status of each; a syntax check does not prove that a boot entry points to working files.

If generation or validation fails, **do not reboot**. Preserve the error and restore the original source, then regenerate and check again:

```
sudo cp -a /etc/default/grub.bl03-original /etc/default/grub
sudo update-grub
sudo grub-script-check /boot/grub/grub.cfg
```

If rollback generation or validation also fails, leave the clone running, keep the console and evidence, and use the separate baseline/recovery route. Do not keep editing generated configuration at random.

After successful generation and validation, reboot the clone with the console open. Observe whether the menu is visible and approximately how long the countdown waits without a keypress. Boot the default entry and verify login and `uname -r`. A console or firmware timing difference can hide a menu; if the expected behavior is absent, compare the actual generated configuration, overrides and selected loader before concluding the edit failed.

Restore the source file using the rollback commands above. Confirm the source matches the backup with `sudo cmp /etc/default/grub.bl03-original /etc/default/grub`, then reboot and observe the restored behavior. A pre-existing hidden or zero-timeout menu may not be visible; a successful login and the restored source plus successful generation are distinct pieces of evidence. Shut down the clone. Leave its disk available for inspection or retire it only after verifying its unique path. The untouched original can then be started for later chapters.

What changes for systemd-boot?

systemd-boot is a UEFI boot manager that discovers entries from the ESP and, where configured, the Extended Boot Loader partition (XBOOTLDR). It can read Boot Loader Specification Type #1 entry files and discover Type #2 unified kernel images. Its documentation describes the locations and features. This is a comparison, not a recommendation to replace Ubuntu's existing loader.

On a system already using systemd-boot, first identify the actual ESP and optional XBOOTLDR mount. Then inspect, where present, `loader/loader.conf`, `loader/entries/` and `EFI/Linux/` on those filesystems. `bootctl status` reports available status; `bootctl list` lists entries it knows how to discover. A list can include discoverable entries even when another loader was used to boot. Absence of a particular directory is not proof of failure if another supported entry mechanism is in use.

Unlike the Ubuntu GRUB workflow, `update-grub` does not generate systemd-boot entries. Configuration and entry ownership may belong to a distribution package, `kernel-install`, an image builder or an administrator. Find that owner before editing. Runtime EFI variables can affect selection too; a file alone may not describe the effective next boot. Use the installed version's manual and distribution procedure for any change and provide a console and rollback before applying it.

Diagnose the correct selection layer

Symptom	First distinction	Evidence
Firmware reports no boot option	Firmware selection versus loader menu	`BootOrder`, selected device and ESP path from the prior chapter
GRUB appears but expected kernel is absent	Loader configuration versus installed artifacts	Generated menu, inputs and `/boot` inventory
GRUB source changed but behavior did not	Input versus generated output versus selected loader	Overrides, generation result, console and active boot path
systemd-boot list differs from current boot	Discoverable entries versus selected loader	`bootctl status`, firmware selection and relevant mounts
A menu entry appears but fails after selection	Menu existence versus working kernel/root	Preserve console error and identify the next failed stage

For the original host, preserve a known working boot choice and console access before any persistent change. Do not use a one-time firmware choice, a loader's one-time selection and a permanent default as interchangeable operations. Recovery of a deliberately broken boot entry is developed after the remaining boot stages are understood.

Practice and recall

Record the selected firmware path, loader evidence, configuration owner, generated or discovered entries, and the clone's before/change/rollback observations. Explain which settings can persist after reboot and which observation applies only to the current boot.

- Firmware selects an EFI application; a loader selects its own boot entry.
- GRUB's source configuration and generated menu have different owners and lifecycles.
- systemd-boot discovers entries from its supported boot partitions and formats.
- A valid menu file and an existing menu entry do not prove a successful boot.
- Test persistent configuration changes on a recoverable clone and verify rollback.

Kernel and Early Userspace

Kernel Images, Command Lines and Matching Modules

A package update installs a new kernel, but `uname -r` still reports the old one. That can be normal: installed files and the kernel currently executing are different states. A boot entry must select a coherent set of artifacts, and the next reboot determines which kernel actually runs.

This chapter inventories the guided Ubuntu `bl02` guest and relates its running kernel to the installed image, initramfs, module tree and boot command line. The exercise is read-only. It does not remove kernels, regenerate initramfs images or change boot arguments. You need console access and the bootloader evidence from the previous chapters. Leave the GRUB test clone shut down and work in the original guest.

A versioned boot set

A **kernel image** is the executable kernel selected for boot. An **initramfs** is an initial filesystem image supplied to the kernel for early userspace; it can contain tools, configuration and modules needed before the real root filesystem is available. A **kernel module** is code that can be loaded into a running kernel when needed. Some functionality is built into the kernel instead and is not a separately loaded module.

For a common Ubuntu installation, versioned artifacts have names resembling:

```
/boot/vmlinuz-<release>
/boot/initrd.img-<release>
/lib/modules/<release>/
```

Here `<release>` stands for the **complete** kernel release string, including distribution and flavor suffixes. Do not type the angle-bracket placeholders as paths. Other distributions, boot mechanisms, architectures and unified kernel images may arrange artifacts differently. The following inspection is specific to the current Ubuntu lab layout.

A loader can pass a command line alongside the selected image. Early userspace and the kernel use some of its arguments at different stages. For example, a `root=` value may help identify the intended root device, but it is not by itself proof of the filesystem ultimately mounted there.

Identify the running kernel and root

Log in to the original `bl02` console and confirm the host and root:

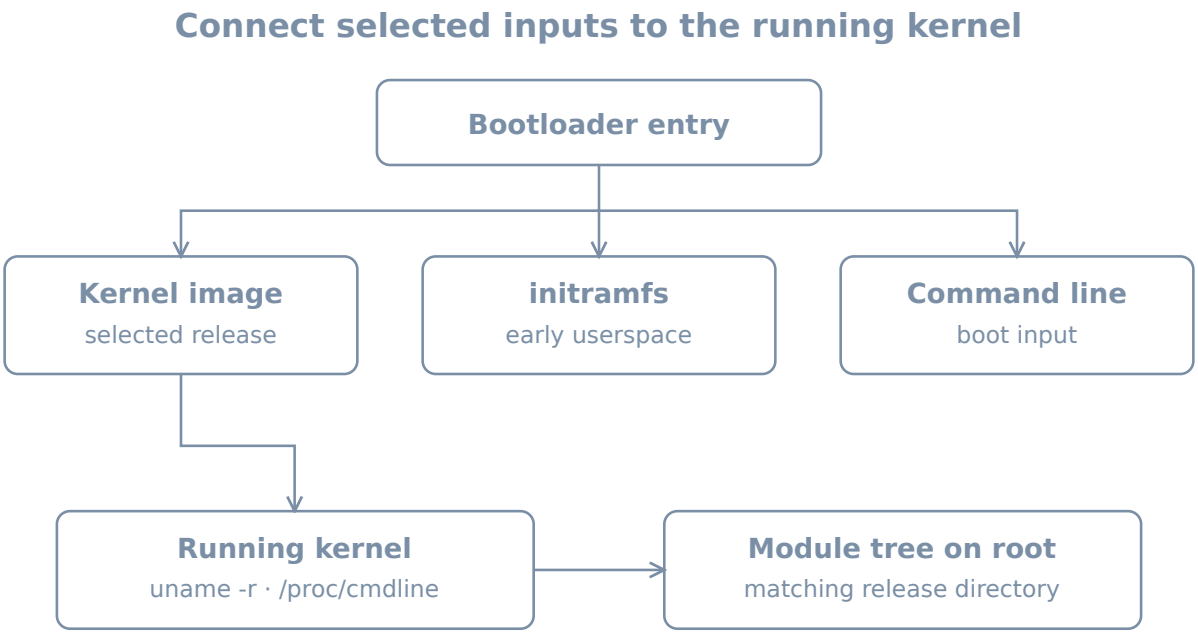


Figure 1: A boot entry selects inputs that must be interpreted as a set.

```
hostnamectl
uname -r
cat /proc/cmdline
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
```

`uname -r` reports the release of the running kernel. `/proc/cmdline` reports the command line exposed by this boot; it is stronger evidence of effective input than a default value in an ungenerated configuration file. `findmnt` shows the current mounted root source and filesystem.

An illustrative output might include:

```
6.x.y-example-generic
BOOT_IMAGE=/vmlinuz-6.x.y-example-generic root=/dev/mapper/example--vg-root ro
```

These are fictional values. Actual releases, paths and arguments will differ. `BOOT_IMAGE` can be supplied by the bootloader as a hint, and its pathname is interpreted in the bootloader's context; it is not guaranteed to be a directly usable Linux path. Compare it with the loader's configuration and mounted `/boot` rather than mechanically prepending a directory.

`ro` on the initial command line can request an initially read-only root. A later mount operation can make the live root read-write, so inspect current mount options rather than concluding that the current root is read-only from the command line alone. Avoid placing passwords or other secrets on a kernel command line: it is visible to processes through `/proc/cmdline` on typical systems and may be retained in logs.

The kernel parameter documentation describes many kernel-recognized arguments and notes that other arguments can be passed to init. An argument's presence is not proof that a particular component recognized or applied it. Confirm the relevant component's documentation and observed behavior.

Inventory files for the running release

Set a shell variable from the actual running kernel and inspect the matching conventional paths:

```
KREL=$(uname -r)
printf 'Running release: %s\n' "$KREL"
ls -l "/boot/vmlinuz-$KREL" "/boot/initrd.img-$KREL"
ls -ld "/lib/modules/$KREL"
findmnt --target /boot --output TARGET,SOURCE,FSTYPE,OPTIONS
```

For the reference installation, the image, initramfs and module directory should normally exist for the running release. If a path is missing, preserve the error and investigate how this particular kernel was booted and packaged. A missing conventional on-disk initramfs does not prove the system never booted: the booted image may have been supplied from another location or the file could have changed after startup.

The `/boot` query reports the filesystem **containing** `/boot`. It may be root or a separate mount. If the prior installation record expects a separate `/boot`, check it with `findmnt --mountpoint /boot`

as well. Looking at an unmounted directory on root can make a healthy separate boot filesystem appear empty.

List candidates without assuming that the highest-looking filename is the current one:

```
ls -lh /boot/vmlinuz-* /boot/initrd.img-*
ls -1 /lib/modules
```

A shell glob with no matches can produce an error; record it instead of treating the literal pattern as a file. Names usually associate a kernel image and initramfs by their full release string. Multiple releases can coexist. A newer installed release is an available candidate only if its files, loader entry and boot dependencies are correct; installation does not itself switch the already running kernel.

Package metadata supplies another view:

```
dpkg-query -W -f='${db:Status-Abbrev} ${binary:Package} ${Version}\n' \
  'linux-image-*' 'linux-modules-*'
```

Rows can include package states other than installed. Read the status abbreviations and actual versions rather than counting every row as an active kernel. If no packages match one pattern, `dpkg-query` may report that fact. Package inventory, files on disk and the running release answer related but different questions.

Follow a module without loading it

The running kernel's module directory contains modules and metadata associated with its release. `lsmod` shows modules currently loaded into the kernel:

```
lsmod | head -20
```

This is a snapshot of loaded modules, not a complete inventory of built-in drivers, available modules or working devices. Earlier device discovery showed how to relate a network interface to its actual driver. If that driver is named `virtio_net` and `modinfo` is available, inspect it without loading or unloading anything:

```
modinfo -k "$(uname -r)" virtio_net
```

Use your observed driver name if different. `modinfo` can report a module's filename, aliases, dependencies, supported parameters and other metadata; fields vary. A built-in driver may be represented as built-in or may lack a separate module file. A module file's existence does not prove that it is loaded, bound to the intended device, compatible with the running kernel or permitted by the host's module-signing policy. The modinfo manual describes lookup by kernel release.

The exact version string matters. Copying an arbitrary `.ko` file into a different release directory does not establish compatibility. An out-of-tree driver may need a build for each kernel and may also need to satisfy signing policy. Diagnose its package, build and signing evidence before changing anything; do not disable security checks or unload a working disk or network driver during this lab.

Distinguish planned arguments from this boot

For the GRUB installation, compare current observations with the configuration sources and generated output:

```
cat /proc/cmdline
sudo grep -nE '^GRUB_CMDLINE_LINUX(_DEFAULT)?=' /etc/default/grub
sudo grep -nE '^[[:space:]]*(linux|initrd)[[:space:]]' /boot/grub/grub.cfg
```

The source assignments express defaults used when configuration is generated. Drop-ins, scripts, menu selection, one-time edits and conditional logic can alter the resulting choice. The generated `linux` and `initrd` lines are candidates; several entries can be present. Do not assume the first line printed is the one that booted. Reconcile the selected menu entry with the console record and `/proc/cmdline`; if the exact selection was not captured, mark it uncertain.

GRUB can use paths relative to a filesystem it selects, while Linux presents mounted paths after boot. A line mentioning `/vmlinuz-...` can refer to a file on a separately mounted `/boot` filesystem. Determine the preceding GRUB search/root commands and the partition map before comparing paths. The command-line arguments are not a safe place to improvise boot fixes on the original host.

Worked diagnosis: new kernel installed, old one running

Suppose package inventory shows releases A and B, but `uname -r` shows A after an update. Ask whether a reboot was expected and whether B was actually selected:

1. Compare the update record and current boot time; do not infer a reboot from package installation.
2. Verify the B image, initramfs and module directory using the **observed** complete B release string, without removing A.
3. Inspect the loader's available entries and console selection. A generated entry does not prove it was successfully booted.
4. Check the current `/proc/cmdline`, `uname -r` and root mount after any planned reboot.
5. If B fails while A works, retain the failure message and the working entry. Investigate the failing stage and matching artifacts before attempting repair.

If an update left no B initramfs or module tree, that is an inconsistency to investigate with package and installation logs. Rebuilding initramfs or repairing a package is a change with prerequisites; the next chapter explains early userspace and its construction. Do not delete a working older kernel merely because the newer one is installed.

Practice and acceptance

Create a table with the current release, current command line, root mount, filesystem containing `/boot`, matching image/initramfs/module paths, and any other installed candidate. Record whether

each observation is runtime evidence, an on-disk artifact or configuration intent. Explain one reason these views could differ without implying corruption.

If you have no second installed kernel, use the fictional A/B scenario above as a reasoning exercise. Do not install or remove kernels just to manufacture one. Keep the record outside the guest and redact sensitive path or identity details before sharing.

Recall card

- Installing a kernel and running it are different events.
- Compare the full release across image, initramfs and module tree; layouts can vary.
- `/proc/cmdline` is current-boot input, while loader configuration describes choices.
- A module on disk, a loaded module and a driver bound to a device are different states.
- Preserve a known working boot path while investigating a failed update.

Initramfs and Reaching the Real Root

The kernel has started, but the installed root filesystem is on an LVM logical volume whose disk has not yet been discovered. The kernel cannot run the installed system's programs from a filesystem it cannot reach. Early userspace bridges that gap: it provides a small temporary environment that can find devices, assemble storage and mount the intended root.

This chapter follows the Ubuntu `bl02` installation. You need its storage and boot records, console access and the full powered-off clone method from earlier. The main inspection is read-only. An optional rebuild runs only on a disposable clone, after checking its recovery path and free space.

From boot image to installed root

The loader supplies a kernel, command line and, in this installation, an initramfs image. The kernel unpacks the initial filesystem into memory and runs its initial program. That early program and its tools discover the devices needed for root, possibly unlock encryption or activate LVM, mount the real root, and hand control to the installed system's init program.

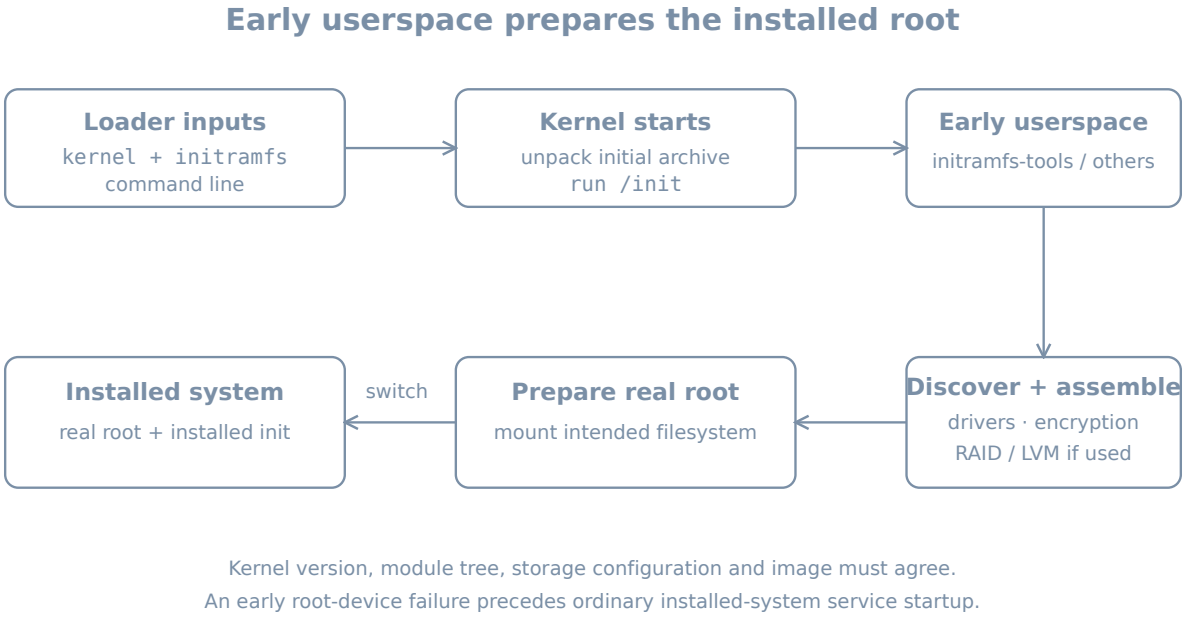


Figure 1: Early userspace prepares the installed root.

An initramfs is an archive used to populate an in-memory filesystem. The familiar Ubuntu filename `initrd.img-<release>` is historical terminology; it does not imply that every machine uses the older block-device initrd mechanism. Systems can use different early-userspace implementations,

or boot without a separate initramfs where their design permits it. The systemd boot overview describes the general stages.

For our guided LVM guest, the dependency chain is approximately virtual disk → partition → physical volume → volume group → root logical volume → root filesystem. An encrypted installation can add an unlock dependency before the volume group. This is a dependency map, not a claim about the exact order of every script on every image.

Inspect the installed image and its inputs

On `bl02`, confirm the current release, root and boot filesystem:

```
hostnamectl
uname -r
cat /proc/cmdline
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
findmnt --target /boot --output TARGET,SOURCE,FSTYPE,OPTIONS
lsblk -o NAME,TYPE,FSTYPE,UUID,MOUNTPOINTS
```

Match the root source with the observed storage tree. A `root=` argument is an input to early boot, while `findmnt` describes the current mount. It is possible for a command line and a mounted source to use different names for the same object. Resolve identifiers before declaring them inconsistent.

On Ubuntu systems using `initramfs-tools`, inspect the configuration and the image associated with the running release:

```
KREL=$(uname -r)
ls -l "/boot/initrd.img-$KREL"
ls -la /etc/initramfs-tools
cat /etc/initramfs-tools/initramfs.conf
sudo lsinitramfs "/boot/initrd.img-$KREL" > "$HOME/bl-initramfs-list.txt"
head -40 "$HOME/bl-initramfs-list.txt"
```

`lsinitramfs` lists archive members; it does not execute them. Check that the listing command succeeded before interpreting the file. If the image or utility is missing, determine the host's actual boot method and packaging rather than creating an empty image. The lsinitramfs manual describes supported image listing and its limitations with multi-segment images.

Look for evidence related to the actual storage stack, for example:

```
grep -Ei '(^|/)(init|lvm|cryptsetup)(/|$|\.)' "$HOME/bl-initramfs-list.txt"
```

No match is not a complete diagnosis: names, packaging, compressed module filenames, built-in drivers and implementations differ. A matching filename establishes presence, not that a script ran successfully or that a device was unlocked. The initramfs-tools documentation separates image-building hooks from boot-time scripts.

Construction and dependencies

A generated initramfs is a snapshot of selected files, scripts and configuration. Changes to early-boot inputs may require regeneration; editing the live root's configuration does not retroactively edit an existing image. Package installation can also trigger generation. The output is tied to a particular kernel release and may contain selected modules for that release.

Keep four questions distinct:

Question	Evidence
What input is configured?	`/etc/initramfs-tools` and the installed package's hooks
What is in the generated image?	`lsinitramfs` for the identified image
Which image did the loader select?	Bootloader entry and console/boot record
Did early userspace reach root?	Successful current root mount and boot evidence

A successful normal boot does not prove that every installed fallback image works. Nor does it prove the image currently on disk is byte-for-byte the one loaded earlier if it was replaced after boot.

Optional lab: rebuild one image on a clone

This exercise writes a boot artifact and reboots. First shut down `bl02`, confirm it is `shut off` and inspect its system disk using the host-side `virsh` procedure from the loader chapter. This clone method applies only to the documented single writable disk, no backing chain, no shared writable disk and no TPM state. Confirm sufficient host space and that the destination name and disk path are new. On the host:

```
sudo virt-clone --connect qemu:///system \
  --original bl02 --name bl03-initramfs-lab \
  --file /var/lib/libvirt/images/bl03-initramfs-lab.qcow2
```

Inspect original and clone disk paths and distinct UEFI NVRAM files in their domain definitions as in the loader chapter. Keep the original off, attach the clone to `blpractice`, and use the graphical console. The clone still reports hostname `bl02`; identify it by the manager's VM name and unique disk.

On the clone, confirm the running release and existing image, and inspect free space:

```
KREL=$(uname -r)
ls -l "/boot/initrd.img-$KREL"
findmnt --target /boot --output TARGET,SOURCE,FSTYPE
df -h /boot /
```

Stop if `/boot` is not the expected filesystem, the image is absent, capacity is insufficient, the root stack differs from the record, or the clone's recovery path is uncertain. Do not remove old kernels to make room. Save the image's pre-change hash and a list of its contents outside the guest using the established evidence procedure:

```
sudo sha256sum "/boot/initrd.img-$KREL"
sudo lsinitramfs "/boot/initrd.img-$KREL" > "$HOME/bl-initramfs-before.txt"
```

With the clone's console and original baseline available, update the **existing** image for the currently running release:

```
sudo update-initramfs -u -k "$KREL"
sudo lsinitramfs "/boot/initrd.img-$KREL" > "$HOME/bl-initramfs-after.txt"
sudo sha256sum "/boot/initrd.img-$KREL"
```

Run each command only after the previous one succeeds. `-u` updates an existing image; `-k` selects the release. Consult the locally installed `man update-initramfs` for the version in use. Hashes and member ordering may change even without an intentional configuration change, so equality is not the success criterion. Compare relevant members, retain the command result, and verify that the bootloader still refers to the intended image.

If generation or inspection fails, do not reboot. Preserve the error and keep the original guest and baseline intact; investigate the clone or discard and recreate it after verifying its unique disk path. After a successful rebuild, reboot the clone through its console and verify login, `uname -r`, the expected root source and `systemctl --failed`. This demonstrates the tested boot path, not every fallback path. Shut down the clone before restarting the original.

Diagnose an early-root failure

Suppose the console reports that the root device cannot be found and offers an early shell. Candidate causes include a wrong root identifier, missing storage driver, incomplete LVM activation, a failed encryption unlock, missing device, or an initramfs that lacks required components.

1. Preserve the exact console error, boot entry, kernel release and time.
2. Identify the expected storage chain from the installation record and compare it with the VM's attached disk.
3. Determine whether the failure is before the real root mount; ordinary service logs may not yet exist on that root.
4. Use a known-good entry or verified recovery media according to the established recovery plan. Inspect the relevant image and configuration offline before changing them.
5. After a justified repair, perform a normal boot and check the root source and dependent services.

An early shell may have different tools and device names from the installed system. Do not format, recreate a volume group, or paste another machine's UUID just to leave the shell. Detailed external-media and offline repair procedures follow later in this part.

Practice and recall

Record the running release, selected image evidence, root storage chain and at least one limitation of the archive listing. If you used the clone, retain the rebuild result and post-reboot root check.

- Early userspace prepares the real root before the installed manager can run.
- Build inputs, archive contents, selected image and successful execution are different observations.
- An image must fit the kernel release and the actual storage dependencies.
- Test rebuilds and recovery on a disposable copy before changing a working boot path.

Root Handoff, PID 1 and the Boot Transaction

The real root is mounted, yet the machine is not necessarily ready for a login or a workload. The installed system manager must coordinate mounts, device events and services. A successful root handoff is one milestone, not the end of startup.

Use the running `bl02` guest and console. This chapter observes the installed systemd manager and its current boot; it does not create or modify units. The preceding initramfs chapter supplies the early-root model.

From temporary root to installed manager

Early userspace prepares the real root and hands over execution to an init program on it. On the reference Ubuntu installation, that installed system manager is systemd. PID 1 is the first userspace process and has special responsibilities, including adopting orphaned processes and coordinating the system's lifecycle. The handoff mechanism and whether systemd also runs in the initramfs depend on the early-userspace implementation; do not assume every initramfs is a systemd initrd.

A **unit** is an object managed by systemd: service, mount, socket, device, target and other types. A **job** is an operation such as starting or stopping a unit. Starting a target can pull other units into a transaction, and systemd orders applicable jobs according to dependency rules. Many jobs can proceed concurrently. The bootup overview describes the major targets and the parallel nature of startup.

Observe PID 1 and this boot

In the guest console:

```
ps -p 1 -o pid,comm,args
readlink -f /proc/1/exe
systemctl is-system-running
systemctl --failed --no-pager
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
```

On the reference host, PID 1 should identify systemd. `is-system-running` may report `running`, `degraded`, `starting` or another state; a nonzero exit status for a state such as `degraded` is diagnostic information, not a command to reset failures. A degraded system can be usable while one unit has failed. The root mount is runtime evidence, not a complete history of how it was found.

Inspect boot messages:

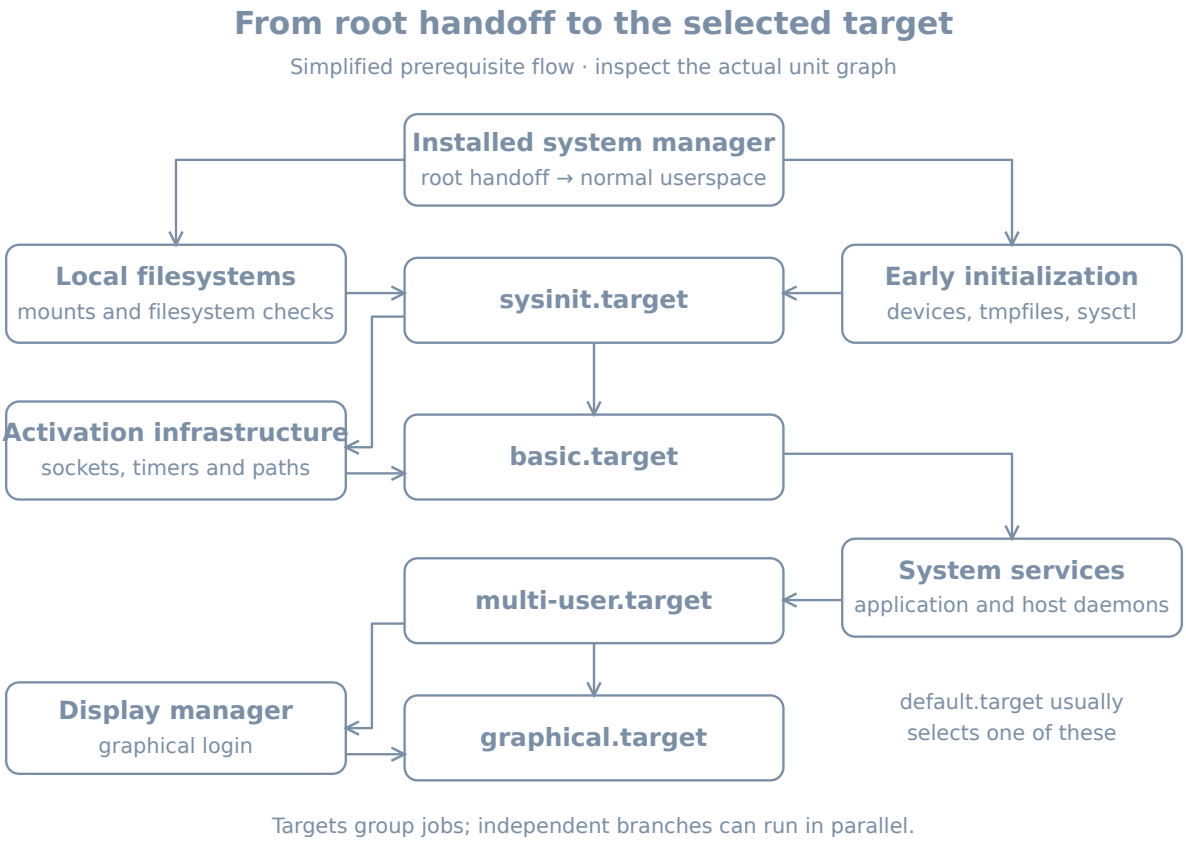


Figure 1: The installed manager coordinates boot toward a selected target.

```
journalctl --list-boots --no-pager
sudo journalctl -b -p warning --no-pager
sudo journalctl -b -u systemd-remount-fs.service --no-pager
```

`-b` selects the current boot and `-p warning` selects warning priority and more severe messages. A unit may not exist, may have completed successfully without much output, or may not be responsible for the symptom. An empty query is an observation. Journal persistence and access depend on configuration; the previous boot may not be retained after a reboot. Do not assume the system journal captures all firmware or early-initramfs messages.

Requirement and ordering are different

A dependency can express that one unit wants or requires another. Ordering expresses which job should precede another **when both jobs are present**. `After=` alone does not pull a unit into the transaction. `Wants=` or `Requires=` alone does not by itself specify a start order. `Requires=` has stronger failure relationships than `Wants=`, but it is not a general substitute for an application-level health check.

For an imaginary service `report.service` using `data.mount`:

Relationship	Effect to reason about
`Wants=data.mount`	Attempt to include the mount without the same failure coupling as a requirement
`Requires=data.mount`	Express a stronger requirement, with documented failure and stop behavior
`After=data.mount`	If both start jobs are present, order the service after the mount job

A real filesystem dependency can be expressed through appropriate mount-related mechanisms; the example isolates the concepts rather than prescribing a unit. The systemd unit reference defines the detailed semantics.

Inspect relationships of a unit present on your host:

```
systemctl list-dependencies --all multi-user.target
systemctl show multi-user.target -p Wants -p Requires -p After -p Before
systemctl list-jobs
```

The dependency tree is not a timeline, and `list-jobs` is only a snapshot of jobs currently queued. Empty output after boot is normal. Properties include implicit as well as explicit relationships; use `systemctl cat UNIT` with an actual unit name to inspect its definitions and drop-ins. A static list cannot prove the service's client-facing behavior.

A transaction can succeed partially

Suppose `multi-user.target` is active while a nonessential service failed. The target's state, the failed unit and the service's client test answer different questions. Find the failed unit with `systemctl --failed`, inspect `systemctl status UNIT` and `journalctl -b -u UNIT` using its actual name, then determine its role and dependencies. Do not clear the failed state as the first step; doing so removes a useful signal without repairing the cause.

An error involving the root filesystem before the installed manager starts belongs to the earlier boot boundary. A failed service after login belongs to the installed manager or application boundary. If uncertain, use the console's last visible message and timestamps to establish the boundary before changing configuration.

Practice and recall

Record PID 1, current root, system state, failed units and one target's `Wants`, `Requires` and `After` properties. Choose one observed relationship and explain why it does or does not establish ordering. Compare that static relationship with current job state and the journal.

- Root handoff starts the installed system's work; it does not establish workload readiness.
- Units represent managed objects; jobs are operations on them.
- Requirement and ordering answer different questions.
- A reached target and an empty job queue are not complete health checks.

Startup and Shutdown

Targets, Activation, Readiness and Boot Timing

A dashboard says the system reached `multi-user.target` in twelve seconds, yet a client request fails for another ten seconds. The two observations can both be correct. A target is a synchronization point in the manager's unit graph; application readiness is a claim that must be checked at the application's boundary.

Use the running `bl02` console and the boot record from the preceding chapter. The commands below inspect state and timing. They do not change the default target or disable any service.

What a target tells you

A target is a systemd unit that groups or orders other units. On a conventional system, `default.target` commonly resolves to `multi-user.target` for a non-graphical host or `graphical.target` for a graphical one. The actual alias and dependencies are configuration, not a universal promise. The systemd special-unit reference documents common targets.

Inspect the default and current target state:

```
systemctl get-default
systemctl list-units --type=target --all --no-pager
systemctl status multi-user.target --no-pager
systemctl --failed --no-pager
```

`get-default` reports the configured default for normal boot. Active targets describe current manager state. A target can be active while a wanted service has failed, while an application is still warming up, or while a service that will be started on demand has not yet been invoked. A graphical target can also include the multi-user target; the two labels do not represent mutually exclusive running modes.

`network.target` and `network-online.target` are especially easy to overread. A reached target is not a guarantee that DNS, a particular remote endpoint or the Internet is reachable. Whether a wait service is configured and what it considers online depend on the network manager and local configuration. Inspect the actual units, then test the dependency your application needs.

Activation and readiness are separate

Some services start during boot. Others are activated by a socket, path, timer, device or an explicit request. A listening socket can exist before the service process, and a service can remain inactive until its first request. The system manager may consider a service started at different points according to its `Type=` and implementation.

For `Type=simple`, manager activation does not wait for a service-specific ready notification. A `Type=notify` service can send a readiness notification, but that signal's usefulness depends on what the service actually checks. A `Type=oneshot` service may complete successfully and exit. The systemd service reference defines the service types and their lifecycle semantics.

Observation	What it supports	What remains to test
Unit is active	Manager considers the unit active	The actual application operation
Socket is listening	A local socket is bound	Whether a request is served correctly
Target is reached	Its activation transaction reached that milestone	Independent or later work and external dependencies
Client check succeeds	The tested operation worked at that time	Other operations, clients and failure modes

Design a readiness check around an expected operation: for example, a query returning a known record, or an HTTP response containing an expected value. Name the client location, credentials, timeout and success condition. A process existing is weaker evidence than the operation it is meant to provide.

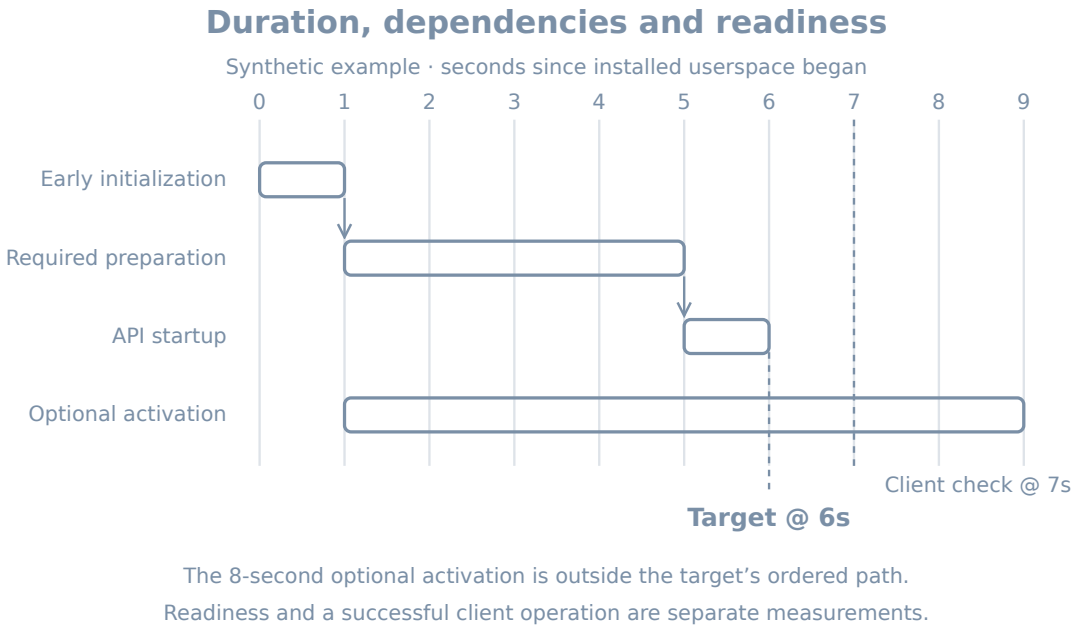


Figure 1: Startup timing and client readiness measure different events.

Read timing without assigning blame too quickly

Run:

```
systemd-analyze time
systemd-analyze blame
systemd-analyze critical-chain
```

The reported phases and available timestamps depend on the firmware, loader and initramfs. `blame` reports unit activation durations, not the complete reason the machine was slow. A long duration can include waiting for a dependency; some unit types do not provide a meaningful activation duration. `critical-chain` is a view of a time-critical ordering chain, not a complete trace of all concurrent work or timed-out jobs. The Ubuntu systemd-analyze manual documents these limits.

If a unit appears slow, inspect its actual status and current-boot log, substituting its observed name:

```
systemctl status UNIT --no-pager
sudo journalctl -b -u UNIT --no-pager
systemctl show UNIT -p Type -p After -p Wants -p Requires
```

Replace `UNIT` with the full unit name; it is a placeholder, not a unit to create. Compare timestamps and ask what the unit was waiting for. A network wait service taking time can reflect a missing link or an intentional network policy. Disabling it may shorten a graph while breaking a service that relies on it.

Worked example

A synthetic timeline shows `multi-user.target` at six seconds and an optional activation finishing at nine. A client request at seven seconds succeeds. It would be wrong to add all unit durations together to claim a fifteen-second boot, or to say that the nine-second unit blocked that client without a dependency and request trace. The target time, optional activation and client test are separate evidence.

For a real incident, define a user-visible start and finish, collect several comparable boots, and control for cache, updates, workload and network conditions. Use a client probe that checks the actual expected response; record failures as well as the first success. More detailed performance methods come later.

Practice and recall

Record the default target, current active targets, system state, `time`, and one entry from `critical-chain`. For that unit, identify one plausible dependency and inspect its log. Explain which observation would prove a particular client operation works and why it is not supplied by the target name alone. Do not change a target or service as part of this inspection.

- A target is a manager milestone, not a complete application health test.
- Activation can be boot-driven or triggered later by another event.
- Service type affects when the manager considers startup complete.
- Timing output is evidence for a question, not a list of services to disable.

Shutdown, Reboot and Services That Delay Shutdown

A VM still shows activity after a shutdown request. Is a user session inhibiting the request, a service stopping slowly, a filesystem busy, or the guest already shut down while the host display remains open? Each requires different evidence. A forced power cut can destroy the information needed to distinguish them.

This chapter uses the disposable `bl03-loader-lab` clone from the earlier GRUB exercise, or a fresh equivalent full powered-off clone of `bl02`. Keep the original shut down while its clone runs because their guest identities are copied. The lab performs normal reboot and poweroff operations on the clone. Keep the VM manager console visible and save any necessary work first.

What a clean transition attempts

A normal shutdown asks the system manager to stop services and sessions, terminate remaining processes, unmount filesystems where possible and complete the requested transition. Dependencies and ordering affect stop jobs too; stop ordering is generally the reverse of start ordering when both jobs are ordered. Some final work happens after ordinary userspace services have stopped. The systemd boot and shutdown overview describes this lifecycle.

Request	Intended outcome
Reboot	Shut down the running system and start it again
Poweroff	Shut down and request power removal
Halt	Stop the system; whether power is removed depends on platform and command behavior
Suspend	Preserve a running session for later resume; not a shutdown
Forced VM power cut	Remove virtual power without waiting for guest cooperation

The VM manager's **shut off** state and a blank console are not identical observations. A guest may have powered off while the console window remains open. Conversely, a blank display does not establish that the guest has stopped writing.

Preflight the transition

On the clone's console:

```
hostnamectl
cat /proc/sys/kernel/random/boot_id
systemctl is-system-running
systemctl --failed --no-pager
systemctl list-jobs
systemd-inhibit --list --no-pager
```

Record the VM identity from the manager and the boot ID from the guest. Inhibitors can request a block or delay for operations such as shutdown; they are distinct from a service's stop job. The systemd-inhibit manual describes their modes. An empty list does not guarantee that every service will stop quickly.

A privileged shutdown request can behave differently from an unprivileged desktop request and may override certain inhibitors. Do not bypass an inhibitor to make this lab faster. Identify its owner and reason; wait for the legitimate operation or coordinate with its owner before a real shutdown.

Reboot and compare boot instances

With the console open, request a normal reboot:

```
sudo systemctl reboot
```

Wait for the guest to return and log in. Check:

```
cat /proc/sys/kernel/random/boot_id
systemctl is-system-running
systemctl --failed --no-pager
journalctl --list-boots --no-pager
```

The boot ID should differ from the pre-reboot value. If the journal retained the previous boot, inspect its final messages:

```
sudo journalctl -b -1 -e --no-pager
```

`-b -1` means the previous boot **in the available journal**, not an absolute guarantee of the immediately preceding physical boot. If the journal was volatile, there may be no previous boot to inspect. Record the unavailable evidence instead of assuming a clean shutdown. Journal timestamps and final messages can show progress, but missing final messages do not alone prove an abrupt power loss.

Now request a normal poweroff:

```
sudo systemctl poweroff
```

In the host's VM manager, wait for the clone to show `shut off`. If needed, check from the host:

```
sudo virsh --connect qemu:///system domstate bl03-loader-lab
```

Use the actual clone name if you created a fresh equivalent. A normal shutdown followed by an observed shut-off domain is the acceptance result. Do not select a forced power action merely because the console has stopped updating for a few seconds.

Diagnose a slow or blocked shutdown

Before starting a shutdown on an accessible system, `systemd-inhibit --list`, `systemctl list-jobs`, service status and logs show different classes of evidence. During a slow transition, the console may show a stopping job and elapsed time. Preserve the unit name and message. If another console session remains usable, inspect jobs and the affected service without restarting it; access may disappear as shutdown progresses.

Observation	Next question
A shutdown inhibitor is listed	Who holds it, what operation is underway, and is it block or delay?
A stop job waits for a service	Is the service finishing work, blocked on a dependency, or failing to exit?
A mount cannot unmount	Which process or nested mount is keeping it busy, and what data is at risk?
Domain is already shut off	Is the manager window simply still open?
A guest stops responding entirely	What was the last console evidence, and what recovery point exists?

A service stop timeout is not automatically proof that its process is safe to kill. An uninterruptible task can be waiting for I/O; storage and remote dependencies require their own investigation. Do not set all timeouts to zero, issue repeated forced resets, or kill random processes. In a real incident, weigh data consistency and availability, record the decision and use the service's recovery procedure. In this disposable clone, the established baseline supplies a bounded reset if a controlled experiment cannot complete.

Practice and recall

Record the clone name, boot IDs before and after reboot, any inhibitors, failed units, prior-boot journal availability and final host-side domain state. Explain why a missing prior journal and an empty inhibitor list cannot each prove a clean shutdown.

- Reboot, poweroff, halt, suspend and forced power removal are different transitions.
- Inhibitors and stopping jobs are different sources of delay.
- Preserve the console message and service evidence before interrupting a stuck transition.
- Verify the guest's actual state from the host after poweroff.

Boot Trust and Recovery

Secure Boot, Unified Images and Measured Boot

A kernel update installs successfully, but a host no longer unlocks its encrypted root automatically. A change can be authorized for execution while still changing measured state used by an unlock policy. To diagnose that situation, separate four questions: what is packaged, what is verified, what is measured, and what policy releases a secret.

This chapter builds the operational model and performs read-only inspection. Use `bl02` and its recorded firmware, bootloader, kernel and storage observations. The original lab does not require Secure Boot, a TPM, a unified kernel image or TPM-backed encryption. An absent feature is a valid observation. Do not change firmware keys, enroll a token, reseal a volume or remove an existing recovery credential in this exercise.

Four related mechanisms

Mechanism	Operational question	What it does not establish by itself
A unified kernel image (UKI)	Which boot resources are packaged in one EFI executable?	That it is signed, trusted or selected
Secure Boot verification	Will a verifier accept the next executable under its configured trust policy?	That every later file and runtime action is protected
Measured boot	What measurements were recorded or extended during boot?	That the measured state was automatically judged trustworthy
TPM-bound unlock policy	Under what conditions can a protected secret be released?	That a recovery credential exists or that all boots will satisfy the policy

These mechanisms can be combined, but none implies all the others. Secure Boot without encrypted storage does not make an offline disk confidential. Encryption does not prove that the booted software was authorized. A measurement is evidence of a state, not an access decision unless another component evaluates it.

Secure Boot and a chain of trust

UEFI Secure Boot applies firmware trust policy to EFI executables. In the Ubuntu UEFI path, firmware can load a signed shim, which participates in the verification chain for subsequent boot

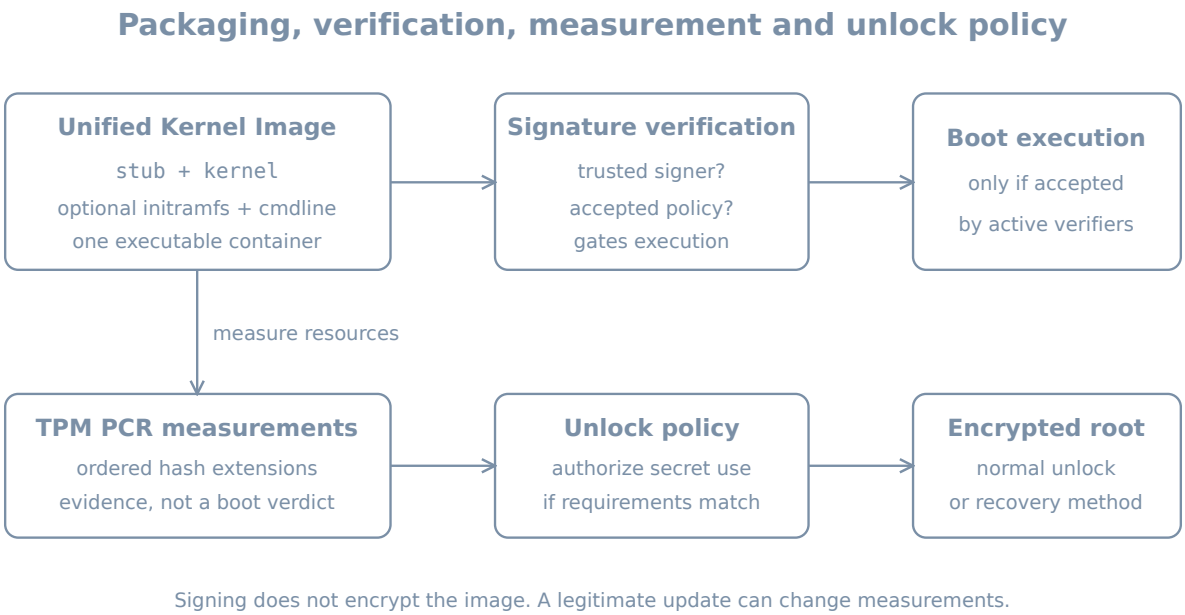


Figure 1: Packaging, verification, measurement and unlock are separate decisions.

components; the actual chain depends on installed software and configuration. Canonical's Secure Boot documentation describes Ubuntu's implementation and key handling.

A signature is useful only with a verifier, trusted key or certificate, and policy. Trust databases and revocation state matter as well as the file's signature. A file's presence on the ESP does not establish that it is signed, accepted by current firmware, selected on the next boot, or the same file that ran earlier.

Secure Boot is not a general guarantee that every byte read after boot is authenticated. Coverage depends on the boot chain, configuration and mechanisms in use. Module-signing enforcement is a related but distinct kernel policy; an out-of-tree module can fail to load even when the kernel booted successfully. Diagnose the exact rejected component and policy rather than disabling verification as a generic repair.

Inspect without changing trust

In the guest:

```
hostnamectl
if test -d /sys/firmware/efi; then
  printf 'EFI firmware interface is present\n'
else
  printf 'EFI firmware interface is unavailable in this boot\n'
fi
if command -v mokutil >/dev/null; then
  mokutil --sb-state
```

```
else
  printf 'mokutil is not installed; Secure Boot state was not checked\n'
fi
```

`mokutil --sb-state` reports the state available through the tool on a supported EFI system; record its output or error. An absent tool is not evidence that Secure Boot is disabled. If the guest firmware was created without Secure Boot support, or the relevant variable interface is unavailable, investigate the actual VM firmware configuration. Do not change the firmware mode or key database to obtain a particular answer.

Record the previously observed `BootCurrent` and EFI path if available. That is the firmware's reported selection for this boot, not a complete proof of every verification step. If a verification failure prevents boot, the firmware or loader console is often more useful than a journal on an inaccessible root.

What a UKI packages

A **unified kernel image** is a UEFI executable containing a boot stub and Linux kernel, optionally an initrd and additional resources such as command-line data. The UKI specification defines its structure. An actual UKI can be started by firmware directly or through a suitable bootloader.

Packaging can simplify distributing and signing a coherent set of boot inputs. It does not mean that every command-line source or external resource is covered by the same signature: that depends on the stub, loader, platform security state and policy. It also does not imply that every initramfs file on an Ubuntu host is inside a UKI. The separate `/boot/vmlinuz-*` and `/boot/initrd.img-*` files inspected earlier are one valid layout.

On a host already using systemd-boot, `bootctl status` and `bootctl list` may help identify its ESP and discoverable entries. Those commands do not prove that a discoverable UKI was the image selected for this boot. Compare firmware selection, loader evidence and the actual boot record. Do not install systemd-boot or create a UKI merely to complete this inspection.

Measurements and policy

A **Trusted Platform Module** (TPM) can maintain platform configuration registers, or PCRs. Measurements are extended into PCRs in sequence: the resulting value depends on both earlier state and the new measurement. A verifier can compare measurements or evaluate a signed policy, and a local unlock mechanism can require an approved state before releasing a secret.

The chosen PCRs, event log, banks, firmware and policy determine what a measurement means. A PCR value without its measurement context is not a complete explanation of what booted. A legitimate firmware, bootloader, configuration or kernel update can change a policy-relevant measurement. A failed automatic unlock therefore calls for investigating the selected policy and update history; it does not automatically establish compromise.

For LUKS2, a TPM-backed enrollment can be one of several unlock methods. The systemd-cryptenroll reference describes both TPM policy options and recovery keys. Such an enrollment is

a change to a real encrypted volume and is **not** part of this lab. A separately protected, tested recovery credential and documented recovery procedure are essential before an operator depends on automated unlock.

Read-only availability check

If the guest has a TPM device exposed, Linux commonly shows an entry under `/sys/class/tpm`:

```
if test -d /sys/class/tpm; then
  ls -la /sys/class/tpm
else
  printf 'No TPM class directory is exposed\n'
fi
```

An empty or missing directory is an observation about this environment, not proof about the physical host's hardware. A virtual TPM, if configured, is a separate virtual device with its own persistent state and recovery implications. Do not attach a new TPM to the existing lab or assume the ordinary VM disk clone includes its state.

On a system whose installed `systemd-analyze` supports it, the following can provide additional TPM availability information:

```
systemd-analyze has-tpm2
```

Record the output **and exit status**; the command can report limited support or return nonzero. If the subcommand is unavailable, use the installed manual and record the limitation. TPM availability does not prove that Secure Boot is enabled, that a PCR policy exists or that disk unlock uses it.

Worked diagnosis: update followed by an unlock prompt

A host normally unlocks automatically; after an update it requests a recovery credential. The administrator should:

1. Preserve the console message and identify the host and encrypted volume without exposing the credential.
2. Use the established recovery credential through the trusted console, if the recovery plan authorizes it. A prompt is not a reason to reformat the disk.
3. Record what changed: firmware, trust databases, loader, kernel, initramfs, UKI or unlock policy. Confirm which components actually booted.
4. Inspect the existing enrollment and policy using the platform's documented tools and authorized procedure. Compare relevant measurement evidence with the expected policy; do not assume a universal PCR set.
5. Plan and test a policy update with a retained recovery path and an explicitly verified subsequent boot.

A valid recovery unlock demonstrates access through that credential; it does not by itself prove why the automatic policy failed. Policy changes, key rotation and signed artifacts require the actual platform's procedures and are developed with storage and security operations later.

Practice and recall

Record the guest's EFI interface, `mokutil` result or observation limit, boot path evidence, TPM availability result and whether this lab actually uses encrypted root. For each item state what it proves and one thing it does not. Keep firmware-variable dumps, device identifiers and recovery material out of shared notes.

- A UKI packages resources; trust depends on the verifier and policy.
- Secure Boot verifies selected executables, while measured boot records state for later evaluation.
- TPM availability does not establish an unlock policy.
- A legitimate update can change measurements; keep a tested recovery path before relying on automatic unlock.
- Inspect the rejected stage and preserve evidence before changing trust or encryption settings.

Rescue, Emergency and External-Media Recovery

A host stops during boot and offers a shell. Before typing a repair command, identify **which system is running** and **which stage failed**. A shell in an initramfs, a systemd emergency shell and a shell from external installation media can have different roots, tools, credentials and access to the installed disk.

This chapter chooses a recovery environment from the observed failure and prepares a disposable guest for the next chapter's controlled repair. You need `bl02`'s firmware and storage record, a verified Ubuntu Server 26.04 installer ISO, console access, and enough host space for a full clone. Keep the original guest and its recovery baseline unchanged. This is an exercise on a clone only; a real incident may require preserving additional evidence before any write.

Identify the failed boundary

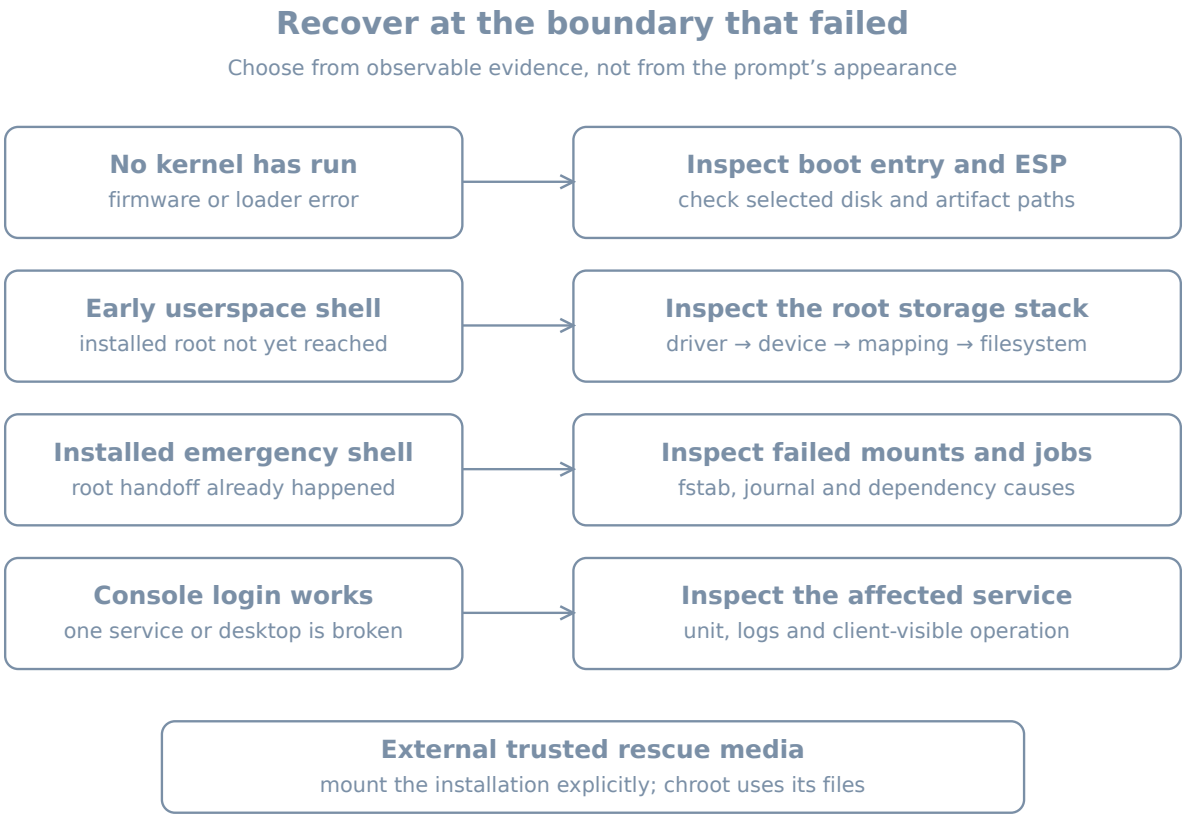


Figure 1: Recover at the boundary where startup stopped.

Last reliable observation	Likely boundary to investigate	Useful evidence
Firmware cannot find a boot option	Firmware, device or EFI path	VM disk attachment, boot entry, ESP identity
Bootloader appears but cannot load an image	Loader configuration or boot filesystem	Selected menu entry and exact error
Kernel starts but root cannot be found	initramfs, device discovery or storage stack	Console message, root argument, storage record
Installed manager reports a failed required mount	Installed root, mount intent or device	Console, fstab and actual filesystem identity
Login works but a service fails	Service or later dependency	Unit status, logs and client test

This table narrows an investigation; a single message does not prove a root cause. Preserve the exact wording, selected VM, timestamp, recent changes and boot entry. A retry or forced reset can hide the first failure. Never format a disk, reset encryption, rewrite all boot variables or disable verification to silence a symptom.

Know which root and manager you are using

Rescue mode is a reduced installed-system environment intended for maintenance; **emergency mode** is more minimal and can be reached when essential startup work fails. Their exact mounts and available authentication depend on the failure and distribution. The systemd special-unit reference describes the targets. An Ubuntu installation may have a locked root password, so an emergency login prompt is not necessarily usable with the ordinary `learner` password. Plan an external route rather than assuming a shell will be granted.

An **initramfs shell** appears earlier, before the installed root has necessarily been mounted. It uses the early environment's tools and device view. An **external live environment** boots its own kernel and root from trusted media; the installed system is a separate disk to identify and mount. `uname -r` in that environment describes the live kernel, not the installed kernel.

A shell prompt does not prove that `/` is the installed root, that it is writable, or that its separate `/boot` and ESP are mounted. Establish that explicitly with `findmnt`, `lsblk`, the VM definition and the installation record. A chroot changes the root path used by a process; it does not boot the installed kernel or give it an independent service manager.

Prepare the controlled fault

On the host, shut down `bl02` cleanly and verify its disk and state using `virsh` as in the prior clone exercises. The following assumes one writable system disk, no backing chain, shared writable storage or TPM state. Verify adequate host space and that both destination names are unused:

```
sudo virsh --connect qemu:///system domstate bl02
sudo virsh --connect qemu:///system domblklist bl02 --details
sudo virt-clone --connect qemu:///system \
```

```
  --original bl02 --name bl03-rescue-lab \
  --file /var/lib/libvirt/images/bl03-rescue-lab.qcow2
```

Only clone after the state reports `shut off`. Compare the original and clone domain XML and disk paths, including distinct UEFI NVRAM files. Attach the clone's NIC to `blpractice` while it is off; no network access is required for the fault. Keep the original off because the cloned guest retains its internal hostname, machine ID and credentials. Use the manager's `bl03-rescue-lab` console and confirm its disk path before proceeding.

Inside the clone, ensure the reserved lab directory and backup file do not exist:

```
sudo test ! -e /mnt/bl-fault && sudo test ! -e /etc/fstab.bl03-before
```

The combined check must succeed. If either fails, stop and inspect rather than overwriting prior work. Choose a deliberately nonexistent filesystem UUID and verify no matching filesystem is reported:

```
sudo blkid -t UUID=11111111-2222-4333-8444-555555555555
```

No output with exit status 2 means `blkid` found no matching device in this view. Any matching output or another error means stop and choose a new unused UUID after checking it. This check does not make the string a general-purpose identifier; it merely bounds this one guest's exercise.

Create the empty mountpoint and a preserved copy of the current configuration:

```
sudo mkdir /mnt/bl-fault
sudo cp -a /etc/fstab /etc/fstab.bl03-before
sudo cmp /etc/fstab /etc/fstab.bl03-before
```

Proceed only after the comparison succeeds. Append exactly one required mount entry with a short device wait:

```
printf '%s\n' 'UUID=11111111-2222-4333-8444-555555555555 /mnt/bl-fault ext4 defaults,x-syste
```

Review `sudo tail -n 3 /etc/fstab` and compare it with the backup. The intended difference is one new line; the system's existing root and boot entries must remain unchanged. The line requests a filesystem that does not exist, and deliberately omits `nofail`. The `x-systemd.device-timeout=` option controls the device wait when interpreted by systemd's fstab generator; see the systemd mount documentation. The directory contains no useful data and is not a root or boot filesystem.

If the file differs in any other way, restore the backup **while the clone is still running**, verify it and stop. Do not run `mount -a`: the intended experiment is the next boot, and mounting all configured filesystems is broader than necessary.

Observe the fault and choose recovery

With the clone console open, reboot normally. Allow time for the device timeout and observe the exact message and resulting mode. A typical systemd installation enters emergency handling

for the missing required mount, but distribution policy and boot configuration can differ. Record the result instead of assuming the prompt is usable. Do not enter credentials into an unidentified environment.

If the host reaches an ordinary login instead, inspect `systemctl --failed`, `findmnt --mountpoint /mnt/bl-fault`, and the current-boot journal from the clone. Reconcile the actual behavior with the appended line. In either case, the next chapter repairs the clone using verified external media, so a locked root account is not a blocker.

After recording the failure, shut down the clone through its available normal controls if possible. If it cannot respond to a normal shutdown, the disposable clone's manager power-off may be used as a last resort after recording that fact; never apply that step to another guest. Wait for `shut off`. Leave its disk attached and keep `bl02` off. The next chapter continues from this stopped, deliberately faulty guest.

Practice and recall

Record the selected clone and disk, the saved fstab path, the fault line, the observed console boundary and whether a normal shutdown succeeded. Explain why the installed root password might not open an emergency shell and why external media can still offer a separate recovery environment.

- Identify firmware, loader, early root, installed mounts and service failures separately.
- A rescue shell's root and kernel depend on where it came from.
- Preserve the fault and recovery evidence before making further changes.
- Keep a known-good original and a verified external route when testing boot failures.

Offline Inspection, Chroot Repair and the Next Normal Boot

The previous exercise added a nonexistent required mount to `bl03-rescue-lab`. The installed root may be intact even if its boot stops at emergency handling. This chapter boots trusted external media, identifies the clone's installed root, restores the saved fstab inside a limited chroot, and verifies the next normal boot.

Only the deliberately faulty clone is a repair target. The original `bl02` stays shut down. You need its storage record, the clone's unique disk path and NVRAM record, the verified Ubuntu Server 26.04 amd64 ISO, VM console access and the fault record. This lab assumes the unencrypted, single-disk guided LVM root from Part 02. If your clone has another layout, use its own documented recovery procedure rather than guessing devices or passwords.

Boot a separate environment

Confirm on the host that the clone is off and that its writable disk is the recorded clone disk:

```
sudo virsh --connect qemu:///system domstate bl03-rescue-lab
sudo virsh --connect qemu:///system domblklist bl03-rescue-lab --details
```

Attach the verified installer ISO as a read-only optical device to this clone. Keep the single system disk attached. Boot the ISO using the VM manager's one-time boot selection and open the console. Do not begin a new installation or accept storage formatting. On Ubuntu Server's live installer, switch to an available installer shell according to its console instructions. The shell runs the live system, not the installed guest. If the ISO or console is not the verified one, stop.

In the live shell inspect:

```
cat /etc/os-release
uname -r
findmnt --target / --output TARGET,SOURCE,FSTYPE
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
sudo pvs
sudo vgs
sudo lvs -o lv_name,vg_name,lv_path,lv_size
```

The live kernel release and live `/` do not describe the installed guest's running kernel or root. Match the disk size, volume-group and logical-volume names to the recorded clone. If the expected LVM tools or logical volume are unavailable, or multiple volumes make the target ambiguous, stop. Activation of a volume group is a separate operation. If the recorded root LV is present but

inactive, and only the verified clone disk is attached, identify its exact volume-group name in `vgs`. Set `VG_NAME` to that observed name in the live shell and activate only that group:

```
printf '%s\n' "$VG_NAME"
test -n "$VG_NAME" && sudo vgchange -ay "$VG_NAME" &&
  sudo lvs -o lv_name,vg_name,lv_path,lv_attr
```

Stop if the name is empty or ambiguous, duplicate volume groups are present, or the activation fails. Do not activate every volume group indiscriminately.

Mount the verified installed root

The goal is to restore one known fstab backup on the root filesystem. No separate `/boot`, ESP or pseudo-filesystem mount is needed for this specific repair.

First ensure `/mnt/bl-target` does not already exist in the live environment:

```
sudo test ! -e /mnt/bl-target
sudo mkdir /mnt/bl-target
```

Run `mkdir` only if the test succeeds. From `lvs`, select the **actual** root LV path that matches the lab record. Assign that observed absolute path to `ROOT_DEVICE` in the live shell; for example, `ROOT_DEVICE=/dev/mapper/observed-name` is only a shape, not a value to copy. Check it before mounting:

```
printf '%s\n' "$ROOT_DEVICE"
test -b "$ROOT_DEVICE"
sudo lsblk -f "$ROOT_DEVICE"
```

Stop if the variable is empty, the device check fails or its identity is not the recorded clone's root. The mount below is intentionally writable because the next step repairs the clone:

```
sudo mount "$ROOT_DEVICE" /mnt/bl-target
findmnt --mountpoint /mnt/bl-target --output TARGET,SOURCE,FSTYPE,OPTIONS
sudo cat /mnt/bl-target/etc/os-release
sudo tail -n 4 /mnt/bl-target/etc/fstab
sudo ls -l /mnt/bl-target/etc/fstab.bl03-before
```

Require an exact mount of the intended device, the expected Ubuntu installation and the deliberate fault line plus backup file. If the evidence differs, unmount and investigate; do not restore a file into a directory on the live root by mistake. If inspection alone is needed in a real incident, use an appropriate read-only procedure first and account for filesystem-specific recovery behavior. This planned lab write is not a general instruction to mount damaged production filesystems read-write.

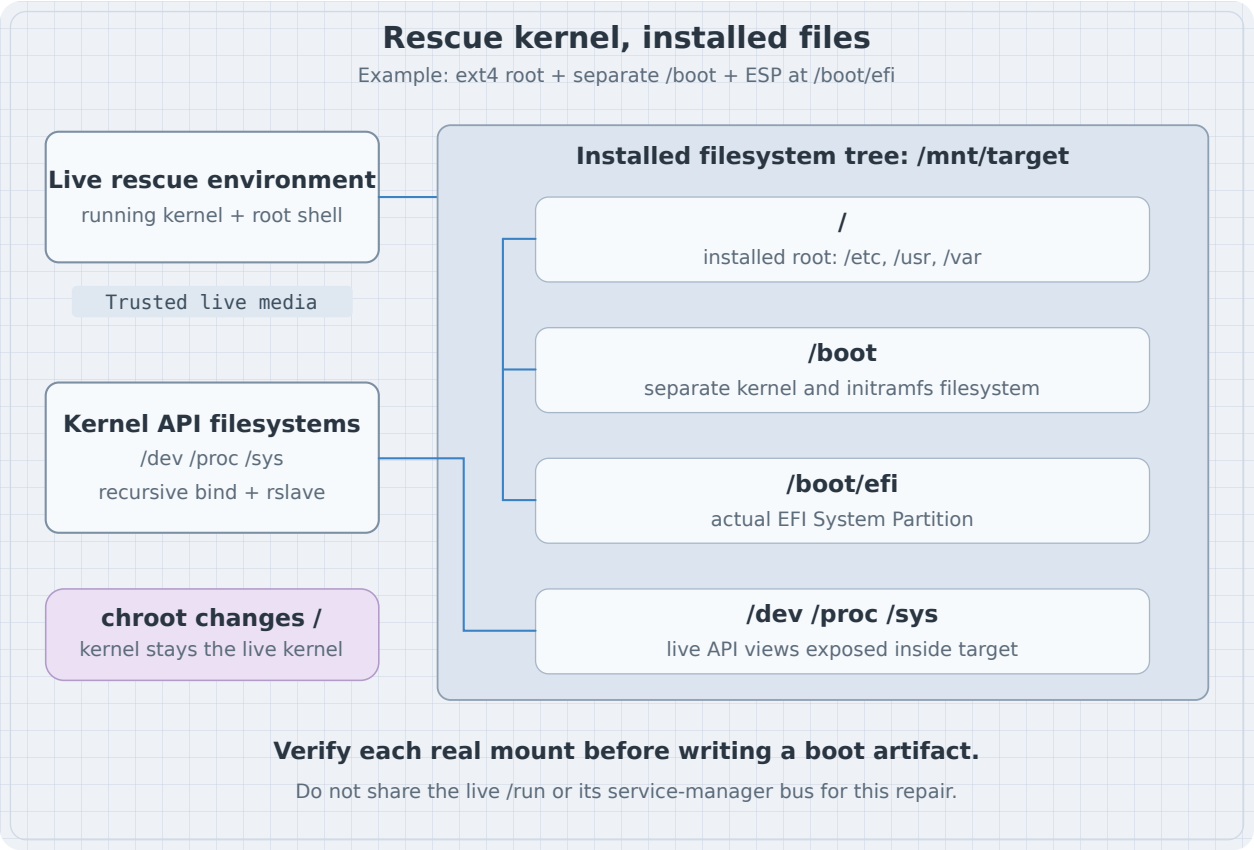


Figure 1: External media and a chroot have different roles.

Repair the specific file

Review the saved backup and fault:

```
sudo diff -u /mnt/bl-target/etc/fstab.bl03-before /mnt/bl-target/etc/fstab
```

The expected difference is the one deliberately appended mount line. `diff` exits with status 1 for differences. If unrelated changes appear, stop and understand them; restoring an old copy could discard legitimate configuration.

Enter a chroot rooted at the mounted clone:

```
sudo chroot /mnt/bl-target /bin/bash
```

The shell now resolves `/etc/fstab` within the installed filesystem, but uses the **live kernel**. It has not booted the installed system, and normal systemd service operations are not available merely because paths changed. The chroot manual describes this changed root directory.

Inside the chroot, verify the fault and backup once more, then restore the lab's saved file:

```
hostname
cat /etc/fstab
ls -l /etc/fstab.bl03-before
cp -a /etc/fstab.bl03-before /etc/fstab
cmp /etc/fstab.bl03-before /etc/fstab
exit
```

The `hostname` command reports the live kernel's hostname, not authoritative proof of the installed guest's hostname. The exact mounted source and inspected files were the target checks. `cp -a` restores the saved file and its attributes; `cmp` should be silent and successful. `exit` returns to the live shell. Do not continue into package, GRUB or initramfs repair: those require additional correctly mounted filesystems and a different plan.

Outside the chroot, verify the target mount and repaired file again:

```
findmnt --mountpoint /mnt/bl-target --output TARGET,SOURCE,FSTYPE
sudo cmp /mnt/bl-target/etc/fstab.bl03-before /mnt/bl-target/etc/fstab &&
  sudo umount /mnt/bl-target
```

Unmount only after leaving the chroot and ensuring no process is using that tree. If unmount fails, inspect the busy mount and processes; do not detach the disk or force a shutdown while writes may still be in progress. The findmnt manual distinguishes an exact mountpoint from the filesystem containing a path.

What a broader chroot repair would require

A bootloader or initramfs repair may need the installed `/boot`, ESP and kernel interfaces such as `/dev`, `/proc` and `/sys` mounted in the right places. Each must be identified and mounted

deliberately; bind-mount propagation, cleanup order, architecture, installed kernel version, package ownership and EFI variables all matter. Do not use the live kernel's `uname -r` as the installed release. Do not share the live `/run` blindly. Firmware variables exposed to the live environment are those of this VM and can be changed by repair tools.

The diagram shows those possible relationships, not an instruction to mount all of them for the fstab exercise. Chroot is not a security boundary or an independently running VM. Restore from a known-good copy when that is the selected plan; use a separate verified backup if the original storage may be damaged. Recovery media and VM clones on the same host do not constitute an independent backup.

Boot normally and verify

Shut down the live environment cleanly. In the VM manager detach the installer ISO, retain the clone disk and restore its normal disk boot selection. Confirm the original `bl02` remains off. Start `bl03-rescue-lab`, log in through its console and inspect:

```
hostnamectl
findmnt --target / --output TARGET,SOURCE,FSTYPE,OPTIONS
systemctl --failed --no-pager
sudo cmp /etc/fstab.bl03-before /etc/fstab
findmnt --mountpoint /mnt/bl-fault
```

The hostname may still say `bl02` because this is a clone. The manager's name and disk path distinguish it. Success requires normal login, the recorded root source, a successful `cmp`, no unexplained failed units, and **no exact mount** at `/mnt/bl-fault`. The last command normally returns no output and a nonzero status when no such mount exists; that is expected here. Also inspect the current-boot journal if a failure remains. A repaired file alone is not proof that the normal boot path works.

Remove the now-unused empty `/mnt/bl-fault` directory from the clone with `sudo rmdir /mnt/bl-fault` only after confirming it is not a mount and contains no files. Retain the backup and notes until the exercise is reviewed. Shut down the clone before starting the original. Do not delete or reformat a disk as part of cleanup.

Recall card

- External media runs its own kernel and root; identify the installed disk separately.
- A writable mount is a deliberate repair action, not proof that a filesystem is healthy.
- Chroot changes path resolution; it does not boot the target system.
- Mount only the components needed for the planned repair and unmount them cleanly.
- The acceptance test is the next normal boot plus the expected root and service state.

Users Identity and Administrative Access

Local Identities and Account Lifecycle

UIDs, GIDs, Groups and Ownership

A file is owned by `learner`, but another process cannot read it. The name printed by a tool is only one part of the answer. Linux checks numeric process credentials against numeric file ownership and permission rules. To explain an access decision, identify the process, the file, its containing directories and the operation being attempted.

This chapter starts the local identity model using the `bl02` console. Its lab creates a small private directory in the current user's home; it does not change other accounts or system files. You need a working `learner` login and the ability to inspect files and run commands.

Names describe numeric identities

A **user ID** (UID) identifies a user to the kernel. A **group ID** (GID) identifies a group. Files record an owner UID and group GID; processes have user and group credentials used in access checks. Tools commonly translate the numbers into names for display. If a name lookup is unavailable, the number can still exist on the file.

The special UID 0 has extensive administrative privilege, but a process's actual authority also depends on capabilities, namespaces and other restrictions. Do not assume that a username containing `admin`, or membership in a group with a familiar name, automatically grants every operation.

```
id
id -u
id -g
id -G
```

`id` shows the current process's identity and group list. `-u` gives the effective UID, `-g` the effective primary GID, and `-G` the group IDs. A process can also have supplementary groups. The account database's current membership and the credentials already held by a running process can differ after an administrative change: start a new session and inspect its actual credentials.

Read ownership and permissions

Create a private lab directory and file:

```
umask 077
mkdir "$HOME/bl-identity-lab"
```

If the directory already exists, stop and inspect it instead of overwriting its contents. Only after `mkdir` succeeds, create and inspect the file:

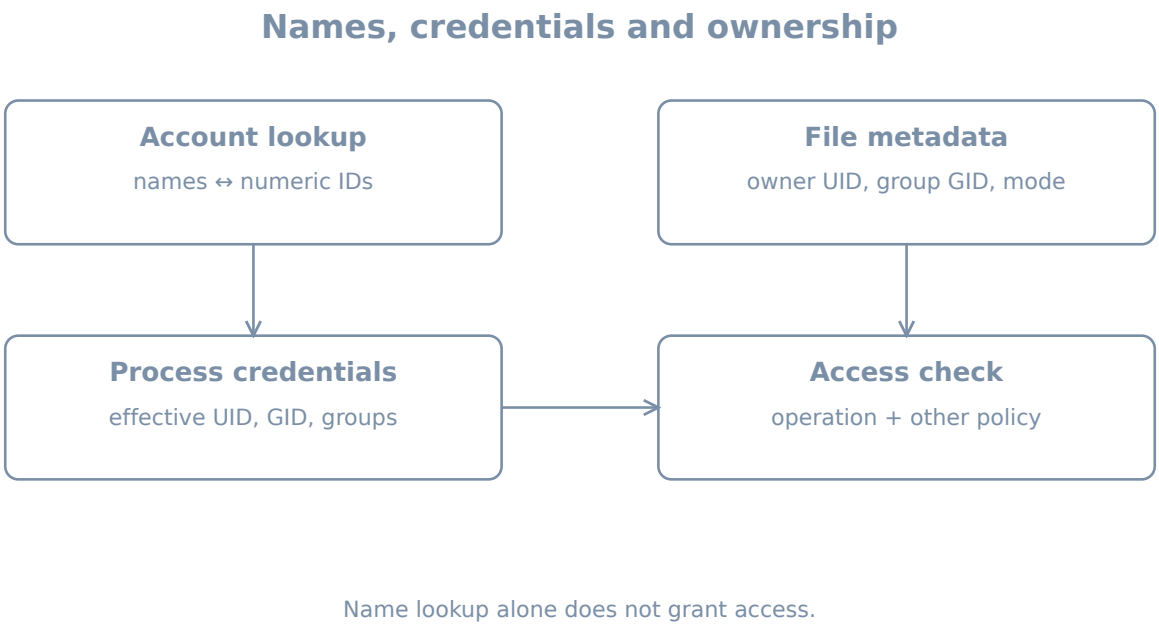


Figure 1: Process credentials and file ownership meet at an access check.

```
printf 'identity lab\n' > "$HOME/bl-identity-lab/note.txt"
ls -l "$HOME/bl-identity-lab/note.txt"
ls -ln "$HOME/bl-identity-lab/note.txt"
stat -c 'uid=%u gid=%g mode=%a name=%n' "$HOME/bl-identity-lab/note.txt"
```

`umask` affects newly created objects in this shell; it does not change existing permissions. The named and numeric `ls` views describe the same file. `stat` prints the stored numeric owner and group, plus the permission mode in octal.

For a simple regular file with mode `0640`, the owner has read and write, the group has read, and others have no permission. For directories, the same letters have different operational meanings: read permits listing names, write permits changes to entries subject to other rules, and execute permits traversal. Access to a file also requires traversal through each parent directory. A readable file behind an inaccessible home directory is not necessarily readable by another user.

A normal mode-bit check considers the applicable owner, group or other class; access control lists and other policies can add rules. Privileged processes and capabilities can alter outcomes. Deeper permissions and policy are covered later; this first model is enough to avoid assuming that a displayed username is the entire authorization rule.

Primary and supplementary groups

The primary group is part of a process's credentials and normally influences the group of newly created files. A parent directory with the set-group-ID bit or filesystem policy can change the

resulting group. Supplementary groups provide additional group memberships for access checks.

Inspect both a process and an account lookup:

```
id
id learner
getent passwd learner
```

The first command reports this process's credentials; `id learner` asks for the named account's identity and group information, which may differ from an already running session. `getent` queries the configured name-service database. These reports do not themselves grant or revoke access.

A file's stored owner is a number. If an account is deleted and that UID later assigned to another account, old files can appear owned by the new account. This is why account retirement must include an inventory and ownership decision; deleting a username does not erase every file created by that user.

Worked diagnosis: group was added but access still fails

An administrator adds a user to a group. The user keeps an old terminal open and still gets permission denied. Possible causes include stale process credentials, missing directory traversal permission, a different group owning the file, an ACL or another policy.

1. In the failing session run `id`; record its effective IDs and supplementary groups.
2. Inspect the intended file and every relevant parent directory with `ls -ld` and `ls -ln`.
3. Start a new authorized login session and run `id` again; a database change does not rewrite the credentials of existing processes.
4. Repeat the specific operation and record the result. If it still fails, inspect the applicable ACL and security policy rather than making the whole tree world-writable.

Do not use `sudo` as the test of ordinary-user access: it tests a different process identity. Likewise, changing a group record does not automatically terminate existing processes or sessions.

Practice and recall

Compare the named and numeric owner/group of `note.txt` with `id`. Explain why changing a displayed username would not itself change the file's stored UID. Verify the directory and file modes, then remove only the lab file and empty directory when done:

```
rm "$HOME/bl-identity-lab/note.txt"
rmdir "$HOME/bl-identity-lab"
```

`rmdir` refuses a nonempty directory. If it fails, inspect the contents instead of deleting unrelated files.

- Processes use numeric credentials; names are lookup results.
- Files store numeric owner and group identities.

- Existing sessions retain their own group credentials until replaced or otherwise changed.
- File access also depends on directory traversal and other policy.

Account Databases, NSS and Identity Lookup

A user is visible to `getent`, but no matching line appears in `/etc/passwd`. That is possible when the configured name service obtains an account from another source. Conversely, finding a local record does not prove that the account can authenticate or access a particular file.

Use `bl02` and the working console account. This chapter only reads local configuration and identity results. It does not connect the lab to a directory service or modify authentication policy.

Local records have different purposes

`/etc/passwd` contains local account information such as login name, numeric UID, primary GID, home directory and login shell. It is commonly readable by local users because programs need identity metadata. The password field commonly contains a placeholder indicating that password data is stored separately; do not infer that an account has a usable password merely from that placeholder.

`/etc/group` records local groups and may list supplementary members. A user's primary group is specified in the account record and may not appear in the group's member list. Protected `/etc/shadow` holds local password-related data and aging fields. It is sensitive: do not print, copy or publish its hashes for this exercise. Additional mechanisms may provide identity or authentication without a local shadow entry.

Inspect your own local records and the system's configuration:

```
getent passwd "$(id -un)"
getent group "$(id -g)"
rg '^passwd:|^group:|^shadow:' /etc/nsswitch.conf
```

If `rg` is unavailable inside the guest, use:

```
grep -E '^(passwd|group|shadow):' /etc/nsswitch.conf
```

`getent` uses the Name Service Switch (NSS), the library mechanism that consults configured sources. The getent manual describes the supported databases. `/etc/nsswitch.conf` controls source order and can include action clauses; its manual defines those rules. Do not change NSS configuration in this chapter.

A passwd-style result has colon-separated fields:

```
name:password-field:UID:GID:comment:home:shell
```

Names and values are examples of field roles, not values to paste into account files. The database reports identity metadata; an actual login also involves authentication, account policy, session setup and the availability of the home directory and shell. Those mechanisms are developed later.

A lookup is not the whole login

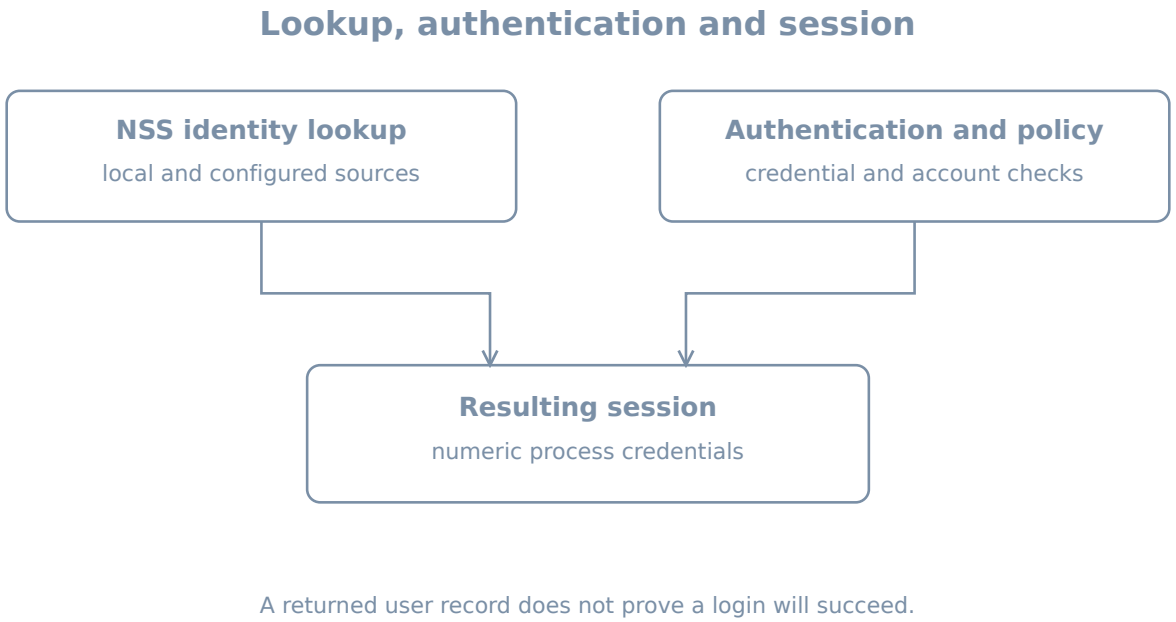


Figure 1: Name lookup and authentication answer different questions.

A successful `getent passwd learner` establishes that this lookup returned a record. It does not establish which credentials are accepted, whether the account is expired, whether the shell can run, or whether a particular service permits login. Conversely, an authentication failure does not automatically mean that identity lookup failed.

Compare the local source with the effective lookup:

```
grep '^learner:' /etc/passwd
getent passwd learner
id learner
```

On this lab's local account, expect a local record and a successful effective lookup. If either is absent, check the selected guest and actual account name before changing anything. A local line and `getent` may be textually equal without proving that no other source contains a duplicate name. Source ordering and status actions matter.

For groups, compare:

```
grep '^learner:' /etc/group
getent group "$(id -g)"
id learner
```

The private primary group may have the same name as the user, but that convention is not guaranteed. A group membership list in `getent group` is not a complete substitute for `id learner`, because the latter also accounts for the primary group and lookup behavior.

Worked diagnosis: a lookup fails

Suppose `getent passwd example` returns no entry. First verify the spelling and guest identity. Then inspect the `passwd:` source order, query the known local file, and record the command's exit status. A remote source, if configured, introduces service and network dependencies; enumeration may be unavailable even when a lookup by key works.

Do not add a duplicate local account or change NSS source order to hide the failure. A reused name or UID can attach existing files to the wrong identity. Determine the authoritative identity source and its health before making a change. Later identity and networking chapters cover directory integration once those dependencies have been introduced.

Practice and recall

Record the source order for `passwd` and `group`, your own effective lookup and the numeric IDs from `id`. Explain why the primary group need not list the username as a supplementary member and why successful lookup does not prove successful login. Redact account details before sharing the record.

- Local account files hold different kinds of data; shadow hashes are sensitive.
- NSS chooses identity sources; `getent` tests an effective lookup.
- Identity lookup, authentication, account policy and authorization are separate steps.
- A database change does not automatically replace credentials in existing sessions.

Account Creation, Locking, Expiration and Removal

An account is no longer needed. Locking its password, expiring the account, ending its sessions and deciding what happens to its files are separate actions. A safe lifecycle starts with a unique identity and ends with a verified disposition of access and data.

This lab uses a **disposable full clone** of `bl02`, named `bl04-accounts-lab`, with a separate disk and UEFI NVRAM. Use the powered-off `virt-clone` procedure from the bootloader chapter, confirm the original is off and the destination name and disk path are new, and attach the clone to `blpractice`. The clone retains the original guest's hostname and credentials; identify it by its manager name and disk path and never run it concurrently with `bl02`. Keep the existing `learner` console and administrative access throughout. Do not run the account changes on a personal host or shared machine.

Plan the identity and its scope

The exercise creates one local user `blstudent` and one supplementary group `blreviewers`. This is a human-style test identity, not an administrative or service account. It has no deliberate sudo grant and no real data. Before creation, ensure both names are unused:

```
getent passwd blstudent
getent group blstudent
getent group blreviewers
test ! -e /home/blstudent
```

Each `getent` query should return no record and exit with status 2 for a missing key; the home-path test must succeed. Check each result separately. An error other than a missing key, an existing name, or a remote identity source you do not understand is a stop condition. Inspect `/etc/nsswitch.conf` as in the previous chapter; a local tool may not know every remote identity that could later appear. Choose a different lab name rather than reusing an existing identity.

Record the available console and baseline. Before real provisioning, also specify a unique numeric UID/GID policy, identity authority, ownership scope, expiration, credentials, approval and recovery owner. Default automatic UID allocation in this disposable lab is not a fleet-wide identity plan.

Create and verify

Ubuntu provides the interactive `adduser` tool; this lab uses explicit lower-level `useradd` options so the intended home and shell are visible. The useradd manual describes the options and notes

Debian-family preferences.

```
sudo useradd -m -U -s /bin/bash blstudent &&
  sudo passwd blstudent &&
  sudo groupadd blreviewers &&
  sudo usermod -aG blreviewers blstudent
```

The `&&` operators stop the sequence if a command fails. If it stops, inspect the partial state before retrying. `-m` requests a home, `-U` a same-named private group, and `-s` the login shell. Enter a unique disposable password at the interactive prompts; do not place it in the command line or record it in notes. `-aG` appends a supplementary group. Using `-G` without `-a` can replace the supplementary group list; the usermod manual documents that distinction.

Inspect the result:

```
getent passwd blstudent
getent group blreviewers
id blstudent
sudo passwd -S blstudent
ls -ldn /home/blstudent
```

Record the observed UID, primary GID, supplementary group and home ownership. Verify a new session from the `learner` console:

```
su - blstudent
```

Authenticate with the lab user's password. In the new shell run `id`, `pwd` and `exit`. The group should appear in the **new process** credentials and the home should be usable. If `su` fails, retain the error and inspect the account, password state, shell and local authentication policy; do not grant sudo or edit protected databases by hand. A new group entry is not proof that an existing session received the group.

Lock, expire and recover deliberately

A password lock disables that local password as an authentication method. It is not a universal account disable: an existing session or a different authentication method may still work. Account expiration is a separate account-policy input, and services can interpret policy differently. The passwd manual describes password lock and status.

With the `blstudent` session exited and the `learner` administrator session retained:

```
sudo passwd -l blstudent
sudo passwd -S blstudent
```

The status should indicate a locked password. From the ordinary `learner` shell, an attempted `su - blstudent` using the previously working password should fail under the lab's local policy. Do not run this test through `sudo`, which uses a different authorization path. If the test behaves differently,

preserve the evidence and stop rather than assuming the account has no access. Restore the prior usable password state for this known lab account:

```
sudo passwd -u blstudent
sudo passwd -S blstudent
```

Unlocking is appropriate here because the account had a working password before the lock. It is not a general instruction to unlock unknown accounts or create passwordless access.

Demonstrate an expired account with an explicit past date, then remove that expiration:

```
sudo usermod --expiredate 1970-01-02 blstudent
sudo chage -l blstudent
sudo usermod --expiredate '' blstudent
sudo chage -l blstudent
```

Read the displayed account-expiration field after each step. Expiration and password locking are distinct controls; neither command proves existing sessions have ended. These changes affect only the test user, never `learner`.

Revoke membership and retire the test user

Remove the test user's supplementary membership and check the database view:

```
sudo gpasswd -d blstudent blreviewers
id blstudent
getent group blreviewers
```

Previously running processes may retain old credentials. For a real departure, coordinate session termination, remote keys, tokens, scheduled jobs, application accounts and service ownership as applicable. Changing a local group list alone is not complete revocation.

Before deleting the lab account, make sure no `blstudent` session remains and record its UID and home path. Remove only the account record, leaving its home as evidence:

```
sudo userdel blstudent
getent passwd blstudent
ls -ldn /home/blstudent
```

The lookup should no longer return the local account; check its exit status. The numeric owner on the retained home illustrates that deleting a name does not remove every file. A different NSS source with the same name would complicate this result; stop if one appears. `userdel` may also remove the user's private group according to local policy. It does not establish that files outside the home, jobs or credentials elsewhere have been handled. The userdel manual describes its scope.

After confirming `blreviewers` has no other intended members or use, remove this lab-only group:

```
getent group blreviewers
sudo groupdel blreviewers
getent group blreviewers
```

Do not use forced deletion or reuse the old UID while retained files remain. Keep the home and its ownership record on this disposable clone until a deliberate cleanup or retirement. Shut the clone down before returning to the original guest. A real account removal requires an explicit data retention, ownership transfer and credential-revocation plan.

Diagnose and recall

If creation fails, distinguish an existing name, unavailable identity source, occupied home path and insufficient privilege. If a login fails, distinguish lookup, password state, expiration, shell and session policy. If access persists after revocation, inspect the **running process** credentials and other authentication paths. Preserve a separate administrator and console recovery path throughout.

- Create identities only after checking name, numeric ownership and source authority.
- Append supplementary groups intentionally; verify a new session's credentials.
- Password lock, account expiration and session termination are different controls.
- Deleting an account does not decide what happens to its files or external credentials.

Home Directories, Skeletons and Session Environments

An account exists and its password is accepted, but the user lands in `/` with a warning that the home directory cannot be entered. Identity lookup, authentication and the session environment have succeeded or failed at different boundaries. The home path in an account record is an intention; the directory, its ownership, its mount and the process environment are separate observations.

Use the disposable `bl04-accounts-lab` clone from the previous chapter, with `bl02` powered off and the clone on `blpractice`. Keep the `learner` console and administrative access. If the clone was retired, create a fresh full powered-off clone using the earlier single-disk, separate-NVRAM procedure. The guest hostname may still be `bl02`; verify the manager's clone name and disk path.

A record, a directory and an environment

The account record contains a home path and login shell. On a conventional system, the home holds a user's persistent files and personal configuration. Its location is not necessarily `/home/NAME`: it may be on another local filesystem or a network mount. `$HOME` is an environment variable inherited or set by session-starting software. It is not itself proof of the account record, a mounted filesystem or correct ownership.

Inspect the current session:

```
id
getent passwd "$(id -un)"
printf 'HOME=%s\nSHELL=%s\n' "$HOME" "$SHELL"
pwd
ls -ldn "$HOME"
findmnt --target "$HOME" --output TARGET,SOURCE,FSTYPE,OPTIONS
umask
```

`pwd` is the current working directory, which can differ from `$HOME`. `$SHELL` commonly identifies a configured or inherited shell, but does not prove which executable is running. A containing-filesystem query does not prove home is its own mount. Compare the numeric owner with `id -u` and consider parent directory traversal. The getent manual describes the effective identity lookup.

Avoid dumping the entire environment into shared notes: it may contain credentials, tokens, private paths or other sensitive values. A user's shell startup files can also change variables after login; a non-login command or service can receive a different environment. This chapter inspects values rather than teaching a separate shell course.

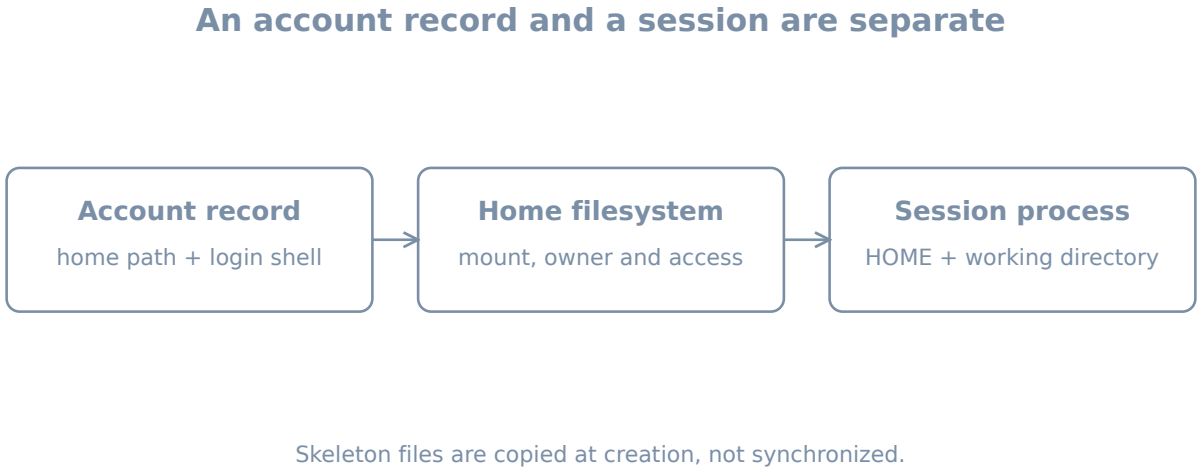


Figure 1: A home path and session are distinct observations.

Skeletons are initial copies

On Ubuntu, account-creation tools can populate a new home from a skeleton directory, commonly `/etc/skel`. These are initial copies, not a synchronization service: later changes to the skeleton do not automatically update existing users' homes. Tool options, pre-existing destination paths and distribution configuration affect the result. The Ubuntu adduser manual describes its account-creation behavior; `useradd` has its own options and defaults.

Inspect the skeleton without changing it:

```
ls -la /etc/skel
getent passwd learner
```

A dot-prefixed name is ordinarily hidden by a plain `ls`. Do not copy private credentials into a shared skeleton, and do not assume a skeleton file has the right settings for every user. Existing home contents may have been customized since account creation.

Lab: create and inspect a fresh home

Use a new lab name `blhome`. Check each condition before creating anything:

```
getent passwd blhome
getent group blhome
test ! -e /home/blhome
```

Both lookups should return no record with status 2, and the home-path test must succeed. An existing name, path or lookup error means stop. Consider configured NSS sources too, as in the account-lifecycle chapter.

Create the local user and set a unique disposable password:

```
sudo useradd -m -U -s /bin/bash blhome && sudo passwd blhome
```

If either step fails, inspect the partial state before retrying. Do not record the password in notes. Inspect the resulting account and directory:

```
getent passwd blhome
id blhome
sudo ls -ldn /home/blhome
sudo ls -la /home/blhome
sudo ls -la /etc/skel
```

Compare filenames and ownership; if a skeleton file is present in both places, compare a specific nonsecret file with `sudo cmp /etc/skel/FILE /home/blhome/FILE` after replacing `FILE` with the actual name. A comparison proves byte equality at that moment, not that the files remain synchronized.

Start a new session from the ordinary `learner` shell:

```
su - blhome
```

After authenticating, run `id`, `pwd`, `printf '%s\n' "$HOME"`, and `findmnt --target "$HOME"`. Confirm the account's observed home and process credentials, then `exit`. The login shell may source startup files; output and exact environment vary with configuration.

Diagnose a missing or inaccessible home

Suppose authentication succeeds but the home is unavailable. Check the account's **actual** home field with `getent passwd`, the mount containing that path with `findmnt`, and the directory plus parents with `ls -ldn`. Compare its numeric owner with the account's UID. A missing network mount, a stale UID, denied traversal and a nonexistent directory require different remedies. Do not recursively `chown` a guessed path, recreate a mountpoint over hidden data, or broaden permissions for all users.

For a real migration, plan the copy method, numeric identities, ACLs, extended attributes, capacity, consistency and the old and new home paths. Verify a fresh login and data access before retirement of the original. Changing the account record alone does not copy files.

For lab cleanup, exit every `blhome` session, record the UID and home, and remove only the account record with `sudo userdel blhome`. The home remains for inspection; it may display a numeric owner. Retain it on this disposable clone until a deliberate cleanup. Do not reuse the UID while its files remain. The original `learner` account must continue to work.

Recall card

- Account home, mounted filesystem, `$HOME` and current working directory are distinct.
- A skeleton supplies initial files; it does not update existing homes.
- Inspect numeric ownership and parent traversal before changing permissions.
- Test a new login session and preserve existing administrative access.

Authentication and Privilege

Passwords, Credential Storage and Policy

A correct-looking username does not guarantee a password login. A local password may be locked, expired or subject to account policy; a service may use another authentication method entirely. Inspect the relevant status without exposing password hashes.

Use the disposable `bl04-accounts-lab` clone and the retained `learner` administrator console. This chapter creates a separate `blpass` test user and leaves it for the next PAM exercise. Keep the original `bl02` shut down while the clone runs. Do not experiment with the only working administrator account.

Passwords are not stored as login text

A local system normally stores a password **verifier**, not the plaintext password. A verifier includes parameters and a salt that allow a candidate password to be checked. Salting makes identical passwords produce different verifier values; a stolen verifier can still be attacked offline. A password hash is therefore sensitive even though it is not the plaintext.

On this Ubuntu-style local-account setup, `/etc/shadow` contains protected password and aging fields. The shadow manual documents their semantics. Do not print or copy its contents into a ticket, book notes or a shared terminal recording. A locked password field prevents use of that UNIX password but does not automatically disable SSH keys, other credentials or already running sessions.

Use status tools to inspect a known local account instead:

```
sudo passwd -S learner
sudo chage -l learner
```

These report status and aging information; do not change `learner`'s password or aging for the exercise. `chage` reports local shadow information and may not describe policy applied by an external identity provider. The chage manual explains password and account expiration fields.

Separate the controls

Control	What it addresses	What it does not automatically do
Password change	Replaces a local password verifier	Revoke every other credential or active session

Control	What it addresses	What it does not automatically do
Password lock	Prevents use of that password verifier	Disable keys, tokens or every service
Password aging	Determines when a password must be changed	Define the account's full lifetime
Account expiration	Restricts account use according to the enforcing service	Terminate all running processes or revoke external identities
Authentication policy	Defines how a service checks a credential	Prove the user is authorized for every requested operation

Strong unique passwords, controlled enrollment, secure storage and recovery procedures all matter. Password policy applies to a specific identity source and login flow: local shadow settings do not define a remote directory's rules. Do not assume a particular hashing algorithm, cost or automatic-update policy from a distribution name; inspect the installed configuration and relevant package documentation when making a change.

Lab: create an account and observe its policy

First verify the new account and private-group names and home path are unused:

```
getent passwd blpass
getent group blpass
test ! -e /home/blpass
```

Check each result. Both lookups should return no record with status 2, and the path test must succeed. An existing identity, path or lookup error is a stop condition. Create the account and set a unique lab password interactively:

```
sudo useradd -m -U -s /bin/bash blpass && sudo passwd blpass
```

If either command fails, inspect partial state before retrying. Never pass the plaintext as an argument or store it in shell history. Confirm status without displaying the verifier:

```
getent passwd blpass
sudo passwd -S blpass
sudo chage -l blpass
```

From the ordinary `learner` shell, run `su - blpass`, authenticate using the new password, run `id`, and `exit`. This confirms the particular local `su` flow and session at that time, not every login service.

Set a bounded maximum password age for this disposable user:

```
sudo chage -M 30 blpass
sudo chage -l blpass
```

Record the displayed maximum and resulting dates. A policy change in `/etc/login.defs` can affect defaults for account creation, but is not evidence that existing records were updated; the login.defs manual documents the shadow suite's settings. PAM and a remote identity provider can apply additional policy. Do not edit global policy to perform this lab.

Keep `blpass` for the next chapter's controlled authentication failure. Its 30-day value is a teaching example, not a recommended universal rotation period. For real users, choose policy based on the environment, current guidance, credential type and incident recovery requirements.

Diagnose and recover a failed password login

1. Identify the service, exact user, time and observed message without recording the password.
2. Verify the account resolves through `getent` and determine whether it is local or externally managed.
3. For the local test account, inspect `passwd -S` and `chage -l`; distinguish locked password, password expiration and account expiration.
4. Check the service's authentication and account policy and the relevant logs. Do not reveal stored verifiers.
5. Use an authorized recovery process and retest the same login path. An administrator password reset is a credential change; protect the new secret and verify the user can use it.

Repeated failed attempts can trigger a configured lockout or alert. Avoid guessing credentials. Preserve the existing administrator session and documented console recovery route before changing authentication policy.

Recall card

- Verifiers, even salted hashes, require protection.
- Password locking, password aging and account expiration are separate controls.
- A successful local login test proves only that specific authentication flow.
- Inspect status and service policy without exposing secrets.

PAM Stacks and Diagnosing Login Failures

The same user can succeed at one login service and fail at another. Those services may invoke different PAM configurations, use different authentication methods or apply different account and session rules. A diagnosis starts with the actual service and its evidence, not a blanket edit to every login path.

Use the disposable `bl04-accounts-lab` clone and the `blpass` account created in the preceding chapter. Keep the working `learner` administrator console open. The exercise locks and unlocks only `blpass`'s previously working local password; it does not edit the PAM stack.

What PAM coordinates

Pluggable Authentication Modules (PAM) provide a framework through which a participating application asks configured modules to perform authentication-related work. A PAM **service name** selects its configuration, usually a file under `/etc/pam.d/`. The caller and configured modules determine the actual behavior.

Four common management groups are:

Group	Question
`auth`	Can the presented authentication method establish the claimed identity?
`account`	Is this account permitted under applicable account policy now?
`password`	How is an authentication token changed?
`session`	What setup and cleanup apply to a session?

The names describe module responsibilities; they do not imply that all applications invoke every group in the same way. Identity lookup can use NSS, and a service can also have non-PAM authorization rules. The PAM configuration manual defines service files, module types and control syntax.

A stack is ordered policy, not just a list of installed modules. Control fields determine how results affect evaluation; `required`, `requisite`, `sufficient`, `optional` and bracketed controls have different behavior. `include` and distribution-specific include syntax can refer to other files. Do not infer an entire login result from one matching line.

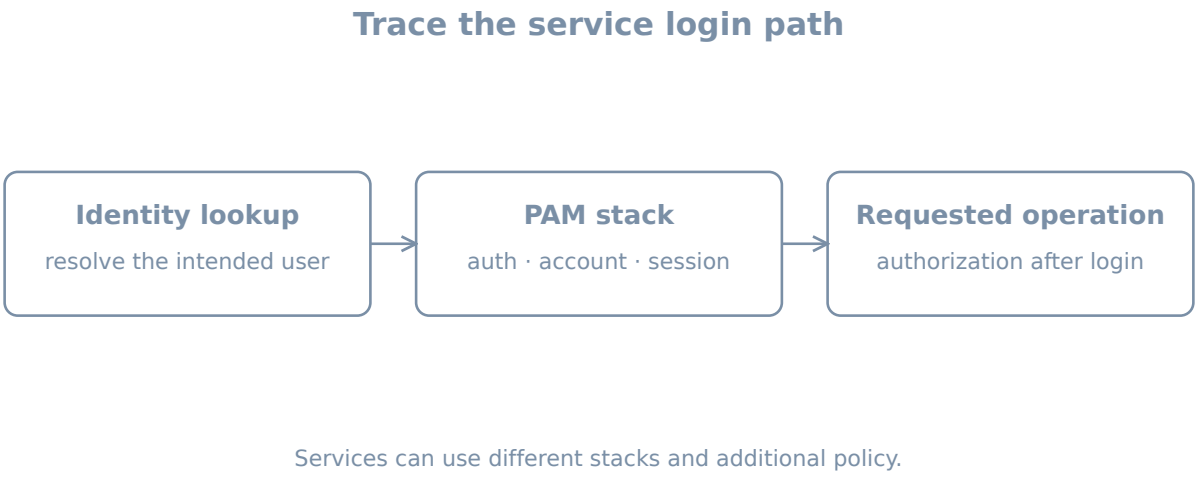


Figure 1: A login has several diagnostic boundaries.

Inspect the relevant service

For the lab's `su` path, inspect the installed files:

```
ls -l /etc/pam.d/su /etc/pam.d/common-*
sudo cat /etc/pam.d/su
sudo cat /etc/pam.d/common-auth
sudo cat /etc/pam.d/common-account
```

Files and includes can differ by installed provider and version. The actual `su` command may come from a different package on another system; inspect it with `command -v su` and `dpkg -S "$(command -v su)"` where applicable. A package lookup may not resolve a symlink in the way you expect; inspect the actual path before drawing conclusions.

Ubuntu's `pam-auth-update` manages packaged profiles and the common configuration files. Its manual explains how it treats local modifications. We do not invoke it here: changing a common stack can affect multiple login and privilege paths, and `--force` can overwrite local policy. A known-good console session and recovery method are essential before a real PAM change.

Lab: observe a bounded failure and recovery

First confirm the test account's known-good login from the ordinary `learner` shell:

```
getent passwd blpass
sudo passwd -S blpass
su - blpass
```

Authenticate with the test password, run `id`, then `exit` back to `learner`. Stop if the baseline login does not work. Do not use `sudo su` for the test: root's invocation follows different privilege and authentication circumstances.

Record the current time, then lock only the test password:

```
date -Is
sudo passwd -l blpass
sudo passwd -S blpass
```

Confirm the locked status. From the non-root `learner` shell, attempt `su - blpass` once with the known password. Expect failure under this local lab's policy. If it succeeds or behaves unexpectedly, exit the test session if one opened and investigate; do not keep trying passwords. Existing sessions and other authentication mechanisms can behave differently.

Inspect recent evidence without exposing credentials:

```
sudo journalctl -b --since '-5 minutes' --no-pager
```

Search the relevant time and service messages in the displayed output. Logging details and identifiers depend on the installed `su`, PAM modules and journal configuration. A message may be absent, and the journal can contain unrelated sensitive data; do not publish it unredacted. A failed attempt, local password status and PAM configuration are complementary observations, not a complete causal proof on their own.

Unlock this test account's previously working password and verify recovery:

```
sudo passwd -u blpass
sudo passwd -S blpass
su - blpass
```

Authenticate once, run `id`, and `exit`. Unlocking is justified because this exercise locked a known usable password. If the original status was unknown, do not blindly unlock it. The test demonstrates this service's behavior; it does not prove that SSH, a graphical login or another service has the same policy.

Follow a failure through the boundaries

For a real report, record the service, user, client, timestamp and exact failure without collecting the secret. Then distinguish:

1. **Identity:** does `getent passwd USER` resolve the intended account from the intended source?
2. **Authentication:** was the applicable credential accepted by the configured method?
3. **Account:** is the account locked, expired or restricted by policy?
4. **Session:** can the service establish the requested session, home and shell?
5. **Authorization:** is the resulting identity permitted to perform the requested operation?

Substitute the actual user in the lookup. A password prompt is not proof that the correct identity source was used; a successful password check is not proof that a session or later authorization succeeded. If the failure affects the administrator path, retain any working privileged console and use the documented recovery plan before editing configuration. Never comment out all PAM checks or paste an unreviewed stack from another distribution.

Leave `blpass` available for the following privilege exercise if needed, or retire it on the disposable clone using the account-lifecycle procedure after all relevant sessions end. Keep its UID and retained files in the record.

Recall card

- PAM policy is selected by service and evaluated as a stack.
- Authentication, account checks, session setup and authorization are distinct.
- Ubuntu common PAM configuration can affect multiple services.
- Test a reversible failure on a separate account and verify the same path after recovery.

Administrative Privilege and Bounded Delegation

An operator needs to run one diagnostic as root. Giving that operator an unrestricted root shell also grants the ability to change the policy that constrained the diagnostic. A useful delegation specifies the caller, target identity, exact executable and arguments, then tests both an allowed and a denied operation.

This lab uses `bl04-accounts-lab`, the existing `learner` administrator console and a new `bloperator` account. Keep `bl02` shut down while the clone runs. Make a known-good clone/baseline before changing policy. If you cannot establish a separate recovery route and a second working administrator session, do not edit sudo policy.

Identify the privilege provider and policy

`sudo` runs a command under a target identity subject to its policy and authentication rules. `su` switches identity using a different flow. Neither a locked root password nor a noninteractive root shell prevents a properly authorized administrator from obtaining root through another mechanism; actual policy matters.

Ubuntu 26.04 uses `sudo-rs` as the default provider, while the traditional implementation remains available. Existing installations can differ. Ubuntu's system utility reference explains provider differences. Inspect the current clone:

```
command -v sudo
readlink -f "$(command -v sudo)"
sudo --version
sudo -l
sudo id -u
```

The final command should print `0`. `sudo -l` describes policy for the current user; an administrator group is only one possible source of permission. Check the installed provider's `sudo --help` and `man sudoers-rs` or corresponding manual before using provider-specific features. Do not switch providers as part of this lab.

The policy may use `/etc/sudoers`, included files, and in sudo-rs an optional `/etc/sudoers-rs` that takes precedence when present. Inspect the actual configuration and whether it includes `/etc/sudoers.d`:

```
sudo ls -l /etc/sudoers /etc/sudoers-rs /etc/sudoers.d
sudo grep -nE '^[[:space:]]*(@includedir|#includedir)' /etc/sudoers
```

An absent optional file can produce an error. If `/etc/sudoers-rs` exists, inspect its include directives too. Do not assume a drop-in is active merely because its file exists. A later matching rule or group membership can broaden a grant, so review the effective policy. The sudo-rs policy manual documents includes and matching.

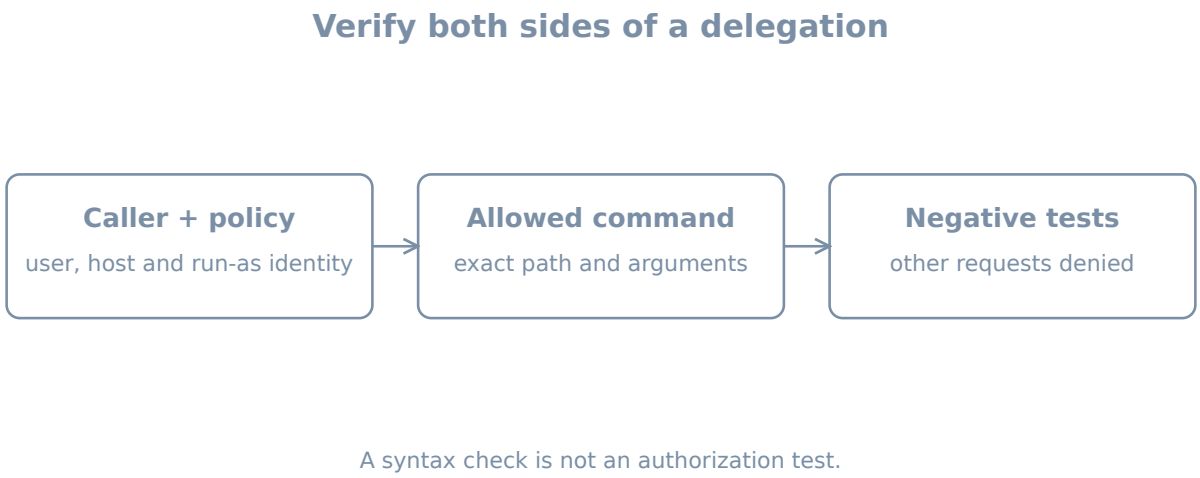


Figure 1: Test both the permitted operation and the boundary.

Create a dedicated test operator

Before creating `bloperator`, check the account, same-named group and home path independently:

```
getent passwd bloperator
getent group bloperator
test ! -e /home/bloperator
```

Require both lookups to return no record with status 2 and the path test to succeed. Stop for an existing name, path or lookup error. Create a local account with a unique disposable password:

```
sudo useradd -m -U -s /bin/bash bloperator && sudo passwd bloperator
```

If a command fails, inspect partial state before retrying. Verify `id bloperator` shows no administrator group, and inspect existing grants for this user from the administrator session:

```
id bloperator
sudo -l -U bloperator
```

A denial or absence of grants is expected before the lab rule. If the user already has broad effective privileges, stop and identify their source. Keep the `learner` session available separately throughout.

Delegate one diagnostic

The example permits only `/usr/bin/id` **without arguments** as root. It demonstrates a policy boundary; running `id` is not a privileged operational task in itself. Confirm the executable is the expected, root-owned system program and that untrusted users cannot modify it or its parent directories:

```
command -v id
ls -l /usr/bin/id
ls -ld /usr /usr/bin
```

Confirm that the active policy includes `/etc/sudoers.d`. If it does not, stop; do not modify a different policy file by guesswork. Ensure the lab drop-in is absent:

```
sudo test ! -e /etc/sudoers.d/bl04-observe
```

Only after that test succeeds, open it using the provider's checked editor:

```
sudo visudo -f /etc/sudoers.d/bl04-observe
```

Enter this single line and save:

```
bloperator ALL=(root) /usr/bin/id ""
```

The empty quoted argument specification means no command-line arguments; omitting it would allow arbitrary arguments to this executable under sudo-rs policy. `ALL` in the host position matches the host, while `(root)` is the target identity. The sudoers-rs manual describes the rule and no-argument syntax. This rule retains the default authentication requirement; `NOPASSWD` is not needed.

Inspect the installed file and validate the complete active policy:

```
sudo ls -l /etc/sudoers.d/bl04-observe
sudo visudo -c
sudo -l -U bloperator
```

The file should be owned by root and not writable by the test user or other untrusted users; use the installed provider's documented ownership and mode requirements. A syntax check is necessary but does not prove the intended permissions or resolve every semantic conflict. If validation fails or the rule is not active, stop and repair from the still-open administrator session or the documented recovery route. Do not close that session and do not use an unchecked editor to bypass a policy error.

Test allow, deny and rollback

From the ordinary `learner` shell, start `su - bloperator` and authenticate as the test user. In that shell:

```
sudo /usr/bin/id
sudo /usr/bin/id -u
sudo /usr/bin/touch /tmp/bl04-should-not-exist
```

Authenticate to sudo with the **operator's** password if prompted. The first command should report root's identity. The second must be denied because it has an argument; the third must be denied because it is a different executable. Stop if either denied test succeeds: another policy rule or group grants more authority than intended. Do not execute additional privileged commands to explore the grant.

A sudo timestamp can allow a later permitted invocation without another prompt; absence of a prompt alone is not proof of `NOPASSWD`. Exit the operator session. From the retained administrator console, remove only this verified lab drop-in and validate:

```
sudo rm /etc/sudoers.d/bl04-observe && sudo visudo -c
sudo -l -U bloperator
```

Reopen a fresh `bloperator` session and verify `sudo /usr/bin/id` is denied. If it is still allowed, inspect effective rules and group membership; do not declare the rollback complete. End the test sessions. Retire the test account using the earlier lifecycle procedure or keep it only on the powered-off disposable clone with its UID and home recorded.

For a real delegation, assess what the executable can do with its inputs, environment, configuration and child processes. Editors, shells and commands that execute arbitrary programs can make an apparently narrow rule equivalent to broad root access. Use a controlled, root-owned diagnostic interface and test the negative cases; syntax validation alone cannot establish safety.

Recall card

- Confirm the actual sudo provider and effective policy before editing.
- A policy specifies caller, host, target identity, executable and arguments.
- `visudo` validation, an allowed test and denied tests provide different evidence.
- Keep a separate administrator and recovery route until rollback is verified.

Service Identities

Service Accounts and Runtime Credentials

A background service needs to write its own state, but it does not need an administrator's interactive login. A dedicated identity limits which files a compromised or malfunctioning process can access and makes ownership visible. It is only one boundary: privileges, supplementary groups, filesystem policy, secrets and the service manager's configuration also matter.

This chapter creates a disposable local service identity and runs a short transient diagnostic under it. Use the `bl04-accounts-lab` clone on `blpractice`, with `bl02` shut down and the `learner` administrator console available. No network service or real secret is installed.

A process identity is not a human session

A **service account** is an identity assigned to a workload. It can own files and be selected by a service manager without being used for interactive login. A locked password and a `nologin` shell reduce ordinary password/shell login paths, but neither prevents an authorized manager or root from starting a process with that UID. They do not revoke SSH keys or other access mechanisms automatically.

A persistent service identity has a stable numeric UID/GID and can own persistent state. A dynamically allocated identity can reduce account management for suitable services, but numeric identity and ownership across restarts, backups and migrations require careful design. Do not copy an ephemeral UID into a different host and assume it represents the same principal. Systemd's execution environment reference documents `User=`, `Group=` and `DynamicUser=`.

A process can still have supplementary groups or capabilities. Verify the effective process context and the actual allowed operation, not just the account's shell. Service supervision, restart behavior and unit hardening receive their own treatment later.

Establish a bounded identity and state directory

Check the proposed user, group and state path individually:

```
getent passwd blsvc
getent group blsvc
test ! -e /var/lib/bl04-service-lab
```

Both lookups should return no record with status 2; the path test must succeed. Stop on any collision or lookup error. Confirm the actual noninteractive shell exists:

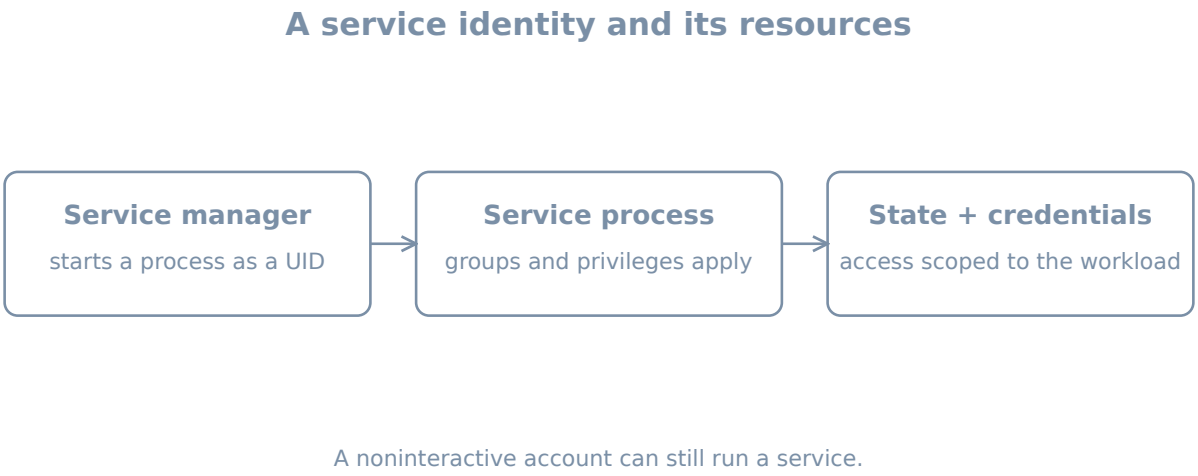


Figure 1: A service identity is one part of the workload boundary.

```
command -v nologin
ls -l /usr/sbin/nologin
```

For this lab use `/usr/sbin/nologin` only if that path exists on the clone. Create a system account with a private group and no home directory:

```
sudo useradd --system --user-group --no-create-home \
  --home-dir /nonexistent --shell /usr/sbin/nologin blsvc
```

Inspect the resulting identity and password status:

```
getent passwd blsvc
id blsvc
sudo passwd -S blsvc
```

Do not set a password or grant sudo membership. An account with no usable password is still a valid process identity. The recorded home path `/nonexistent` is an account-field value; this command does not create that directory. Do not create a home merely to make it resemble a human account.

Create only the new lab state directory, owned by this identity:

```
sudo install -d -o blsvc -g blsvc -m 0750 /var/lib/bl04-service-lab
sudo ls -ldn /var/lib/bl04-service-lab
```

Run this only after the earlier absence check succeeds. The mode grants owner access and group read/traversal; other access depends on parent directories and any additional policy. Record the numeric UID/GID, path and intended owner. A separate identity is useful only if the files and operations are scoped to it.

Run a transient workload and verify ownership

First ask systemd to run `id` as the service identity:

```
sudo systemd-run --collect --pipe --uid=blsvc --gid=blsvc /usr/bin/id
```

The output should show the `blsvc` numeric identity and group. `--pipe` connects the command's output and waits for it; `--collect` allows the transient unit to be cleaned up after it finishes. The systemd-run manual documents these options. This is an administrative test of service execution, not an interactive login as `blsvc`.

Now create a harmless file under the permitted state directory:

```
sudo systemd-run --collect --pipe --uid=blsvc --gid=blsvc \
  /usr/bin/touch /var/lib/bl04-service-lab/probe
sudo ls -ln /var/lib/bl04-service-lab/probe
```

Require a successful command and compare the file's numeric owner and group with `id blsvc`. If it fails, inspect the exact error, account lookup and directory traversal/ownership. Do not make the directory world-writable or run the workload as root to hide a permission failure. Check the manager's journal for the transient unit name printed by `systemd-run` if output does not explain the failure.

This test demonstrates creation of one file under one identity. It does not establish isolation from every other path or prove that a future service has correct dependencies, restart policy, resource limits or security configuration.

Handle credentials as runtime data

A workload may need a database password, private key or API token that is distinct from its UNIX identity. Treat the service account and that credential as separate assets with separate rotation, access, audit and revocation plans. Avoid putting secrets in a unit's command line, publicly readable configuration, logs or a shared environment dump.

Systemd supports service credentials such as `LoadCredential=` and encrypted credential mechanisms; the systemd-creds reference describes their role. The source file's protection, the service's ability to read the delivered credential, backup handling, rotation and revocation still need design. Passing a credential to a process does not protect it from that process or from a sufficiently privileged administrator. This chapter does not create or print a secret.

Retirement and diagnosis

Before retiring a real service identity, stop and disable the workload through its owner, verify no processes or timers still use it, inventory files and external credentials, retain or transfer state according to policy, revoke credentials, and only then remove the identity. A numeric UID can outlive its name on disk and later be reused.

For this lab, the transient commands have ended. Record the UID and `/var/lib/bl04-service-lab` ownership; leave the account and directory on the disposable clone until an explicit cleanup decision. Shut down the clone before restarting `bl02`. Do not delete a service account while a real workload is running.

If a service cannot start, distinguish: identity lookup failed; its configured user lacks path traversal; the required state mount is missing; a credential was not delivered; or the program itself exited. Inspect the unit's effective configuration and journal and repeat the intended operation after the narrow fix. Changing its user to root is not a diagnosis.

Recall card

- A noninteractive account can still run a service process.
- Stable ownership, supplementary groups and privileges determine the practical boundary.
- A successful transient test proves only the operation it performed.
- Runtime credentials have a lifecycle separate from the UNIX identity.
- Stop workloads and account for retained state before removing an identity.

Sessions and Central Identity

Sessions, logind and Authorization Beyond the Terminal

An unprivileged desktop user requests a system operation and sees an authentication prompt. That does not necessarily mean the application ran `sudo`. A privileged service can receive a request over an interprocess channel and ask an authorization authority whether the requesting process may perform a particular action.

This chapter examines sessions and polkit using the existing `bl02` console. The lab is read-only: it lists observed sessions and, if installed, registered authorization actions. It does not terminate sessions, change rules or trigger a privileged operation. A graphical desktop and an authentication agent are optional; a text-only server may expose less session information.

User, process, session and seat

A **user account** is an identity record. A **process** has current numeric credentials. A **session** is a grouping of activity associated with a login or similar interaction. One user can have more than one session, and a service can run under an account without being an interactive login session. A **seat** represents a set of local interaction devices; a remote session need not have a local seat.

On a system using systemd-logind, the login manager tracks sessions and user activity and coordinates some operations such as power and device access. It does not replace all process credential checks or service-specific authorization. The loginctl manual describes the management interface.

Inspect the current host:

```
hostnamectl
id
printf 'Session ID: %s\n' "${XDG_SESSION_ID:-not supplied}"
loginctl list-sessions --no-pager
loginctl list-users --no-pager
```

The environment variable is a hint supplied to some sessions, not a universal session detector. `su`, service processes, containers or an installation without an appropriate PAM/logind integration may not yield a distinct registered session. An empty list or inaccessible manager is an observation to explain, not proof that no process exists.

If `list-sessions` shows a session for the console, use its **observed** session ID in the following command, replacing `SESSION_ID`:

```
loginctl show-session SESSION_ID \
  -p Id -p User -p Name -p Type -p Class \
  -p Remote -p Active -p State
```

Do not substitute a username for a session ID. Values can be empty or unavailable, and `Active` is a session property, not proof that every process or service belonging to that user is healthy. The session ID is local runtime state; it can change after logout or reboot. Avoid `terminate-session`, `kill-session` and `lock-session` during this inspection.

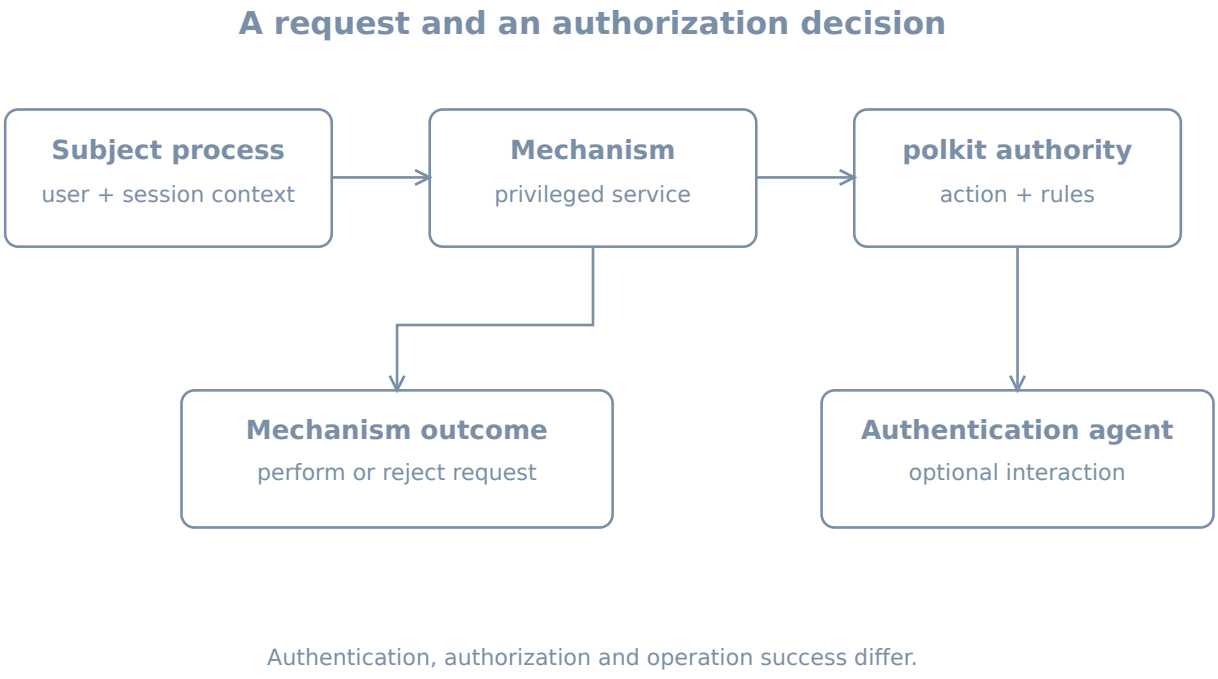


Figure 1: A service can request an authorization decision about a session's process.

How a polkit request is evaluated

polkit supplies an authorization interface for participating privileged services. In a common arrangement, an unprivileged **subject** asks a privileged **mechanism** to perform an action. The mechanism asks the polkit authority for a decision about that subject and named action. A policy can allow, deny or require authentication. When interaction is permitted, an authentication agent may prompt the user or an administrator.

The polkit reference describes subjects, mechanisms, actions, rules and agents. A policy decision depends on the installed rules, the specific action and subject, and potentially session context. The identity used for authentication can differ from the original requesting subject. Authentication demonstrates a credential check; authorization determines whether the operation is permitted.

`sudo` and polkit are distinct paths. A sudo rule for one command does not automatically authorize a polkit action, and a polkit authorization does not generally grant a root shell. Both can involve PAM or an administrator credential, but their callers, policies and audit evidence differ.

Inspect actions without exercising one

If the tools and service are present:

```
command -v pkaction
systemctl status polkit.service --no-pager
pkaction
```

The unit may be absent, inactive or started on demand; `status` can return nonzero. If `pkaction` or the authority is unavailable, record that limitation and complete the conceptual exercise below. Do not install a desktop or change service policy merely to match an example.

Choose a harmless **registered action identifier** from the output. Assign its exact observed name to `ACTION` in your shell, then inspect its declaration:

```
test -n "${ACTION:-}" && pkaction --action-id "$ACTION" --verbose
```

Stop if `ACTION` is empty or the action does not exist. The pkaction manual describes the listing. The displayed implicit defaults for active, inactive or other subjects are not a complete prediction of the effective decision: installed authorization rules, subject details, authentication agents and the mechanism's request can alter the result. Listing an action does not execute it.

A server console may have no graphical agent. A denied request, a request requiring authentication without an agent, and a failed credential check are different situations. Do not make an action universally allowed or run a privileged operation simply to see a prompt. Action definitions and local rules can reveal system details; review before sharing.

Worked diagnosis: a request cannot be authorized

Suppose a user reports "authorization failed" when requesting a device operation:

1. Identify the exact application, action, user, session, time and message. Do not collect the password or token.
2. Confirm the process's current UID and groups and, if tracked, its session's local/remote and active properties.
3. Check that the mechanism and authority are available and identify the requested action from the application or logs.
4. Inspect that action's declaration and the site's applicable rules; distinguish an explicit denial from an authentication challenge or an unavailable agent.
5. Test the **same** operation through an authorized, controlled workflow after a narrowly scoped correction. Verify the actual result and relevant logs.

A successful `sudo -l` is not a substitute for this evidence. Conversely, a polkit authorization does not prove the requested service successfully completed its operation. Do not terminate the user's session or edit rules until the effect and recovery path are understood.

Practice and recall

Record the current account, any observed session ID and properties, the presence or absence of polkit tools, and one registered action's stated defaults if available. Explain why those defaults cannot alone predict an effective request. On a text-only host with no polkit, use the worked example to identify the missing evidence instead of fabricating a result.

- An account, process, session and seat are distinct objects.
- polkit evaluates a named action for a subject through a participating mechanism.
- Session context and an authentication agent can affect a request.
- Authentication, authorization and successful completion of the operation are separate observations.

Central Identity: LDAP, Kerberos and SSSD Foundations

A local account works at the console, but a centrally managed user cannot log in. The problem might be name lookup, directory availability, time, ticket acquisition, account policy, a missing home or service authorization. Central identity adds shared dependencies; it does not eliminate the local access and recovery questions already learned.

This chapter introduces those dependencies and performs a read-only inventory of the existing `bl02` host. It does not join a domain, install a directory server, create a Kerberos realm or change PAM/NSS. The main lab remains usable with no network identity infrastructure. Actual directory integration is revisited after the book develops network services, DNS, time and certificate operation.

What central identity changes

A local account database is maintained on one host. A central identity service can provide user and group information, authentication or access policy to multiple hosts. The exact architecture varies: a directory may store identities, an authentication service may issue credentials, and a Linux client may cache and translate results.

Component	Typical role	Distinguish from
LDAP directory	Search and manage structured directory entries and attributes	A guarantee that a particular login is authorized
Kerberos realm and KDC	Authenticate principals and issue time-limited tickets	A complete POSIX UID/GID database or filesystem permission policy
SSSD on a Linux client	Integrate supported identity, authentication and access providers with local interfaces	The remote directory or KDC itself
NSS and PAM	Expose identity lookup and authentication/account interfaces to applications	The remote source's availability and the application's own policy

LDAP is a directory access protocol; it can be used in several configurations and is not synonymous with Kerberos. The LDAP protocol specification describes its directory operations. Protect directory communication with the deployment's verified TLS and trust configuration; an encrypted connection to an unverified endpoint does not establish the intended server's identity.

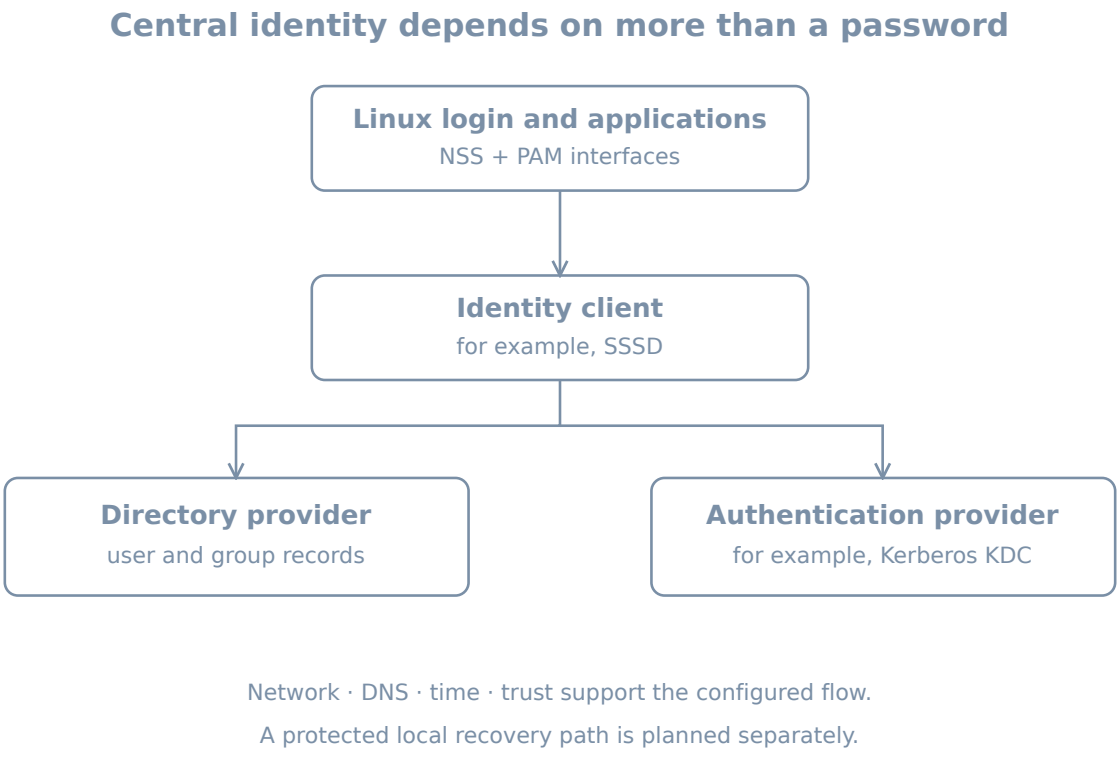


Figure 1: Central identity introduces several independent dependencies.

Kerberos in the minimum useful model

A **principal** is a named identity in a Kerberos realm. A **key distribution center** (KDC) provides authentication and ticket-granting services. A client can obtain a ticket-granting ticket and use it to request a ticket for a particular service. The service must have the appropriate identity and key material to validate its ticket. A **keytab** holds long-term principal keys and must be protected like other credentials.

A ticket has a lifetime and is not a replacement for local authorization. DNS and time are operational dependencies: clients must locate the intended service, and unacceptable clock skew can prevent authentication. Realm naming conventions and discovery rules depend on the deployment. Ubuntu's Kerberos introduction provides the terminology; its server guide identifies DNS and time as prerequisites.

If a client already has the Kerberos utility and a legitimate credential cache, `klist` can display ticket metadata:

```
if command -v klist >/dev/null; then
  klist
else
  printf 'klist is not installed; no ticket inspection performed\n'
fi
```

An empty cache or a nonzero result is expected on a local-only lab host. Ticket names, addresses and timestamps can be sensitive; redact before sharing. `klist` does not create a ticket. Do not use options that display key material or run `kinit` with an unknown principal simply to produce output. The MIT klist reference distinguishes ticket-cache and keytab inspection.

Where SSSD fits

SSSD can provide NSS and PAM integration with supported remote identity and authentication sources. It can also cache information and, when configured and permitted, support some offline logins. What is cached, for whom, for how long and under which policy are deployment-specific. A successful cached lookup or offline login does not prove that the directory or KDC is currently reachable. Canonical's SSSD overview describes these interfaces and common provider combinations.

On a configured host, `/etc/sssd/sssd.conf` and optional snippets describe domains and providers; their ownership and restricted access matter. The Ubuntu SSSD configuration manual documents file permissions and snippet precedence. Configuration may contain sensitive topology or credential references. Do not publish it, clear caches, change providers or restart SSSD during the introductory inspection.

Numeric identity is an operational dependency. If two hosts map one central user to different UIDs, shared files can appear to belong to the wrong principal. If a local and central account collide by name or number, source order can change which record an application sees. Choose and verify a consistent mapping policy before migrating ownership or enabling shared storage.

Read-only local inventory

On `bl02`, establish the existing local baseline:

```
hostnamectl
id
getent passwd "$(id -un)"
grep -E '^(passwd|group|shadow):' /etc/nsswitch.conf
timedatectl status
systemctl status sssd.service --no-pager
```

SSSD may not be installed or active; `status` can return nonzero. A source named `sss` in NSS configuration is not by itself proof that an SSSD domain is configured or reachable. The clock display is evidence about local time and reported synchronization, not a guarantee that a particular KDC agrees with it. The current user's successful local lookup does not prove remote identity works.

If this host has **no** central identity configured, record "not configured or not established by this inspection" and continue with the synthetic exercise. Do not install packages, join a realm, alter the local administrator or expose the guest to a real directory to manufacture a result.

Worked diagnosis: central lookup works but login fails

Consider a fictional user `alex@example.test` on a host configured for a directory and Kerberos. The name is illustrative; do not query a real environment without authorization.

1. `getent passwd 'alex@example.test'` returns a UID and home. This supports a lookup result, potentially from cache; it does not prove a live directory connection.
2. A login attempt fails. Record the actual service, error and time. Check account policy and its PAM path, and distinguish authentication from session/home failure.
3. Inspect configured DNS names and reachability, time synchronization and SSSD status using the site's approved procedure. Compare the actual provider and current state rather than guessing from package names.
4. If Kerberos is in use, inspect ticket metadata for the affected authorized session where appropriate; absence of a ticket, an expired ticket, a KDC error and a service authorization denial require different follow-up.
5. After the targeted correction, repeat the same login and intended operation from the same client context. Verify UID/GID, home access and service authorization, and document whether the result was online or cached.

Do not clear all identity caches as a first diagnostic step. It can discard useful state and affect offline access. Do not create a same-named local user as a shortcut: a collision can change identity resolution and file ownership. For a planned deployment, retain a tested local administrative recovery path that does not depend on the central service, while protecting that path and auditing its use.

Design exercise and acceptance

Without creating infrastructure, write a one-page dependency plan for two future Linux hosts:

- Identify the authority for user/group records, authentication and access policy; state whether they are the same service.
- Define how names and numeric IDs remain consistent, and how homes and file ownership are provisioned.
- Record DNS, network, time and certificate/trust dependencies and who maintains them.
- State offline-login policy, cache limits, credential/keytab protection and a tested local recovery route.
- Specify a positive login and client-operation test, a denied-access test, and evidence to collect if directory or KDC access fails.

Success for this chapter is a coherent plan and an honest local inventory, not a simulated claim that a central login has been tested. The integrated implementation belongs after its networking and trust prerequisites are available.

Recall card

- Directory lookup, ticket authentication, account policy and application authorization are separate functions.
- SSSD is a client integration component; cached success may not mean the provider is online.
- DNS, time, trust and consistent numeric IDs are operational dependencies.
- Preserve a protected local recovery path and test the actual login and service operation.

Packages Software and Updates

Package State and Transactions

Package Metadata, File Ownership and Installed State

A command works on a host, but the operator does not know which package installed it or whether that package is currently supported by an enabled repository. Those are separate questions. The executable, the package database, repository metadata and the running process each provide different evidence.

This chapter inventories the Ubuntu `bl02` guest without changing packages. You need console access, the first-boot update record and the ability to run a command. Later labs use a disposable clone for changes.

From software to a running process

A **package** is a distribution artifact containing files and metadata used to install and maintain software. Its metadata can name a version, architecture, dependencies and configuration files. Ubuntu's Debian package system records local package state in dpkg's database. APT uses repository metadata and local state to plan transactions.

An upstream release number and the distribution package version need not be identical. A distribution can backport fixes while retaining an upstream-looking version. A version string alone does not prove whether a specific fix is present; consult the package changelog and the distribution's security information for the exact release and package.

An installed package is not necessarily running, and a running process can continue executing an older mapped executable after files are upgraded. A repository candidate is a proposed available version, not proof that this host installed it.

Query the installed database

On `bl02`, confirm the host and package architecture:

```
hostnamectl
cat /etc/os-release
dpkg --print-architecture
dpkg --print-foreign-architectures
```

A package's architecture may be `all`, the native architecture or another configured architecture. An empty foreign-architecture list is normal.

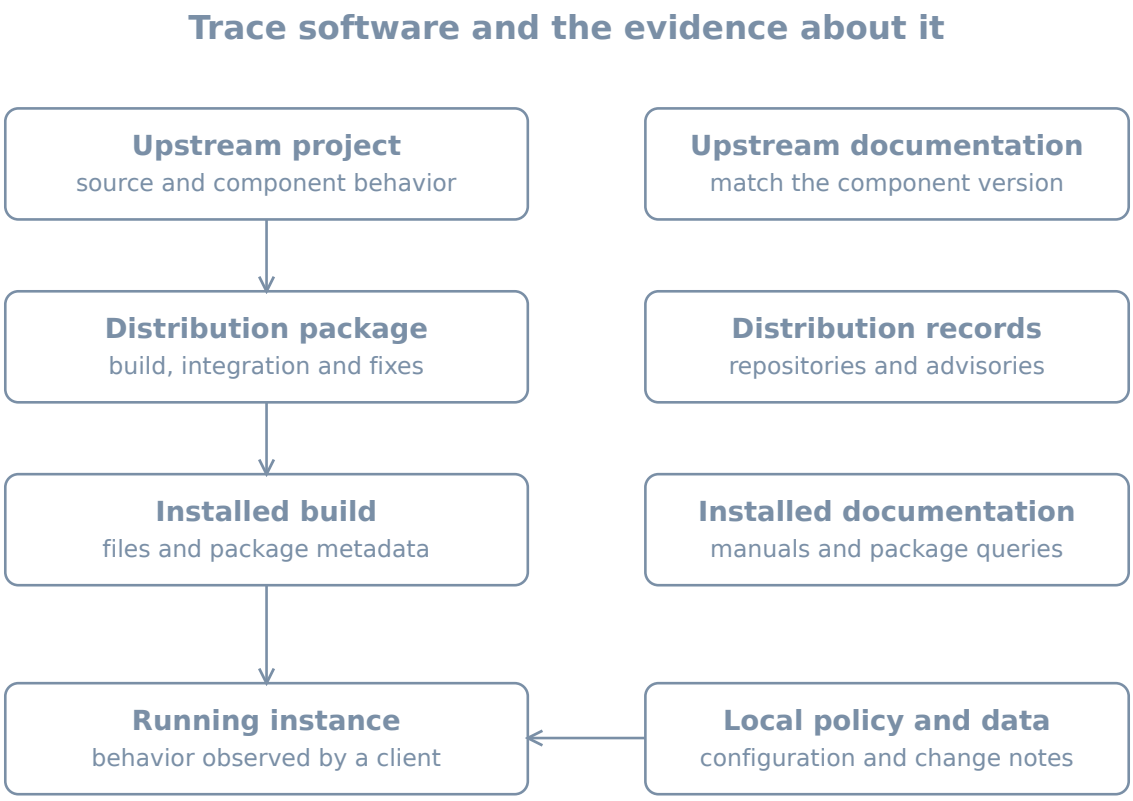


Figure 1: Software has several layers of ownership and evidence.

Use the installed `apt` package as a stable example:

```
dpkg-query -l apt
dpkg-query -W -f='${binary:Package} ${Version} ${Architecture} ${db:Status-Abbrev}\n' apt
dpkg-query -s apt
```

In `dpkg-query -l`, the columns represent desired action, current status and error flag. The commonly seen `ii` means selected for installation and currently installed; a package can instead be unpacked, half-configured or have configuration files left after removal. The dpkg-query manual defines the states. The formatted query is easier to compare than assuming that every listed package is fully installed.

Inspect a package's recorded files and dependencies:

```
dpkg-query -L apt
apt-cache depends apt
apt-cache policy apt
```

`-L` lists paths registered to the package, not necessarily every file created later by a maintainer script or an alternatives mechanism. `apt-cache depends` reads available metadata; `apt-cache policy` compares installed and candidate versions using the current APT cache. That cache can be stale or incomplete. A package may have several architecture-qualified names or multiple packages contributing to a working program.

Map an executable to its package

Ask the shell what it would execute, then inspect the path and package ownership:

```
command -v apt
readlink -f "$(command -v apt)"
dpkg-query -S /usr/bin/apt
```

The query path is the conventional Ubuntu lab path; first confirm that it exists and is the actual path on this host. If it differs, query the observed path instead. Symlinks, alternatives, diversions and the merged `/usr` layout can make the invoked path and the package-recorded path differ. `dpkg-query -S` searches package database records; it is not a full filesystem scan or a proof that the current file contents are authentic. A generated file, local installation, container image or Snap application may have no matching dpkg owner.

For a package file, inspect its installed metadata separately from the file's current bytes. Package verification tools have limits: not all files have recorded checksums, and local configuration is expected to differ. Do not overwrite a suspected file until you know its owner, expected configuration and recovery plan.

Worked diagnosis: “the package owns the wrong version”

Suppose `apt-cache policy apt` shows installed version A and candidate B, while the command’s `--version` reports A. This may simply mean B is available but not installed. Check dpkg status, the active executable path and the cache timestamp. If a daemon reports an older version after a successful upgrade, check when its process started and plan the service restart or reboot required by its owner; do not infer that package installation failed from a long-running process alone.

If `dpkg-query -S` cannot find a path, inspect whether it is a symlink, an alternative, generated state or software installed outside dpkg. Do not delete the file as “unmanaged” without understanding its role.

Practice and recall

Record the host, architecture, installed `apt` package version and state, candidate version, executable path and package ownership result. Identify one observation that would require a separate runtime check. Redact internal repository details before sharing.

- Package state, candidate version, filesystem path and running process are separate observations.
- `dpkg-query` reports recorded ownership and state, not a complete history of every file.
- APT metadata reflects the most recent successful refresh, not necessarily current remote state.
- Use the distribution’s package and security information to interpret versions and fixes.

dpkg, APT and Dependency Resolution

Installing a program can require several other packages and can run configuration scripts. APT plans and obtains a transaction; dpkg manages individual package unpacking and configuration. A preview helps the operator inspect the proposed changes, but the transaction's actual result and the program's behavior still need verification.

This chapter creates `bl05-packages-lab`, a disposable full clone of powered-off `bl02`. It installs the small `tree` package and leaves it installed for a later hold exercise. Use the established single-disk, no-shared-disk, no-backing-chain and no-TPM clone procedure. Confirm the original is shut off, sufficient host space is available, and the destination name and disk path do not exist. On the host:

```
sudo virsh --connect qemu:///system domstate bl02
sudo virsh --connect qemu:///system domblklist bl02 --details
sudo virt-clone --connect qemu:///system \
  --original bl02 --name bl05-packages-lab \
  --file /var/lib/libvirt/images/bl05-packages-lab.qcow2
```

Only run the clone command after confirming `shut off` and the expected system disk. Inspect both domain definitions and confirm distinct disk and NVRAM paths. Keep `bl02` off while the clone runs. Attach the clone to the recorded setup NAT network for package access and keep its graphical console open. The copied guest hostname may still be `bl02`; the manager name and disk path identify the clone.

What each tool knows

A `.deb` package carries files and control metadata. `dpkg` records selections, unpacked files and configuration state. APT reads enabled sources and cached package indexes, considers dependency relationships and proposes a set of packages to install, upgrade or remove; it then invokes package installation machinery. The Ubuntu package-management guide describes this division.

`Depends` expresses a required relationship for package configuration; alternatives and virtual packages can provide multiple ways to satisfy it. `Pre-Depends`, conflicts and version constraints can affect ordering or feasibility. A package manager reasons about declared relationships, not every site-specific application dependency or service health condition.

Refresh and preview on the clone

Check the clone, console, disk space and network before changing package state:

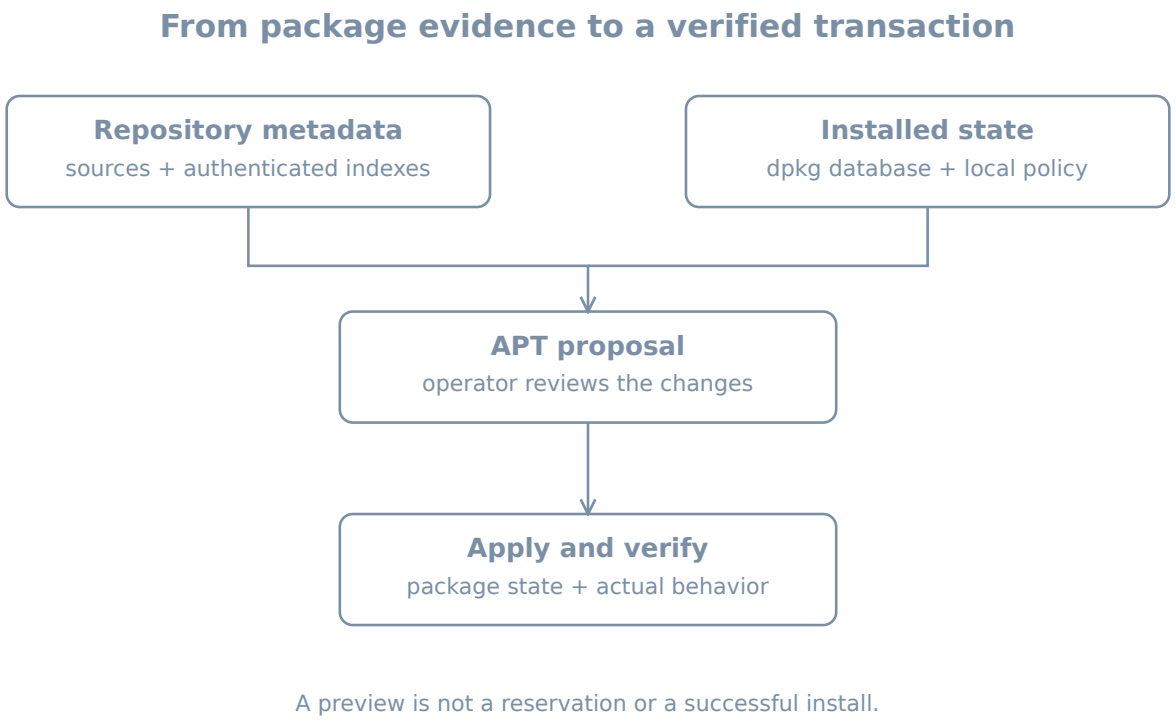


Figure 1: APT's plan, dpkg state and runtime verification are distinct stages.

```
hostnamectl
sudo id -u
df -h / /var
ip -brief address
ip route
sudo dpkg --audit
```

An empty audit normally means no reported partially installed package state, not that every application is healthy. Address and route output do not prove repository access. Review the configured sources using the first-boot procedure, then refresh:

```
sudo apt update
```

Require a clean result; a partial refresh is not a clean baseline. Preserve errors instead of bypassing signature checks. APT's manual distinguishes refreshing package indexes from installing upgrades.

Check whether `tree` is already installed or held:

```
dpkg-query -W -f='${db:Status-Abbrev} ${binary:Package} ${Version}\n' tree
apt-mark showhold
apt-cache policy tree
```

`dpkg-query` can report no matching package and return nonzero. Proceed only if `tree` is not installed, is not held, and a candidate is available. If it is already installed or held, record the difference and use a newly prepared clone instead of removing or unholding an existing package. Confirm the clone has no production data or service dependency on this test package.

Preview:

```
sudo apt-get -s install tree
```

`-s` simulates a transaction and does not apply package changes. Read the complete proposal: new packages, upgrades, removals and errors. A simulation is not a reservation; another package job or a metadata change can alter the eventual transaction. If the proposal is unexpectedly broad or removes packages, stop and investigate. Do not accept a destructive change just because a simulation returned output.

Install, verify and keep the test package

After reviewing the simulation, run the interactive transaction:

```
sudo apt install tree
```

Review the actual confirmation before accepting. If a package manager is already running, identify it and wait; do not delete lock files. Keep the transaction output and investigate any error before proceeding. Installation scripts may run as part of configuration and can have effects outside the package's file list.

Verify the result:

```
dpkg-query -l tree
dpkg-query -W -f='${db:Status-Abbrev} ${Version}\n' tree
command -v tree
tree --version
dpkg-query -S /usr/bin/tree
sudo dpkg --audit
```

Expect a fully installed package, a command that runs and a recorded package owner for the observed executable. Confirm the actual path before using the conventional `/usr/bin/tree` query. An empty audit and successful `--version` are useful but do not prove every feature or external dependency.

APT distinguishes `upgrade` from `full-upgrade`: the latter may remove packages to resolve the transaction, while the former does not remove installed packages. The APT manual documents the distinction. Do not switch to a more disruptive command simply to eliminate a pending count.

Diagnose an incomplete transaction

If installation stops, record the exact operation, package state and logs before retrying:

```
sudo dpkg --audit
dpkg-query -l tree
sudo tail -n 80 /var/log/apt/history.log
sudo tail -n 80 /var/log/apt/term.log
```

An unpacked or half-configured status differs from fully installed. Disk capacity, an interrupted maintainer script, dependency problems and another active package job need different responses. Do not delete lock files, remove package database files or force-install unrelated packages. A later chapter practices recovery of an interrupted transaction with a deliberately bounded fault.

Keep `tree` and the clone for the version and hold exercise. Record the pre-install absence, proposed and actual transaction, installed version, command result and cleanup obligation. A clone on the same host is not an independent backup.

Recall card

- APT proposes dependency-aware transactions; dpkg records and configures package state.
- Refreshing indexes, simulating, installing and verifying are separate operations.
- A simulation can become stale before the actual transaction.
- A package can be unpacked but not fully configured.
- Preserve errors and inspect state before attempting repair.

Sources Selection and Configuration

Repositories, Signatures and Source Configuration

A package name can be available from more than one repository. APT needs to know where metadata comes from, which signing keys are trusted for it, and which versions are candidates. A reachable URL alone does not establish trusted software provenance.

This chapter inspects the Ubuntu `bl05-packages-lab` clone and its configured package sources. It does not add a repository or import a key. Keep the package-lab console open, the original `bl02` off and the setup NAT network attached as in the preceding chapter.

The authenticated repository chain

A repository publishes release metadata describing package indexes; those indexes describe package files and their checksums. APT verifies repository authentication using the configured trust material and then uses the authenticated metadata chain when retrieving packages. The APT secure manual describes the archive-authentication model.

A signature establishes that metadata was signed by a key accepted by the configured policy; it does not prove that the software is safe, that a key was obtained from the right publisher, or that the running application is uncompromised. A compromised trusted signing key or repository remains a risk. HTTPS protects transport to a validated endpoint, while repository signatures provide a separate metadata-authentication mechanism; neither replaces choosing a trustworthy source and key.

Read configured sources

APT can read a traditional one-line `.list` file and deb822-style `.sources` files. On a fresh Ubuntu 26.04 installation, `ubuntu.sources` is common, but imported or upgraded systems can differ. Inspect the actual files:

```
ls -l /etc/apt/sources.list /etc/apt/sources.list.d
sudo cat /etc/apt/sources.list.d/ubuntu.sources
```

An absent optional file produces an error; list and inspect the files that actually exist. Source entries can contain private mirror names or credentials; restrict and redact before sharing. The sources.list manual defines source formats and options.

A deb822 entry can describe `Types`, `URIs`, `Suites`, `Components`, architectures, an enabled state and `Signed-By`. For this lab, record the observed release suites, components, mirror and any scoped

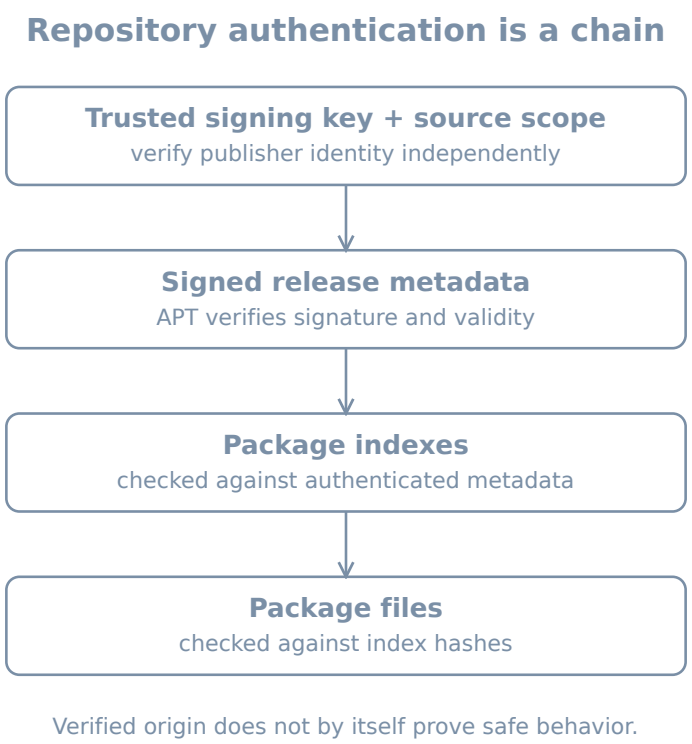


Figure 1: Trust attaches to metadata, indexes and package files.

keyring. A source configuration is **intent**. `apt-cache policy` and `apt update` provide additional evidence about loaded indexes and refresh results.

A `Signed-By` value can restrict which keyring or key may authenticate that source. Keyrings, permissions and their contents must be checked according to the installed APT version and publisher instructions; do not assume every source uses the same key. A third-party signing key should be independently verified through a trusted channel and scoped to the intended source. Avoid adding an unverified key to a broad global trust store.

Connect a package candidate to sources

After the clean metadata refresh in the prior chapter:

```
apt-cache policy tree
apt-cache policy
apt-config dump | grep -E 'Dir::Etc::(sourcelist|sourceparts|trusted)'
```

The first command reports the package's installed version, candidate, priorities and available origins as known to APT. The second summarizes configured package origins and priorities. `apt-config dump` exposes selected configuration paths; the filter can return no lines. Interpret all results as the local cache and configuration, not an independent attestation of a remote server's current content.

`Origin`, `Label`, `Suite`, `Codename` and a repository URI answer different questions. An authenticated Release file contains publisher-provided labels; a familiar label alone is not a fingerprint. A candidate can be the same version available from more than one source, and this policy display is not definitive historical proof of which source supplied an already installed package.

Diagnose a failed refresh

Suppose `apt update` reports a missing key, expired metadata or a failed signature. Preserve the exact error and source. Then:

1. Confirm the guest's time with `timedatectl status` and the intended source definition.
2. Check whether the configured keyring and its access permissions match the trusted publisher's current instructions and your change record.
3. Distinguish a network or mirror failure from a signature or metadata-validity failure; read the complete APT result.
4. Apply a verified, authorized source or key correction using the distribution's documented process, then run `apt update` again and require a clean result.

Do not disable authentication, accept an unknown key, or add an unrelated repository to make an update finish. A key rotation requires a trusted verification channel and an operational plan, not a fingerprint copied from the failing mirror itself. A stale cached candidate can still appear after a failed refresh; record the failure and do not treat that candidate as freshly verified.

Practice and recall

Record the enabled source files, observed suites and components, any `Signed-By` scope, last clean refresh result, and the `tree` candidate and origins. Explain why a matching package name, reachable mirror and signed metadata each support different claims. Keep private source URLs and credentials out of shared evidence.

- A repository signature authenticates metadata under a configured trust policy.
- Source configuration, cached metadata and installed package history are distinct.
- Scope and independently verify third-party keys.
- Resolve trust errors rather than bypassing them.

Version Selection, Holds, Pinning and Dependency Conflicts

An update is available but a package remains at its installed version. The cause may be a hold, candidate policy, phased rollout, dependency constraint or a failed transaction. A single "kept back" message does not identify which one.

Use `bl05-packages-lab`, where the previous transaction installed `tree` after verifying that it was initially absent and not held. Keep the `learner` console, original `bl02` powered off and the package-lab evidence. This chapter briefly holds and unholds only `tree`; it does not change repository priorities or force a downgrade.

Installed, candidate and available

```
dpkg-query -W -f='${db:Status-Abbrev} ${Version}\n' tree
apt-cache policy tree
apt-mark showhold
sudo apt-get -s upgrade
```

The **installed** version is local package state. The **candidate** is APT's current preferred version under its available metadata and policy. A simulated upgrade shows a proposed transaction, including packages that may be deferred. Read the complete result; a hold can affect automatic upgrade behavior even when a newer candidate exists. Other causes include dependency changes, pin priorities, phased updates or a policy that requires a different transaction type.

A simulation does not reserve packages or prevent another job from changing the state. `apt-cache policy` does not prove that a candidate was installed, tested or supplied by a particular repository in an earlier transaction.

Priorities and pins

APT preferences assign priorities to versions. Default priorities vary with installed state, target release and archive metadata; phased updates can also affect eligibility. APT normally avoids downgrades unless a sufficiently high priority or explicit choice changes that behavior. When priorities tie, version ordering matters. The APT preferences manual documents the complete algorithm and warns that careless preferences can make packages uninstallable.

Inspect existing preferences without editing them:

```
ls -l /etc/apt/preferences /etc/apt/preferences.d
apt-cache policy tree
```

An absent optional file is normal. Review actual `.pref` files only if present, and protect private repository details. A pin is a selection rule, not a guarantee that the resulting combination is supported or secure. Mixing unrelated release suites can select incompatible libraries; do not copy a pin from another host or assign an extreme priority just to force a desired answer.

A **hold** is a package selection state that suppresses certain automatic package changes. It is not a security update plan, a dependency lockfile or an absolute barrier to every explicit administrative operation. Record why it exists, who owns it, when to review it, and which security fixes are deferred. The apt-mark manual describes hold and unhold.

Lab: hold and release the disposable package

First reconfirm the exact target and preconditions:

```
hostnamectl
dpkg-query -l tree
apt-mark showhold
```

Proceed only on the identified package clone, with `tree` fully installed and absent from the hold list. If it was already held, do not overwrite another operator's policy. Record the installed version and pre-existing hold list.

```
sudo apt-mark hold tree
apt-mark showhold
apt-cache policy tree
```

Confirm `tree` appears in the hold list. The candidate output may remain unchanged: a hold and candidate selection are different views. You need not have a newer version available to demonstrate the hold state; do not fabricate an upgrade result.

Restore the original state:

```
sudo apt-mark unhold tree
apt-mark showhold
dpkg-query -l tree
```

Verify that `tree` is no longer held and is still installed. If unhold fails, preserve the output and leave the clone's record open; do not report rollback complete. No package version was changed by these hold commands.

Investigate a dependency conflict

For a reported package, inspect its installed state, candidate and declared relationships:

```
apt-cache policy tree
apt-cache depends tree
apt-cache rdepends tree
sudo apt-get -s install tree
```

A `Depends` line can include alternatives and version constraints; reverse dependencies show metadata relationships, not proof that every listed package is installed or in active use. Review the full simulated proposal and its error. A request for an explicit `package=version` is still subject to available metadata and dependency constraints, and a real installation can change or remove other packages. Do not force a version or downgrade the lab's core libraries to make the simulation succeed.

For a real conflict, record the enabled sources, installed and candidate versions, holds, preferences and the exact resolver message. Determine a supported set of packages, service impact and recovery path before changing policy. A version rollback can leave newer data formats or configuration incompatible with the older program; a package downgrade is not a complete application rollback.

Keep `tree` on the clone for the next configuration chapter, then remove it as directed there. Keep the original guest unchanged.

Recall card

- Installed version, APT candidate and hold state answer different questions.
- Pins affect selection; they do not prove compatibility or safety.
- A hold needs an owner, expiry/review decision and security-update plan.
- Read the whole resolver proposal and preserve a recovery path before changing versions.

Configuration Files, Alternatives and Package Hooks

An upgrade asks whether to keep a locally modified configuration file. Choosing the package maintainer's version can remove local behavior; keeping the local version can omit a new required setting. The correct choice depends on the actual change, the service and a tested rollback plan.

This chapter uses `bl05-packages-lab` from the preceding chapters. It inspects package configuration and alternatives, then removes the `tree` package that the lab deliberately installed. It does not alter a real service configuration or force a conffile prompt.

A conffile is a package-managed configuration file

A Debian package can mark selected files as **conffiles**. dpkg tracks the installed package version and can ask what to do when both the administrator and a new package version changed a conffile. Not every file under `/etc` is a dpkg conffile: packages and applications also generate or manage configuration by other mechanisms.

Inspect the package's recorded conffile information:

```
dpkg-query -W -f='${binary:Package}\n${Conffiles}\n' apt
```

The output can contain paths and recorded checksums; it does not show an intelligible three-way diff or prove that a service accepts its current configuration. If the field is empty, choose another **installed** package known to have conffiles rather than fabricating a path. The dpkg manual describes conffile handling and package states.

Before a real upgrade, record the service's current configuration, package version, dependencies, validation command and recovery path. During a conffile prompt, inspect the displayed difference and consult the package's instructions. Common choices can keep the local file, install the maintainer's file, or inspect differences; available prompts and defaults depend on the operation and options. Keep a protected backup and test the resulting service behavior. Blindly accepting either side is not a merge strategy.

Files with suffixes such as `.dpkg-dist`, `.dpkg-old` or `.dpkg-new` can be evidence of a package operation, but their presence and exact meaning depend on the operation. Review the actual files before deleting or merging them. Configuration can contain secrets; restrict access and redact differences before sharing.

Maintainer scripts and triggers

Packages can run maintainer scripts during installation, configuration, upgrade and removal. Triggers allow one package to request deferred work from another; package state can remain pending while that work is incomplete. These scripts can affect generated files or services that are not listed as ordinary package payload paths.

Inspect the available control files for the installed `apt` package:

```
dpkg-query --control-list apt
```

If `postinst` appears in the list, inspect it without executing it:

```
dpkg-query --control-show apt postinst
```

A missing script is normal for some packages. The dpkg-query manual documents these read-only control-file queries. A script's content is not a transaction preview; its execution depends on package state, arguments, dependencies and environment. Never run a maintainer script manually just to see what it does.

When a transaction fails, capture the package state and exact script error. A partially configured package and a fully configured package with a broken application are different conditions. The later interrupted-transaction chapter covers controlled recovery; do not force a package state or remove scripts from the database to silence an error.

Alternatives select an implementation

The alternatives system can present a common name that points through managed symbolic links to one of several implementations. Automatic mode generally follows configured priorities; manual mode retains a selected choice until an administrator changes it. Inspect the `editor` group if present:

```
update-alternatives --display editor
ls -l /usr/bin/editor /etc/alternatives/editor
readlink -f /usr/bin/editor
```

An unavailable alternatives group or path is a result to investigate, not a reason to create one. The update-alternatives manual describes link groups and modes. Package ownership of the final executable, an alternatives selection, and a user's `EDITOR` environment variable can all affect what is actually invoked. Inspect the relevant caller before changing the system default.

This operation is read-only. A real alternatives change should identify the owner and consumers, record the previous mode and selection, test the affected application and provide a rollback. Not every Ubuntu command is managed by alternatives; for example, distribution provider selection may use other packaging mechanisms.

Lab: finish and remove the test package

The earlier chapters installed `tree` only after confirming it was absent, then removed its temporary hold. Reconfirm those assumptions on the **clone**:

```
hostnamectl
dpkg-query -l tree
apt-mark showhold
```

Proceed only if the manager identifies `bl05-packages-lab`, `tree` is the test package installed during this lab and it is no longer held. If the original state is uncertain, stop; do not remove a package merely because it is named `tree`.

Preview removal:

```
sudo apt-get -s remove tree
```

Read the whole proposed transaction. If it removes anything other than the intended test package or reports unexpected dependency consequences, stop and investigate. When the actual proposal is understood:

```
sudo apt remove tree
```

Review the real confirmation before accepting. Do not run `autoremove` or `purge` as part of this exercise: dependencies or configuration can have other owners and retention requirements. Verify the package state and command result:

```
dpkg-query -l tree
command -v tree
sudo dpkg --audit
```

`command -v tree` may return no output and nonzero if no other executable with that name is on `PATH`; that is expected for this clean lab. A removed package can leave configuration records, and another installation mechanism could provide a command with the same name. Record the actual state and investigate any discrepancy. End with the clone powered off before restarting `bl02`.

Worked diagnosis: a service changed after upgrade

Suppose a service fails after its package upgraded. Compare the transaction time, package version, service logs and configuration validation. Check whether a conffile was kept, replaced or left with an alternate version, and whether a maintainer script or trigger failed. Restore or merge only after understanding which configuration belongs to the new version. Then restart through the service's documented procedure and repeat a client-side acceptance test.

Replacing all of `/etc` from an old backup or downgrading a package without accounting for data-format changes can create further failures. Preserve the evidence and use the service's supported recovery plan.

Practice and recall

Record one package's conffile field, one available control-file name, the `editor` alternatives state if present, and the test package's before/after removal state. Explain why none of those alone proves the application is working.

- A conffile is one package-management mechanism, not every configuration file.
- Maintainer scripts and triggers can change state beyond a file list.
- Alternatives, package ownership and an environment override are different selection layers.
- Preview removal and verify the actual resulting state; do not assume purge or autoremove is harmless.

Updates and Kernel Lifecycle

Security Updates and Maintenance Windows

A security fix is useful only when it reaches the affected system and the relevant code is running. Package availability, a successful installation, a service restart and a verified recovery are separate events. This chapter turns them into an observable maintenance procedure.

Decide what must change

A distribution often backports security fixes without changing the upstream-looking version to the latest upstream release. Compare the installed package version and its Ubuntu security notice or changelog; an upstream version string alone does not establish exposure. Check the release, enabled repositories, support status and whether the affected component is actually used. Ubuntu's security-update documentation describes its archive and extended security coverage.

A maintenance window is an agreed period for a change and its verification. Establish a responsible operator, expected disruption, dependencies, a restorable backup, access through a console if networking fails, and a stop condition. For a fleet, update a representative canary first and measure results before proceeding. A backup is only a recovery plan if its restore has been tested and the application data is consistent.

Observe policy and evidence

On `bl05-packages-lab`, start the previously prepared clone with the original `bl02` powered off. The two share an identity and must not be online together. Confirm the lab identity and inspect:

```
hostnamectl
cat /etc/os-release
apt-cache policy
apt-mark showhold
systemctl list-timers 'apt-*'
apt-config dump | grep -E 'APT::Periodic|Unattended-Upgrade'
```

A missing timer or an empty match is evidence to investigate, not proof that no other automation exists. Configuration may be spread across files and list options can accumulate. Review the effective settings and their source files; a later scalar assignment can override an earlier one, while list entries may be added. Avoid placing credentials in shared logs.

```
dpkg-query -W unattended-upgrades
systemctl status apt-daily.timer apt-daily-upgrade.timer --no-pager
```

```
sudo journalctl -u apt-daily-upgrade.service --since '7 days ago' --no-pager
sudo ls -l /var/log/unattended-upgrades/
```

Some images omit these packages, timers or logs. A timer's next activation does not prove the previous run succeeded, nor that it covered a particular repository. If present, inspect the unattended-upgrades log and compare its timestamp and actual package changes with the security policy. APT periodic downloads, unattended installation, Snap refreshes and reboot scheduling are distinct controls. The Ubuntu documentation recommends local drop-ins instead of editing the packaged `50unattended-upgrades` file; review the active origins and reboot policy rather than assuming defaults apply to every image or third-party source.

Perform a controlled update on the clone

The clone needs working DNS and access to its configured, trusted repositories. Record its installed versions and running kernel; ensure the virtual console works and a recovery copy exists. `apt update` changes the local package indexes and requires network access:

```
uname -r
sudo apt update
apt list --upgradable
sudo apt-get -s upgrade
```

Read the complete simulation, including held-back packages, conffile implications and any service or kernel changes. A simulation is not a reservation: repository state, other package operations or local policy can change before execution. `upgrade` and `full-upgrade` have different removal behavior; do not substitute the latter merely to make the list empty.

If the lab's proposed changes are understood and a maintenance window is available, run `sudo apt upgrade` on the **clone**, read the actual proposal and confirmation, and decline if it differs unexpectedly. Do not interrupt a running package transaction. Record the result and investigate errors before attempting another operation:

```
sudo dpkg --audit
apt list --upgradable
journalctl -b -p warning --no-pager
```

Some updates may remain phased, held or unavailable through the configured sources. Check `apt-cache policy PACKAGE` for a specific package; do not assume an empty audit proves every workload healthy. Test the service or command affected by each update and record versions, timings and any required restart or reboot. The next chapter handles the kernel decision. Leave the clone available for the next lab.

Diagnose a missed update

If an advisory's fix is absent, first establish the distribution release, architecture, installed version and advisory's fixed version. Then inspect sources, `apt-cache policy`, hold state, timer/service

logs and network or signature errors. A fresh index cannot fix an untrusted or unreachable repository, and an installed library does not automatically replace code already mapped into every running process. Decide on a service restart or reboot with the workload owner, then verify the running service and client-visible behavior.

Practice and recall

Write a one-page maintenance record with preconditions, the simulated proposal, actual result, restart decision and acceptance check. Explain why automatic installation does not guarantee automatic remediation and why a clean package database is only one success signal.

Kernel Updates, Reboots and Livepatch

A new kernel package can be installed while the machine still runs the old kernel. The installed files are the next-boot candidates; `uname -r` reports the running kernel. A safe update keeps these facts separate and preserves a workable route back to the console.

Inspect the running and installed kernels

On `bl05-packages-lab`, inspect without changing boot selection:

```
uname -r
dpkg-query -W 'linux-image-*' 2>/dev/null
ls -l /boot/vmlinuz-* /boot/initrd.img-* 2>/dev/null
ls -l /var/run/reboot-required* 2>/dev/null
```

The package query may include records for packages that are not installed. Inspect their status with `dpkg-query -l PACKAGE`. The reboot-required files are useful signals when present, not a complete list of all reasons to restart and not proof of which change triggered them. Missing files or unmatched globs may produce errors on a particular image. `/boot` can be separate and can fill independently of the root filesystem:

```
findmnt /boot
findmnt /boot/efi
df -h /boot /boot/efi
```

An absent separate mount is not necessarily an error; verify the actual layout before changing it. Meta-packages track a kernel series and flavor, while versioned image, modules and headers packages serve different roles. Out-of-tree modules, including DKMS-built drivers, must be compatible with the target kernel and the host's Secure Boot policy. See the earlier boot chapters for initramfs and boot-entry evidence.

Plan the transition

Before applying updates, record the current kernel, the intended target, storage space, bootloader entry and fallback console. A fallback requires an installed, bootable previous kernel with its matching modules and initramfs; an old filename alone is not a tested fallback. Preserve access to any encrypted root unlock mechanism and confirm a recovery environment is available. Review package proposals before removing old kernels: keeping every version indefinitely can fill `/boot`, while deleting the only tested fallback can make recovery harder.

After the controlled update in the preceding chapter, compare installed versioned images with `uname -r`. If the next boot should use a newer kernel, arrange a reboot within the window:

```
sudo systemctl reboot
```

Run this only on the lab clone after confirming its identity and access through the VM console. A reboot ends sessions and interrupts workloads. Reconnect after startup and verify:

```
uname -r
systemctl is-system-running
journalctl -b -p warning --no-pager
sudo dpkg --audit
```

A `degraded` state warrants investigation; unrelated warnings require context. Verify the actual workload, network and any external kernel modules, not just the version. If the new kernel fails, use the known bootloader fallback through the console, capture its failure evidence and repair the cause before trying again. Do not blindly restore an older kernel indefinitely or disable boot security to make an unsigned driver work.

Livepatch changes the timing, not the lifecycle

Canonical Livepatch can apply selected security fixes to a supported running Ubuntu kernel. Coverage, kernel support and subscription availability vary; it does not cover all kernel fixes or eliminate normal package updates and planned reboots. Check the service's actual status and advisory coverage, not merely the presence of a client package. Ubuntu's security-update documentation describes the service and its limits. Other Linux systems have different live-patching implementations and support policies.

Worked diagnosis: package updated, vulnerability remains

An administrator installs a fixed kernel image but the scanner still sees the old running release. Compare `uname -r` with the installed version, inspect the planned boot entry and determine whether the relevant issue is covered by an active live patch. If a reboot is required, schedule one, then verify the running release and service acceptance. If the booted version remains old, investigate the actual boot selection and boot logs rather than repeatedly reinstalling the image.

Practice and recall

Record running and installed kernels before and after the lab reboot. Identify one reason a version string is insufficient to establish security status and one reason a fallback kernel is not a substitute for a tested restore.

Other Formats and Software Supply

Snaps, Flatpak and Other Software Formats

A host can have multiple software inventories. A program named `firefox` or `editor` may be provided by a Debian package, a Snap, a Flatpak, or a file installed by an administrator. Updating only APT does not update every one of them.

Identify the owner and update path

APT and dpkg manage Debian packages and their dependencies. Snap packages are managed by snapd, with revisions, channels, interfaces and a separate refresh mechanism. Flatpak applications and shared runtimes are managed per system or per user through configured remotes. Standalone binaries, language-specific environments and container images have still other owners and update paths. A container image is not a replacement for updating the host kernel or runtime.

Start by identifying a particular command instead of guessing its format:

```
type -a firefox
command -v firefox
readlink -f "$(command -v firefox)"
```

If the command is missing, `command -v` returns nonzero and there is no resolved path to inspect. A launcher or wrapper may hand off to another manager; a Debian-owned launcher does not prove the application payload is a Debian package. Follow the actual execution path and compare inventories:

```
dpkg-query -S /usr/bin/firefox
snap list
flatpak list
```

The example path may be absent or unowned; the `snap` or `flatpak` commands may not be installed. Run the relevant query only when its manager exists, and inspect the path actually found. Manager inventory and application behavior both matter.

Snap: revisions, confinement and refresh

Where snapd is installed, inspect without installing or changing anything:

```
snap list --all
snap refresh --time
```

```
snap changes
```

A Snap has a publisher, channel and revision; `snap list --all` may show retained disabled revisions. Snaps normally refresh automatically. Strict confinement uses interfaces to mediate access, whereas classic confinement grants broader host access; confinement is not a guarantee that an application is safe. A refresh can affect running services or application sessions. Snap's update documentation explains schedules, holds and their scope.

A targeted refresh can bypass a hold on that specific snap; do not treat a hold as an immutable freeze. Reverting to a retained revision can restore some revision-specific data, but common application data and external databases can remain changed. Test the application's own recovery procedure and compatibility before relying on a revert. See Snap revisions for data behavior.

Flatpak: applications, runtimes and remotes

If Flatpak is installed, inventory both applications and runtimes, and distinguish the user and system installations:

```
flatpak remotes --show-details
flatpak list --app
flatpak list --runtime
```

A remote is a software source with its own trust and lifecycle. An application can depend on a runtime maintained separately from the host's Debian libraries. Permissions and desktop portals mediate access, but available grants and overrides change the effective boundary. Inspect an installed application's declared permissions with `flatpak info --show-permissions APP_ID`; use an actual application ID from the inventory. See Flatpak's concepts and command reference.

No installation or permission change is required for this exercise. On a server, avoid installing an additional manager simply to run these queries. Missing tools and empty lists are legitimate inventory results.

Make one maintenance inventory

For each application record its executable or launch method, manager, publisher/source, version or revision, automatic update policy, data location, privileges, restart needs and removal/restore method. Do not assume `apt remove` removes a Snap or that removing an application removes its data. A script in `/usr/local`, a language virtual environment or a vendor tarball may have no central update service at all; assign an owner and patch process explicitly.

Worked diagnosis and practice

A desktop application is reported vulnerable after `apt upgrade` succeeds. Determine which application is actually launched, inspect its manager and source, and compare its installed revision against

the publisher's advisory. Check whether a pending refresh requires the application to exit. Update using that manager's documented process in a maintenance window, then verify the running application. On the lab clone, create a two-column inventory of installed managers and sources; explain what each manager does **not** update.

Source Builds, Local Packages and Reproducibility

Building software locally transfers responsibility for its source, build dependencies, updates and removal to the operator. A successful compile does not establish that the source was authentic, that the result is reproducible or that it will receive security fixes.

Record the supply chain

Record the project's identity, release or commit, source URL, verified signature or hash from an independently authenticated channel, license, target architecture, compiler, dependency versions and build instructions. Build in an isolated disposable environment with no production credentials and limited network access. Build scripts are executable code: an untrusted source tree must not be run with administrator privileges. Inspect the build system and use the upstream project's documented tests.

A reproducible build means independent builds from equivalent declared inputs produce bit-identical artifacts. It does not establish that the source is benign. Timestamps, generated file order, toolchain versions, paths and network-fetched dependencies can make outputs differ. `SOURCE_DATE_EPOCH` supplies a deterministic timestamp to tools that support it; it does not make an arbitrary build reproducible by itself. See the SOURCE_DATE_EPOCH specification.

Stage instead of installing into the host

Many build systems support a staging directory (often `DESTDIR`) that receives files under their intended final paths without writing into `/usr` on the build host. Check the project's documentation: support and syntax vary. Avoid `sudo make install` as an untracked shortcut, especially when a distribution package already owns the same paths. Build artifacts for deployment should have a documented owner, version, dependencies, upgrade path and removal procedure.

A Debian package records ownership and metadata in the package database. Creating one does not automatically solve dependency selection, signatures, upgrades, triggers or service policy. Debian source packaging and a clean build environment are the appropriate next step for a maintained application. The dpkg-deb manual documents the archive format and inspection commands.

Lab: build and inspect a tiny local package

On the disposable `bl05-packages-lab` clone, create an isolated working directory owned by the current user. This example packages a text file and has no maintainer scripts; it does not install anything. `dpkg-deb` must already be available.

```
labdir=$(mktemp -d)
printf 'Working directory: %s\n' "$labdir"
mkdir -p "$labdir/pkg/DEBIAN" "$labdir/pkg/usr/share/bl05-demo"
cat > "$labdir/pkg/DEBIAN/control" <<'CONTROL'
Package: bl05-demo
Version: 1.0-1
Architecture: all
Maintainer: Lab Operator <lab@example.invalid>
Description: Disposable local packaging example
 A small file for inspecting a package without installing it.
CONTROL
printf 'Package ownership demonstration\n' > "$labdir/pkg/usr/share/bl05-demo/readme"
dpkg-deb --root-owner-group --build "$labdir/pkg" "$labdir/bl05-demo_1.0-1_all.deb"
dpkg-deb --info "$labdir/bl05-demo_1.0-1_all.deb"
dpkg-deb --contents "$labdir/bl05-demo_1.0-1_all.deb"
sha256sum "$labdir/bl05-demo_1.0-1_all.deb"
```

Examine the package name, version, architecture and file paths. `all` means the package is architecture-independent, not that every dependency or interpreter would work on all hosts. The hash identifies these bytes; it is not a signature or evidence of trust. Rebuilding and comparing hashes is an experiment, not proof of independently reproducible builds unless inputs and environments are controlled independently.

Record the printed working directory. Remove it only after checking that it is the disposable directory just created:

```
printf '%s\n' "$labdir"
# After checking the path is the disposable directory:
rm -r -- "$labdir"
```

Do not reuse this removal command in a later shell where `labdir` is missing or has a different value. A real deployment would use a reviewed package with appropriate dependency metadata, authenticated distribution, an upgrade policy and acceptance tests. Installing a local `.deb` can execute its maintainer scripts as root; inspect and trust the whole package before installation.

Diagnose and practice

If an application works on the build host but fails elsewhere, compare architecture, interpreter, shared-library requirements, build flags, paths and runtime dependencies. A successful build may have used undeclared libraries already present on the builder. Rebuild in a clean environment

and verify on a separate test host before changing dependency metadata. Explain the differences between file ownership, source authenticity and artifact reproducibility.

Offline Repositories, Mirrors and Disconnected Installation

A disconnected host still needs a way to receive authenticated software and security fixes. A cache, a mirror and a portable bundle solve different problems: a cache retains requested objects, a mirror copies a defined repository set, and a bundle carries a selected transaction. None is current or trustworthy merely because it is nearby.

Design the supply route

Record the target release and architecture, enabled components, installed packages, storage capacity and update cadence. Decide who approves sources and signing keys, how metadata and packages cross the boundary, who verifies them, and how old snapshots are retained for recovery. A mirror can carry signed upstream metadata unchanged; an internally assembled repository needs its own signed release metadata and an independently provisioned, scoped trust key. Protect the signing key separately from the distribution server.

APT authenticates repository release metadata, which identifies index hashes; indexes in turn identify package hashes. Transport encryption can protect the connection but does not replace repository authentication. Conversely, valid signatures do not establish that software is appropriate for the target or that its metadata is fresh. A release's expiry can make an old snapshot unusable; handle freshness through an explicit policy rather than disabling verification to get past an error. The APT authentication manual explains this trust chain.

A cache saves bandwidth only for objects it has retained and may need network access on a miss. A mirror needs sufficient space for all selected architectures, components and versions; incomplete synchronization can temporarily mix metadata with missing packages. Publish a completed, verified snapshot atomically and test it before clients consume it. Preserve source packages and required licenses when your policy requires them.

Inventory a target without changing it

Use the `bl05-packages-lab` clone. Keep its network configured for this observation; no sources will be added or disabled. Record release and architecture along with current sources and package state:

```
cat /etc/os-release
dpkg --print-architecture
dpkg --print-foreign-architectures
```

```
apt-cache policy
dpkg-query -W -f='${binary:Package}\t${Version}\t${db:Status-Status}\n' \
  > "$HOME/bl05-package-inventory.tsv"
```

The inventory file is written in the lab user's home and may disclose installed software. It is not a complete list of repository configuration, package selections, dependencies or application data. Keep it with the target's change record and restrict distribution as appropriate. Review enabled source files and scoped signing keys as described earlier in this part. Inspect the package candidates required by a proposed change:

```
apt-cache policy tree
sudo apt-get -s install tree
```

This simulation uses the target's current local indexes and state. It neither downloads packages nor guarantees that another host will resolve the same transaction. `tree` is only an example; the preceding lab removed it. Do not install it during this inspection.

Prepare and verify a transfer

A connected staging system must use a repository snapshot and target-compatible release, architectures, source configuration and package state. Resolve the **whole** transaction, including dependencies, on a representative target or controlled equivalent. Export the required signed metadata, indexes and package files using your chosen mirror or bundle tooling. Authenticate the portable media or transfer manifest independently; then verify repository signatures and hashes at the receiving side. A checksum copied beside a file on the same untrusted medium cannot by itself authenticate either file.

Test the procedure on a disposable host with its external network disconnected. Configure the approved local source using a scoped `Signed-By` key that was provisioned through a separate trusted channel; do not use `trusted=yes` to bypass a signature failure. Run `apt update`, inspect `apt-cache policy` and simulate the planned installation. If resolution wants a missing dependency or another repository, stop and rebuild the bundle or snapshot; do not silently reconnect the supposedly isolated host. Apply only a reviewed proposal, then validate package state and the workload. Record the snapshot identity, transfer date, signatures, versions, results and next update deadline.

The exact source stanza depends on how the repository is published. The Ubuntu sources.list manual describes deb822 sources and `Signed-By`. A directory of `.deb` files alone is not a signed APT repository. For a full implementation, also plan key rotation, revocation, repository expiry, storage monitoring, audit trails and emergency transport.

Diagnose and practice

If `apt update` reports a signature error, preserve the exact message and check the source URL, key identity, release metadata, clock and snapshot completeness. An expired release, wrong key and corrupted metadata require different fixes. If installation fails with an unavailable dependency,

compare the candidate versions and architecture with the snapshot manifest; copying only the requested package rarely captures the full dependency closure.

Draw a supply route from upstream to the isolated host and mark the trust check and responsible owner at each crossing. Describe how you would test a stale or incomplete snapshot without changing a production host.

Transaction Recovery

Interrupted Package Transactions and Recovery

An interrupted update may leave a package unpacked but not configured, a service stopped, or an application with data already migrated. Repair begins by preserving evidence and identifying the failed stage. Package database consistency is a necessary check, not a complete application rollback.

Establish the failure boundary

If another package process is working, let it finish; identify it and the associated service instead of deleting lock files. Check space on both the filesystem holding `/var` and any separate `/boot`. Capture the failing command, the error, package status, recent APT and dpkg logs, source configuration and any relevant service logs before retrying:

```
sudo dpkg --audit
sudo tail -n 100 /var/log/dpkg.log
sudo tail -n 100 /var/log/apt/term.log
df -h / /var /boot
systemctl list-jobs
```

Logs may be absent or rotated; protect them if they contain sensitive information. Inspect the specific package using `dpkg-query -l PACKAGE` and `apt-cache policy PACKAGE`, replacing `PACKAGE` with the actual name. The dpkg manual explains package states and configuration. A failed network download, a full filesystem, a dependency conflict and a maintainer-script failure require different repairs.

Fix the identified cause first. For a reviewed, interrupted configuration, `sudo dpkg --configure PACKAGE` can retry that package's configuration; `sudo dpkg --configure -a` may configure **all** pending packages and run their scripts. `sudo apt-get -s -f install` previews APT's proposal to fix broken dependencies. The actual `apt-get -f install` can change or remove packages and run scripts, so review its full proposal and stop if it is unexpected. Neither command repairs the underlying storage fault or makes an unsafe script safe. Do not use force options or edit package database files as a routine fix.

Lab: recover a deliberately failed configuration

Use the disposable `bl05-packages-lab` clone, not a production host. Keep `bl02` off, confirm the clone in the VM manager and have its console and a tested recovery copy available. This exercise creates and installs a tiny package whose post-install script intentionally fails until a marker exists. It will modify the clone's package database and `/run`. `dpkg-deb` and `dpkg` must be present.

First ensure `sudo dpkg --audit` is empty, no package operation is running, `bl05-recovery-demo` has no existing installed or residual record, and `/run/bl05-recovery-ready` does not exist. If any condition fails, stop and investigate instead of overwriting another owner's state:

```
sudo dpkg --audit
dpkg-query -l bl05-recovery-demo
ls -l /run/bl05-recovery-ready
```

The last two queries are expected to report no match. A package in `rc` or any other recorded state is not a clean baseline. Build the known lab package as the unprivileged user:

```
labdir=$(mktemp -d)
printf 'Lab directory: %s\n' "$labdir"
mkdir -p "$labdir/pkg/DEBIAN" "$labdir/pkg/usr/share/bl05-recovery-demo"
cat > "$labdir/pkg/DEBIAN/control" <<'CONTROL'
Package: bl05-recovery-demo
Version: 1.0-1
Architecture: all
Maintainer: Lab Operator <lab@example.invalid>
Description: Disposable package recovery exercise
 A deliberately failing post-install script for a lab clone.
CONTROL
cat > "$labdir/pkg/DEBIAN/postinst" <<'SCRIPT'
#!/bin/sh
set -e
if [ "$1" = configure ] && [ ! -f /run/bl05-recovery-ready ]; then
    echo 'Lab prerequisite is missing' >&2
    exit 1
fi
SCRIPT
chmod 755 "$labdir/pkg/DEBIAN/postinst"
printf 'Recovery exercise\n' > "$labdir/pkg/usr/share/bl05-recovery-demo/readme"
dpkg-deb --root-owner-group --build "$labdir/pkg" "$labdir/demo.deb"
dpkg-deb --info "$labdir/demo.deb"
dpkg-deb --contents "$labdir/demo.deb"
```

Review the package and script above before installing: package scripts execute as root. If the build failed or the contents differ, stop. The next command intentionally returns an error and leaves this package in an incomplete state:

```
sudo dpkg -i "$labdir/demo.deb"
sudo dpkg --audit
dpkg-query -l bl05-recovery-demo
```

Confirm the failure mentions the lab prerequisite and the audit names the lab package. If anything else failed, stop and investigate. Supply the deliberately missing prerequisite, then retry only this package:

```
sudo touch /run/bl05-recovery-ready
sudo dpkg --configure bl05-recovery-demo
dpkg-query -l bl05-recovery-demo
sudo dpkg --audit
```

A successful retry should show the package fully installed and no audit issues. If it does not, preserve the output and do not continue to cleanup until the cause is understood. Remove only the lab package and marker, then inspect the result:

```
sudo dpkg -r bl05-recovery-demo
sudo rm -- /run/bl05-recovery-ready
sudo dpkg --audit
dpkg-query -l bl05-recovery-demo
printf 'Remove this disposable directory only after checking: %s\n' "$labdir"
```

Remove the printed temporary build directory after confirming it is the one created for this exercise. If the shell session is lost, recover the recorded path; do not guess. Power off the clone before starting `bl02`.

Rollback has boundaries

Removing or downgrading a package does not necessarily reverse database migrations, external side effects, generated files or a changed configuration. An older package may not be available or compatible with newer data. Recovery may require a coordinated restore of software, configuration and application data from a tested, consistent backup; a same-host VM clone is not an independent backup. Verify both package state and the application through a client after recovery.

Practice and recall

Record the intentional error, status before and after configuration, the exact prerequisite and final audit. Explain why the targeted retry was appropriate here and why it must not be generalized into running arbitrary maintainer scripts by hand. Compare the package-state repair with an application-data restore.

Services Systemd and Scheduled Work

Service Supervision and Execution

Service Supervision, Units and Managers

A server can be running as a process without being supervised or ready to serve a client. A service manager connects desired activation, process lifecycle, logs and failure information. This chapter builds a small local service that subsequent chapters will inspect and improve.

Unit state is not application health

On a systemd host, a **unit** describes an object managed by systemd. Service units supervise processes; other unit types represent sockets, timers, mounts, devices and targets. The system manager handles system units, while a user's manager handles that user's units. `systemctl --user` addresses the current user's manager when a user session and bus are available; it is not the same manager as `sudo systemctl`.

Inspect the system without making changes:

```
ps -p 1 -o pid,comm,args
systemctl list-units --type=service --state=running
systemctl list-unit-files --type=service
systemctl --failed
```

Loaded units and installed unit files are different inventories. A unit's `active` state, its enabled state and its client-visible health are separate facts. `enabled` generally describes links for future activation, not a promise that the service is running now; a disabled unit can still be started manually or by another unit. A `static` unit may be activated by a dependency despite lacking direct enablement. systemctl's manual documents these states.

Prepare a separate service lab

Create `bl06-service-lab` as a full clone of powered-off `bl02`, following the same single-disk, no-shared-disk, no-backing-chain and no-TPM precautions used for `bl05-packages-lab`. Confirm sufficient space and that the destination name and disk path do not already exist:

```
sudo virsh --connect qemu:///system domstate bl02
sudo virsh --connect qemu:///system domblklist bl02 --details
sudo virt-clone --connect qemu:///system \
  --original bl02 --name bl06-service-lab \
  --file /var/lib/libvirt/images/bl06-service-lab.qcow2
```

Run the clone command only after checking that the original is shut off and has the expected disk layout. Inspect both domains for distinct disks and NVRAM; keep `bl02` and other clones off when this clone runs, because the copied guest has the same identity. The guest hostname may still be `bl02`: identify it through the VM manager and console, not the hostname alone. Keep a recovery copy and console access. The clone needs Python 3; check `command -v python3` and `/usr/bin/python3 --version`. Do not install tools solely to hide an unexpected baseline.

Create a bounded example service

The example serves existing documentation on loopback port 18765, only inside the VM. Python's demonstration HTTP server is not a production web server. Verify that port is unused and the name is not already owned before proceeding:

```
sudo ss -ltnp 'sport = :18765'
systemctl cat bl06-demo.service
ls -ld /usr/share/doc
```

An empty listener result and a unit-not-found result are expected. If the port, unit, or `/etc/systemd/system/bl06-demo.service` already exists, stop and choose a fresh clone. Create the local unit:

```
sudo tee /etc/systemd/system/bl06-demo.service > /dev/null <<'UNIT'
[Unit]
Description=BL06 loopback documentation demonstration

[Service]
Type=exec
DynamicUser=yes
WorkingDirectory=/usr/share/doc
ExecStart=/usr/bin/python3 -m http.server 18765 --bind 127.0.0.1
Restart=on-failure
RestartSec=2s

[Install]
WantedBy=multi-user.target
UNIT
sudo systemd-analyze verify /etc/systemd/system/bl06-demo.service
```

If verification reports errors, fix them before loading. `DynamicUser=yes` uses a transient service identity; this service needs only read access to the documentation directory. It is deliberately not enabled at boot. Load the definition and start it:

```
sudo systemctl daemon-reload
sudo systemctl start bl06-demo.service
systemctl status bl06-demo.service --no-pager
systemctl is-enabled bl06-demo.service
```

```
sudo journalctl -u bl06-demo.service -b --no-pager
```

`is-enabled` returns a nonzero status for a disabled unit; that is expected here. Test from inside the same guest:

```
python3 - <<'PYCODE'
import urllib.request
response = urllib.request.urlopen("http://127.0.0.1:18765/", timeout=5)
print(response.status)
PYCODE
```

Expect HTTP 200 for the directory listing. If it fails, inspect the unit status, log, listener address and permissions. Do not expose this example to an untrusted network. Keep the unit for the next chapters; later exercises will remove it.

Practice and recall

Record the manager, unit file, main PID, active and enabled states, listener and client response. Explain why starting a process and obtaining a successful HTTP response answer different questions. Compare `systemctl list-units` with `systemctl list-unit-files` and describe why their counts differ.

Execution Environment, Identity and Exit Status

A program that works in an interactive shell can fail as a service because its identity, working directory, environment or access to files differs. A unit must describe the execution context rather than depend on the operator's current shell.

Inspect what the manager will run

On `bl06-service-lab`, inspect the example created in the previous chapter:

```
systemctl cat bl06-demo.service
systemctl show bl06-demo.service \
  -p Type -p ExecStart -p User -p DynamicUser -p WorkingDirectory \
  -p MainPID -p Result -p ExecMainCode -p ExecMainStatus
systemctl status bl06-demo.service --no-pager
```

`systemctl cat` shows the fragment and drop-ins; `show` exposes effective properties and runtime results. The dynamic user's actual identity can be allocated at runtime and is not a persistent human account. A PID is a point-in-time observation and can change on restart. If access is denied or the service is absent, resolve the lab setup before continuing.

`ExecStart=` is parsed by systemd, not automatically by an interactive shell. Shell features such as redirection, pipelines and command substitution do not work as they do in a shell unless a shell is explicitly invoked; avoid constructing shell strings from untrusted input. Use absolute executable paths, and understand the unit's `WorkingDirectory=` and `User=`. The systemd execution manual describes execution context and environment.

Environment and privileges

`Environment=` and `EnvironmentFile=` can provide configuration. They are not a safe secret store: environment values can be exposed through manager interfaces or inherited by child processes. Prefer service credential mechanisms and appropriate file permissions for secrets. A user session's `PATH`, home directory and shell initialization files should not be assumed to exist in a system service. `WorkingDirectory=` failure can prevent execution entirely; permissions must allow the selected service identity to traverse every parent directory and read the needed files.

Run the application with the least privilege it needs. A service account, a dynamic identity and a human user's identity have different persistence and access properties. Dynamic identities are convenient for this stateless read-only example; persistent state requires explicitly managed directories and ownership. Do not use root merely because a permission error disappeared.

Read exit evidence

```
systemctl show bl06-demo.service \
  -p ActiveState -p SubState -p Result -p ExecMainCode -p ExecMainStatus
sudo journalctl -u bl06-demo.service -b -n 40 --no-pager
```

`ExecMainCode` and `ExecMainStatus` describe how the main process exited when that information is available; the unit's `Result` describes the manager's outcome. A service that never executed because setup failed has different evidence from one that executed and returned an error. A successful process exit can still be wrong for a long-running service, and `active` alone is not a functional check.

For the lab server, compare the process context with the unit properties and repeat the HTTP request from the previous chapter. Do not change the service identity or write to `/usr/share/doc`; that directory is package-managed. Later chapters address persistent service data and sandboxing.

Worked diagnosis and practice

Suppose a service reports `status=200/CHDIR`. Inspect the effective working directory and journal, determine whether the path exists and is accessible to the service's identity, and fix the documented path or permissions at their owning layer. Re-run a client test after restart. If the status instead indicates an executable or permission failure, follow that evidence rather than changing unrelated paths. Record the lab unit's effective execution context and explain why an environment variable is unsuitable for a password.

Readiness and Dependencies

Startup Types, Readiness, Watchdogs and Restarts

A process can exist before it can accept work. A database might still be replaying a log; a web application might be waiting for its configuration. Ordering another service after a process launch is useful only if the manager has a meaningful definition of readiness.

How a service reports startup

Systemd's `Type=` selects a startup protocol. With `simple`, the manager considers the service started after the process is forked, before confirming that the executable ran. `exec` waits until the executable has been invoked successfully, but does not prove that it has bound a port or can answer a request. `oneshot` waits for its start command to exit; it is appropriate for bounded work, not a daemon that stays in the foreground. `notify` expects an application that implements the notification protocol to send `READY=1` when it considers itself ready. Legacy daemons may use `forking`, with their own process-identification considerations.

The precise semantics, timeouts and other types are in the systemd.service manual. Do not switch a program to `Type=notify` unless it actually sends the required notification. A readiness signal is the application's assertion; an independent client check can still reveal a broken dependency or wrong response.

Examine the lab service

The `bl06-demo` Python HTTP server uses `Type=exec`. On the lab clone:

```
systemctl show bl06-demo.service \
  -p Type -p ActiveState -p SubState -p MainPID -p Restart -p NRestarts
sudo ss -ltnp 'sport = :18765'
python3 - <<'PYCODE'
import urllib.request
response = urllib.request.urlopen("http://127.0.0.1:18765/", timeout=5)
print(response.status)
PYCODE
```

The service state and listener are useful evidence; the HTTP response tests an actual request. They are snapshots and can change between commands. Python's basic server does not send systemd readiness notifications. For a real service, choose a documented startup protocol and test the required dependency and response semantics, including failure and timeouts.

Restart policy and watchdogs

`Restart=on-failure` requests a restart after qualifying failures; it does not make a failed dependency healthy and does not guarantee an endless retry loop. Start-rate limiting can suppress repeated starts. `RestartSec=` spaces attempts. Choose a policy with the service's data integrity and failure mode in mind, and inspect both the service logs and the manager's restart counters. A deliberately stopped service should not be used as a test of failure recovery.

A watchdog is a deadline for periodic notifications from a cooperating application. `WatchdogSec=` alone cannot teach an application to send watchdog messages. A process may send heartbeats while a dependent network service or client path is broken, so supplement the watchdog with meaningful health checks. Do not add watchdog settings to the Python lab server.

To inspect restart evidence without injecting a fault:

```
systemctl show bl06-demo.service -p Restart -p RestartUSec -p NRestarts
sudo journalctl -u bl06-demo.service -b -n 50 --no-pager
```

A high restart count suggests examining the first failure, not merely resetting the counter. If the unit reaches its start limit, diagnose and correct the cause before `reset-failed` or another start. The reset clears manager failure/rate-limit state; it does not fix the underlying application.

Worked diagnosis and practice

A dependent unit starts after a web service but its first request fails. Inspect the web unit's type and startup logs, listener and client response; `After=` on a `Type=exec` service provides ordering through execution, not proof that the web server is ready. Use the application's supported readiness protocol or design retries with bounded timeouts. Record the lab service's type, restart policy and HTTP response, and explain what each tells you and what it cannot establish.

Requirements, Ordering and Failure Propagation

A service can need another unit to be started, need it to start first, or need it to remain active. Those are separate relationships. Confusing them can produce a boot that looks successful while the application starts before its prerequisite is ready.

Build the dependency model

`Wants=` pulls another unit into the transaction but is weak: its failure does not by itself prevent the requesting unit from starting. `Requires=` pulls in a stronger requirement; combined with `After=`, failure to activate the required unit prevents the ordered dependent from starting. `After=` and `Before=` specify ordering only and do not themselves pull another unit in. Shutdown order reverses the start order. `BindsTo=` with `After=` expresses a stronger ongoing association, but even an active unit is not necessarily an available remote endpoint. Conditions and unexpected deactivation have subtleties; use the systemd.unit manual when designing a real unit.

`network.target` does not guarantee an assigned address or remote connectivity. `network-online.target` is an optional synchronization point with meaning determined by the installed network stack and wait service; it cannot promise a remote database or DNS server will stay reachable. Applications need timeouts, retry and failure handling as well as unit dependencies.

Inspect the demo's actual relationships:

```
systemctl list-dependencies bl06-demo.service
systemctl show bl06-demo.service -p Wants -p Requires -p After -p Before
```

Implicit dependencies and defaults can add relationships absent from the unit file. The listing describes manager relationships, not the complete application dependency graph.

Lab: observe a failed prerequisite

On `bl06-service-lab`, create two temporary system units under `/run/systemd/system`; they disappear at reboot and are not enabled. The prerequisite exits unsuccessfully, and the dependent requires it and is ordered after it. First check both names and files are unused:

```
systemctl cat bl06-prereq.service bl06-dependent.service
ls -l /run/systemd/system/bl06-prereq.service \
  /run/systemd/system/bl06-dependent.service
```

Unit-not-found and missing-file messages are expected. If either unit or file exists, stop rather than overwrite it. Confirm `/usr/bin/false` and `/usr/bin/true` exist, then write the units:

```
sudo tee /run/systemd/system/bl06-prereq.service > /dev/null <<'UNIT'
[Unit]
Description=BL06 intentionally failing prerequisite
[Service]
Type=oneshot
ExecStart=/usr/bin/false
UNIT
sudo tee /run/systemd/system/bl06-dependent.service > /dev/null <<'UNIT'
[Unit]
Description=BL06 ordered dependent
Requires=bl06-prereq.service
After=bl06-prereq.service
[Service]
Type=oneshot
ExecStart=/usr/bin/true
RemainAfterExit=yes
UNIT
sudo systemd-analyze verify /run/systemd/system/bl06-prereq.service \
  /run/systemd/system/bl06-dependent.service
```

Resolve verification errors before continuing. Reload and start the dependent:

```
sudo systemctl daemon-reload
sudo systemctl start bl06-dependent.service
systemctl status bl06-prereq.service bl06-dependent.service --no-pager
sudo journalctl -b -u bl06-prereq.service -u bl06-dependent.service --no-pager
```

The start is expected to fail: the ordered requirement fails before the dependent can run. Compare unit result and journal messages. The dependent's own program has not demonstrated a failure; it was prevented from running by the dependency transaction. If the result differs, inspect the effective unit files and manager state before changing them.

After recording evidence, remove only the two units created in this exercise and clear their failure state:

```
sudo systemctl stop bl06-dependent.service bl06-prereq.service
sudo systemctl reset-failed bl06-dependent.service bl06-prereq.service
sudo rm -- /run/systemd/system/bl06-dependent.service \
  /run/systemd/system/bl06-prereq.service
sudo systemctl daemon-reload
```

Keep `bl06-demo.service` for subsequent chapters. No real system prerequisite was modified. In a production incident, preserve failure evidence before resetting status and check the implications of stopping a unit and its dependents.

Practice and recall

Draw two arrows: one for pulling in a prerequisite and one for ordering it. Explain what changes if `Requires=` becomes `Wants=`, and why `After=` alone would not start the prerequisite. State what this lab does and does not prove about a continuously running remote dependency.

Running Multiple Kernels on One Bare-Metal Server

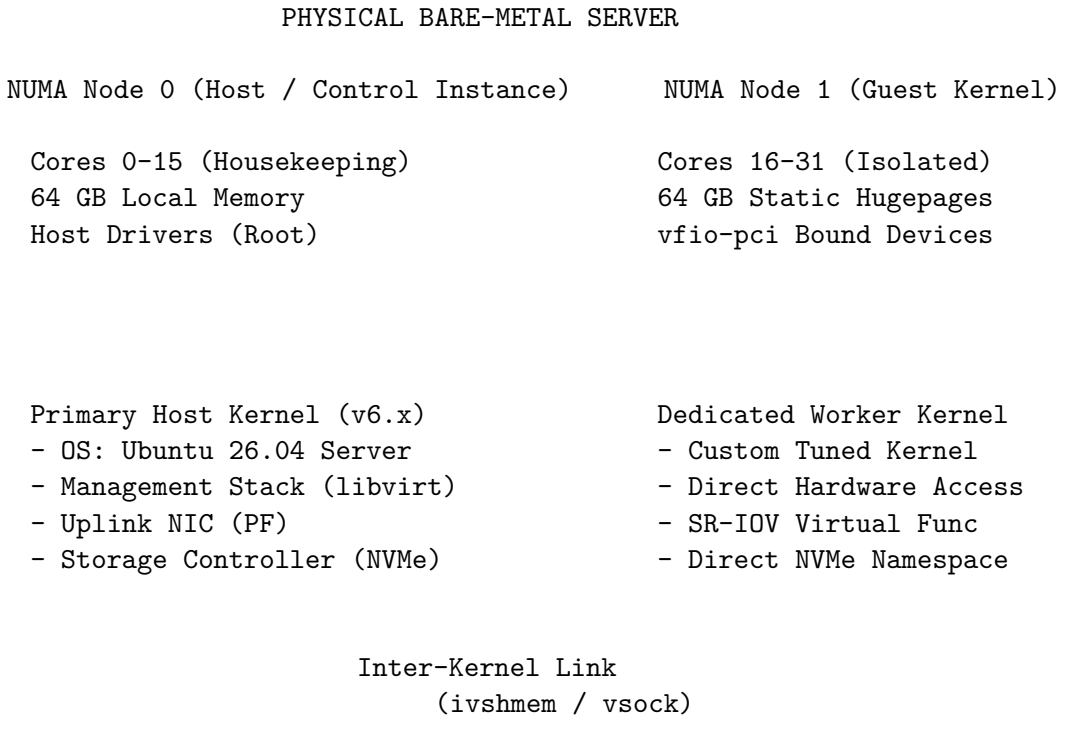
Running Multiple Kernels on One Bare-Metal Server

Running Multiple Kernels on One Bare-Metal Server

Modern bare-metal servers offer extraordinary core counts, multi-terabyte memory configurations, and high-bandwidth PCIe fabrics. However, the standard monolithic Linux architecture operates under the assumption that a single kernel image owns and arbitrates all physical machine resources. In high-performance, real-time, isolated multi-tenant, and fault-tolerant production environments, running multiple independent kernel instances on one physical machine provides unprecedented isolation and predictability without the overhead of enterprise hypervisor virtualization layers.

This section covers the architecture, mechanics, isolation primitives, performance characteristics, and production operations required to partition bare-metal hardware and run multiple operating system kernels concurrently and transitions safely.

Curriculum Architecture



Chapter Progression

Chapter	Title	Scope & Core Mechanism
Intro	Introduction: The Multikernel Problem	Why multiple kernels, definition of multikernel, approach landscape
01	Linux Kernel Fundamentals	Monolithic resource monopoly, boot sequence, hardware enumeration, NUMA
02	The Boot Process in Depth	EFI environment, bootloaders, cmdline isolation, initramfs, userspace handoff
03	Virtualization vs True Bare-Metal Multi-Kernel	Type-1 hypervisors, KVM architecture, containers vs microVMs vs partitioning
04	kexec and Kernel-to-Kernel Transitions	kexec mechanics, kdump crash dumping, fast reboots, limitations
05	Hardware-Assisted Multi-Kernel with KVM/QEMU	KVM setup, CPU pinning, NUMA-aware allocation, static memory guarantees
06	Device Assignment and I/O Isolation	VFIO, SR-IOV virtual functions, dedicated NICs, NVMe partitioning, GPUs
07	Networking Between Kernel Instances	Internal fabrics, external routable links, vhost-net, ivshmem, virtio-vsock
08	Resource Isolation Strategies	`isolcpus`, `nohz_full`, cpusets, IRQ affinity routing, LLC cache isolation
09	Security Analysis and Threat Modeling	VM escape CVEs, Secure Boot, IOMMU DMA attack mitigation, memory encryption
10	Observability and Operations	Telemetry, cross-kernel logging, kdump integration, PMU hardware counters
11	Performance Engineering	Virtualization tax quantification, TLB shootdowns, NUMA latency, virtio vs VFIO
12	Production Operations and Lifecycle Management	Rolling kernel upgrades, livepatch, automated config, maintenance windows
13	Troubleshooting Guide	Early boot hangs, VFIO reset errors, network packet drops, panic diagnosis

Chapter	Title	Scope & Core Mechanism
14	Architecture Comparisons and Decision Framework	Multi-kernel vs bare-metal servers, hypervisors vs containers, decision matrix
15	Complete Project Plan: From Requirements to Decommissioning	5-phase execution plan: assessment, lab validation, build, rollout, retirement
Outro	Conclusion	Future disaggregated hardware, CXL evolution, and production principles

Introduction: The Multikernel Problem

Introduction: The Multikernel Problem

The modern server compute model is dominated by high-density multi-socket CPU architectures, massive physical memory footprints, and complex PCIe topologies. In this landscape, operating systems face an architectural paradox: the monolithic Linux kernel was designed under the assumption of complete, centralized ownership of physical hardware. As hardware scales to hundreds of logical cores and terabytes of RAM across non-uniform memory access (NUMA) nodes, monolithic kernel scalability bottlenecks, shared-lock contention, and single-point-of-failure risks become acute.

This chapter frames the multikernel problem, explains the motivation for hosting multiple operating system kernels on a single physical bare-metal server, clarifies architectural terminology, and charts the landscape of approaches.

Why You Want Multiple Kernels

Deploying multiple independent kernel instances on single physical bare-metal hardware addresses specific production requirements that cannot be satisfied by standard single-kernel containerization or traditional bloated virtualization:

1. **Mixed-Criticality Workload Consolidation**: Enterprises frequently need to run deterministic, real-time control applications (requiring PREEMPT_RT or specialized low-latency kernels) alongside standard throughput-oriented batch processing or database workloads. Running both workloads under a single kernel subjects the deterministic tasks to kernel lock contention, scheduling jitter, and unpredictable memory reclaim stalls.
2. **Strict Fault Domain Isolation**: In a single-kernel system, a kernel panic or unhandled Machine Check Exception (MCE) in any driver or subsystem halts the entire physical node, taking down every hosted process. Multi-kernel configurations decouple the failure domains: a crash, kernel panic, or runaway memory leak in one kernel instance leaves the other instances executing uninterrupted.
3. **Conflicting Kernel Configuration & Version Dependencies**: Certain enterprise workloads, proprietary hardware drivers, or specialized storage fabrics require specific kernel versions, patch levels, or conflicting compile-time options (such as page sizes: 4KB vs. 64KB pages on ARM64). Multi-kernel architectures allow disparate kernel versions and configurations to co-exist natively on one machine.
4. **Zero-Overhead Multi-Tenancy**: Traditional enterprise hypervisors introduce a 5% to 15% virtualization tax in CPU scheduling overhead, nested page table walks, and shared hypervisor lock contention. By dedicating physical cores, NUMA nodes, and PCIe devices directly to partitioned kernels, systems achieve near-100% bare-metal performance while enforcing hardware-grade isolation.

5. **Security and Attack Surface Reduction**: Shared-kernel containerization shares the underlying kernel syscall boundary, memory subsystem, and device drivers across tenants. A privilege escalation vulnerability or kernel exploit in a container compromises the entire node. Multiple isolated kernels isolate security boundaries at the hardware-enforced CPU privilege rings and IOMMU layers.

What Multikernel Actually Means

The term **Multikernel** originated in academic operating system research (notably the *Barrelfish* operating system from ETH Zurich and Microsoft Research), which argued that modern multicore servers should be treated not as a single symmetric multiprocessor (SMP) system, but as a distributed network of independent cores communicating via message passing.

In the context of production Linux systems engineering on bare-metal hardware, **Multikernel** refers to architectures where:

- Multiple independent OS kernel images execute on the same physical host simultaneously or transitionally.
- Memory is statically partitioned or hardware-enforced such that each kernel owns its distinct physical address ranges.
- CPU cores are statically partitioned into non-overlapping sets, eliminating inter-kernel CPU scheduling contention.
- Peripheral devices (NICs, NVMe controllers, GPUs) are explicitly assigned to specific kernels via IOMMU groups and PCI passthrough rather than multiplexed through software emulators.
- Inter-kernel coordination occurs over explicit network protocols, message queues, or hardware-mediated shared memory rather than shared OS data structures.

The Landscape of Approaches

Running multiple kernels on one bare-metal server spans several technological mechanisms:

Approach	Concurrency	Hardware Isolation Mechanism	Typical Use Case
True Hardware Partitioning (LPAR/BPAR)	Concurrent	Hardware backplane & firmware domain routing	Mainframes, high-end mission-critical RISC/x86
Hardware-Assisted KVM/QEMU	Concurrent	Hardware VMX/SVM root vs non-root, EPT/NPT, VFIO	Cloud infrastructure, isolated bare-metal micro-clouds
MicroVMs (Firecracker / Cloud-Hypervisor)	Concurrent	Minimalist VMM, virtio-over-MMIO, restricted device model	Ephemeral workloads, serverless runtimes, secure sandboxing
Type-1 Bare-Metal Hypervisors (Xen / ESXi)	Concurrent	Thin bare-metal control layer (Ring -1)	Enterprise infrastructure consolidation

Approach	Concurrency	Hardware Isolation Mechanism	Typical Use Case
Sequential Kernel Transition (`kexec`)	Sequential	Firmware-bypassing in-memory kernel handoff	Fast reboots, crash dumping (`kdump`), live firmware updates
Shared-Kernel Containers (Namespaces / cgroups)	Single Kernel	Software isolation primitives within one kernel image	Microservices, stateless application deployment

How to Use This Book

This book is organized progressively into four architectural tiers:

1. **Foundations (Chapters 1–2)**: Deep exploration of the Linux kernel's initialization sequence, hardware discovery, memory allocation, EFI firmware handoff, and boot parameters.
2. **Core Multikernel Mechanics (Chapters 3–8)**: Comparing hypervisor models against bare-metal partitioning, mastering `kexec`, configuring KVM/QEMU for zero-overhead hardware virtualization, isolating PCIe devices with VFIO and SR-IOV, designing inter-kernel networks, and enforcing strict CPU/NUMA isolation.
3. **Operational Hardening (Chapters 9–13)**: Security threat modeling, cross-kernel telemetry and observability, performance benchmarking, lifecycle management, and systematic failure troubleshooting.
4. **Architecture & Project Planning (Chapters 14–15 & Conclusion)**: Decision matrices for architecture selection, and an end-to-end 5-phase operational deployment plan.

Chapter 1: Linux Kernel Fundamentals

Chapter 1: Linux Kernel Fundamentals

Before architecting a system that runs multiple kernels, we must understand why the standard Linux kernel expects to operate as an absolute monopolist over the host's physical resources. By examining the hardware initialization path, CPU topology discovery, and physical memory allocation algorithms, we can identify exactly which hardware structures must be partitioned, isolated, or virtualized to support concurrent kernel execution.

The Kernel as Resource Monopolist

The Linux kernel is historically a monolithic supervisor program designed to execute at the highest processor privilege level (Ring 0 on x86_64, EL1 on ARM64). In this design:

- **Single Address Space Authority**: The kernel assumes complete ownership of the physical address space. Physical addresses $0x00000000$ through the highest physical memory address are tracked by the kernel's internal page structures (`struct page` and large memory folios).
- **Interrupt Routing Monopoly**: The interrupt controller (such as the x86 Advanced Programmable Interrupt Controller - APIC, or ARM Generic Interrupt Controller - GIC) has its interrupt descriptor table (IDT) or vector table mapped and managed solely by the primary kernel.
- **Hardware Register Arbitrator**: System management registers, MSRs (Model Specific Registers), clock generators, and PCIe configuration space accesses are assumed to have a single authoritative master.

When two uncoordinated kernels attempt to write to the same hardware registers, program the same interrupt vectors, or allocate the same physical memory pages, immediate memory corruption and unrecoverable processor state exceptions occur.

Boot and Initialization Sequence

The Linux boot process follows a deterministic sequence from processor reset vector to userspace:

```
[Firmware / UEFI]    [Bootloader (GRUB/systemd-boot)]    [Kernel Entry (_start)]



[setup_arch()]    Initialize architecture, detect memory map from UEFI
[trap_init()]     Configure exception handling and interrupt vectors
[mm_init()]       Set up page allocator, zones, and buddy system
```

```
[sched_init()]    Initialize runqueues and CFS/EEVDF scheduler
[rest_init()]     Spawn kernel_init (PID 1) and kthreadd (PID 2)
```

At `setup_arch()`, the kernel parses the EFI memory map, claims all usable memory regions, and marks ACPI reserved tables. Any secondary kernel attempting to boot must receive its own clean, non-overlapping memory map.

Hardware Discovery and Enumeration

During early initialization, the kernel identifies and configures connected peripherals:

- **ACPI Table Parsing**: The kernel parses the Root System Description Pointer (RSDP) to discover the Multiple APIC Description Table (MADT), Differentiated System Description Table (DSDT), and SRAT (System Resource Affinity Table).
- **PCIe Bus Scanning**: The kernel reads the PCIe Extended Configuration Space (via MMCONFIG / ECAM) to enumerate all devices, functions, and bridges starting at Bus 0, Device 0, Function 0.
- **Resource Allocation**: The kernel assigns Base Address Registers (BARs) and allocates memory-mapped I/O (MMIO) and I/O port windows to each discovered PCIe device.

In a multi-kernel environment, peripheral enumeration must either be strictly virtualized or physically isolated using hardware IOMMU boundaries so that each kernel discovers only its assigned hardware subset.

CPU Topology and NUMA Architecture

Modern enterprise servers feature Non-Uniform Memory Access (NUMA) topologies where physical memory is distributed across multiple memory controllers attached directly to specific CPU sockets or dies:

- **NUMA Nodes**: A physical CPU socket or die and its directly attached local memory channels form a NUMA node. Accessing local memory yields minimal latency (~80ns), while accessing remote memory over the inter-socket bus (Intel UPI or AMD Infinity Fabric) incurs significant latency penalties (~140ns+).
- **Cores, Packages, and SMT**: A physical processor package contains multiple physical execution cores, each often split into two Symmetric Multithreading (SMT) logical threads. SMT threads share L1/L2 caches and execution execution units.
- **Core Complexes (CCX)**: Modern AMD EPYC architectures group 8 cores into Core Complexes sharing an L3 cache slice.

Multi-kernel architectures leverage NUMA boundaries naturally: a dedicated kernel can be bound completely to NUMA Node 1 (cores and local RAM), while the control kernel executes exclusively on NUMA Node 0, avoiding all cross-interconnect memory traffic.

Memory Management Fundamentals

The kernel organizes physical memory into distinct allocation tiers:

- **Zones**: Memory is partitioned into functional zones: `ZONE_DMA` (first 16MB for legacy ISA), `ZONE_DMA32` (memory under 4GB), and `ZONE_NORMAL` (directly mapped 64-bit physical memory).
- **Buddy Allocator**: The primary allocator manages memory in power-of-two page blocks ($4\text{ KB} \times 2^n$) up to `MAX_ORDER` (typically 4MB).
- **Slab/Slub Allocators**: Small kernel object caches (e.g., `task_struct`, `mm_struct`) are carved out of buddy pages to prevent external fragmentation.
- **Folios**: Modern Linux kernels (v6.x+) utilize large memory folios to manage compound pages efficiently across the page cache and anonymous memory.

To safely execute multiple kernels, memory isolation requires static physical carving—either via bootloader memory reservation parameters (`memmap=`), hypervisor-enforced Extended Page Tables (EPT), or hardware physical memory partitioning.

Chapter 2: The Boot Process in Depth

Chapter 2: The Boot Process in Depth

To understand how multiple kernels can be booted, partitioned, or transitioned sequentially on a physical bare-metal system, we must examine the boot path in rigorous detail. This chapter explores the UEFI environment, bootloader handoff protocols, kernel command-line configuration for resource isolation, initramfs architecture, and the transition to userspace.

Firmware and the EFI Environment

Modern x86_64 and ARM64 servers utilize Unified Extensible Firmware Interface (UEFI) specification v2.8+ firmware:

- **UEFI System Table**: The firmware exposes Boot Services (available only before the OS exits firmware) and Runtime Services (such as NVRAM variable access, available throughout OS runtime).
- **EFI System Partition (ESP)**: A standard FAT32 partition (GUID: `C12A7328-F81F-11D2-BA4B-00A0C93E`) containing UEFI binaries (`/EFI/BOOT/BOOTX64.EFI`, `/EFI/ubuntu/grubx64.efi`).
- **NVRAM Boot Variables**: Non-volatile variables (`Boot0001`, `BootOrder`) managed via `efibootmgr` determine the firmware boot priority and path to initial executables.

When bootstrapping an isolated multi-kernel environment, the UEFI firmware can either launch a primary bare-metal kernel (which subsequently provisions guest kernels), or a hypervisor shim layer (such as Xen or a minimal hypervisor bootloader) that partitions hardware before either kernel begins execution.

Bootloaders: GRUB, systemd-boot, and Others

The bootloader bridges firmware services and the operating system kernel:

- **GRUB2**: Complex, modular bootloader supporting complex scripting, dynamic menu generation, network PXE booting, and diverse filesystem drivers.
- **systemd-boot**: Minimalist, text-only UEFI boot manager implementing the Boot Loader Specification (BLS). Fast and deterministic, it relies strictly on UEFI filesystem drivers.
- **Unified Kernel Images (UKI)**: Single UEFI PE binaries combining the Linux kernel (`vmlinuz`), initramfs, boot command line, and an optional splash image into an authenticated, digitally signed executable.

Bootloader configuration allows operators to define distinct boot profiles: one for single-kernel production, one for hardware isolation and VFIO passthrough staging, and one for hypervisor-based partitioning.

Kernel Command-Line Parameters

Kernel command-line parameters passed via the bootloader configure hardware reservation and isolation mechanisms before any device drivers load. For multi-kernel host preparation, key parameters include:

Parameter	Purpose	Example Value
`isolcpus`	Isolates specified CPU cores from the CFS scheduler	`isolcpus=managed_irq,domain,16-31`
`nohz_full`	Enables tickless execution on specified cores	`nohz_full=16-31`
`rcu_nocbs`	Offloads RCU callback processing from isolated cores	`rcu_nocbs=16-31`
`intel_iommu` / `amd_iommu`	Enables hardware IOMMU translation units	`intel_iommu=on iommu=pt`
`hugepagesz` / `hugepages`	Reserves static 1GB or 2MB hugepages at boot	`hugepagesz=1G default_hugepagesz=1G hugepages=64`
`vfio-pci.ids`	Binds target PCIe device IDs directly to VFIO	`vfio-pci.ids=10de:2204,10de:1aef`
`memmap`	Carves out or reserves physical memory address ranges	`memmap=64G$64G`

Configuring these parameters ensures the primary host kernel does not bind to hardware resources intended for secondary isolated kernels.

Initramfs Construction and Role

The initial ramdisk (initramfs) is a gzipped or zstd-compressed CPIO archive loaded into memory alongside the kernel:

1. **Driver Staging**: Supplies kernel modules required to access the root storage fabric (such as `nvme`, `mpt3sas`, `multipath`, `virtio_blk`).
2. **Device Discovery & Crypto**: Executes `udevd` to discover block devices, opens LUKS encrypted volumes via `cryptsetup`, and assembles LVM or mdadm RAID arrays.
3. **VFIO Device Unbinding**: In multi-kernel architectures, early initramfs scripts can unbind specific PCIe devices from native drivers before root mount, binding them to `vfio-pci` to prevent host driver claiming.
4. **The `switch_root` Operation**: Mounts the real root filesystem, deletes the initramfs contents, moves the root mount to `/`, and executes the init binary (`/sbin/init` or `/lib/systemd/systemd`).

The Handoff to Userspace

The handoff from kernel to userspace occurs when `kernel_init()` invokes `run_init_process("/sbin/init")`:

- The process receives Process ID 1 (PID 1) and inherits execution context.
- Systemd establishes basic cgroup hierarchies, mounts essential virtual filesystems (`/proc`, `/sys`, `/dev`), and starts the `systemd-journald` logging service.
- Systemd evaluates target dependency units, brings up local networking, and starts virtualization or partitioning daemons (such as `libvirtd`, `qemu`, or custom partition controllers).
- In a multi-kernel server, a designated systemd target (`multikernel.target`) can automatically initialize hugepage pools, verify VFIO binding, and launch secondary kernel instances.

Chapter 3: Virtualization vs True Bare-Metal Multi-Kernel

Chapter 3: Virtualization vs True Bare-Metal Multi-Kernel

Achieving multi-kernel execution on physical hardware requires choosing between hardware-assisted virtualization, bare-metal hypervisors, lightweight microVMs, and true physical hardware partitioning. This chapter breaks down the architectural trade-offs, execution models, and isolation mechanics of each paradigm.

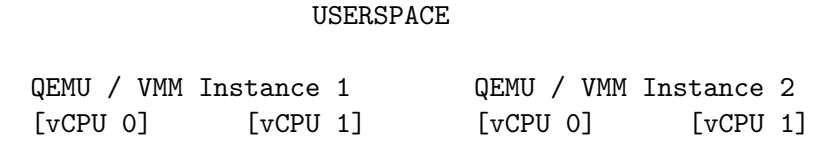
Type-1 Hypervisors: ESXi, Xen, KVM in Bare-Metal Mode

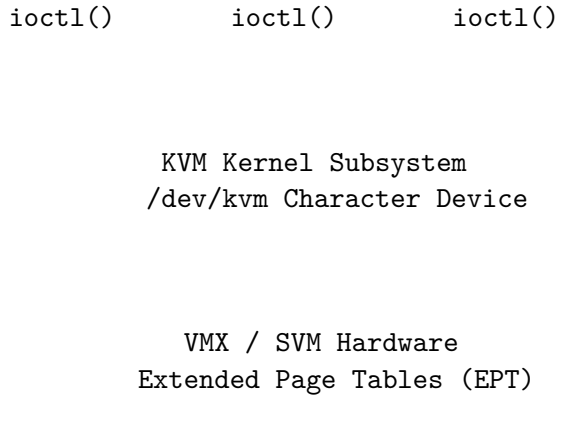
Hypervisors are classically categorized by their relationship to the underlying physical hardware:

- **Type-1 (Bare-Metal) Hypervisors**: Run directly on bare-metal hardware without an underlying general-purpose host OS.
  - **VMware ESXi**: Proprietary, closed-source bare-metal hypervisor (VMkernel). Tight hardware compatibility list (HCL), monolithic VM management.
  - **Xen**: Open-source microkernel hypervisor running in CPU Ring -1 (VMX root mode). Distinguishes between the privileged management domain (`dom0`) and unprivileged guest domains (`domU`). Hardware drivers run primarily in `dom0`.
- **KVM in Bare-Metal Mode**: Although technically a Linux kernel module (`kvm.ko`), KVM transforms a stripped-down Linux host into a Type-1 hypervisor. The Linux kernel runs directly on physical hardware in VMX root mode; guest kernels execute in VMX non-root mode. When tuned with CPU pinning, static memory allocation, and PCIe passthrough, KVM provides bare-metal performance with the maturity of the Linux driver ecosystem.

KVM Architecture Deep Dive

KVM (Kernel-based Virtual Machine) operates as a collaboration between a kernel-space module and a userspace Virtual Machine Monitor (VMM, typically QEMU, Cloud-Hypervisor, or Firecracker):





1. **The `/dev/kvm` Interface**: The VMM opens `/dev/kvm` and issues `KVM_CREATE_VM` to instantiate a virtual machine descriptor.
2. **vCPU Creation**: `KVM_CREATE_VCPU` creates execution threads. Each vCPU is an ordinary POSIX thread in userspace scheduled onto physical CPU cores.
3. **Execution Loop**: The VMM enters `KVM_RUN`. The processor executes a `VMLAUNCH`/`VMRESUME` instruction, transitioning the core into VMX non-root mode to execute guest kernel code natively at full clock speed.
4. **VM-Exits**: When the guest kernel executes privileged instructions (e.g., programming I/O ports, modifying CR3 page directory base register without shadowing, or triggering hardware interrupts), the hardware triggers a **VM-Exit**, returning control to KVM.

Linux Containers and Shared-Kernel Architectures

Linux containers (Docker, Podman, LXC/LXD) isolate processes using kernel software boundaries:

- **Namespaces**: Seven distinct namespaces (`pid`, `net`, `mnt`, `ipc`, `uts`, `user`, `cgroup`) partition the process's view of global system resources.
- **cgroups v2**: Enforce resource limits, accounting, and bandwidth throttling across CPU, memory, I/O, and pids.

Why Containers Fail the Multi-Kernel Requirement: - All containers share the single host kernel binary and runtime memory. - A kernel panic crashes every container simultaneously. - Different kernel versions, custom module requirements, and kernel command-line isolation parameters cannot be applied per-container. - Containers share the kernel syscall attack surface.

MicroVMs and Lightweight Virtualization

MicroVMs (such as **AWS Firecracker** and **Cloud-Hypervisor**) represent a convergence of container density and hypervisor-grade isolation:

- **Stripped-Down Device Model**: They completely eliminate legacy PC emulation (no IDE controllers, no floppy drives, no ACPI power states, no PCI buses).

- **Virtio-Over-MMIO**: Device I/O is handled strictly through memory-mapped I/O (MMIO) virtio queues.
- **Sub-50ms Boot Time**: Lightweight guest kernels boot from memory in tens of milliseconds with minimal memory footprint (~5MB overhead per instance).
- **Rust-Based Memory Safety**: The VMM implementation in Rust eliminates entire classes of buffer overflow and memory corruption vulnerabilities.

True Bare-Metal Partitioning Concepts

True bare-metal hardware partitioning achieves multi-kernel execution without any hypervisor software:

- **Physical Board-Level Partitioning**: Enterprise hardware (such as IBM Power Systems LPAR, HP Superdome nPars) uses physical hardware switches and crossbar fabric routing to split a chassis into electrically isolated hardware partitions.
- **Firmware-Partitioned Multikernels**: Modern systems use customized firmware (or early bootloader partitions) to carve non-overlapping physical CPU ranges and memory addresses, booting two distinct kernels side-by-side on bare silicon.
- **PCIe Fabric Partitioning**: Modern PCIe switches (such as Microchip Pax and Broadcom ExpressFabric) partition physical PCIe lanes and root complexes into distinct virtual hierarchies routed to separate physical hosts or processing clusters.

Chapter 4: kexec and Kernel-to-Kernel Transitions

Chapter 4: kexec and Kernel-to-Kernel Transitions

While hardware-assisted virtualization enables concurrent multi-kernel execution, many production operations require **sequential kernel transitions** on bare-metal hardware. Performing a standard server reboot triggers prolonged firmware POST memory training, UEFI device enumeration, and IPMI handshakes that can take 10 to 30 minutes on large multi-socket systems. The Linux `kexec` subsystem bypasses hardware firmware entirely, directly booting a new kernel from memory in under two seconds.

How kexec Works

The `kexec` mechanism operates in two distinct phases: load and execute.

```
[Running Kernel A]

          (kexec_load syscall)
[Stage Kernel Image, Initramfs & Cmdline into Reserved Memory Pages]

          (reboot syscall with LINUX_REBOOT_CMD_KEXEC)
[Quiesce Hardware & Drain Device DMA]

          (Disable Interrupts / Reset APIC)
[Jump to Sub-Trampoline Code]

          (Overwrites Memory with Staged Image)
[Entry Point of Kernel B (_start)]
```

1. **System Call `kexec_load()`**: The user space utility loads the new kernel (`vmlinuz`), initramfs, and kernel command-line strings into memory pages marked safe by the running kernel.
2. **Device Quiescence**: When execution is triggered, the kernel invokes `device_shutdown()` on all registered bus drivers, cleanly stopping DMA engines, clearing PCI device registers, and masking interrupts.
3. **Execution Jump**: The CPU transitions to real-mode or identity-mapped assembly code, disables the MMU, copies the staged kernel into its final physical load address, and jumps directly to the entry point of the new kernel.

kexec-tools Configuration and Usage

The `kexec-tools` package provides userspace control over kexec operations:

```
# 1. Install kexec-tools
sudo apt-get update && sudo apt-get install -y kexec-tools

# 2. Stage the new kernel into memory
sudo kexec -l /boot/vmlinuz-6.8.0-generic \
  --initrd=/boot/initrd.img-6.8.0-generic \
  --command-line="$(cat /proc/cmdline)"

# 3. Verify that the kernel is loaded
cat /sys/kernel/kexec_loaded
# Output: 1

# 4. Trigger instantaneous transition to the new kernel
sudo kexec -e
```

When systemd is configured with `systemctl kexec`, it unmounts filesystems, terminates services cleanly, and issues the kexec execute call automatically.

kdump for Crash Dumping

`kexec` serves as the foundation for enterprise Linux crash analysis (`kdump`):

- **Memory Reservation**: The host kernel reserves a contiguous block of physical memory at boot using the `crashkernel=` parameter (e.g., `crashkernel=512M`).
- **Crash Kernel Staging**: Early in userspace boot, the `kdump-tools` service loads a minimal capture kernel into this reserved memory region via `kexec -p`.
- **NMI / Panic Trigger**: When a fatal kernel panic or hardware Machine Check Exception occurs, the panicked kernel immediately executes `crash_kexec()`, jumping to the pre-loaded capture kernel without touching the primary memory image.
- **Dumping `/proc/vmcore`**: The capture kernel boots using only its reserved 512MB RAM, exposes the panicked kernel's entire physical memory at `/proc/vmcore`, and uses `makedumpfile` to compress and write the crash dump to disk or network storage.

Limitations of kexec for Multi-Kernel Goals

While indispensable for fast lifecycle transitions and crash analysis, `kexec` has fundamental architectural limitations:

- **Sequential, Not Concurrent**: `kexec` replaces the currently running kernel. It cannot execute two kernels side-by-side simultaneously.

- **Hardware State Leakage**: Bypassing hardware firmware initialization means peripheral controllers (especially complex PCIe cards, RAID HBAs, and GPUs) may retain dirty state, requiring driver-level reset routines (`FLR` - Function Level Reset) to avoid driver crashes in the new kernel.
- **In-Flight DMA Risks**: If an ill-behaved driver fails to cancel DMA mappings before transition, physical memory corruption can occur during the handoff.

Production Use Cases for kexec

1. **Zero-POST Kernel Upgrades**: Upgrading the production kernel on 4-socket, 4TB RAM bare-metal database nodes without paying the 20-minute firmware reboot penalty.
2. **Crash Analytics Pipelines**: Automated generation and network forwarding of diagnostic core dumps upon kernel panic across large bare-metal fleets.
3. **Staged Provisioning Transitions**: Booting an ephemeral discovery/provisioning OS in RAM, and immediately transitioning via kexec into the deployed target OS on disk.

Chapter 5: Hardware-Assisted Multi-Kernel with KVM/QEMU

Chapter 5: Hardware-Assisted Multi-Kernel with KVM/QEMU

When true concurrent multi-kernel execution is required on commodity x86_64 or ARM64 servers, hardware-assisted virtualization through KVM and QEMU provides the most robust, battle-tested foundation. However, running a secondary kernel with bare-metal performance guarantees requires abandoning default cloud-style overcommit behaviors. Every vCPU must be strictly pinned, memory statically backed by hugepages, and NUMA locality rigorously preserved.

KVM Setup and Kernel Module Configuration

To prepare an Ubuntu 26.04 host for low-latency multi-kernel operations:

1. **Verify Hardware Virtualization Extensions**:

```
egrep -c '(vmx|svm)' /proc/cpuinfo
# Must return core count > 0
```

2. **Load and Configure KVM Modules**: Create `/etc/modprobe.d/kvm.conf`:

```
# For Intel platforms
options kvm_intel nested=1
options kvm_intel preemption_timer=0
options kvm_intel enable_shadow_vmcs=1
options kvm ignore_msrs=1
```

3. **Verify Module Initialization**:

```
sudo modprobe kvm_intel
ls -la /dev/kvm
# Verify crw-rw----+ root kvm
```

Guest Kernel Installation and Configuration

Secondary kernels running alongside the host should be purpose-built to eliminate unnecessary drivers, emulated device probing, and ACPI bloat:

- **Minimalist `.config`**: Compile the guest kernel with only `CONFIG_VIRTIO`, `CONFIG_VFIO`, and specific hardware drivers needed for passed-through PCIe devices.

- **Direct Kernel Boot**: Rather than emulating a virtual disk containing a virtual bootloader (GRUB), QEMU can boot the guest kernel image directly from the host filesystem using the `-kernel`, `-initrd`, and `-append` arguments. This eliminates seconds of virtual firmware negotiation.

CPU Pinning and NUMA-Aware VM Placement

To eliminate scheduling latency, context-switching overhead, and cross-socket bus contention:

1. **Identify Host NUMA Topology**:

```
numactl --hardware
# Node 0: CPUs 0-15, 64GB RAM
# Node 1: CPUs 16-31, 64GB RAM
```

2. **Isolate Physical Cores for the Secondary Kernel**: Host boot command line: `isolcpus=domain,managed_irq,16-31 nohz_full=16-31 rcu_nocbs=16-31`.

3. **Pin QEMU vCPUs to Dedicated Physical Cores**: Each guest vCPU thread is mapped 1:1 to an isolated physical core. In libvirt XML:

```
<vcpu placement='static'>16</vcpu>
<cputune>
  <vcpupin vcpu='0' cpuset='16'/>
  <vcpupin vcpu='1' cpuset='17'/>
  <vcpupin vcpu='2' cpuset='18'/>
  <!-- ... through core 31 ... -->
  <emulatorpin cpuset='0-1'/>
</cputune>
<numatune>
  <memory mode='strict' nodeset='1'/>
</numatune>
```

Memory Ballooning vs Static Allocation

Default virtualization relies on memory ballooning (`virtio-balloon`) to dynamically reclaim memory from idle guests, and KSM (Kernel Samepage Merging) to deduplicate shared memory pages. **Both must be disabled for multi-kernel bare-metal workloads**:

- **Memory Ballooning Penalties**: Causes random guest kernel page allocation latency spikes when the balloon driver inflates.
- **KSM Overhead**: Periodically scans physical memory, invalidating page table entries and triggering TLB shootdowns.

- **Static Hugepages Allocation**: Back the secondary kernel's entire memory footprint with 1GB static hugepages allocated exclusively from NUMA Node 1:

```
echo 64 | sudo tee /sys/devices/system/node/node1/hugepages/hugepages-1048576kB/nr_huge
```

In QEMU/libvirt configuration:

```
<memoryBacking>
  <hugepages>
    <page size='1' unit='G' nodeset='1'/>
  </hugepages>
  <locked/>
</memoryBacking>
```

The `<locked/>` directive executes `mlock()`, guaranteeing the secondary kernel's pages are permanently locked in physical RAM and cannot be swapped out under any circumstance.

Complete Working Deployment Example

Here is a standalone bash script launching an independent Linux kernel on NUMA Node 1:

```
#!/usr/bin/env bash
set -euo pipefail

GUEST_KERNEL="/opt/kernels/vmlinuz-6.8.0-rt"
GUEST_INITRD="/opt/kernels/initrd.img-6.8.0-rt"
GUEST_CMDLINE="console=ttyS0 root=/dev/nvme0n1p1 rw isolcpus=1-15 nohz_full=1-15"

taskset -c 16-31 qemu-system-x86_64 \
  -name worker-kernel-node1,debug-threads=on \
  -enable-kvm \
  -cpu host,migratable=off,+invtsc \
  -smp 16,sockets=1,cores=16,threads=1 \
  -m 64G \
  -object memory-backend-memfd,id=mem1,size=64G,hugetlb=on,hugetlbsize=1G,host-nodes=1,polic
  -numa node,memdev=mem1,nodeid=0 \
  -kernel "$GUEST_KERNEL" \
  -initrd "$GUEST_INITRD" \
  -append "$GUEST_CMDLINE" \
  -nographic \
  -serial mon:stdio
```

This configuration delivers deterministic microsecond execution directly on bare metal alongside the primary management kernel.

Chapter 6: Device Assignment and I/O Isolation

Chapter 6: Device Assignment and I/O Isolation

Emulating virtual I/O devices through software models introduces CPU traps, memory copies, and latency jitter. To achieve true bare-metal throughput and deterministic latency across concurrent kernels, physical peripheral devices must be decoupled from the host and assigned directly to secondary kernel instances. This chapter covers the mechanics of VFIO (Virtual Function I/O), IOMMU group validation, SR-IOV network slicing, storage controller dedication, and accelerator assignment.

PCI Passthrough (VFIO) Fundamentals

The Virtual Function I/O (VFIO) framework is the standard Linux subsystem for exposing direct device access to userspace drivers and virtual machine instances in a secure, memory-protected manner:

- **IOMMU Protection**: VFIO relies on hardware IOMMU (Intel VT-d or AMD-Vi) units to program DMA remapping tables. A device passed to a guest kernel can only perform DMA transfers into physical memory pages explicitly mapped to that guest.
- **IOMMU Groups**: The hardware enforces isolation at the granularity of IOMMU groups. An IOMMU group represents the smallest set of physical devices that cannot be differentiated from one another by the PCIe root complex (often due to PCIe bridges lacking Access Control Services - ACS).
- **Rule of VFIO Assignment**: Every device in an IOMMU group must either be assigned to the same guest or bound to the `vfio-pci` stub driver on the host.

To inspect IOMMU groups:

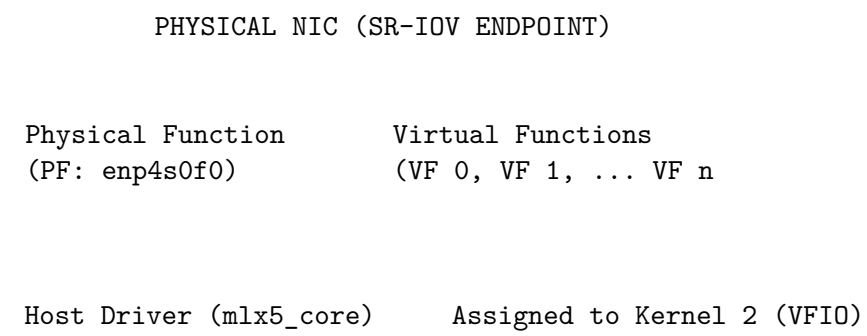
```
for d in /sys/kernel/iommu_groups/*/devices/*; do
  n=${d#*/iommu_groups/*}; n=${n%%/*}
  printf 'IOMMU Group %s ' "$n"
  lspci -nns "${d##*/}"
done
```

SR-IOV and Virtual Functions

Single Root I/O Virtualization (SR-IOV) is a PCIe standard that allows a single physical PCIe endpoint to partition itself into multiple distinct hardware interfaces:

- **Physical Function (PF)**: The primary PCIe device containing full configuration space, configuration registers, and administrative control.

- **Virtual Functions (VF)**: Lightweight PCIe functions carved out of the PF, each with its own independent configuration space, Base Address Registers (BARs), and transmit/receive queue pairs.



Enabling VFs on an enterprise NIC (e.g., Mellanox ConnectX-6 or Intel E810):

```
# Provision 4 Virtual Functions on physical interface enp4s0f0
echo 4 | sudo tee /sys/class/net/enp4s0f0/device/sriov_numvfs

# Verify new PCIe addresses
lspci | grep -i "Virtual Function"
```

Network Interface Assignment

Once VFs are spawned, bind a target VF to `vfio-pci`:

```
# 1. Unbind VF from default kernel driver
echo "0000:04:00.1" | sudo tee /sys/bus/pci/devices/0000:04:00.1/driver/unbind

# 2. Bind to vfio-pci
echo "15b3 101e" | sudo tee /sys/bus/pci/drivers/vfio-pci/new_id
echo "0000:04:00.1" | sudo tee /sys/bus/pci/drivers/vfio-pci/bind
```

The secondary kernel now owns the virtual network interface directly. Packets bypass the host operating system entirely, moving directly between physical wire and the secondary kernel's memory rings with sub-microsecond latency.

Storage Device Partitioning

Multi-kernel storage assignment balances capacity and I/O isolation:

1. **Direct NVMe Controller Passthrough**: Pass an entire physical PCIe NVMe M.2 or U.2 drive to the secondary kernel via VFIO. The guest kernel loads its native NVMe driver, programs submission/completion queues directly, and achieves full drive IOPS (e.g., 1.5M IOPS).
2. **NVMe Namespace Delegation**: Enterprise NVMe drives support hardware namespaces. One controller exposes multiple distinct namespaces (`/dev/nvme0n1`, `/dev/nvme0n2`). Individual namespaces can be formatted and mapped to secondary kernels via virtio-blk or raw block passthrough.
3. **Raw Block Delegation (`virtio-scsi` / `virtio-blk`)**: Pass a dedicated physical disk partition directly with cache disabled (`cache=none,aio=native`), bypassing the host page cache.

Specialized Hardware: GPUs, FPGAs, Accelerators

Modern AI and low-latency financial systems dedicate accelerators directly to specific kernels:

- **GPU Passthrough**: Isolate NVIDIA/AMD PCIe GPUs and their corresponding HDMI/audio endpoints into VFIO, preventing host `nouveau` or `amdgpu` drivers from claiming them.
- **PCIe Bifurcation**: Configure BIOS bifurcation settings (e.g., splitting a 16x PCIe slot into 4x/4x/4x/4x) to route physical lanes to independent riser cards.
- **Interrupt Routing (MSI-X)**: Ensure physical interrupt lines are delivered directly to the target CPU cores via hardware interrupt remapping without host intervention.

Chapter 7: Networking Between Kernel Instances

Chapter 7: Networking Between Kernel Instances

When multiple operating system kernels execute concurrently on a single bare-metal chassis, they rarely operate in total isolation. Control planes, storage synchronization, telemetry aggregation, and distributed service communication demand high-bandwidth, ultra-low-latency inter-kernel connectivity. This chapter covers internal virtual network architectures, external network topologies, performance optimization, and memory-mapped inter-kernel communication.

Internal Networking Architectures

Connecting co-located kernel instances can be achieved through multiple internal mechanisms:

1. **Linux Bridge with Tap Interfaces**:
    - Standard, simple bridge (`br0`) bridging host tap devices.
    - Low complexity, but packets traverse host software bridge code, incurring latency overhead (~15–25µs).
2. **Macvtap (Direct Bridge Mode)**:
    - Replaces the software bridge by attaching guest tap endpoints directly to a physical host network interface.
    - In `bridge` mode, co-located endpoints communicate directly within the network driver stack without external switch reflection.
3. **Open vSwitch (OVS) with DPDK Datapath**:
    - Kernel-bypass software switching using polling threads.
    - Ideal for complex VLAN, VXLAN, or flow-table routing between multiple instances, but consumes dedicated CPU polling cores.

External Network Configuration

For production multi-kernel servers connected to data center fabrics:

- **Dedicated Physical Uplinks**: Server Motherboards with multiple 25G/100G interfaces dedicate physical Port 0 to the primary management kernel and Port 1 to the secondary worker kernel.
- **VLAN Trunking over SR-IOV**: A single physical 100G uplink carries 802.1Q tagged traffic. The host administrator configures the physical function (PF) to assign VLAN tags automatically to specific Virtual Functions (VFs):

```
sudo ip link set enp4s0f0 vf 0 vlan 100 qos 5
sudo ip link set enp4s0f0 vf 1 vlan 200 qos 3
```

The guest kernels receive clean, untagged traffic natively isolated at the switch/NIC silicon layer.

Network Performance Optimization

Where virtual devices (`virtio-net`) must be used instead of SR-IOV:

- **vhost-net Acceleration**: Move packet processing from QEMU userspace directly into a dedicated kernel thread (`vhost-<pid>`).
- **Multi-Queue Virtio**: Match the number of virtio transmit/receive queues to the number of allocated vCPUs (`queues=16`), distributing packet processing across all guest cores via Receive Side Scaling (RSS).
- **Offload Configuration**: Enable TCP Segmentation Offload (TSO), Generic Segmentation Offload (GSO), and Generic Receive Offload (GRO) to maximize single-stream throughput up to 40Gbps+.

Inter-Kernel Communication Patterns

For microsecond-scale inter-kernel coordination:

- **Inter-VM Shared Memory (IVSHMEM)**: Exposes a shared POSIX shared memory region (`/dev/shm`) directly as a PCI MMIO device inside secondary kernels. Instances read and write to the same physical RAM pages without any TCP/IP or network stack overhead (<1µs latency).
- **Virtio-VSOCK**: Zero-configuration socket family (`AF_VSOCK`) enabling point-to-point stream and datagram communication between host and guest kernels:

```
struct sockaddr_vm sa = {
    .svm_family = AF_VSOCK,
    .svm_cid = VMADDR_CID_HOST,
    .svm_port = 1234
};
connect(fd, (struct sockaddr*)&sa, sizeof(sa));
```

Bypasses IP routing, firewall tables, and packet serialization.

Complete Networking Configuration Example

The following script configures an isolated low-latency internal network bridge and a secondary high-speed vsock channel:

```
#!/usr/bin/env bash
set -euo pipefail

# 1. Create isolated internal bridge
sudo ip link add br-internal type bridge
sudo ip addr add 192.168.100.1/24 dev br-internal
sudo ip link set br-internal up

# 2. Spawn tap device with multi-queue support
sudo ip tuntap add dev tap0 mode tap vnet_hdr multi_queue
sudo ip link set tap0 master br-internal up

# 3. Launch secondary kernel with vhost-net and vsock
qemu-system-x86_64 \
  -enable-kvm \
  -smp 8 \
  -m 16G \
  -netdev tap,id=net0,ifname=tap0,script=no,downscript=no,vhost=on,queues=8 \
  -device virtio-net-pci,netdev=net0,mq=on,vectors=18 \
  -device vhost-vsock-pci,id=vhost-vsock-pci0,guest-cid=3 \
  -kernel /opt/kernels/vmlinuz-worker \
  -append "console=ttyS0 ip=192.168.100.2::192.168.100.1:255.255.255.0::eth0:none"
```

This configuration yields full internal network routing alongside high-speed programmatic vsock channels.

Chapter 8: Resource Isolation Strategies

Chapter 8: Resource Isolation Strategies

The primary objective of running multiple kernels on a single physical host is to eliminate cross-workload interference (the "noisy neighbour" effect). On high-performance hardware, interference rarely arises from raw CPU cycle exhaustion; rather, it manifests in shared resource contention: CPU cache thrashing, interrupt routing disruptions, NUMA interconnect saturation, and storage queue locking. This chapter details kernel tuning and isolation primitives to achieve deterministic performance.

CPU Isolation: isolcpus, nohz_full, rcu_nocbs

Linux kernels provide specialized boot parameters to shield specific CPU cores from general operating system interference:

1. `isolcpus=domain,managed_irq,<cpulist>`:
    - Removes the specified cores from the kernel's Completely Fair Scheduler (CFS/EEVDF) load-balancing domains.
    - The kernel will never automatically place or migrate any process onto these cores. Only tasks explicitly bound via `taskset`, `sched_setaffinity()`, or cpusets will execute here.
2. `nohz_full=<cpulist>`:
    - Puts specified cores into adaptive-ticks mode.
    - When exactly one task is running on an isolated core, the periodic scheduling timer tick (1000 Hz or 250 Hz) is completely turned off, eliminating timer interrupt jitter.
3. `rcu_nocbs=<cpulist>`:
    - Offloads Read-Copy-Update (RCU) callback invocations to housekeeping cores. RCU processing threads will never execute on isolated cores, preserving strict CPU execution budgets.

Example host kernel boot line:

```
isolcpus=domain,managed_irq,16-31 nohz_full=16-31 rcu_nocbs=16-31
```

Cpusets and Cgroup-Based CPU Management

Cgroups v2 `cpuset` controllers provide dynamic hierarchical partitioning of CPU cores and memory nodes:

```
# 1. Create a dedicated cpuset for the primary management stack
sudo mkdir -p /sys/fs/cgroup/management
echo "0-15" | sudo tee /sys/fs/cgroup/management/cpuset.cpus
echo "0" | sudo tee /sys/fs/cgroup/management/cpuset.mems

# 2. Create an isolated cpuset partition for secondary kernel worker instances
sudo mkdir -p /sys/fs/cgroup/isolated-workers
echo "root" | sudo tee /sys/fs/cgroup/isolated-workers/cpuset.cpus.partition
echo "16-31" | sudo tee /sys/fs/cgroup/isolated-workers/cpuset.cpus
echo "1" | sudo tee /sys/fs/cgroup/isolated-workers/cpuset.mems
```

Setting `cpuset.cpus.partition` to `root` marks these cores as an isolated partition, preventing any child or parent groups from sharing those CPU resources.

Memory Isolation and NUMA Policies

Cross-socket memory accesses incur severe interconnect delays and bandwidth throttling:

- **Strict NUMA Allocation**: Bind memory strictly to Node 1 via `numactl --membind=1`. If memory is exhausted on Node 1, allocations fail immediately rather than allocating slow remote memory from Node 0.
- **Hugepages Isolation**: Pre-allocate 1GB hugepages at boot time. Hugepages bypass the kernel's transparent hugepage (THP) daemon (`khugepaged`), eliminating memory compaction and page migration overhead.

Interrupt Routing and IRQ Affinity

By default, the `irqbalance` daemon distributes hardware interrupts across all available CPU cores:

- **The Problem**: A high-volume network interface or storage controller can fire 100,000+ interrupts per second, repeatedly interrupting tasks executing on isolated cores.
- **The Solution**: Mask isolated cores out of the `irqbalance` banned list, and pin device interrupts explicitly via `/proc/irq/<irq>/smp_affinity`:

```
# Exclude cores 16-31 from irqbalance in /etc/default/irqbalance
IRQBALANCE_BANNED_CPUS="00000000,ffff0000"

# Manually route host management NIC interrupts to Core 0
echo 1 | sudo tee /proc/irq/42/smp_affinity
```

I/O Scheduling and Storage Contention

When storage devices are shared (rather than passed through via dedicated NVMe controllers):

- **I/O Scheduler Selection**: Use `none` (passthrough) for high-speed NVMe drives to eliminate kernel I/O scheduling latency. Use `bfq` or `mq-deadline` on slower SATA SSDs to enforce fair queuing.
- **I/O Weight and Throttling**: Use cgroup v2 `io.weight` and `io.max` to prevent batch background processes in one kernel environment from starving low-latency transactions in another.

What Can and Cannot Be Safely Shared

Hardware Resource	Can Be Shared?	Mechanism / Risk
CPU Execution Pipeline	**No**	Core must be dedicated. SMT sibling threads must also be isolated.
Physical Memory (RAM)	**No**	Static physical reservation or EPT isolation required. No overcommit.
L3 Cache (LLC)	**Conditionally**	Shared by default on a die. Requires Intel RDT / AMD CAT to partition.
Memory Bus Bandwidth	**Conditionally**	Heavy memory streaming in one instance can saturate physical bus bandwidth.
PCIe Bus / Root Port	**Yes**	Multiple devices share PCIe bandwidth if root complex throughput is sufficient.
Power Budget (TDP)	**Yes (Shared)**	Heavy AVX-512 loads on one socket can cause thermal downclocking across cores.

Chapter 9: Security Analysis and Threat Modeling

Chapter 9: Security Analysis and Threat Modeling

Consolidating distinct workloads under a multi-kernel architecture requires rigorous security boundary analysis. Unlike shared-kernel container environments, multi-kernel topologies establish hardware-enforced CPU isolation and physical memory segregation. However, shared physical components—such as CPU microarchitectural buffers, PCIe root complexes, and hypervisor control surfaces—present unique attack surfaces. This chapter conducts a comprehensive security review and threat modeling analysis.

Attack Surfaces in Multi-Kernel Architectures

In a multi-kernel system, threats fall into three primary categories:

```
1. Hypervisor / VMM Control Plane Exploitation
   (QEMU bugs, KVM ioctl vulnerability, device model flaw)

2. Hardware Microarchitectural Side Channels
   (Spectre, Meltdown, MDS, L1TF, Downfall, Inception)

3. Peripheral DMA / PCIe Fabric Attacks
   (Direct memory access forgery, rogue interrupt injection)
```

1. **VMM / Device Model Surface**: Userspace emulators (QEMU) expose large C-based codebases. Flaws in virtio device emulation or PCI bus emulation can permit code execution in host userspace context.
2. **Side-Channel Leakage**: Concurrent execution on SMT sibling threads sharing L1/L2 caches enables cross-thread timing attacks that infer private cryptographic keys.
3. **PCIe DMA Attacks**: A malicious guest driver on a passed-through physical device might attempt to issue DMA requests targeting host memory pages outside its allocated boundary.

VM Escape Vulnerabilities and Recent CVEs

Reviewing historical vulnerabilities highlights boundary failure points:

- **VENOM (CVE-2015-3456)**: Buffer overflow in QEMU's legacy floppy disk controller code allowing root code execution on the host. *Mitigation: Compile QEMU with all legacy devices stripped, or use lightweight Rust VMMs (Firecracker).*
- **virtio-net Buffer Overflows**: Flaws in packet header parsing across guest-host boundaries. *Mitigation: Use vhost-net or direct VFIO SR-IOV passthrough.*
- **KVM Kernel Privilege Escalations (CVE-2022-27666 / CVE-2023-32233)**: Vulnerabilities in netfilter or paging code within the Linux kernel itself.

Secure Boot and Kernel Trust

To guarantee the integrity of every executing kernel instance:

- **UEFI Secure Boot Chain**: The physical host validates bootloader signatures via Microsoft KEK/DB or custom Machine Owner Keys (MOK).
- **Signed Unified Kernel Images (UKIs)**: Secondary kernels are packaged as authenticated UKIs signed by the enterprise internal root of trust.
- **TPM 2.0 Measured Boot**: Platform Configuration Registers (PCRs) record cryptographic measurements of firmware, bootloader, kernel, and command lines. Remote attestation verifies the host is in a trusted state before secrets are unsealed.

DMA Protection and IOMMU Security

The hardware IOMMU is the primary defensive barrier for device assignment:

- **IOMMU Page Tables**: Every DMA transaction originating from a PCIe device carries a requester ID (Bus:Device:Function). The IOMMU translates device addresses to host physical addresses using per-domain page tables.
- **Strict Isolation**: A device passed to a secondary kernel cannot access host memory or memory assigned to other kernels, preventing malicious DMA overwrites.
- **Interrupt Remapping**: Hardware interrupt remapping units validate MSI/MSI-X vectors, preventing rogue devices from injecting arbitrary interrupts into the host kernel.

Secrets and Credential Isolation

Hardware memory encryption ensures data protection even against a compromised host hypervisor:

- **AMD SEV-SNP (Secure Encrypted Virtualization - Secure Nested Paging)**: Encrypts guest RAM with unique hardware keys stored in the AMD Secure Processor. Prevents hypervisor reads, memory replay attacks, and state modification.
- **Intel TDX (Trust Domain Extensions)**: Isolates guest kernels into Trust Domains (TDs), shielding memory and CPU register state from the host VMM.

Comparing Multi-Kernel vs VM Security Posture

Feature	Containers	Traditional VMs	Dedicated Multi-Kernel (VFIO/NUMA)
Boundary Enforcement	Kernel software	Hypervisor + Hardware	Hardware VMX/SVM + IOMMU + Hardware Crypto
Shared Kernel Code	100%	0% (Separate kernels)	0% (Separate kernels)
Device Model Attack Surface	Low	High (Emulated devices)	Zero (Direct hardware passthrough)
Microarchitectural Protection	Weak	Moderate	Strong (Physical SMT & NUMA isolation)
Compromise Blast Radius	Entire host node	Limited to VM (unless escape)	Limited to instance; hardware-enforced

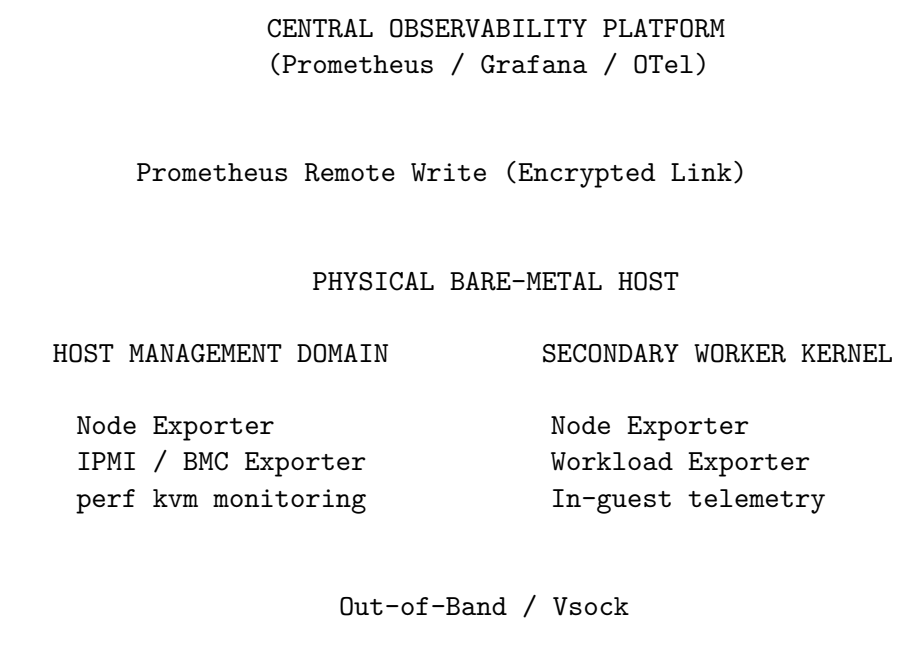
Chapter 10: Observability and Operations

Chapter 10: Observability and Operations

Operating multiple independent kernels on one physical server complicates telemetry, logging, crash diagnostics, and performance profiling. Because guest kernels execute in their own isolated address spaces, standard host monitoring tools cannot directly inspect their processes, memory allocations, or internal kernel states. This chapter establishes an end-to-end observability and telemetry architecture for multi-kernel environments.

Metrics and Monitoring Architecture

A robust telemetry pipeline combines hardware, host, and guest metric layers:



1. **Host-Level Metrics**: IPMI hardware sensors (chassis temperatures, fan speeds, power draw per socket), host CPU frequencies, and KVM VM-exit rates.
2. **Guest-Level Metrics**: In-guest `node_exporter` or OpenTelemetry collectors emitting OS counters (load averages, memory fragmentation, disk queue depths) over internal network interfaces or dedicated vsock sockets.
3. **Hardware PMU Counters**: Monitoring Performance Monitoring Unit (PMU) counters across sockets to detect memory bus saturation and cache line contention.

Logging Strategy Across Kernel Instances

Centralizing log streams without creating network loops or cross-kernel dependency stalls:

- **Isolated Journald Forwarding**: Secondary kernels forward systemd journal records to the host using `systemd-journal-remote` over the dedicated internal link (`br-internal` or vsock).
- **Serial Console Multiplexing**: QEMU instances output serial consoles to local Unix domain sockets or ptys (`-serial chardev:char0`), allowing the host system to capture early boot messages and panic logs even when network stacks are uninitialized.
- **Out-of-Band IPMI Serial-over-LAN (SoL)**: Captures host kernel output in the event of complete physical node lockups.

Crash Diagnostics and kdump Integration

Managing crashes across independent kernel instances:

- **Guest Kernel Panics**: Configure secondary kernels with independent `kdump` configurations. When a secondary kernel panics, it triggers a warm jump to its own local crash kernel, writing the dump to its dedicated NVMe or network storage volume without affecting the host.
- **Host Panics with Running Guests**: If the host kernel panics, it executes host-level kdump. Host kdump scripts should attempt to send graceful ACPI shutdown signals or freeze guest vCPUs before capturing the host memory image.

Performance Profiling Methodology

Investigating latency spikes and overhead in multi-kernel systems:

1. `perf kvm`: Profile guest and host cycles simultaneously:

```
# Record host and guest cycles with symbols
sudo perf kvm --host --guest record -a -F 99 sleep 10

# Analyze vm-exit causes and execution hotspots
sudo perf kvm stat report
```

2. **eBPF Tracing**: Trace hardware interrupt delays and vCPU wake-up latencies using BCC and bpftrace scripts targeting `kvm:kvm_exit` and `sched:sched_switch`.
3. **Hardware PMU Profiling**: Track Last-Level Cache (LLC) misses with `perf stat -e LLC-load-misses,LLC-store-misses`.

Health Checks and Alerting Design

Automated failure detection and recovery policies:

- **Hardware Watchdogs**: Modern motherboards feature IPMI hardware watchdogs. If the host kernel hangs, the watchdog resets the chassis after a configurable timeout (e.g., 60 seconds).
- **Software Watchdogs (`virtio-watchdog`)**: Guests configure virtual watchdog devices. If a secondary kernel locks up, QEMU receives an event (`WATCHDOG_ACTION_RESET`) and automatically reboots the secondary kernel instance.
- **Heartbeat Daemon over Vsock**: A lightweight watchdog daemon continuously exchanges heartbeats between host and guest. Missing three consecutive heartbeats initiates automated remediation.

Chapter 11: Performance Engineering

Chapter 11: Performance Engineering

Running multiple kernels on bare metal is often driven by stringent performance requirements. While hardware-assisted virtualization provides near-native execution speed, unoptimized configurations introduce subtle latencies, cache pollution, and I/O bottlenecks. This chapter quantifies the "virtualization tax," evaluates cache and NUMA effects, benchmarks storage and networking datapaths, and presents a rigorous methodology for measuring multi-kernel performance.

Boot Time Analysis

In ephemeral or fast-recovery architectures, kernel boot time is a critical operational SLA:

- **Firmware Overhead**: Physical UEFI initialization takes 60–600 seconds on server hardware.
- **Direct Kernel Boot vs. Virtual Firmware**: Direct kernel booting via QEMU (`-kernel`) reduces guest boot time from ~12 seconds (with SeaBIOS or OVMF) to <1.2 seconds.
- **Kernel Image Trimming**: Disabling unneeded drivers, modules, and early splash screens reduces kernel decompression and hardware probing time. Using uncompressed `vmlinux` with modern fast NVMe storage can yield boot times under 150ms.
- **Initramfs Optimization**: Constructing minimal static initramfs binaries (e.g., using BusyBox or custom Go/Rust binaries) cuts early userspace startup to tens of milliseconds.

CPU Overhead and Virtualization Tax

When CPU cores are strictly pinned (1:1 vCPU-to-pCPU mapping), where does overhead come from?

1. **VM-Exits**: A context switch between VMX non-root mode (guest execution) and VMX root mode (KVM handler) costs approximately 800 to 1,500 CPU cycles. Minimizing VM-exits is the primary goal of low-latency virtualization.
2. **Two-Dimensional Paging (EPT/NPT)**: Translating guest virtual addresses (GVA) to guest physical addresses (GPA), and then to host physical addresses (HPA), requires traversing nested page tables. In the worst case, a TLB miss costs up to 24 memory accesses.
    - *Mitigation: Backing memory with 1GB hugepages reduces page table depth to 2 levels and maximizes TLB coverage.*
3. **Timer Virtualization**: Modern processors feature hardware TSC scaling and Invariant TSC (`+invtsc`). Enabling this flag exposes the physical RDTSC instruction directly to the guest without hypervisor interception.

Cache Effects and NUMA Locality

Cross-node memory access severely degrades throughput:

```
Direct Local Memory Access (Node 1)    : ~75 ns latency, 150 GB/s bandwidth
Cross-Socket UPI/Fabric Access (Node 0): ~145 ns latency, 45 GB/s bandwidth
```

- **NUMA Misplacement Penalties**: If a secondary kernel running on Socket 1's cores allocates memory from Socket 0, every memory read traverses the inter-socket bus, halving bandwidth and doubling latency.
- **L3 Cache Slicing**: On multi-die architectures (AMD EPYC CCDs), tasks must be pinned within the same Core Complex (CCX) to avoid intra-socket L3 cache miss penalties.

I/O Performance: Virtio vs Passthrough

Empirical throughput and latency comparisons for 100GbE networking and NVMe storage:

I/O Datapath	Throughput (100GbE / NVMe)	P99 Latency	CPU Utilization
Emulated (e1000 / IDE)	1.8 Gbps / 45 MB/s	850 µs	High (100% core saturation)
Virtio (virtio-net / virtio-blk)	38 Gbps / 1,800 MB/s	35 µs	Moderate (vhost-net kernel thread)
SR-IOV Virtual Function (VFIO)	94 Gbps / N/A	3.2 µs	Minimal (zero hypervisor overhead)
PCI Passthrough (Direct NVMe)	N/A / 6,800 MB/s	8.5 µs	Native (exact bare-metal equivalence)

PCI passthrough and SR-IOV eliminate the hypervisor from the I/O datapath entirely, delivering 98–99% of raw bare-metal hardware capacity.

Benchmarking Methodology for Multi-Kernel Systems

Designing statistically valid benchmarks to evaluate multi-kernel isolation:

1. **Establish Single-Kernel Baseline**: Benchmark the physical server under single-kernel native Linux using standard tools (`fio`, `iperf3`, `wrk`, `stress-ng`).
2. **Co-Located Idle Test**: Deploy the secondary kernel in an idle state and re-run host benchmarks to measure baseline hypervisor noise.
3. **Noisy Neighbour Stress Test**: Generate intense cache-thrashing, memory bus saturation, and I/O pressure on Kernel A while recording the latency distribution of deterministic tasks on Kernel B.
4. **Latency Percentile Profiling**: Focus on high percentiles (P99, P99.9, P99.99) rather than averages. Multi-kernel isolation proves its value at the tail.

Chapter 12: Production Operations and Lifecycle Management

Chapter 12: Production Operations and Lifecycle Management

Operating multi-kernel bare-metal systems in 24/7 production environments requires deliberate operational discipline. Traditional single-server maintenance routines—such as rebooting for kernel security updates or upgrading device firmware—must be redesigned to preserve workload uptime across independent kernel instances. This chapter covers lifecycle management, rolling updates, configuration automation, disaster recovery, and decommissioning.

Kernel Upgrade Strategies

Updating operating system kernels without disrupting production services:

1. **Guest Kernel Upgrades**:
    - Secondary worker kernels can be restarted, patched, or kexec-rebooted independently without interrupting the primary management kernel or other peer instances.
    - Using direct kernel boot (`-kernel`), upgrading a secondary kernel requires simply pointing the VMM startup configuration to the new `/opt/kernels/vmlinuz-new` binary and triggering an in-guest restart.
2. **Host Hypervisor Kernel Upgrades**:
    - Upgrading the underlying host kernel requires a physical node restart.
    - In clustered deployments, workloads running in secondary kernels are drained or live-migrated to peer physical hosts before the host reboot.
3. **Livepatching (Canonical Livepatch / kpatch)**:
    - For critical CVEs, inject binary patches directly into running host and guest kernels without rebooting. Livepatch replaces vulnerable kernel functions at runtime using ftrace hook redirection.

Configuration Management at Scale

Declarative infrastructure-as-code ensures multi-kernel configurations remain reproducible:

- **Ansible / Puppet Automation**: Automated playbooks template `/etc/default/grub`, compile isolated cgroup hierarchies, configure `irqbalance` exclusion masks, and generate libvirt XML descriptors.

- **Drift Detection**: Automated audit jobs continuously verify that host kernel command lines match declared isolation parameters and that no stray processes are running on isolated CPU sets.

Disaster Recovery Architecture

Preparing for catastrophic hardware or storage loss:

- **State vs. Stateless Kernels**: Where possible, design secondary worker kernels to be completely stateless, mounting immutable root images (SquashFS / UKI) and writing data to network storage (Ceph / NFS) or dedicated partitioned NVMe arrays.
- **Host Configuration Backups**: Version-control the host's complete declarative configuration (network netplan files, VFIO bindings, udev rules, and QEMU launch scripts) in Git.
- **Cold-Start Rebuild Runbooks**: Document and regularly drill bare-metal reconstruction from clean PXE installation to multi-kernel deployment.

Maintenance Windows and Change Management

Managing physical node updates safely:

- **Drain Phase**: Inform upstream load balancers or cluster schedulers to divert traffic away from the node's secondary kernel instances.
- **Quiesce & Snapshot**: Stop running services cleanly, flush filesystem journals, and unmount storage volumes.
- **Firmware & BIOS Updates**: Apply vendor BIOS/BMC firmware updates via Redfish API or vendor tools (Dell iDRAC, HPE iLO).
- **Verification Gate**: Boot the physical host, run automated validation scripts checking CPU isolation, NUMA allocation, and VFIO device bindings, and then restart the secondary kernel instances.

Decommissioning and Migration Procedures

When retiring or repurposing bare-metal hardware:

- **Workload Migration**: Reallocate instances to newer hardware architectures.
- **Device Unbinding and Sanitization**: Unbind PCIe devices from VFIO, return them to standard kernel drivers, and execute cryptographic ATA/NVMe Sanitize commands (`nvme format /dev/nvmeX --ses=2`) to permanently purge data.
- **Firmware Reset**: Clear NVRAM boot variables, disable custom Secure Boot keys, and reset BMC credentials before hardware return.

Chapter 13: Troubleshooting Guide

Chapter 13: Troubleshooting Guide

Operating multi-kernel bare-metal systems introduces complex failure modes spanning firmware handoffs, IOMMU translation failures, PCIe reset issues, and cross-kernel resource contention. This chapter provides an evidence-based diagnostic guide to resolve the most common and challenging failure scenarios.

Boot Failures and Recovery

Symptom: Secondary Kernel Freezes on Early Boot

- **Diagnosis**: The secondary kernel hangs immediately after decompression with no output on the console.
- **Root Cause**: Console output is directed to a virtual VGA or serial port that is unmapped in the VMM configuration, or ACPI power tables conflict with guest direct boot.
- **Fix**: Add `earlyprintk=serial,ttyS0,115200 console=ttyS0 keep_bootcon` to the guest kernel command line. Ensure QEMU includes `-serial mon:stdio -nographic`. If ACPI hangs, test with `acpi=off` to isolate ACPI table parsing bugs.

Symptom: Host Panics During Kernel Boot with `isolcpus`

- **Diagnosis**: Host kernel enters a panic or hangs during SMP bringup when `isolcpus` or `nohz_full` is specified.
- **Root Cause**: Core 0 (the primary bootstrap processor - BSP) was inadvertently included in the isolation mask.
- **Fix**: Core 0 must always remain a housekeeping core. Ensure isolation ranges begin at Core 1 or higher (e.g., `isolcpus=domain,managed_irq,16-31`).

Device Assignment and VFIO Issues

Symptom: `Device is ineligible for IOMMU domain attach`

- **Diagnosis**: QEMU fails to start with error: `Failed to assign device: Device is ineligible for IOMMU domain attach due to other devices in group`.
- **Root Cause**: The PCIe device belongs to an IOMMU group containing other devices (e.g., PCIe root ports or onboard audio) that are still bound to host drivers.

- **Fix**: Check `ls /sys/bus/pci/devices/<pci_id>/iommu_group/devices/`. Either bind every device in that group to `vfio-pci`, install the device into an ACS-isolated PCIe slot directly connected to the CPU root complex, or enable PCIe ACS override patches (for lab/testing environments only).

Symptom: PCIe Device Fails Function-Level Reset (FLR)

- **Diagnosis**: Guest reboots fail with `Device failed to reset after FLR; device will be marked dead`.
- **Root Cause**: The physical PCIe controller does not implement standard PCIe FLR, or retains internal state across resets (common on certain AMD GPUs and RAID controllers).
- **Fix**: Use vendor-specific reset quirks (e.g., `vendor-reset` kernel module for AMD GPUs) or configure secondary bus reset (SBR) via `setpci`.

Networking Problems

Symptom: Asymmetric Routing or Dropped Packets on Inter-Kernel Bridge

- **Diagnosis**: Secondary kernel can ping host, but host cannot ping secondary kernel; or high packet drop rate under load.
- **Root Cause**: Netfilter/iptables rules on the host are filtering bridge traffic via the `br_netfilter` module, or MTU mismatches exist between tap interface and bridge.
- **Fix**:

```
# Disable bridge netfilter packet mangling
sudo sysctl -w net.bridge.bridge-nf-call-iptables=0
sudo sysctl -w net.bridge.bridge-nf-call-ip6tables=0

# Ensure matching MTUs across bridge and tap
sudo ip link set dev br-internal mtu 9000
sudo ip link set dev tap0 mtu 9000
```

Performance Degradation Diagnosis

Symptom: Unexplained High Latency Jitter (P99 Spikes)

- **Diagnosis**: Real-time tasks running in the secondary kernel experience 50µs+ latency spikes.
- **Investigation Steps**:

1. Check for unexpected VM-exits using `perf kvm stat live`. Look for excessive `CR3_LOAD` (TLB flushes) or `IO_INSTRUCTION` exits.
2. Verify that SMT sibling threads are not running unpinned host workloads.
3. Verify that `irqbalance` is not sending physical interrupts to isolated cores.

4. Check for Intel Turbo Boost frequency throttling or CPU C-state sleep transitions. Force C-state limiting with `intel_idle.max_cstate=1 processor.max_cstate=1`.

Kernel Panics and Crashes

Symptom: Host Freezes During Secondary Kernel Shutdown

- **Diagnosis**: The entire physical server locks up when stopping a secondary kernel instance.
- **Root Cause**: A passed-through PCIe device issued a DMA transaction to an unmapped host physical address immediately after the guest tore down its memory tables.
- **Fix**: Update motherboard BIOS/firmware to resolve IOMMU DMAR bugs. Verify that the host kernel command line includes `iommu=pt` to ensure physical translation bypass is enabled for host devices.

Resource Exhaustion

Symptom: `Cannot allocate memory` When Allocating Hugepages

- **Diagnosis**: Attempting to allocate static 1GB hugepages fails even though `free -m` reports sufficient free RAM.
- **Root Cause**: Memory fragmentation has broken contiguous physical memory blocks required for 1GB allocations.
- **Fix**: Pre-allocate hugepages at boot time via the kernel command line (`hugepagesz=1G default_hugepagesz=1G hugepages=64`) before physical memory becomes fragmented.

Chapter 14: Architecture Comparisons and Decision Framework

Chapter 14: Architecture Comparisons and Decision Framework

Systems architects frequently face the challenge of choosing the optimal compute isolation architecture for critical enterprise workloads. Operating systems engineering presents multiple viable deployment models: purchasing additional physical servers, deploying enterprise Type-1 hypervisors (ESXi/Xen), leveraging lightweight microVMs or unikernels, running shared-kernel containers, or architecting a dedicated multi-kernel bare-metal partition. This chapter establishes a formal decision framework and comparison matrix.

Multi-Kernel vs Multiple Physical Servers

Evaluation Factor	Multiple Physical 1U Servers	Single Bare-Metal Multi-Kernel Host
Capital Expense (CapEx)	High (Redundant PSUs, chassis, NICs, racks)	Low (Consolidated dense 2U/4U hardware)
Operational Expense (OpEx)	High power, cooling, and data center switch port costs	Low power, consolidated power supplies
Inter-Kernel Latency	Network-bound (10–50 µs over switch fabric)	Memory-bound (<1 µs via IVSHMEM / vsock)
Hardware Utilization	Low (stranded CPU/RAM on underutilized nodes)	High (dynamic or static allocation flexibility)
Hardware Fault Domain	Independent chassis (complete electrical isolation)	Shared physical motherboard, PSU, and backplane

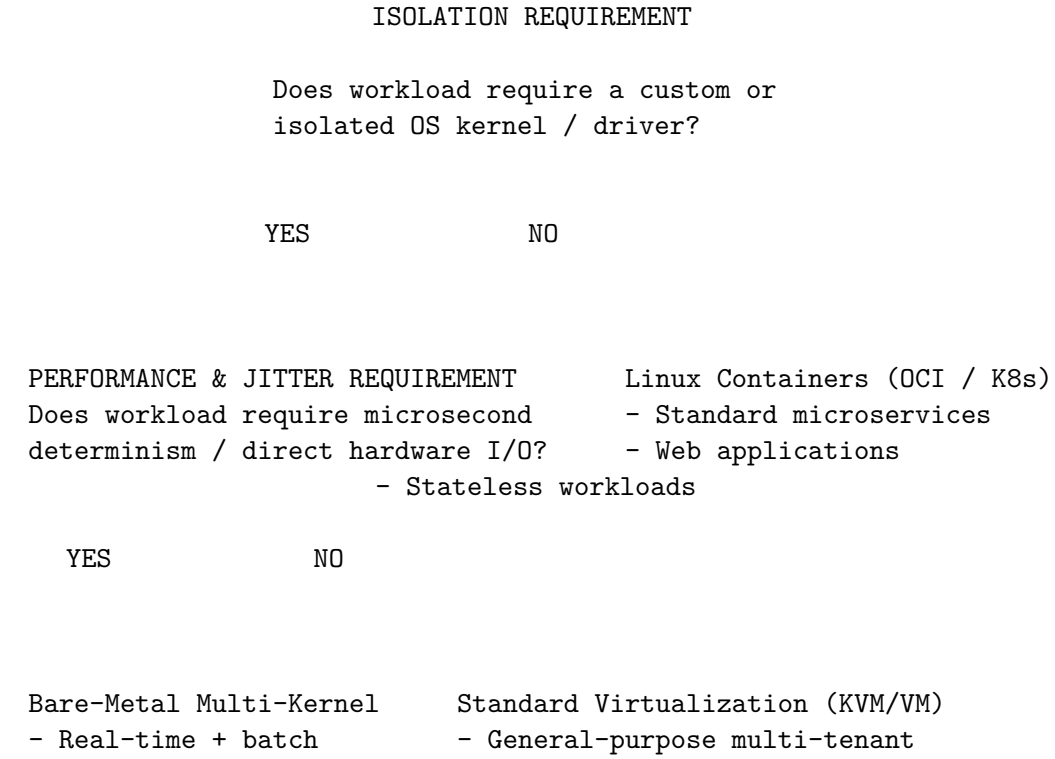
Takeaway: Multiple physical servers remain necessary only when complete chassis-level electrical or physical geographic fault isolation is required. For compute consolidation, multi-kernel topologies yield dramatic CapEx/OpEx savings.

KVM vs Xen vs ESXi for Multi-Kernel Workloads

- **VMware ESXi**: Excellent enterprise GUI management, but proprietary licensing, closed driver model, and high virtualization tax for low-latency/real-time tuning.
- **Xen**: Strong historical microkernel architecture, but complex operational management, declining hardware vendor support compared to KVM, and performance overhead in `dom0` I/O bounce buffering.

- **KVM in Bare-Metal Mode**: Native Linux driver ecosystem, direct integration with systemd, zero software licensing costs, and unmatched low-latency performance through VFIO, static hugepages, and CPU pinning.

Containers vs VMs vs Multi-Kernel: When to Use What



MicroVMs and Unikernels as Alternatives

- **Unikernels (OSv, MirageOS)**: Single-address-space library operating systems compiled directly with a single application. Extreme performance and minimal attack surface, but zero debugging tools, no POSIX shell, and difficult maintenance.
- **MicroVMs (Firecracker)**: Excellent for fast multi-tenant sandboxing and serverless functions (<5ms cold start), but restricted hardware device models preclude direct high-performance GPU/SR-IOV hardware assignment.

Hardware Partitioning Approaches

- **Proprietary Hardware Partitioning (IBM LPAR, HP nPars)**: Hardware backplane level splitting. Extremely costly and tied to proprietary architectures.

- **Linux Software-Defined Multi-Kernel**: Uses standard off-the-shelf commodity hardware (Dell PowerEdge, HPE ProLiant, Supermicro), standard Linux kernel primitives, and open-source tools to achieve 98% of proprietary hardware partitioning capability at a fraction of the cost.

A Decision Matrix for Architecture Selection

Criteria / Constraint	Recommended Architecture
Need different kernel versions or PREEMPT_RT on bare metal	**Multi-Kernel (KVM/VFIO Dedicated)**
Sub-microsecond inter-workload data exchange	**Multi-Kernel (IVSHMEM)**
100Gbps+ line-rate network throughput without CPU overhead	**Multi-Kernel (SR-IOV Passthrough)**
Hundreds of ephemeral microservices on one server	**Linux Containers / Kubernetes**
General-purpose multi-tenant hosting with overcommit	**Standard KVM / Proxmox / OpenStack**
Strict physical disaster survival	**Geographically Dispersed Physical Servers**

Chapter 15: Complete Project Plan: From Requirements to Decommissioning

Chapter 15: Complete Project Plan: From Requirements to Decommissioning

Translating multi-kernel concepts into a production deployment requires a structured, multi-phase project plan. This chapter outlines an enterprise-grade 5-phase execution plan: from initial hardware capability assessment and lab validation, to kernel configuration, automated rollout, and operational decommissioning.

Phase 1: Requirements Gathering and Hardware Assessment

Objective: Determine workload performance constraints and verify physical server capabilities before procurement or deployment.

1. **Workload Profiling**:
    - Quantify core requirements, memory bandwidth needs, storage IOPS, and network throughput for both primary and secondary workloads.
    - Establish baseline SLAs: maximum acceptable interrupt latency jitter (e.g., $< 15\mu$s P99), network throughput requirements, and recovery time objectives (RTO).
2. **Hardware Topology Audit**:
    - Verify CPU hardware virtualization support: `vmx` (Intel) or `svm` (AMD), Extended Page Tables (`ept`), and Invariant TSC (`invtsc`).
    - Audit NUMA topology: `numactl --hardware` or `lstopo` to map core packages, memory controllers, and PCIe slot affinities.
    - Verify IOMMU grouping and Access Control Services (ACS): ensure targeted PCIe slots occupy clean, isolated IOMMU groups.
    - Verify SR-IOV support on target Network Interface Cards (NICs).

Phase 2: Architecture Selection and Lab Validation

Objective: Validate the chosen isolation topology in a non-production staging environment.

1. **Topology Blueprint**:
    - Designate CPU allocation: Core 0–15 for Host Management (Node 0); Core 16–31 for Secondary Kernel (Node 1).
    - Designate Memory allocation: 64GB host memory; 64GB 1GB static hugepages bound to Node 1.

- Designate Device assignment: VFIO binding for target NVMe controller and SR-IOV Virtual Function.

2. **Lab Prototyping**:
   - Deploy prototype configuration on a staging server.
   - Inject synthetic load (e.g., `stress-ng` memory and CPU thrashing on Node 0) while executing microsecond latency tests (`cyclictest`) on the secondary kernel.
   - Verify that P99 latency on Node 1 remains under target thresholds despite saturated host bus activity.

Phase 3: Kernel Preparation and Configuration

Objective: Build, harden, and sign production kernel binaries and system configurations.

1. **Guest Kernel Build**:
   - Compile tailored production kernel with `CONFIG_PREEMPT_RT=y` (if real-time) or stripped driver set.
   - Build minimal static initramfs containing only required storage and network drivers.
   - Sign the generated UKI / kernel binary with internal enterprise Secure Boot keys.
2. **Host OS Provisioning**:
   - Configure host bootloader command line with `isolcpus`, `nohz_full`, `rcu_nocbs`, `intel_iommu=on`, and `hugepages`.
   - Implement `udev` rules to automatically bind target PCIe devices to `vfio-pci` upon boot.
   - Create systemd service units to initialize isolated cgroup partitions and hugepage pools before instance launch.

Phase 4: Deployment and Verification

Objective: Automated, repeatable deployment to production bare-metal servers.

1. **Automated Rollout via Configuration Management**:
   - Execute Ansible playbooks to apply host OS settings, stage kernel images, and configure QEMU/libvirt launcher scripts.
   - Trigger host reboot to verify clean bootloader parameter parsing.
2. **Production Verification Gates**:
   - Run automated verification scripts:
     - Check `cat /proc/cmdline` for correct isolation flags.
     - Verify `/sys/kernel/iommu_groups/` bindings.
     - Verify hugepage reservation on NUMA Node 1.
     - Launch secondary kernel and execute automated smoke tests (ping, NVMe read/write, vsock ping).
3. **Telemetry & Monitoring Integration**:

- Connect host and guest Prometheus exporters, verify Grafana dashboards, and test alert routing.

Phase 5: Operations, Maintenance, and Decommissioning

Objective: Sustained 24/7 operations, patching, disaster recovery, and hardware retirement.

1. **Daily Operational Care**:
   - Monitor hardware PMU counters for cache contention anomalies.
   - Audit systemd journal logs across host and guest instances.
   - Execute monthly failover and crash dump (`kdump`) verification drills.
2. **Patch Management & Change Execution**:
   - Apply livepatch security updates for zero-downtime maintenance.
   - Schedule maintenance windows for physical host BIOS/BMC firmware updates following standard drain runbooks.
3. **Hardware Decommissioning**:
   - Drain workloads to replacement hardware.
   - Execute cryptographic sanitization (`nvme sanitize`) on all storage drives.
   - Clear NVRAM and reset BMC to factory defaults before physical chassis decommissioning.

Conclusion

Conclusion

Running multiple operating system kernels on a single physical bare-metal server bridges the gap between the operational efficiency of hardware consolidation and the uncompromising performance of dedicated bare metal. By replacing the traditional monolithic model—where a single kernel is expected to arbitrate every hardware resource—with deliberate, hardware-enforced partitioning, systems engineers can achieve deterministic low latency, strict security boundaries, and robust fault isolation.

Key Architectural Takeaways

1. **Hardware as a Network of Resources**: Modern multi-socket, multi-core servers resemble distributed systems on a single silicon fabric. Treating them as partitioned domains with explicit memory and interconnect boundaries aligns software architecture with physical silicon realities.
2. **Elimination of the Virtualization Tax**: Hardware-assisted virtualization (KVM) combined with CPU pinning (`isolcpus`, `nohz_full`), static hugepages, and direct PCI passthrough (VFIO / SR-IOV) reduces hypervisor overhead from 15% to less than 1–2%, delivering true bare-metal throughput.
3. **Failure Domain Decoupling**: Isolate unrecoverable crashes, driver panics, or experimental runtimes to a secondary instance while preserving the integrity and availability of the primary host management layer.
4. **Hardware-Enforced Security**: Transitioning from software namespace isolation (containers) to hardware VMX/SVM privilege modes and IOMMU DMA translation establishes verifiable, cryptographically attestation-ready multi-tenant security boundaries.

The Future of Multikernel Systems

The architecture of multi-kernel computing continues to accelerate with emerging hardware standards:

- **Compute Express Link (CXL)**: CXL 2.0 and 3.0 introduce cache-coherent memory pooling and device disaggregation over PCIe Gen 5/6 fabrics. Multiple physical servers and multiple co-located kernels can dynamically map and share disaggregated memory pools with hardware-enforced cache coherency.
- **Heterogeneous Compute Engines**: Modern server motherboards incorporate diverse processing architectures: x86/ARM host CPUs alongside specialized Data Processing Units (DPUs / SmartNICs) and AI accelerators running independent embedded Linux kernels. Multi-kernel management becomes standard operating practice across the entire chassis fabric.

- **Evolving Kernel Primitives**: Developments in the upstream Linux kernel—including user-space scheduler extensions (`sched_ext`), dynamic hugepage folios, and hardware-accelerated virtio queues—continue to lower the cost of domain isolation.

Operational Synthesis

To succeed with multi-kernel deployments in production:

- **Automate Everything**: Treat kernel command-line arguments, udev VFIO bindings, and CPU pinning configurations as declarative code.
- **Measure at the Tail**: Evaluate system health using P99 and P99.99 latency distributions under noisy neighbour stress tests, not idle averages.
- **Respect Hardware Topologies**: Always align CPU cores, memory channels, and PCIe devices along identical NUMA node boundaries to prevent catastrophic cross-bus contention.

With these principles and technical primitives in place, the multi-kernel bare-metal server stands as one of the most powerful, predictable, and resilient architectures in modern systems engineering.

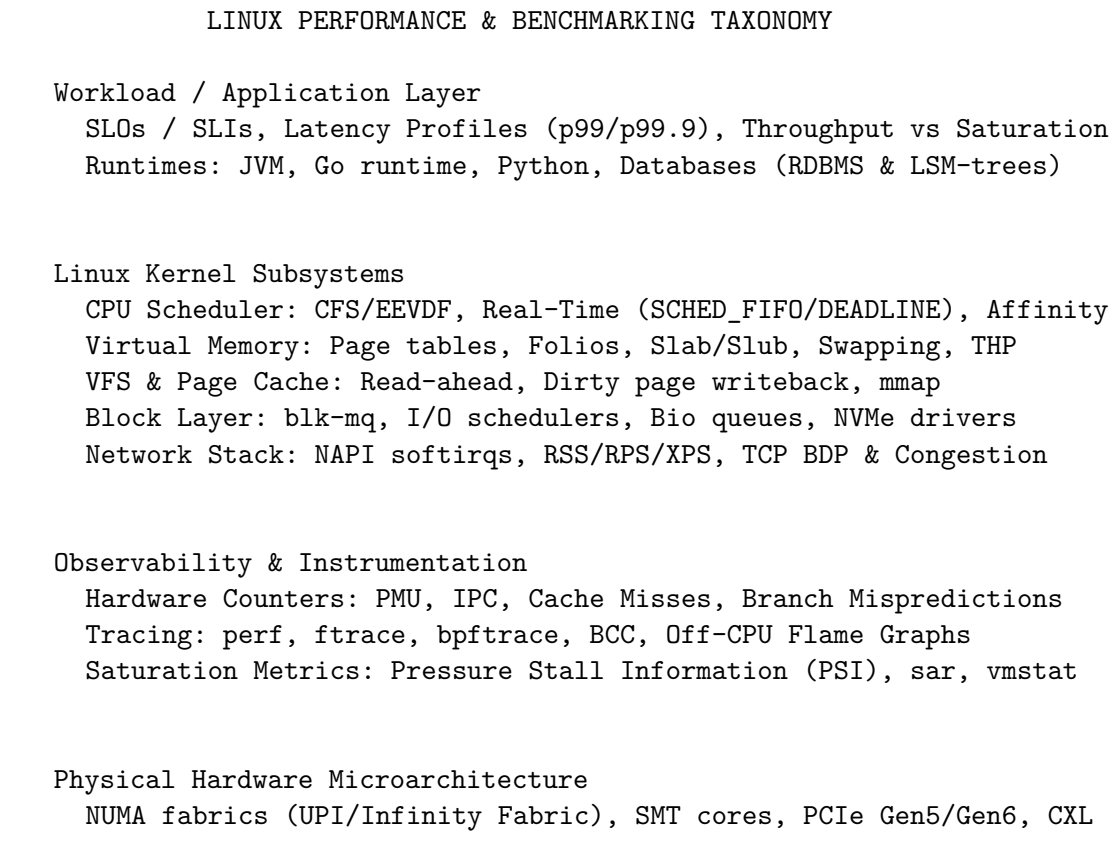
Linux Performance Engineering & Server Benchmarking

Linux Performance Engineering & Server Benchmarking

Linux Performance Engineering & Server Benchmarking

Performance is not a switch you flip or a list of magical sysctl parameters copied from internet forums. Performance is an emergent property of the entire system—the interaction between user code, language runtimes, kernel subsystems, hardware microarchitecture, interconnect topologies, and persistent storage media.

This section provides a rigorous, deep-dive curriculum into Linux performance engineering and server benchmarking. Spanning 28 structured chapters, it guides systems engineers, SREs, kernel developers, and infrastructure architects from first principles and statistical measurement methodologies to bare-metal hardware profiling, kernel subsystem internals, eBPF tracing, high-load benchmarking, and production cluster optimization.



Curriculum Breakdown

Chapter	Title	Scope & Subsystems
Intro	Introduction: The Discipline of Performance Engineering	System properties, USE/RED metrics, measurement imperative, cognitive traps
01	The Performance Engineering Methodology	Workload characterization, SLI/SLO baselines, experimental controls, statistics
02	Linux Kernel Architecture and Performance	Boundary costs, system calls, tasks, context switching, softirqs, lock contention
03	CPU Architecture and Performance Counters	Superscalar pipelines, cache hierarchies, branch prediction, SMT, PMU counters
04	CPU Scheduling and Process Placement	CFS/EEVDF mechanics, virtual runtime, RT scheduling, affinity, NUMA balancing
05	Memory Management: Virtual Memory and Pages	Page tables, minor/major faults, Buddy allocator, swap/zswap, Huge Pages, RSS/PSS
06	NUMA Architecture and Memory Placement	Interconnect latency, local vs remote RAM, numactl policies, perf c2c, vNUMA
07	The Page Cache and Memory-Mapped Files	Folios, read-ahead, write-behind, mmap vs read, kswapd eviction, dirty writeback
08	I/O Subsystems: From Block Layer to Disk	VFS-to-disk path, blk-mq, I/O schedulers, bio splitting/merging, queue depths
09	Storage Technologies and Performance Characteristics	HDD seek physics, NAND/SSD wear leveling, NVMe queue pairs, RAID, cloud EBS
10	Filesystems and Mount Options	ext4 vs XFS vs btrfs, journaling modes, fsync cost, noatime, barrier tuning
11	The Networking Stack and Performance	Socket buffers, NAPI softirqs, RSS/RPS/XPS, TCP window scaling, UDP overhead
12	Network Performance Tuning in Practice	TCP sysctls, BDP calculation, CUBIC vs BBR, multiqueue NICs, validation

Chapter	Title	Scope & Subsystems
13	System Call Tracing and Process Analysis	strace ptrace cost, perf trace, ftrace function_graph, dynamic kprobes/uprobes
14	eBPF and BPF-Based Observability	In-kernel VM, verifier, BPF maps, bpftrace scripts, BCC tools, CO-RE/libbpf
15	Performance Analysis Tools and Flame Graphs	On-CPU vs Off-CPU flame graphs, perf record/report DWARF, hotspot call paths
16	System Monitoring and Observability	Classic tools (vmstat/iostat), modern tools, Pressure Stall Information (PSI)
17	Benchmarking: Design and Execution	Workload synthesis, testbed isolation, warmup/cooldown, statistical confidence
18	Benchmarking Tools and Suites	fio storage jobs, stress-ng, sysbench OLTP, iperf3, lmbench, wrk, Phoronix
19	CPU and Memory Tuning for Production	CPU governors, IRQ affinity, isolcpus/nohz_full, 1GB hugepages, vm sysctl knobs
20	I/O and Filesystem Tuning for Production	Scheduler selection, mount flags, dirty page thresholds, NVMe queue polling
21	cgroups, systemd and Resource Management	cgroups v2 hierarchy, cpu.max quota throttling, memory.high/max, io limits
22	Containers, Virtualization and Noisy Neighbors	Container vs VM overhead, hypervisor exits, cache thrashing, Intel RDT/MBA
23	Databases and High-Performance Applications	PostgreSQL/MySQL memory & I/O patterns, shared buffers, futexes, runtime GC
24	Distributed Systems and Cluster Performance	Network serialization, PTP clock skew, Raft/Paxos disk fsync, cluster topology
25	Production Case Studies	Real-world post-mortems: interrupt storms, memory swap crisis, NUMA skew, EBS stall
26	Capacity Planning and Performance Regression	Little's Law, queueing models (M/M/c), baseline tracking, CI performance gates
27	Advanced Topics and Emerging Technologies	PREEMPT_RT, kernel bypass (DPDK/XDP), x86 vs ARM64, RDMA, CXL interconnects

Chapter	Title	Scope & Subsystems
28	Conclusion: Putting It All Together	The engineer's mindset, Amdahl's Law, premature optimization, performance culture

Introduction: The Discipline of Performance Engineering

Introduction: The Discipline of Performance Engineering

Performance engineering is often treated as an emergency triage exercise: a production incident strikes, latency spikes, and engineers scramble to tweak sysctl parameters, increase thread pools, or add hardware. This reactive posture is both costly and ineffective. Genuine performance engineering is a continuous discipline rooted in scientific measurement, queueing theory, architectural awareness, and systematic hypothesis testing.

This introduction establishes the philosophical and theoretical foundation required before modifying a single kernel knob or running a benchmark harness.

Performance Is a System Property, Not a Configuration

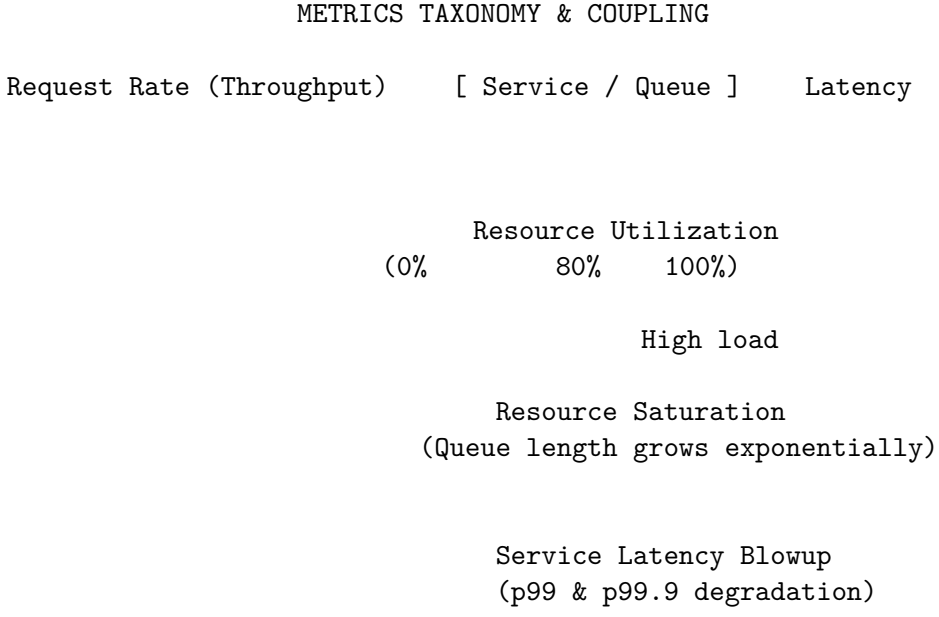
A widespread misconception in systems administration is that "tuning" consists of finding a secret set of configuration parameters that unlocks maximum hardware performance. In reality:

1. **Trade-offs Govern Every Parameter**: Increasing network socket buffer sizes (`net.ipv4.tcp_rmem`) improves bulk transfer throughput over high-latency wide-area networks, but on memory-constrained servers under high concurrency, it causes kernel memory exhaustion and triggers bufferbloat.
2. **Context-Specific Efficiency**: An I/O scheduler optimized for spinning disks (like BFQ) degrades throughput when applied to multi-queue NVMe solid-state storage.
3. **Emergent Behavior**: Performance is the result of non-linear interactions across layers:
    - Application logic and memory allocation patterns
    - Language runtime execution (garbage collectors, thread schedulers)
    - Standard C library memory management (`glibc ptmalloc` vs `mimalloc` vs `jemalloc`)
    - Kernel virtual memory management, page caching, and scheduler placement
    - Hardware caches, interconnect buses, and storage media latency

Treating performance as a static configuration leads to cargo-cult engineering—copying sysctl snippets from blog posts that were tuned for completely different workloads, hardware generations, and kernel versions.

Latency, Throughput, Utilization, Saturation, Availability

To reason about systems rigorously, performance engineers rely on standardized metric models: **USE** (Utilization, Saturation, and Errors) for hardware and kernel resources, and **RED** (Rate, Errors, and Duration) for application requests.



Core Dimensions Defined

- **Latency (Duration)**: The time taken to process an operation. Latency must always be represented as a distribution (percentiles such as median/p50, p95, p99, and p99.9), never as a single arithmetic mean. Averages hide tail latency anomalies caused by stop-the-world GC pauses, kernel lock contention, or page reclaim stalls.
- **Throughput (Rate)**: The volume of completed work units per second (e.g., HTTP requests/sec, database transactions/sec, gigabits/sec).
- **Utilization**: The fraction of time a resource is actively performing work during an observation window. For resources that handle concurrent operations (like multi-core CPUs or multi-queue SSDs), utilization measures capacity consumption.
- **Saturation**: The degree to which extra work is queued because a resource is operating at full capacity. Once utilization approaches high thresholds, queueing delays spike exponentially according to queueing theory ($W = \frac{\rho}{\mu(1-\rho)}$).
- **Availability (Errors)**: The proportion of operations that complete successfully without errors or timeouts.

The Measurement Imperative

The first law of performance engineering is simple: **if you did not measure it, you do not know it.**

Intuition and guesswork about performance bottlenecks are notoriously unreliable. Modern systems are so layered that human intuition consistently fails to predict where cycles are actually burned:

- An application might report high CPU usage, yet hardware performance counters reveal that the processor is stalled 60% of cycles waiting for main memory cache lines due to poor data locality.
- A database might appear disk-bound due to slow query response times, but kernel tracing reveals that threads are blocking on `futex` mutex contention inside userspace locking primitives.

Every performance investigation must begin with telemetry and profiling before any code or configuration change is attempted. Once a change is made, the identical measurement harness must be run under controlled conditions to prove whether the intervention helped, had no effect, or introduced a regression.

Common Pitfalls and How to Avoid Them

Pitfall	Manifestation	Engineering Remedy
Averaging Fallacy	Relying on mean latency (e.g., "our average response time is 15ms")	Measure high-order percentiles (p95, p99, p99.9, p99.99) and visualize histograms.
Streetlight Effect	Looking only at metrics that are easy to view (e.g., standard top CPU %)	Use systemic methodologies (Brendan Gregg's USE Method) to systematically inspect every resource.
Coordinated Omission	Benchmark client pauses while awaiting responses, skipping scheduled requests during server stalls	Use non-blocking, open-model load generators (e.g., wrk2, k6 open-model) that schedule requests independently of server latency.
Observer Effect	High-overhead tracing tools (e.g., naive strace on hot loops) distorting the measurement	Use low-overhead sampling profilers (`perf`) and in-kernel eBPF aggregations that filter events before userspace emission.
Microbenchmark Illusion	Isolated code runs blazing fast in microbenchmarks, but regresses under production concurrency	Validate changes under realistic concurrent workloads with realistic cache pressure, thread contention, and I/O noise.

Mastering performance engineering requires replacing superstitious tuning with verifiable evidence. The next chapter presents the formal methodology used to drive structured investigations from initial symptom to root cause.

Chapter 1: The Performance Engineering Methodology

Chapter 1: The Performance Engineering Methodology

Performance investigations without an established methodology degrade into ad-hoc trial and error. Engineers jump between random metrics, execute uncoordinated configuration changes, and mistakenly attribute natural system variance to their latest intervention.

This chapter defines an end-to-end, reproducible performance engineering methodology that guides engineers from workload characterization to hypothesis testing, statistical validation, and root cause diagnosis.

Characterizing the Workload

Before analyzing bottlenecks, you must understand the shape and demands of the workload running on the server. Workload characterization answers four fundamental questions:

1. **Who is generating the load?** (Client concurrency, request arrival distributions, burstiness, batch vs interactive).
2. **Why is the load generated?** (Payload size, database read-to-write ratios, cache hit ratios, transaction complexity).
3. **What resources are requested?** (CPU instruction mix, memory allocation rates, network packet rates, random vs sequential storage I/O).
4. **How does the load change over time?** (Diurnal traffic cycles, periodic cron jobs, background compaction, garbage collection).

```
# Example: High-level workload characterization using basic system tools
# Observe overall load averages, active task counts, and runqueue pressure
uptime

# Observe context switch rates, interrupt rates, and paging activity
vmstat 1 5
```

Defining Success: SLOs, SLIs and Performance Targets

Performance tuning must be anchored to concrete engineering objectives:

- **Service Level Indicator (SLI)**: A carefully defined quantitative measure of service performance. Example: *The 99th percentile response latency of `GET /orders` measured over a rolling 5-minute window at the edge gateway.*

- **Service Level Objective (SLO)**: A target value or range for an SLI agreed upon by stakeholders. Example: *p99 latency < 45ms for 99.9% of all 5-minute windows.*
- **Performance Target**: An internal optimization goal. Example: *Reduce p99 tail latency from 80ms to 35ms at 15,000 requests per second while keeping server CPU utilization under 70%.*

Without defined SLOs and targets, engineering teams risk over-optimizing components that have negligible impact on user-perceived performance or business outcomes.

Baseline Creation and Reference Measurements

A baseline is a frozen, documented snapshot of system performance under known, controlled conditions. Baselines serve as the invariant reference point against which all past and future changes are compared.

A robust baseline record includes:

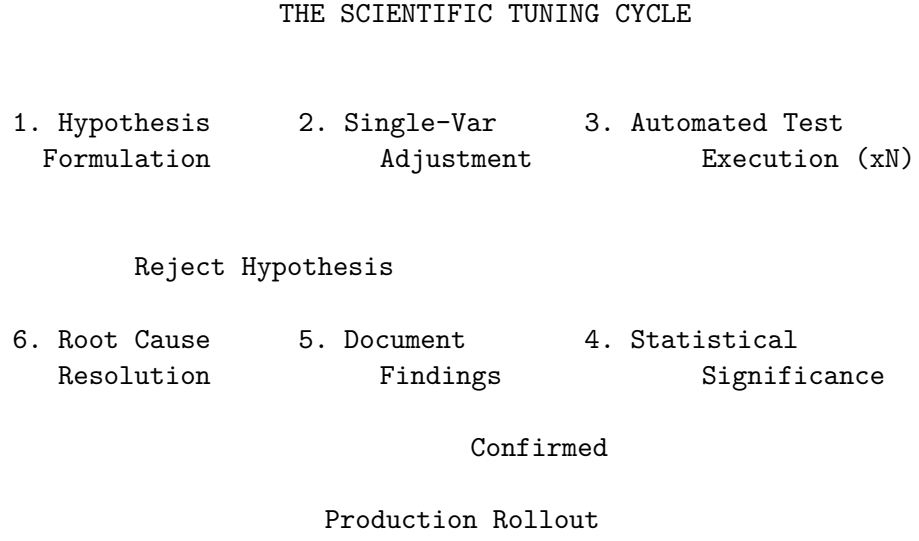
1. **Hardware & Environment Specification**: CPU model, stepping, microcode version, memory topology, NUMA layout, disk controller firmware, network interface cards.
2. **Software Stack Versions**: Kernel version (`uname -a`), distribution release, C library version, compiler flags, application runtime/JVM version.
3. **Host Configuration Snapshot**: Complete dump of `/proc/cmdline`, active sysctl parameters (`sysctl -a`), mounted filesystems and options (`mount -v`), and systemd unit limits.
4. **Telemetry Profiles**: Pre-test system metrics, CPU utilization by core, memory allocation distribution, baseline I/O latency curves under idle conditions.

```
# Archive system configuration for baseline records
uname -a > baseline-env.txt
cat /proc/cmdline >> baseline-env.txt
sysctl -a > baseline-sysctl.txt 2>/dev/null
lscpu >> baseline-env.txt
numactl --hardware >> baseline-env.txt
```

Experimental Design: Variables, Controls and Reproducibility

Performance engineering follows the scientific method. When conducting experiments:

- **Single-Variable Rule**: Alter exactly one independent variable per test run (e.g., change `vm.dirty_ratio` while holding all other variables constant). If you adjust an I/O scheduler, a mount option, and a memory sysctl simultaneously, you cannot isolate causality.
- **Control Group**: Maintain an unmodified control node alongside the experimental candidate node, driven by the identical load generator stream.
- **Reproducibility**: Automation scripts must build the test environment, seed data fixtures, warm caches, execute the workload, and collect metrics identically across runs.



Statistical Rigor: Variance, Outliers and Significance

A single benchmark run is meaningless. Environmental noise (background interrupts, thermal throttling, cloud multi-tenancy jitter) introduces variance.

1. **Multiple Repetitions**: Always run tests at least $N = 5$ or $N = 10$ times.
2. **Dispersion Metrics**: Calculate the median, interquartile range (IQR), and standard deviation (σ).
3. **Statistical Hypothesis Testing**: Use the two-sample Student's t-test or Welch's t-test (for unequal variances) to determine if the observed latency reduction is statistically significant ($p < 0.01$) or merely random noise.
4. **Outlier Filtering**: Never discard outliers blindly. In latency engineering, the "outlier" is frequently the primary bug: a lock convoy, a stop-the-world garbage collection pause, or a major page fault on a cold memory folio.

The Diagnosis Loop: From Symptom to Root Cause

When faced with degraded performance in production, follow this disciplined diagnostic loop:

1. **Verify the Problem**: Confirm symptoms against SLIs and client metrics. Eliminate client-side network issues or load balancer anomalies.
2. **Top-Down Resource Triage (USE Method)**:
    - For every component (CPU, Memory, Disk, Network, Buses):
        - Is Utilization near 100%?
        - Is there Saturation (runqueues, wait times, backlog buffers)?
        - Are there Error counters incrementing?

3. **Bottleneck Drill-Down**:
    - If CPU is saturated: profile with `perf` and off-CPU sampling to find hot code paths or thread blocking.
    - If I/O is saturated: inspect block queue latency with `biolatency` and check filesystem dirty page writeback.
    - If Memory is saturated: inspect page fault rates, kswapd activity, and memory cgroup pressure.
4. **Formulate a Falsifiable Hypothesis**: State clearly: *"Thread pool exhaustion in service X is caused by slow database queries holding connections, not by CPU starvation."*
5. **Test and Verify**: Apply the targeted fix, measure with identical metrics, and verify that the symptom is permanently resolved.

Chapter 2: Linux Kernel Architecture and Performance

Chapter 2: Linux Kernel Architecture and Performance

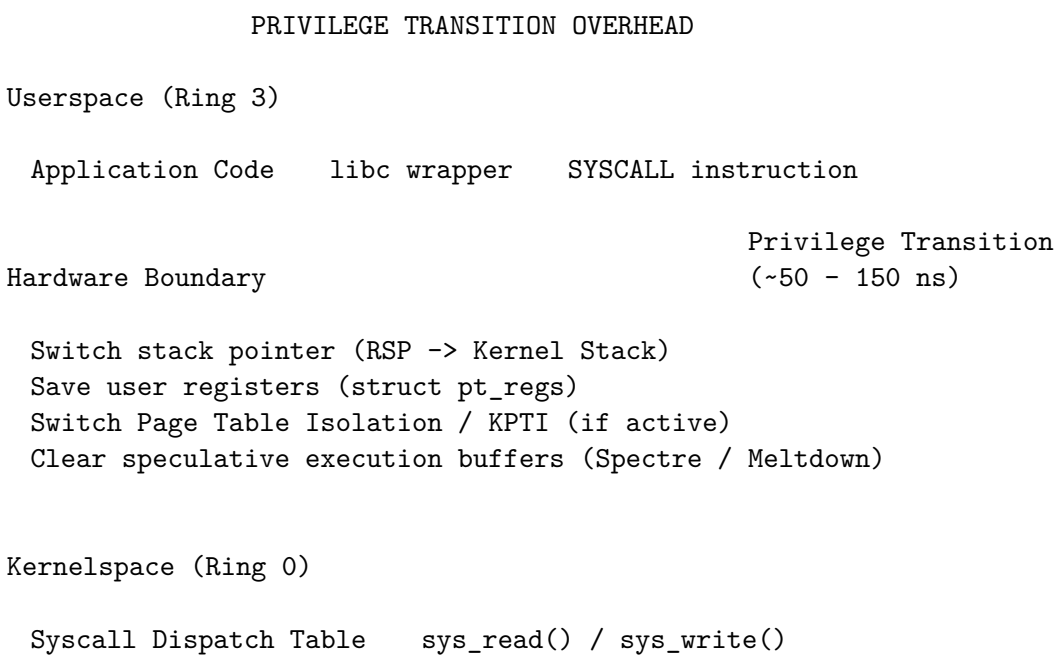
To extract maximum performance from Linux, an engineer must understand the operating system not as an opaque black box, but as a concurrent supervisor managing hardware privilege boundaries. Every interaction between an application and the outside world—allocating memory, sending packets, writing to disks—incurs architectural overhead across the hardware privilege boundary.

This chapter analyzes the internal mechanics and latency costs of kernel boundaries, system calls, task structures, context switches, interrupt handling, and kernel synchronization primitives.

Kernel Space and User Space: The Boundary Cost

Modern processors enforce security and memory isolation using hardware privilege levels:

- **Ring 0 / EL1 (Kernel Mode)**: Unrestricted execution privileges, direct hardware access, full page table manipulation.
- **Ring 3 / EL0 (User Mode)**: Restricted execution environment, isolated virtual address spaces, hardware memory protection faults on unauthorized accesses.



The transition between Ring 3 and Ring 0 is not free. Modern mitigation mechanisms (Kernel Page Table Isolation - KPTI, retpolines, speculative store bypass disabling) increased syscall entry/exit costs from ~50ns to over 150ns on affected x86 architectures. High-frequency syscall loops (such as calling `gettimeofday` or tiny `read()` calls in a tight loop) can burn massive CPU cycles purely on boundary crossing.

Fast Userspace Alternatives: vDSO

To eliminate boundary overhead for frequent read-only queries, the kernel exports the **vDSO (virtual Dynamic Shared Object)**. Memory pages containing current time values (`clock_gettime`, `gettimeofday`) and CPU identity (`getcpu`) are mapped directly into userspace address spaces, allowing libc to return hardware-accurate results without entering Ring 0.

System Calls: Entry, Exit and Overhead

On x86_64, system calls enter via the `SYSCALL` instruction and return via `SYSRET`. The kernel saves caller-saved registers into a `struct pt_regs` structure on the task's kernel stack:

```
// Simplified representation of pt_regs saved during syscall entry
struct pt_regs {
    unsigned long r15, r14, r13, r12, rbp, rbx;
    unsigned long r11, r10, r9, r8, ax, cx, dx, si, di;
    unsigned long orig_ax;
    unsigned long ip, cs, flags, sp, ss;
};
```

1. **Parameter Passing**: On x86_64, system call arguments pass in registers `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, and the syscall number passes in `rax`.
2. **Dispatch Overhead**: The kernel indexes into the `sys_call_table` array, executes the handler, and verifies return values.
3. **Copy Overhead**: For read/write operations, data must be copied between user buffers and kernel page cache buffers via `copy_to_user()` and `copy_from_user()`, validating memory bounds on each byte.

Processes, Threads and the Task Structure

In the Linux kernel, there is no fundamental architectural difference between a "process" and a "thread". Both are represented by instances of `struct task_struct`:

- A **Process** is a task that owns a unique virtual memory descriptor (`struct mm_struct`), unique file descriptor table (`struct files_struct`), and unique signal handlers.
- A **Thread** is a task created via `clone()` with shared memory and descriptor flags (`CLONE_VM`, `CLONE_FILES`, `CLONE_FS`, `CLONE_SIGHAND`), sharing its parent's `mm_struct`.

Because tasks are the universal schedulable unit, the kernel scheduler treats threads and processes identically when placing them on CPU runqueues.

Context Switching: Mechanics and Cost

A context switch occurs when the CPU halts execution of one task and resumes execution of another. Context switches are either:

- **Voluntary**: The task voluntarily yields the CPU by sleeping, waiting for I/O completion, or blocking on a lock (`schedule()`).
- **Involuntary**: The scheduler preempts the task because its allocated time quantum expired, or a higher-priority task became runnable.

```
# View context switch rates system-wide (csw/s)
vmstat 1 3

# View voluntary vs involuntary context switches per process
pidstat -w -p <PID> 1 3
```

The Two Costs of Context Switches

1. **Direct Cost (Registers & Stacks)**: Saving CPU registers, swapping the kernel stack pointer, and switching architecture-specific hardware state (e.g., FPU/AVX vector registers). This takes ~1 to 3 microseconds.
2. **Indirect Cost (Cache Eviction)**: Swapping the `CR3` register to point to a new page table invalidates the Translation Lookaside Buffer (TLB). As the new task runs, it suffers cold L1/L2/L3 cache misses as its working set replaces the old task's cache lines. Indirect cache pollution costs often exceed 10–30 microseconds of degraded execution speed.

Interrupts, Softirqs and Bottom Halves

Hardware devices notify the CPU of events (e.g., incoming network packets, completed NVMe blocks) via hardware interrupts (IRQs). To prevent long-running interrupt handlers from freezing the system, Linux splits interrupt processing into two tiers:

1. **Top Half (Hard IRQ)**: Executes immediately with interrupts disabled on the receiving CPU. It acknowledges the hardware controller, acknowledges the interrupt, moves data into a temporary buffer, schedules the bottom half, and exits in microseconds.
2. **Bottom Half (Softirq / Tasklet / Workqueue)**: Executes asynchronously with interrupts enabled:
    - **Softirqs**: Fixed in-kernel vectors (`NET_RX_SOFTIRQ`, `BLOCK_SOFTIRQ`, `SCHED_SOFTIRQ`, `RCU_SOFTIRQ`). High throughput, runs in interrupt context.
    - **ksoftirqd**: Dedicated per-CPU kernel threads (`ksoftirqd/N`) that execute when softirq load is so heavy that running them in-line would starve userspace tasks.

```
# Inspect softirq activity across all CPUs
cat /proc/softirqs
```

If `/proc/softirqs` shows `NET_RX` concentrated entirely on CPU 0, that core will suffer softirq starvation, dropping packets while other cores remain idle.

Synchronization Primitives: Locks, Spinlocks and Contention

Concurrent kernel execution requires synchronization across multiple CPU cores:

Primitive	Execution Context	Mechanism on Contention	Overhead / Use Case
Spinlock (`spinlock_t`)	Atomic / Interrupt context	Busy-waits (spins in a tight CPU loop)	Ultra-fast (<10ns) for brief critical sections. Must never sleep. High CPU burn if contested.
Mutex (`struct mutex`)	Sleepable / Process context	Enqueues task onto a wait queue and context switches	Used for long critical sections. Incurs context switch overhead on contention.
Futex (Fast Userspace Mutex)	Userspace / Kernel hybrid	Uncontested: atomic CAS in userspace. Contested: kernel syscall to sleep.	The foundation of pthread mutexes. Zero syscall cost when uncontested.
Read-Copy Update (RCU)	Read-mostly data structures	Readers execute lock-free without atomics. Writers stage copies and defer reclamation.	Extreme read scalability across thousands of cores (routing tables, VFS dentries).

Contention on spinlocks or mutexes destroys horizontal CPU scalability. As core counts scale from 16 to 128 cores, lock convoys and cache coherency bounce can cause a 10x degradation in throughput. Profiling tools like `perf lock` and `bpftrace` are essential for identifying contested lock addresses in production.

Chapter 3: CPU Architecture and Performance Counters

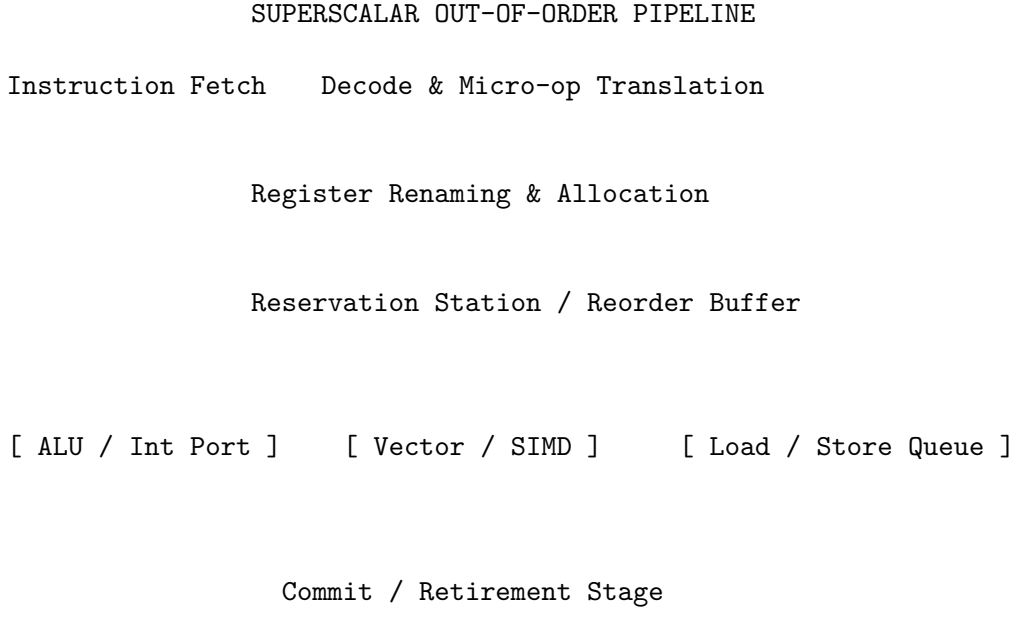
Chapter 3: CPU Architecture and Performance Counters

Modern CPUs are extraordinarily complex superscalar, out-of-order execution engines. Understanding clock frequency alone tells you almost nothing about how effectively software utilizes the processor. Two workloads consuming 100% CPU on `top` can deliver radically different actual performance depending on hardware pipeline stalls, cache misses, branch mispredictions, and memory bus saturation.

This chapter breaks down modern processor microarchitecture and demonstrates how to measure hardware-level efficiency using Performance Monitoring Units (PMUs) and hardware counters.

Modern CPU Microarchitecture: Pipelines and Parallelism

A modern processor core does not execute instructions sequentially one at a time. Instead, it utilizes deep pipelines and superscalar execution:



- **Superscalar Execution**: Modern cores can decode, dispatch, and retire multiple instructions per clock cycle (often 4 to 8 micro-ops per cycle).

- **Out-of-Order (OoO) Execution**: If an instruction is waiting for data to load from main memory, the core dynamically finds and executes subsequent independent instructions to keep execution units saturated.
- **Retirement**: Instructions execute speculatively and out of order, but results are committed to architectural registers in strict original program order.

Instructions Per Cycle (IPC)

The ultimate metric of CPU execution efficiency is **IPC (Instructions Per Cycle)**:

$$\text{IPC} = \frac{\text{Instructions Retired}}{\text{CPU Clock Cycles}}$$

- **IPC $>$ 1.5 - 2.5**: Excellent pipeline utilization. Workload is compute-bound and cache-friendly.
- **IPC $<$ 0.7**: Poor execution efficiency. The CPU is spending the majority of its clock cycles stalled waiting for memory lines or branch resolution.

Cache Hierarchy: L1, L2, L3, TLB and Cache Misses

Memory access latency is the primary bottleneck in modern computing. Accessing physical RAM requires hundreds of clock cycles, while L1 cache takes only a handful:

Level	Typical Capacity	Access Latency	Scope
L1 Data / Instruction (L1d/L1i)	32 KB - 64 KB per core	~4 - 5 clock cycles (~1 ns)	Private to core
L2 Cache	512 KB - 1 MB per core	~12 - 14 clock cycles (~3 - 4 ns)	Private to core
L3 Cache (Last Level Cache - LLC)	32 MB - 256 MB per socket	~40 - 60 clock cycles (~12 - 18 ns)	Shared across cores in CCX/Socket
Main Memory (DRAM)	64 GB - 2 TB	~200 - 300 clock cycles (~60 - 90 ns)	Shared via memory controllers
Remote NUMA DRAM	Multi-socket nodes	~400 - 600 clock cycles (~120 - 180 ns)	Traverses UPI/Infinity Fabric

```
# Inspect hardware cache topology
lscpu -C
```

Cache Lines and False Sharing

Memory moves between DRAM and CPU caches in fixed 64-byte chunks called **cache lines**.

- **True Sharing**: Multiple cores read and write shared data structures protected by locks.

- **False Sharing**: Two independent threads running on separate cores modify completely distinct variables that happen to reside within the same 64-byte cache line. The cache coherence protocol (MESI/MOESI) invalidates the cache line across cores on every write, causing severe bus contention and latency spikes.

Branch Prediction and Speculative Execution

Because pipelines are 14 to 20 stages deep, the processor cannot wait for conditional branches (`if/else`, loops) to evaluate before fetching subsequent instructions.

- **Branch Predictor**: Sophisticated neural-network and pattern-history tables predict which branch path will be taken with >95% accuracy.
- **Speculative Execution**: The CPU executes the predicted path ahead of time. If the prediction was correct, the results commit instantly.
- **Branch Misprediction Penalty**: If the prediction was wrong, the entire speculative pipeline must be flushed and refetched, burning 15 to 20 clock cycles of pure stall overhead.

Cores, Hyperthreading and SMT

Simultaneous Multithreading (SMT), known as Hyper-Threading on Intel, exposes two logical processors per physical core:

- SMT duplicates architectural register state (instruction pointers, general registers) while sharing the underlying execution units, pipelines, and L1/L2 caches.
- **Throughput vs Latency Trade-off**: SMT increases total system throughput by 20%–35% by keeping execution units busy during single-thread memory stalls. However, SMT increases tail latency jitter for latency-critical workloads due to resource contention between co-located threads.

In high-frequency trading and hard real-time environments, engineers often disable SMT via `/sys/devices/system/cpu/smt/control` to eliminate noisy neighbor effects between hyperthreads.

CPU Frequency Scaling and Power States

Processors dynamically adjust clock frequencies and power consumption to balance performance and thermal envelopes:

- **C-States (Idle States)**: $C0$ is fully active. Deeper states ($C1, C6, C8$) power down internal core circuitry and flush caches. Waking from deep C-states takes 10 to 100 microseconds, introducing latency spikes on bursty workloads.
- **P-States (Performance States)**: Dynamic Voltage and Frequency Scaling (DVFS). P0 is maximum turbo frequency; higher numbers indicate lower clock speeds.
- **Linux Governors**:

- `powersave`: Minimizes power draw (often default on Intel systems running `intel_pstate`).
- `performance`: Locks the CPU at its maximum non-turbo or turbo frequency.
- `schedutil`: Adjusts frequency dynamically based on CFS scheduler runqueue utilization.

```
# Check current CPU frequency and active governor
cpupower frequency-info

# Lock all cores to maximum performance mode
sudo cpupower frequency-set -g performance
```

Performance Counters and Hardware Events

Every modern enterprise CPU includes a dedicated **Performance Monitoring Unit (PMU)** capable of counting hardware events with zero impact on application behavior:

```
# Measure high-level hardware efficiency of a command
perf stat -B -e \
  cycles,instructions,cache-references,cache-misses,branches,branch-misses \
  stress-ng --cpu 4 --timeout 5s
```

Example output analysis:

```
     4,120,490,120      cycles                    #    3.210 GHz
     5,892,102,400      instructions              #    1.43  insn per cycle
        18,490,120      cache-references          #   14.400 M/sec
         2,104,890      cache-misses              #   11.38% of all L1D hits
       840,290,100      branches                  #  654.400 M/sec
        12,400,200      branch-misses             #    1.48% of all branches
```

By profiling IPC, LLC cache miss rates, and branch miss percentages, performance engineers can immediately pinpoint whether an application bottleneck is bound by compute pipelines, memory bandwidth, or unpredictable control flow.

Chapter 4: CPU Scheduling and Process Placement

Chapter 4: CPU Scheduling and Process Placement

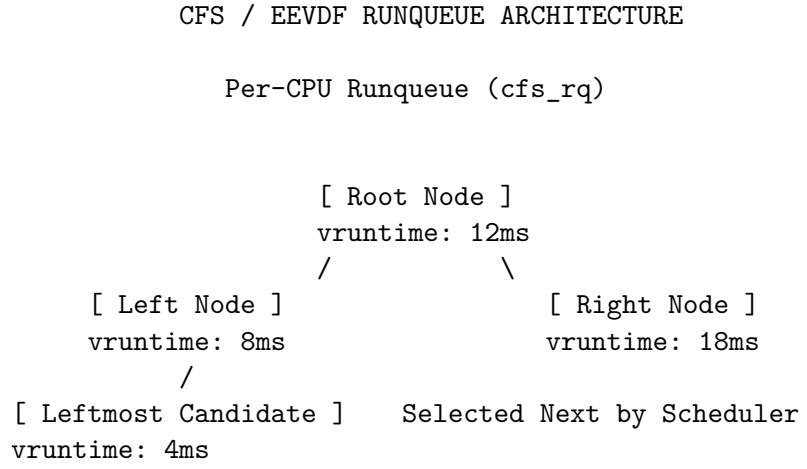
The Linux CPU scheduler is responsible for allocating processor time across competing threads, ensuring fair resource distribution while minimizing context switch latency and maximizing cache locality. In modern Linux (kernel 6.6+), the scheduling subsystem evolved from the classic Completely Fair Scheduler (CFS) to the Earliest Eligible Virtual Deadline First (EEVDF) scheduler, refining latency guarantees for interactive and latency-sensitive workloads.

This chapter details the internal mechanics of Linux CPU scheduling, real-time priorities, core affinity, task isolation, and cross-NUMA load balancing.

The Completely Fair Scheduler: Internal Mechanics

The fundamental goal of CFS is to simulate an "ideal multi-tasking CPU" on real hardware:

- In an ideal system with N runnable tasks, every task would run simultaneously at $\frac{1}{N}$ speed.
- On real hardware with discrete cores, CFS tracks how much time each task has consumed and picks the task that has received the least CPU time.
- **Red-Black Tree Runqueues**: CFS maintains a per-CPU time-ordered red-black tree (`struct cfs_rq`). The leftmost node in the tree is the task with the smallest virtual runtime; it is chosen for execution next ($O(1)$ pick, $O(\log N)$ insert).



The Transition to EEVDF (Linux 6.6+)

While CFS ensured proportional throughput fairness over longer windows, it often penalized latency-sensitive tasks that slept frequently. EEVDF addresses this by introducing: - **Virtual Time**: Tracks fair progress. - **Lag**: The difference between the time a task should have received according to its weight and the actual time it consumed. - **Virtual Deadline**: When a task's allocated time slice must finish. EEVDF selects the eligible task with the earliest virtual deadline, providing dramatically more deterministic wakeup latency.

Virtual Runtime, Timeslices and Fairness

- `vruntime` **(Virtual Runtime)**: Measures the execution time consumed by a task, scaled inversely by its nice level (priority weight). Tasks with higher priority (lower nice value) accumulate `vruntime` more slowly, allowing them to run longer and more frequently before falling behind in the red-black tree.
- **Timeslice Calculation**: The dynamic slice assigned to a task is calculated based on `sysctl_sched_latency` divided by the number of active runnable tasks, bounded by `sysctl_sched_min_granularity`.

```
# View active scheduler configuration knobs
sysctl kernel.sched_latency_ns 2>/dev/null || sysctl kernel.sched_base_slice_ns 2>/dev/null
```

Real-Time Scheduling: SCHED_FIFO, SCHED_RR, SCHED_DEADLINE

For workloads that cannot tolerate the dynamic timeslicing of CFS/EEVDF, Linux provides real-time scheduling classes that strictly preempt normal tasks:

Class	Policy	Preemption Behavior	Target Use Case
SCHED_FIFO	First-In, First-Out	Runs until it voluntarily yields, blocks on I/O, or is preempted by a higher-priority RT task. No timeslicing.	High-frequency trading, audio DSP, industrial robotics.
SCHED_RR	Round-Robin	Similar to FIFO, but preempted when its fixed round-robin timeslice expires if equal-priority tasks exist.	Real-time multi-threaded pipelines.

Class	Policy	Preemption Behavior	Target Use Case
SCHED_DEADLINE	Earliest Deadline First (EDF)	Constant Bandwidth Server (CBS). Specified by `runtime`, `deadline`, and `period`.	Hard real-time audio/video streaming, avionics.

```
# Run a process with real-time FIFO priority 80
sudo chrt --fifo 80 ./latency-critical-worker

# Inspect scheduling policy and priority of an existing PID
chrt -p <PID>
```

> [!CAUTION] A misbehaving `SCHED_FIFO` thread that enters an infinite busy-wait loop will starve all normal userspace processes and kernel background threads on that core, freezing system management unless `kernel.sched_rt_runtime_us` bandwidth throttling is enabled.

CPU Affinity, Isolation and Task Migration

By default, the scheduler migrates tasks between CPU cores to balance load. However, task migration flushes private L1/L2 caches and forces remote memory access across interconnects.

Pinning Tasks with CPU Affinity

Using `sched_setaffinity` or `taskset`, an engineer can bind a process to a dedicated set of CPU cores:

```
# Launch an application bound strictly to physical cores 2 and 3
taskset -c 2,3 ./database-engine

# Query existing core affinity of a running PID
taskset -cp <PID>
```

Full Core Isolation (`isolcpus` & `nohz_full`)

For deterministic microsecond latency, physical cores can be completely isolated from the standard scheduler and kernel housekeeping:

1. `isolcpus=24-31`: Removes cores 24 through 31 from the general CFS load-balancing domain. No standard tasks will be scheduled there unless explicitly pinned with `taskset`.
2. `nohz_full=24-31`: Stops the scheduler timer tick on isolated cores whenever exactly one task is runnable, eliminating periodic 1000Hz timer interrupt interruptions.

3. `rcu_nocbs=24-31`: Offloads Read-Copy Update (RCU) callback processing to housekeeping cores.

Load Balancing Across NUMA Nodes

The Linux scheduler organizes CPU cores into a hierarchical tree of **Scheduling Domains** (`sched_domain`): - SMT Domain (hyperthreads on the same core, shared L1/L2) - MC Domain (cores sharing L3 cache) - NUMA Domain (cores sharing local memory controller) - System Domain (interconnect hops to remote sockets)

Load balancing occurs frequently within SMT and MC domains because the cache penalty of moving a thread to an adjacent core sharing the same L3 cache is negligible. Conversely, cross-NUMA balancing is deferred and amortized because moving a thread across sockets forces all its existing memory pages to be accessed remotely over inter-socket buses.

Chapter 5: Memory Management: Virtual Memory and Pages

Chapter 5: Memory Management: Virtual Memory and Pages

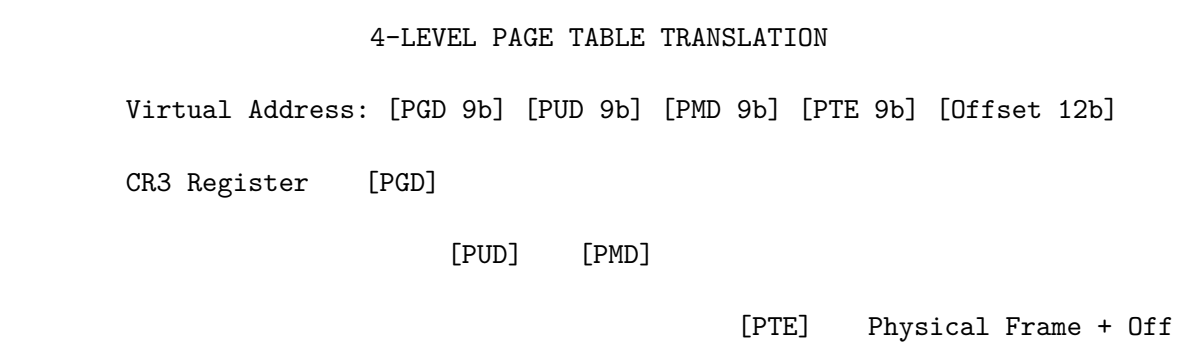
Memory management is frequently the defining factor in overall system throughput and tail latency. Linux provides every userspace process with an isolated, contiguous virtual address space, mapping virtual memory addresses to physical RAM on demand through multi-level hardware page tables.

This chapter breaks down the mechanics of virtual address translation, page faults, kernel allocators, swap behavior, huge pages, and accurate memory accounting.

Virtual Memory: Address Spaces and Page Tables

On modern x86_64 systems, virtual addresses are mapped to physical frames via multi-level page tables managed by the hardware Memory Management Unit (MMU):

- **4-Level Paging (48-bit VA)**: 256 TB address space. Walks four levels: PGD (Page Global Directory) → P4D → PUD (Page Upper Directory) → PMD (Page Middle Directory) → PTE (Page Table Entry).
- **5-Level Paging (57-bit VA / LA57)**: 128 PB address space, adding a fifth translation tier for multi-terabyte systems.



The Translation Lookaside Buffer (TLB)

Walking four levels of page tables in DRAM incurs up to 400 clock cycles of latency per memory access. To mitigate this, processors cache translations in the **TLB (Translation Lookaside Buffer)**. TLB hits resolve in ~1 clock cycle. Minimizing TLB misses is a critical optimization target for memory-intensive applications.

Page Faults: Minor and Major Faults

Linux uses optimistic, on-demand memory allocation. When an application calls `malloc()` or `mmap()`, the kernel allocates virtual address space ranges (`struct vm_area_struct` or VMA), but no physical RAM is assigned until the thread first touches the address, generating a **Page Fault** exception:

- **Minor Page Fault**: The requested memory page is already present in physical RAM (e.g., mapped from the Page Cache, an anonymous zero-fill page, or shared memory). The kernel simply updates the process's PTE and resumes execution in microseconds.
- **Major Page Fault**: The page is not in physical RAM and must be read synchronously from disk or network storage (either from a file-backed binary or swap space). The thread blocks on disk I/O, stalling for milliseconds.

```
# Observe minor (minflt/s) and major (majflt/s) page faults
sar -B 1 3
```

Physical Memory: Allocation and Management

Physical memory allocation is managed by two complementary subsystems:

1. **The Buddy Allocator**: Manages raw physical memory pages in power-of-two blocks ($2^0 = 4\text{KB}, 2^1 = 8\text{KB}, \ldots, 2^{10} = 4\text{MB}$). It avoids external fragmentation by splitting and merging adjacent buddy blocks across zones (`ZONE_DMA32`, `ZONE_NORMAL`, `ZONE_MOVABLE`).
2. **The Slab/Slub Allocator**: Sits on top of the buddy allocator to allocate small, frequently used kernel data structures (dentries, inodes, socket buffers, task structs) without wasting full 4KB pages.
3. **Memory Folios**: Modern Linux (6.x+) abstracts compound pages into *folios*, guaranteeing contiguous physical memory chunks and drastically reducing internal kernel metadata tracking overhead.

```
# View active kernel slab allocations
sudo slabtop -s c
```

Swapping: Mechanics, Costs and Modern Behavior

When memory demand exceeds physical capacity, the kernel reclaims memory:

- File-backed clean pages are discarded immediately (they can be re-read from disk).
- Anonymous dirty pages (heap, stack, anonymous mmap) must be written to **swap space**.

```
# Key memory reclamation sysctl parameters
cat /proc/sys/vm/swappiness             # Default 60 (aggressiveness of swapping anon vs pag
cat /proc/sys/vm/vfs_cache_pressure     # Default 100 (reclaim priority for dentries/inodes)
```

```
cat /proc/sys/vm/watermark_scale_factor  # Proximity threshold for waking kswapd background
```

Modern Swap Best Practice: `zswap` and `zram`

Traditional disk swap introduces catastrophic latency spikes. Modern production Linux leverages **zswap** (a compressed in-memory write-back cache for swap pages) or **zram** (a compressed RAM block device). Compressing a 4KB page with LZ4/ZSTD in CPU cache requires ~2 microseconds—three orders of magnitude faster than writing to NVMe.

Huge Pages and Transparent Huge Pages

Standard pages are 4KB. A process with 64GB of RAM requires $16,777,216$ PTEs (128MB of page table overhead) and places immense pressure on the TLB.

- **2MB Huge Pages**: Translates directly at the PMD level, eliminating PTE table walks. 64GB requires only 32,768 entries.
- **1GB Huge Pages**: Translates at the PUD level on supported hardware. Ideal for multi-gigabyte database buffer pools and hypervisor guest memory.

Transparent Huge Pages (THP) vs Explicit HugeTLB

- **Explicit HugeTLB (`hugetlbfs`)**: Pre-allocated at boot. Never fragmented, never swapped. Required for high-performance databases (PostgreSQL, DPDK, QEMU/KVM).
- **Transparent Huge Pages (THP)**: The kernel automatically attempts to allocate 2MB pages for anonymous allocations.

  [!WARNING] While THP helps compute workloads, it can introduce latency spikes in latency-critical databases (like Redis or Cassandra) during background page compaction. Set `transparent_hugepage=madvise` to let applications explicitly opt in via `madvise(MADV_HUGEPAGE)`.

Memory Accounting: RSS, PSS and Kernel Metrics

Evaluating memory consumption using simple `ps` or `top` metrics is often misleading due to shared libraries and copy-on-write mappings:

Metric	Definition	Significance
VIRT (VSZ)	Total virtual address space mapped	Includes unallocated virtual mappings and open files. Unrelated to actual RAM use.

Metric	Definition	Significance
RSS (Resident Set Size)	Physical RAM currently mapped by the process	Overcounts memory shared across processes (e.g., shared libc is counted in every process's RSS).
PSS (Proportional Set Size)	Private RAM + (Shared RAM / number of sharing processes)	The most accurate metric for individual process physical memory consumption.
USS (Unique Set Size)	Private RAM exclusively mapped by this process	The exact amount of memory freed if the process terminates.

```
# View detailed PSS / Private clean and dirty pages for a process
sudo grep -A 12 "^Private" /proc/<PID>/smaps_rollup
```

Chapter 6: NUMA Architecture and Memory Placement

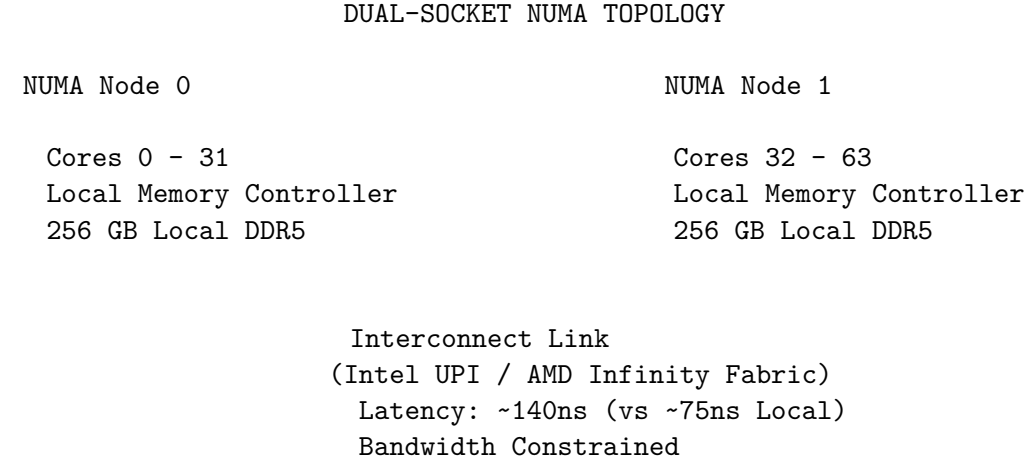
Chapter 6: NUMA Architecture and Memory Placement

On modern multi-socket and high-core-count servers, uniform memory access is physically impossible. Memory controllers are integrated directly into processor sockets and multi-chip modules (CCDs). This creates a Non-Uniform Memory Access (NUMA) architecture where the latency and bandwidth of accessing physical memory depend heavily on which CPU core issues the load instruction and which physical memory channel holds the data.

This chapter explores the performance implications of NUMA topologies, diagnostics, allocation policies, and optimization techniques.

What NUMA Means for Performance

In a NUMA system, the machine is partitioned into multiple **NUMA nodes**. Each node consists of a group of CPU cores and their physically adjacent memory channels:



Accessing memory on the local node incurs minimal latency (~75ns). Accessing memory attached to a remote socket requires traversing the inter-socket interconnect (Intel Ultra Path Interconnect - UPI, or AMD Infinity Fabric), adding 60ns–100ns of latency and consuming inter-socket interconnect bandwidth.

NUMA Nodes, Local vs Remote Memory Access

The kernel discovers NUMA distances from the ACPI System Locality Distance Information Table (SLIT):

```
# Display NUMA hardware topology, core mapping, and distance matrix
numactl --hardware
```

Example SLIT distance matrix:

```
node distances:
node   0   1
  0:  10  21
  1:  21  10
```

A value of `10` represents normalized local access latency. A distance of `21` indicates that remote accesses incur more than double the access cost.

NUMA-Aware Allocation and Policies

Linux provides memory allocation policies that control where new pages are physically allocated:

- **Local Allocation (Default)**: The kernel attempts to allocate memory on the same NUMA node as the thread executing the allocation (`MPOL_DEFAULT`).
- **Interleaved (`MPOL_INTERLEAVE`)**: Allocations alternate across specified NUMA nodes in round-robin fashion (page by page). Ideal for large multi-threaded shared caches that exceed single-node memory capacity.
- **Preferred (`MPOL_PREFERRED`)**: Strongly prefers allocating on a specific node, falling back to other nodes if memory is exhausted.
- **Bind (`MPOL_BIND`)**: Strictly allocates memory from specified nodes. Fails if memory on those nodes is exhausted.

Diagnosing NUMA Imbalance

When memory is allocated unevenly, or threads run on Node 0 while accessing memory on Node 1, severe performance degradation occurs.

```
# Check memory allocation and hit/miss counters per NUMA node
numastat -c
```

Key counters to observe: - `numa_hit`: Pages allocated successfully on the local node. - `numa_miss`: Pages allocated on a remote node because the intended local node was full. - `numa_foreign`: Pages intended for another node that ended up allocated here. - `local_node` vs `other_node`: High ratios of `other_node` indicate severe cross-NUMA traffic.

Profiling Cache-to-Cache Traffic with `perf c2c`

The `perf c2c` (cache-to-cache) tool samples memory load/store events to detect cross-socket cache line bouncing:

```
# Record memory access events to detect false sharing and remote NUMA contention
sudo perf c2c record -F 60000 -- ./application-workload
sudo perf c2c report --stdio
```

Optimizing for NUMA: Placement and Binding

Binding with `numactl`

To eliminate cross-NUMA latency, launch critical workloads bound entirely to a single NUMA node (both CPU cores and memory):

```
# Execute pinned to Node 0 cores and Node 0 memory
numactl --cpunodebind=0 --membind=0 ./redis-server /etc/redis/node0.conf

# Interleave memory across nodes for large in-memory databases (e.g., PostgreSQL)
numactl --interleave=all ./postgres -D /data/postgres
```

Automatic NUMA Balancing (`kernel.numa_balancing`)

Linux includes an autonomous balancing daemon that periodically unmaps memory pages to generate minor page faults. By inspecting which CPU core touched the page, the kernel dynamically migrates pages to the accessing node.

> [!TIP] While helpful for generic batch workloads, Automatic NUMA Balancing introduces periodic TLB shootdown and migration overhead. For predictable low latency in databases or trading systems, disable automatic balancing (`sysctl kernel.numa_balancing=0`) and enforce static binding using `numactl` or systemd cpusets.

NUMA in Virtualized and Cloud Environments

In virtualized environments (KVM, VMware, AWS/GCP bare-metal and large instances):

- **vNUMA Topology**: Hypervisors must expose a virtual NUMA topology to the guest that accurately mirrors the physical NUMA layout of the underlying bare metal.
- **Cross-NUMA Overcommit**: If a virtual machine is assigned 32 vCPUs spanning two physical NUMA nodes without vNUMA awareness, the guest operating system treats memory as uniform, leading to pathological cross-socket interconnect thrashing.

- **Cloud Instance Selection**: Large cloud instances (e.g., AWS c6i.32xlarge, m6a.48xlarge) span multiple NUMA sockets. Infrastructure engineers must verify socket layout using `lscpu` and configure application worker thread pools per NUMA socket.

Chapter 7: The Page Cache and Memory-Mapped Files

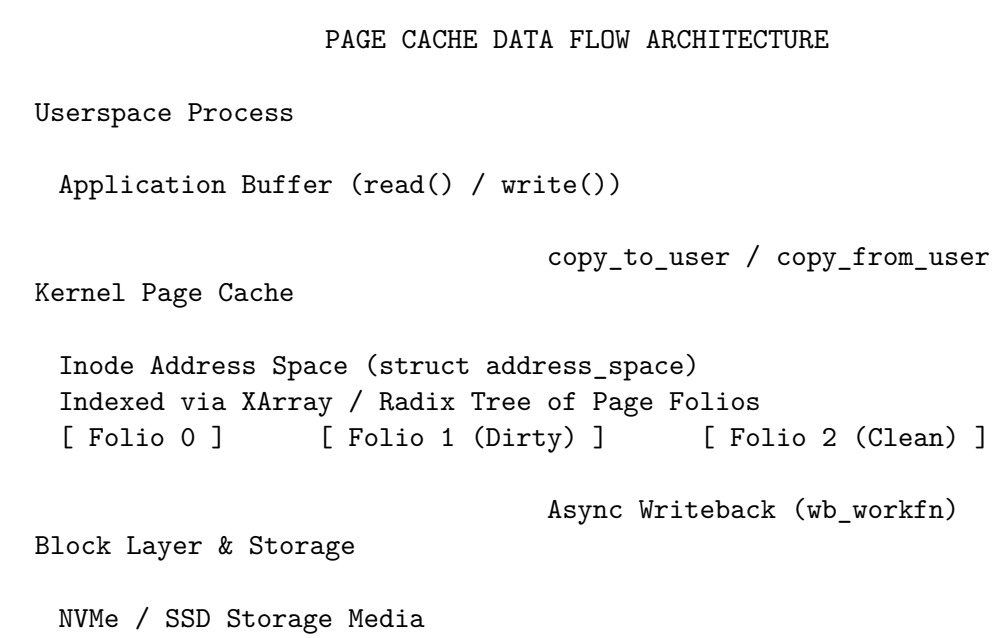
Chapter 7: The Page Cache and Memory-Mapped Files

The Linux Page Cache is the central caching layer between persistent block storage and application address spaces. By keeping frequently accessed file data in unused physical memory, the Page Cache accelerates read and write performance by orders of magnitude compared to direct disk access.

This chapter examines the internal architecture of the Page Cache, read-ahead algorithms, memory-mapped files (`mmap`), cache pressure eviction, and dirty page writeback tuning.

Page Cache: Architecture and Behavior

Whenever an application issues a standard `read()` or `write()` system call, the kernel operates on the Page Cache rather than talking directly to the disk:



- **Address Space Indexing**: Every cached file is represented by a `struct address_space` attached to its filesystem inode. Cached pages/folios are indexed by their file offset using an efficient in-memory **XArray** (formerly radix tree).

- **Read Path**: The kernel searches the XArray for the requested file offset. If found (Page Cache Hit), data is copied directly to the user buffer in nanoseconds without generating disk I/O. If missing (Page Cache Miss), the kernel triggers a synchronous read from block storage.
- **Write Path**: Writes update the in-memory page folio and mark it **Dirty**. The `write()` system call returns immediately to userspace. Disk writes are deferred and flushed asynchronously by background kernel threads.

Read-Ahead and Write-Behind

Adaptive Read-Ahead

When the kernel detects sequential read patterns, it speculatively loads upcoming file blocks into the Page Cache before the application requests them:

- **Initial Window**: Starts small (e.g., 16KB).
- **Expansion Window**: Scales dynamically up to `read_ahead_kb` (typically 128KB–512KB) as long as sequential hits continue.
- **Application Hints (`posix_fadvise`)**: Applications can explicitly instruct the kernel on expected access patterns:
  - `POSIX_FADV_SEQUENTIAL`: Doubles the read-ahead window.
  - `POSIX_FADV_RANDOM`: Disables read-ahead entirely to avoid wasting I/O bandwidth.
  - `POSIX_FADV_WILLNEED`: Triggers non-blocking background population of the Page Cache.

```
# Query and adjust device read-ahead window (in 512-byte sectors)
sudo blockdev --getra /dev/nvme0n1
sudo blockdev --setra 1024 /dev/nvme0n1    # Set to 512 KB
```

Memory-Mapped Files and mmap

Instead of copying data between kernel buffers and userspace buffers via `read()` and `write()`, applications can map files directly into their virtual address spaces using `mmap()`:

```
// Map 1GB file into userspace address space
int fd = open("large_data.bin", O_RDWR);
void *addr = mmap(NULL, 1024*1024*1024, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
```

`mmap` vs `read()` / `write()` Trade-offs

- **Zero-Copy Access**: Memory-mapped reads avoid copying bytes through `copy_to_user()`. Userspace pointer dereferences directly touch the Page Cache frame.
- **TLB Shootdown and Page Fault Cost**: Initial access triggers minor page faults to populate the process's page table entries. On multi-threaded systems with frequent unmapping, TLB shootdown interrupts across cores can negate zero-copy savings.

- Modern high-performance storage engines (e.g., LMDB, RocksDB, Kafka) select `mmap` or standard I/O based on working-set-to-RAM ratios.

Cache Pressure and Eviction Policies

Under memory pressure, the kernel must reclaim memory pages. Linux categorizes pages using Least Recently Used (LRU) lists:

1. **Active List**: Pages touched recently. Protected from immediate eviction.
2. **Inactive List**: Eviction candidates. Clean inactive pages are dropped immediately; dirty pages are queued for writeback.

The balance between reclaiming file-backed Page Cache pages versus anonymous heap/stack pages is controlled by `vm.vfs_cache_pressure` and `vm.swappiness`:

```
# Adjust inode/dentry cache reclamation priority (default 100)
# Values > 100 reclaim dentries/inodes more aggressively
sudo sysctl vm.vfs_cache_pressure=100
```

Dirty Page Management and Writeback

Dirty pages accumulate in RAM until flushed by background writeback workers (`kworker/flush`). Misconfigured dirty ratios cause massive "write stalls" where interactive latency blows up because userspace processes are forced to freeze and write dirty pages to disk synchronously.

Critical Writeback Knobs

```
# Percentage of available memory where background writeback starts
cat /proc/sys/vm/dirty_background_ratio    # Typical default: 10%

# Hard percentage where userspace processes block and perform synchronous writes
cat /proc/sys/vm/dirty_ratio               # Typical default: 20%

# Maximum age of dirty data before writeback is forced (in hundredths of a second)
cat /proc/sys/vm/dirty_expire_centisecs    # Typical default: 3000 (30 seconds)
```

> [!IMPORTANT] On modern enterprise servers with 512GB+ RAM, default ratios are disastrous: `dirty_ratio=20` means the kernel permits 100GB of dirty data to accumulate in RAM before blocking applications. When flushing starts, storage queues choke and latency spikes to tens of seconds. **Production Fix**: Use byte-based limits on large RAM hosts:
>
> ```
> sudo sysctl -w vm.dirty_background_bytes=268435456   # 256 MB
> sudo sysctl -w vm.dirty_bytes=1073741824              # 1 GB
> ```

Chapter 8: I/O Subsystems: From Block Layer to Disk

Chapter 8: I/O Subsystems: From Block Layer to Disk

The Linux storage I/O subsystem bridges high-level POSIX filesystem operations with physical storage controllers. Over the last decade, storage bandwidth evolved from hundreds of IOPS on spinning rust (HDDs) to millions of IOPS on multi-channel NVMe SSDs. This transition forced a total architectural overhaul of the Linux block layer from the single-lock legacy request queue to the multi-queue (`blk-mq`) architecture.

This chapter traces an I/O request from the initial system call down to the physical disk controller, examining request queues, I/O schedulers, bio splitting, and completion interrupts.

The I/O Path: From open() to the Disk

Every storage I/O operation traverses distinct kernel layers:

```
                        STORAGE I/O DATA PATH

Userspace (open / read / write / fsync / io_uring)


Virtual Filesystem (VFS) Layer (sys_write / vfs_write)


Concrete Filesystem (ext4, XFS, btrfs)
- Translates file offset to logical block addresses (LBAs)
- Allocates extents and builds bio structures


Multi-Queue Block Layer (blk-mq)
- Software Staging Queues (per-CPU)    I/O Scheduler (Optional)
- Hardware Dispatch Queues (per-channel)


Device Driver (nvme, mpt3sas, virtio-blk)
- Maps DMA scatter-gather lists to controller ring buffers


Physical Storage Controller & Flash/Disk Media
```

The Block Layer and Request Queues (`blk-mq`)

In the legacy Linux block layer, all CPU cores submitted requests to a single global request queue protected by a single spinlock. On modern NVMe devices capable of 1M+ IOPS, that single lock became an immediate bottleneck.

The modern **Multi-Queue Block Layer (`blk-mq`)** eliminates this contention through a two-tiered queue design:

1. **Software Staging Queues**: Per-CPU or per-core software queues. Tasks submit requests locklessly to their local CPU queue.
2. **Hardware Dispatch Queues**: Mapped directly to the physical submission queues of the storage hardware controller (e.g., NVMe controllers support up to 64,000 independent hardware queues).

I/O Schedulers: noop, deadline, cfq, mq-deadline, kyber, bfq

I/O schedulers decide the order and timing of requests dispatched to storage devices:

Scheduler	Architecture	Primary Strength	Recommended Use Case
`none`	Direct pass-through	Zero software queue overhead, lowest CPU utilization	High-speed NVMe drives with native hardware queues
`mq-deadline`	Deadline queues (read/write)	Prevents read starvation by prioritizing reads over writes	SATA SSDs, SAS SSDs, fast cloud block devices
`bfq` (Budget Fair Queue)	Proportional weight sharing	Interactive responsiveness and fairness under heavy write load	Desktop workstations, SATA spinning disks (HDDs)
`kyber`	Latency-target feedback loop	Regulates queue depths to maintain strict read latency limits	Shared cloud nodes, fast flash with high write concurrency

```
# Query active and available schedulers for a device
cat /sys/block/nvme0n1/queue/scheduler

# Switch scheduler dynamically (e.g., set to 'none' on NVMe)
echo "none" | sudo tee /sys/block/nvme0n1/queue/scheduler
```

Block Devices and Queueing Models

A storage device's ability to absorb parallel requests is governed by queue depth:

- **Queue Depth (`nr_requests`)**: The maximum number of requests the block layer permits to be concurrently staged.
- **Little's Law for Storage**:

$$\text{Throughput} = \frac{\text{Queue Depth}}{\text{Latency}}$$

  To achieve 1,000,000 IOPS on a device with 100μs latency, the host must sustain an active queue depth of:

$$QD = 1,000,000 \times 0.0001 = 100$$

  If the application sends synchronous single-threaded requests with $QD = 1$, the maximum possible throughput is capped at only 10,000 IOPS regardless of how fast the NVMe drive is.

I/O Merge, Splitting and Bio Structures

The fundamental unit of I/O in the Linux block layer is `struct bio`:

- A **bio** represents an in-flight I/O request composed of one or more memory vectors (scatter-gather list of physical memory folios).
- **Bio Merging**: If an application submits adjacent small block requests, the block layer merges them into a single contiguous `struct request` before sending it to the controller, amortizing DMA setup overhead.
- **Bio Splitting**: If a bio exceeds the device's maximum transfer size (`max_sectors_kb`), the block layer splits it into multiple smaller requests.

```
# Inspect maximum request and sector limits
cat /sys/block/nvme0n1/queue/max_sectors_kb
cat /sys/block/nvme0n1/queue/hw_sector_size
```

I/O Completion and Interrupt Handling

When the storage controller completes an I/O operation:

1. **Hardware Interrupt**: The controller fires an MSI-X interrupt targeted to the CPU core that submitted the request.
2. **Interrupt Service Routine (ISR)**: The driver acknowledges the controller's completion queue.
3. **Softirq Callback**: `BLOCK_SOFTIRQ` executes, completing the `bio`, invoking filesystem end-I/O callbacks, and waking up sleeping threads or completing `io_uring` ring entries.

Hybrid Polling Mode for Extreme Latency

On modern ultra-low-latency NVMe and Optane devices, the overhead of interrupt handling (~2–5μs) is larger than the media access latency. Linux supports **polling mode** (`io_uring` with `IORING_SETUP_IOPOLL` or `/sys/block/<dev>/queue/io_poll`), where the application thread directly polls the NVMe completion queue, achieving sub-10μs deterministic latency.

Chapter 9: Storage Technologies and Performance Characteristics

Chapter 9: Storage Technologies and Performance Characteristics

Storage media physical characteristics determine the fundamental limits of latency, throughput, and parallelism. An application architect cannot optimize storage I/O without knowing whether the underlying device is a mechanical spinning platter, a multi-layer 3D TLC NAND flash drive, or a virtualized cloud network block volume with provisioned burst credits.

This chapter compares the physical constraints and architectural behaviors of HDDs, SATA/SAS SSDs, NVMe devices, RAID controllers, and cloud storage volumes.

HDDs: Seek Latency, Rotational Latency and Throughput

Mechanical Hard Disk Drives (HDDs) are governed by physical mechanics:

- **Seek Latency**: Moving the mechanical actuator arm across concentric tracks (~3–10ms).
- **Rotational Latency**: Waiting for the requested sector to rotate under the drive head (at 7200 RPM, average rotation takes 4.16ms).
- **Total Random Access Latency**: Typically 8ms to 15ms per random operation. This limits a single HDD to approximately **75 to 150 random IOPS**.
- **Sequential Throughput**: When the drive head reads continuous sectors along a track, bandwidth reaches 150MB/s to 260MB/s.

  [!NOTE] On HDDs, random I/O is catastrophically slower than sequential I/O (a 1000x difference in effective throughput). Filesystems and database engines designed for HDDs go to great lengths to convert random writes into sequential appends (e.g., WAL logs, LSM-trees).

SSDs: NAND Characteristics, Wear Leveling and GC

Solid-State Drives (SSDs) use non-volatile NAND flash memory:

- **Page Reads vs Block Erases**: NAND flash reads and programs data in **Pages** (typically 4KB to 16KB). However, flash cells cannot be overwritten in place; they must first be erased in large **Erase Blocks** (typically 4MB to 16MB).
- **Garbage Collection (GC)**: As files are updated, existing pages become invalid. The drive controller periodically copies surviving valid pages to a new erase block and erases the old block in the background. Heavy GC activity causes severe latency spikes and write amplification.

- **Write Amplification Factor (WAF)**: The ratio of bytes written to NAND flash versus bytes written by the operating system:

$$\text{WAF} = \frac{\text{Bytes Written to NAND}}{\text{Bytes Written by Host}}$$

- **TRIM / Discard**: Informs the SSD controller which logical blocks no longer hold valid data, enabling efficient background GC without copying dead pages.

NVMe: Protocol Advantages and Queue Architecture

Non-Volatile Memory Express (NVMe) was designed from scratch for solid-state storage, replacing legacy SATA/AHCI protocols that were engineered around spinning disks:

```
                    AHCI (SATA) vs NVMe PROTOCOL

Legacy AHCI (SATA)                  Modern NVMe
  1 Command Queue                    Up to 64,000 Command Queues
  Max Queue Depth: 32 commands       Max Queue Depth: 64,000 cmds
  Requires single register lock      Lockless per-CPU queue pairs
  High CPU interrupt overhead        Direct PCIe Gen4/Gen5 fabric

Latency: ~500µs                     Latency: ~10µs - 50µs
Max Throughput: ~550 MB/s           Max Throughput: 7,000 - 14,000+
```

By placing submission and completion queue pairs on individual CPU cores, NVMe eliminates software lock contention, achieving millions of IOPS per server.

RAID and Hardware Controllers

RAID arrays combine multiple physical drives to balance fault tolerance, capacity, and performance:

RAID Level	Min Disks	Read Performance	Write Performance	Failure Tolerance
RAID 0	2	$N\times$ throughput	$N\times$ throughput	0 drives (loss of any drive destroys array)
RAID 1	2	$N\times$ throughput	Single drive throughput	$N-1$ drives
RAID 10	4	$N\times$ throughput	$\frac{N}{2}\times$ throughput	1 drive per mirror set (up to $\frac{N}{2}$)

RAID Level	Min Disks	Read Performance	Write Performance	Failure Tolerance
RAID 5	3	$(N-1)\times$ throughput	Penalized by parity write ($\approx \frac{1}{4}$ speed)	1 drive
RAID 6	4	$(N-2)\times$ throughput	Severe double parity write penalty	2 drives

Hardware Controller Cache Modes

Hardware RAID controllers feature onboard DRAM caches backed by batteries or supercapacitors (BBU / Flash-Backed Write Cache):

- **Write-Back (WB)**: The controller acknowledges writes as soon as data hits the battery-backed DRAM, delivering sub-millisecond write latency and absorbing write bursts.
- **Write-Through (WT)**: The controller waits until data is physically written to the underlying disks before acknowledging the write. Safe, but degrades write latency to raw disk speeds.

  [!CAUTION] If a RAID controller's battery fails, it typically falls back from Write-Back to Write-Through automatically, resulting in an unannounced 90% drop in database write throughput. Always monitor controller battery health via `megacli` or `storcli`.

Cloud Block Storage and Shared Storage

Cloud virtual disks (AWS EBS, GCP Persistent Disk, Azure Managed Disks, Ceph/SAN) are not local disks; they are network-attached storage services accessed over virtualized Ethernet fabrics:

- **Baseline vs Burst IOPS**: Cloud volumes provide baseline IOPS tied to allocated volume size, with burst credits that deplete during prolonged write operations.
- **EC2 / Instance Throttling**: Even if an EBS volume supports 64,000 IOPS, the host EC2 instance has hard limits on EBS bandwidth and network throughput (EBS-optimized bandwidth limits).
- **Network Latency Floor**: Because every request travels over an internal data center network, the latency floor is rarely lower than 500μs–1ms, compared to 15μs on local bare-metal NVMe.

Choosing Storage for the Workload

Workload Type	Ideal Storage Architecture	Key Considerations
High-Throughput Databases (OLTP)	Local NVMe in RAID 10 or raw NVMe with software replication	Prioritize low p99 write latency and high random IOPS; configure write-back or battery-backed storage.
Time-Series / Append Logs (Kafka)	Striped NVMe or SAS SSDs (RAID 10 / filesystem extents)	High sequential write throughput; maximize Page Cache dirty writeback bandwidth.
Warm Data & Object Stores	High-density SATA enterprise HDDs in RAID 6 or erasure-coded ZFS	Prioritize lowest $/TB cost; sequential reads dominate; random reads buffered by NVMe cache tiers.
Stateless Cloud Applications	Provisioned Cloud SSDs with predictable IOPS	Ensure VM network bandwidth does not throttle disk queue depth; monitor EBS burst exhaustion.

Chapter 10: Filesystems and Mount Options

Chapter 10: Filesystems and Mount Options

The filesystem is responsible for structuring raw block storage into hierarchical directories and files, managing free space allocation, maintaining metadata consistency, and coordinating concurrent access. Choosing the appropriate filesystem and configuring mount options can alter I/O throughput and latency by more than 40%.

This chapter compares the internal architectures of ext4, XFS, and btrfs, analyzes journaling costs, evaluates critical mount flags, and details filesystem caching mechanisms.

ext4: Journaling, Allocation and Performance

The Fourth Extended Filesystem (ext4) is the default general-purpose filesystem for many Linux distributions:

- **Block Groups and Extents**: Physical space is divided into Block Groups. ext4 replaced legacy indirect block pointers with contiguous **Extents** (mapping up to 128MB of contiguous physical blocks per extent record), significantly reducing metadata overhead.
- **Delayed Allocation (`delalloc`)**: ext4 delays allocating physical blocks on disk until data is actually written out from the Page Cache by writeback threads. This allows the allocator to group adjacent writes into large contiguous extents, preventing fragmentation.
- **Directory Indexing (`dir_index`)**: Uses hashed B-trees (HTrees) to accelerate lookups in directories containing hundreds of thousands of files.

XFS: Design Principles and Scalability

XFS was engineered by Silicon Graphics (SGI) for extreme scalability, massive storage capacity, and highly parallel workloads:

- **Allocation Groups (AGs)**: XFS divides the storage space into multiple completely independent Allocation Groups (typically 4 to 32 AGs). Each AG possesses its own free-space B+trees and inode allocation maps.
- **Parallel Allocation**: Unlike ext4, which can suffer lock contention when hundreds of threads allocate blocks concurrently, XFS allows concurrent threads to allocate space across separate Allocation Groups in parallel without lock collisions.
- **Dynamic Inodes**: Inodes are allocated dynamically across AGs as needed, avoiding the fixed inode count limitations of traditional filesystems.

  [!TIP] For high-concurrency database workloads (PostgreSQL, MySQL/InnoDB) and multi-terabyte flash arrays, **XFS** is universally preferred over ext4 due to its lockless Allocation Group architecture.

btrfs and Copy-on-Write Semantics

Btrfs (B-tree Filesystem) uses a Copy-on-Write (CoW) architecture combined with integrated volume management:

- **CoW Mechanics**: Existing data blocks are never overwritten in place. When a block is modified, the new data is written to a completely free block, and metadata pointers are updated up the root tree.
- **Snapshots and Checksums**: Provides near-instantaneous subvolume snapshots and data/metadata CRC32c/XXH64 checksumming for silent data corruption detection.
- **Performance Trade-offs**: CoW introduces severe fragmentation when handling random write-in-place workloads (e.g., database files or VM disk images). While snapshots and checksums provide strong integrity guarantees, btrfs typically exhibits lower peak random write throughput compared to XFS.

Filesystem Journaling and Write Performance

To maintain filesystem consistency across sudden crashes and power losses, filesystems maintain an append-only circular log called a **Journal**:

```
                    JOURNALING MODES (EXT4)

data=journal
  Both metadata AND file data written to journal first
  Safest, but cuts write throughput by ~50% (double write penalty)

data=ordered (Default)
  File data written to main storage first
  Metadata written to journal before committing transaction

data=writeback
  Only metadata is journaled; file data written whenever
  Highest throughput; risk of stale data leaks on sudden crash
```

The Cost of `fsync()`

When an application calls `fsync()` or `fdatasync()`: 1. The kernel flushes all dirty Page Cache pages for the file to disk. 2. The filesystem flushes the journal transaction to storage. 3. The block layer issues a hardware **Cache Flush Barrier** to the disk controller, forcing the drive to dump its internal volatile write cache.

Frequent `fsync()` calls in synchronous loops throttle throughput to the physical disk flush latency (~500μs–5ms per call).

Mount Options: Performance Implications

Mount options instruct the VFS and filesystem drivers how to treat metadata updates and caching:

Mount Flag	Default	Performance Optimization	Risk / Trade-off
`noatime`	`relatime`	Eliminates disk writes whenever a file is read	None for 99.9% of servers (breaks legacy mutt mail). Essential for performance.
`nodiratime`	Enabled	Stops updating directory access times on reads	Subsumed by `noatime`.
`barrier=0` / `nobarrier`	`barrier=1`	Disables hardware write cache flush commands	**Extreme data loss risk**: silent filesystem corruption on power loss unless protected by reliable BBU.
`commit=N`	`commit=5`	Extends sync interval for journal commits (e.g. `commit=30`)	Batches journal updates; increases window of lost data during crashes to N seconds.
`discard` vs `fstrim`	Variable	Continuous sync discard degrades write latency	Replace continuous `discard` mount flag with periodic weekly `fstrim.timer`.

```
# Recommended production mount entry for high-performance XFS on NVMe
/dev/nvme0n1p1 /data xfs defaults,noatime,allocsize=64M 0 0
```

Filesystem-Level Caching and Coherency

- **Dentry Cache (dcache)**: Caches directory entries (`struct dentry`), mapping path strings to inodes in memory. Eliminates disk lookups for path resolution.
- **Inode Cache (icache)**: Keeps active filesystem inode metadata in memory.
- **Direct I/O (`O_DIRECT`)**: Bypasses both the Page Cache and filesystem read-ahead, issuing DMA transfers directly between userspace memory buffers and the storage controller. Used by databases (Oracle, ScyllaDB, PostgreSQL with direct I/O) to manage their own caching algorithms without double-buffering in the Page Cache.

Chapter 11: The Networking Stack and Performance

Chapter 11: The Networking Stack and Performance

Modern data center networks operate at line rates of 25Gbps, 100Gbps, and 400Gbps. At 100Gbps, a server can receive over 14 million packets per second (Mpps) for standard 1500-byte frames, or up to 148 Mpps for 64-byte minimum packets. Processing that packet volume through the kernel requires deep understanding of hardware NIC queues, interrupt mitigation, softirqs, socket buffers, and TCP congestion algorithms.

This chapter details the packet journey through the Linux network stack and examines the architectural levers used to maximize network throughput and minimize packet latency.

The Networking Data Path

The Linux network receive path moves packets through hardware, driver, and protocol layers:

```
                    NETWORK RECEIVE PATH (RX)

1. Physical Wire / Fiber    NIC MAC/PHY


2. NIC DMA Engine        Copies frame to Host RAM RX Ring Buffer


3. Hardware MSI-X IRQ     Hard IRQ Handler schedules NAPI


4. NET_RX_SOFTIRQ        napi_poll() drains RX ring budget
                               Allocates struct sk_buff (skb)

5. IP Layer (ip_rcv)      Route lookup, checksum verification


6. TCP Layer (tcp_v4_rcv)    Reassembly, flow control, ACKs
                               Copies payload into Socket Buffer

7. Userspace App        Woken up via epoll / reads data
```

- **Socket Buffer (`struct sk_buff`)**: The core network data structure in Linux. An `sk_buff` (or *skb*) wraps packet headers and data buffers, traveling through protocol layers without copying the underlying packet payload.

Socket Buffering and Memory

Every network socket allocates dedicated memory buffers for receiving and transmitting data:

- **Receive Buffer (`SO_RCVBUF`)**: Holds incoming data that has been acknowledged by TCP but not yet read by userspace.
- **Send Buffer (`SO_SNDBUF`)**: Holds outbound data queued for transmission or awaiting peer acknowledgment for retransmission.
- **TCP Autotuning**: Modern Linux automatically adjusts socket buffer sizes dynamically based on the current connection's Bandwidth-Delay Product (BDP):

```
# Display min, default, and max TCP buffer sizes in bytes
cat /proc/sys/net/ipv4/tcp_rmem   # Read buffer (e.g. 4096  131072  6291456)
cat /proc/sys/net/ipv4/tcp_wmem   # Write buffer (e.g. 4096  16384   4194304)
```

NIC Interrupts and Softirq Processing

Generating a hardware interrupt for every incoming packet at 100Gbps would bring any multi-core CPU to an immediate halt. Linux solves this with the **NAPI (New API)** subsystem:

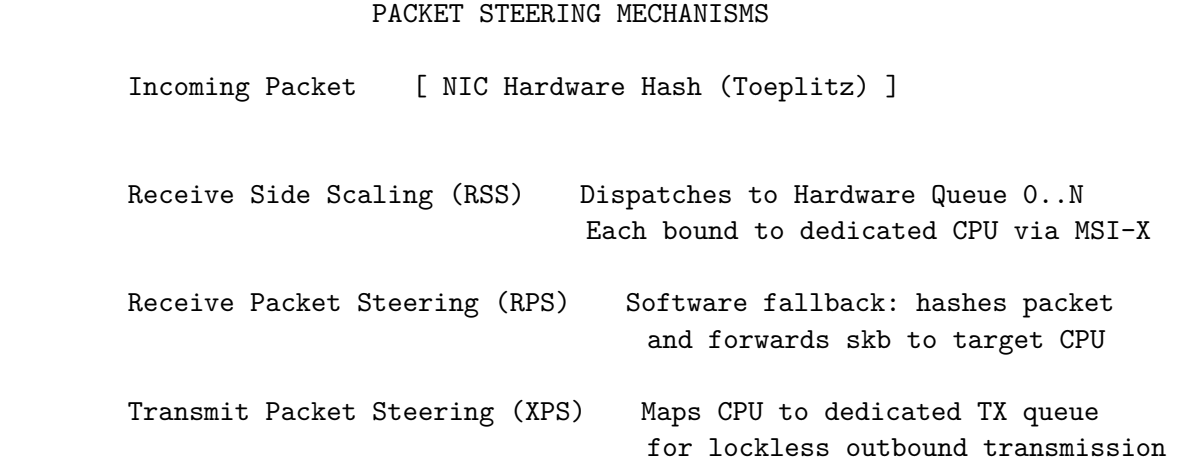
1. The first incoming packet triggers a hardware interrupt.
2. The interrupt handler disables further device interrupts and schedules `NET_RX_SOFTIRQ`.
3. The softirq daemon polls the NIC's ring buffer in batches (controlled by `netdev_budget`, default 300 packets) using high-speed polling loops.
4. Once the ring buffer is empty, interrupts are re-enabled.

```
# Monitor softirq saturation on CPU cores
cat /proc/softirqs | grep NET_RX
```

If a single core shows 100% CPU utilization in `si` (software interrupt) on `top`, that core is saturated by network packet processing.

RSS, RPS, XPS: Distributing Network Load

To scale packet processing across dozens of CPU cores, Linux employs hardware and software steering mechanisms:



- **RSS (Receive Side Scaling)**: Hardware-level distribution. Modern multi-queue NICs hash packet 4-tuples (IP/Port pairs) and route packets directly to distinct hardware queues, each tied to an independent CPU core via dedicated MSI-X interrupts.
- **RPS (Receive Packet Steering)**: Software RSS emulation. Distributes packets across CPUs when NIC hardware lacks multi-queue support.
- **XPS (Transmit Packet Steering)**: Assigns dedicated transmission queues per CPU core, eliminating lock contention on outbound traffic.

TCP Congestion Control and Buffer Tuning

TCP congestion control algorithms govern how aggressively a sender increases its transmission window (`cwnd`) and how it reacts to packet loss or delay:

- **CUBIC**: Default on Linux for years. Uses a cubic function of time since the last loss event to scale `cwnd`. Performs exceptionally well in high-bandwidth, low-latency data center environments, but can suffer bufferbloat on lossy wide-area links.
- **BBR (Bottleneck Bandwidth and RTT)**: Developed by Google. Models the physical delivery rate and round-trip time of the network path directly, transmitting at the bottleneck bandwidth rather than filling network buffers until packet drops occur. BBR delivers dramatically higher throughput over lossy or high-latency WAN links.

```
# Check available and active congestion control algorithms
sysctl net.ipv4.tcp_available_congestion_control
sysctl net.ipv4.tcp_congestion_control
```

UDP and Raw Sockets Performance

Unlike TCP, UDP has no flow control, acknowledgments, or window retransmissions:

- **Packet Drops**: If the receiving application reads from the UDP socket more slowly than incoming packets arrive, the socket receive buffer (`SO_RCVBUF`) overflows, and incoming packets are silently discarded by the kernel (`RcvbufErrors` in `/proc/net/snmp`).
- **Syscall Overhead**: Sending or receiving millions of UDP packets using traditional single-packet `recv()` / `send()` syscalls burns massive CPU cycles.
- **Batching Syscalls**: Modern applications use `recvmmsg()` and `sendmmsg()` to transmit or receive hundreds of packets in a single system call, reducing boundary crossing overhead by up to 80%.

Chapter 12: Network Performance Tuning in Practice

Chapter 12: Network Performance Tuning in Practice

Theoretical knowledge of the Linux network stack must be translated into concrete, reproducible configuration settings for production hosts. Misconfigured network parameters can cap a 100Gbps network link to 1Gbps, exhaust ephemeral ports, trigger bufferbloat, or crash a server under connection spikes.

This chapter details practical network tuning recipes, formulas for socket buffer sizing, congestion control selection, hardware multiqueue configuration, and validation methodologies.

Core TCP Parameters and Their Effects

The following kernel parameters control connection lifecycle, backlog queues, and memory consumption:

```
# Increase backlog queues for incoming SYN packets and established connections
sudo sysctl -w net.core.somaxconn=4096            # Max socket listen backlog
sudo sysctl -w net.ipv4.tcp_max_syn_backlog=8192  # Max half-open SYN connection queue

# Reuse sockets in TIME_WAIT state for new outgoing connections (safe for clients)
sudo sysctl -w net.ipv4.tcp_tw_reuse=1

# Reduce TIME_WAIT timeout from 60s to 30s to free socket structures faster
sudo sysctl -w net.ipv4.tcp_fin_timeout=30

# Expand ephemeral port range for high-concurrency client/proxy hosts
sudo sysctl -w net.ipv4.ip_local_port_range="10240 65535"

# Disable TCP slow start after idle to prevent latency penalties on idle connections
sudo sysctl -w net.ipv4.tcp_slow_start_after_idle=0
```

Socket Buffer Sizing Strategy

To maximize single-stream TCP throughput over long distances, socket buffers must be sized to accommodate the **Bandwidth-Delay Product (BDP)**:

$$\text{BDP} = \text{Bandwidth (bits/sec)} \times \text{Round-Trip Time (RTT in seconds)}$$

Practical Sizing Calculation

Suppose a 10Gbps link connects two data centers with an RTT of 40ms (0.040 seconds):

$$\text{BDP} = 10 \times 10^9 \text{ bps} \times 0.040 \text{ s} = 400,000,000 \text{ bits} = 50,000,000 \text{ bytes} \approx 50 \text{ MB}$$

If the maximum receive buffer (`tcp_rmem` max) is left at the typical default of 6MB, the TCP window will cap out at 6MB, limiting the maximum achievable throughput to:

$$\text{Max Throughput} = \frac{6 \text{ MB}}{0.040 \text{ s}} = 150 \text{ MB/s} = 1.2 \text{ Gbps}$$

The remaining 8.8 Gbps of link capacity will sit completely unused!

```
# Production tuning for high-BDP 10G/40G links
sudo sysctl -w net.core.rmem_max=67108864          # 64 MB
sudo sysctl -w net.core.wmem_max=67108864          # 64 MB
sudo sysctl -w net.ipv4.tcp_rmem="4096 87380 67108864"
sudo sysctl -w net.ipv4.tcp_wmem="4096 65536 67108864"
```

> [!CAUTION] On reverse-proxy servers handling 500,000 concurrent client connections, setting `tcp_rmem` max to 64MB risks instant kernel Out-Of-Memory (OOM) crashes if buffer usage expands. High-concurrency edge nodes require smaller conservative buffer limits (e.g., 2MB to 4MB max).

Congestion Control Algorithm Selection

Modern Linux supports dynamically loading congestion control modules:

```
# Enable BBR congestion control
sudo modprobe tcp_bbr
echo "tcp_bbr" | sudo tee -a /etc/modules-load.d/bbr.conf
sudo sysctl -w net.ipv4.tcp_congestion_control=bbr

# Verify active algorithm
sysctl net.ipv4.tcp_congestion_control
```

- **Use BBR**: For WAN services, internet-facing APIs, cross-region replication, and lossy networks. BBR prevents packet loss from throttling bandwidth and maintains minimal queueing delay.
- **Use CUBIC**: For ultra-low-latency intra-datacenter networks where packet loss is near zero and packet queueing in switch buffers is tightly bounded.

Multiqueue Configuration and RSS Tuning

High-speed NICs require symmetric allocation of queues across physical CPU cores:

```
# Check maximum supported vs currently configured hardware queues
ethtool -l eth0

# Set hardware queues to match physical core count (e.g., 16 queues)
sudo ethtool -L eth0 combined 16

# Increase NIC ring buffer sizes to absorb traffic bursts without packet drops
ethtool -g eth0
sudo ethtool -G eth0 rx 4096 tx 4096
```

Pinning IRQs with `irqbalance` or Static Scripts

Ensure that network interrupts are pinned to the CPU cores belonging to the same NUMA node as the PCIe slot hosting the NIC:

```
# Verify NUMA node of network interface
cat /sys/class/net/eth0/device/numa_node

# View current IRQ affinities for NIC queues
grep eth0 /proc/interrupts
```

Validating Network Performance Changes

Never assume a network tuning parameter succeeded without rigorous verification:

```
# Test raw bandwidth and throughput using iperf3
# Server:
iperf3 -s -p 5201

# Client (multiple parallel streams to saturate link):
iperf3 -c 10.0.0.1 -p 5201 -P 8 -t 30

# Inspect TCP retransmissions, drops, and buffer errors during the test
nstat -z | grep -E "TcpRetransSegs|TcpOutRsts|TcpInErrors"

# Check for NIC-level packet drops and buffer overflows
ethtool -S eth0 | grep -E "drop|miss|error|fifo"
```

If `TcpRetransSegs` or `rx_dropped` increment during the benchmark, the system is suffering from buffer starvation, interrupt drops, or switch-level congestion.

Chapter 13: System Call Tracing and Process Analysis

Chapter 13: System Call Tracing and Process Analysis

When an application misbehaves, degrades in latency, or hangs, performance engineers must observe the boundaries where userspace requests kernel services. Tracing reveals the exact arguments, execution durations, return codes, and call stacks of every system call and internal kernel function.

This chapter details the mechanics, operational overhead, and best practices of system call tracing, dynamic probes, ftrace, perf trace, and traditional tracing frameworks.

strace and ltrace: Mechanics and Use Cases

`strace` is the ubiquitous first tool engineers reach for when debugging process behavior. However, using `strace` in production without understanding its internal mechanics is dangerous:

- **Mechanics of `ptrace()`**: `strace` relies on the `ptrace()` system call. For every single system call executed by the target process, `ptrace` stops the process twice: once upon syscall entry, and once upon syscall exit.
- **The Overhead Trap**: Stopping a process twice per syscall forces multiple context switches, TLB flushes, and inter-process signals. On an application executing 100,000 syscalls per second, `strace` can slow execution down by **10x to 100x**, often turning minor latency anomalies into severe production outages.

Safe Production Use of `strace`

If `strace` must be used in production, strictly limit its scope:

```
# Summarize syscall counts and execution time without printing every call (-c)
strace -c -p <PID> -f

# Filter for specific problematic syscalls (-e) and record per-call latency (-T)
strace -e trace=openat,futex,read -T -p <PID>
```

`ltrace` operates similarly to intercept dynamic library (`libc`, `libssl`) calls, sharing the same `ptrace` overhead profile.

perf: Architecture and Profiling Modes

The Linux `perf` tool (part of the kernel source tree) is a high-performance profiler that operates directly via the `perf_event_open()` system call, sampling CPU hardware counters and kernel tracepoints with negligible overhead (<1–2% CPU load):

```
# Low-overhead syscall tracing with 'perf trace' (replaces strace safely)
sudo perf trace -p <PID> --duration 5.0   # Only print syscalls taking > 5ms

# Profile top system calls system-wide for 10 seconds
sudo perf top -e raw_syscalls:sys_enter
```

ftrace: Kernel Tracing Infrastructure

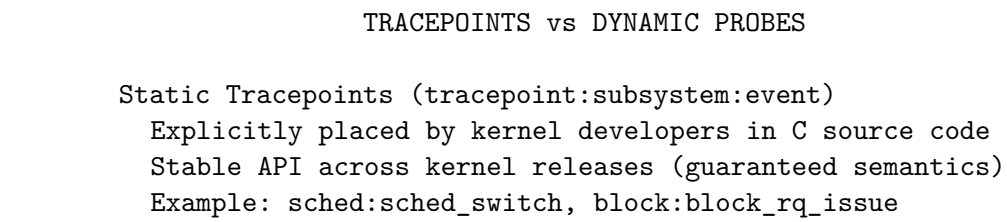
`ftrace` is the built-in function tracer built directly into the Linux kernel. It operates via `tracefs` (typically mounted at `/sys/kernel/tracing` or `/sys/kernel/debug/tracing`):

- **Zero-Overhead Inactive Probes**: Modern Linux kernels use `gcc -pg -mfentry` to place 5-byte NOP instructions at the prologue of every kernel function. When tracing is inactive, execution cost is zero. When activated, the kernel replaces NOPs with jump instructions to tracing trampolines dynamically.
- **`function_graph` Tracer**: Traces function call entries and exits, measuring the execution duration of kernel functions with nanosecond precision.

```
# Tracing kernel function graph via tracefs
cd /sys/kernel/tracing
echo 0 > tracing_on
echo function_graph > current_tracer
echo vfs_read > set_graph_function
echo 1 > tracing_on
head -n 30 trace
echo 0 > tracing_on
```

Tracepoints and Dynamic Probes

Linux provides two distinct mechanisms for instrumenting execution:



```
Dynamic Probes (kprobes & uprobes)
  Injected dynamically at runtime at ANY instruction address
  Breakpoint instruction (INT3 / BRK) patched into running code
  kprobes: Instruments arbitrary kernel functions
  uprobes: Instruments arbitrary userspace binaries and libraries
```

```bash
# Add a dynamic kprobe to measure do_sys_openat2 latency using perf
sudo perf probe --add 'do_sys_openat2 dfd:s32 filename:string'
sudo perf record -e probe:do_sys_openat2 -a sleep 5
sudo perf script
```

SystemTap: Overview and Comparison

SystemTap was the pioneering dynamic tracing infrastructure for Linux. It compiles user-written scripts into kernel C modules, loads them into the running kernel, and executes instrumentation handlers.

- **Strengths**: Powerful language, deep support for legacy RHEL/CentOS distributions.
- **Weaknesses**: Requires kernel development packages (`kernel-devel`, `kernel-debuginfo`), compilation takes tens of seconds, and a buggy script can cause kernel crashes.
- **Modern Consensus**: In modern production engineering (kernel 5.x/6.x+), SystemTap has been almost completely superseded by **eBPF** and **bpftrace**.

Building a Tracing Strategy

To minimize production risk while debugging performance:

1. **Level 1 (Counters & Metrics)**: Start with non-invasive metrics (`vmstat`, `sar`, `node_exporter`, PSI).
2. **Level 2 (Sampling Profiling)**: Use `perf record -F 99` or `perf top` to find hot functions without instrumenting individual calls.
3. **Level 3 (Static Tracepoints)**: Use `perf trace` or `bpftrace` on established tracepoints (`sched:sched_switch`, `block:block_rq_issue`).
4. **Level 4 (Dynamic Probes)**: Use `kprobes`/`uprobes` only when hunting specific function arguments or rare latency outliers.
5. **Level 5 (strace)**: Restrict `strace` to short diagnostic bursts with tight filtering (`-e trace=... -c`), never on high-throughput production request loops.

Chapter 14: eBPF and BPF-Based Observability

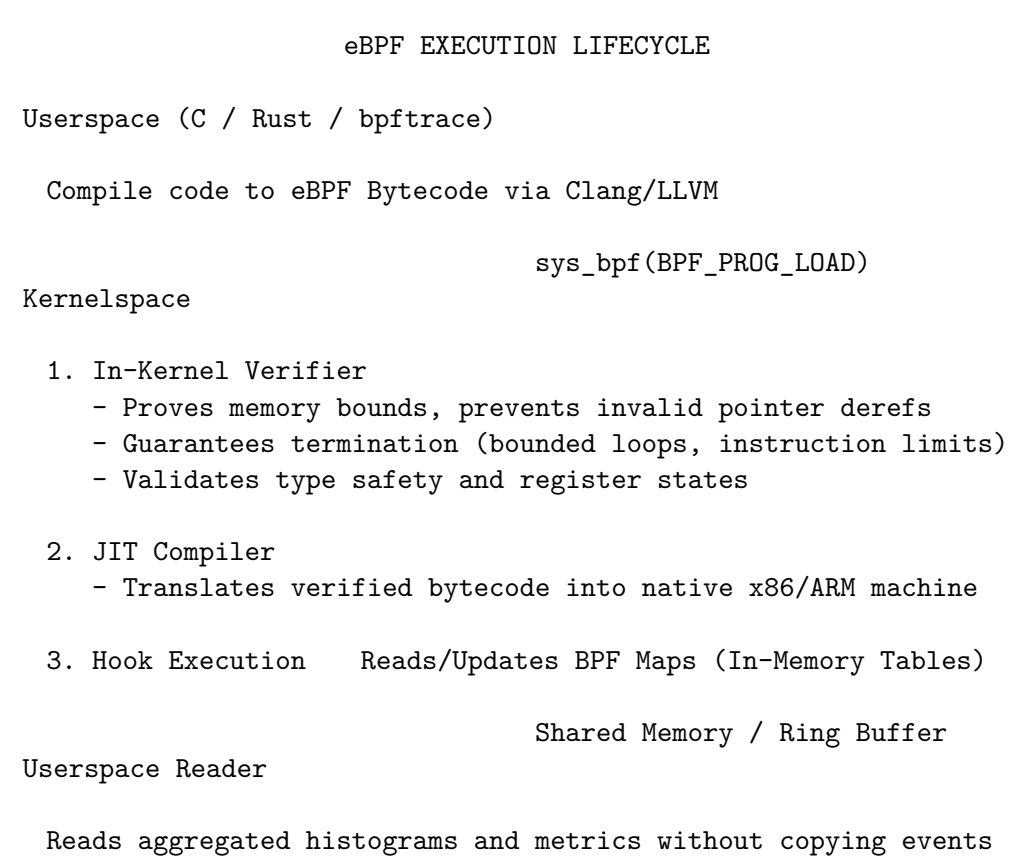
Chapter 14: eBPF and BPF-Based Observability

Extended Berkeley Packet Filter (eBPF) revolutionized Linux systems engineering by turning the operating system kernel into a safely programmable platform. Instead of modifying kernel source code, loading fragile out-of-tree kernel modules, or incurring `ptrace` context-switch penalties, engineers can load sandboxed bytecode directly into the kernel to measure, filter, and trace events at line rate.

This chapter details the internal eBPF architecture, the in-kernel verifier, BPF map data structures, BCC diagnostic tools, bpftrace one-liners, and production safety considerations.

eBPF Architecture and Safety Model

eBPF programs are event-driven: they attach to specific kernel hooks (kprobes, tracepoints, socket filters, XDP, perf events, or cgroups) and execute whenever the associated event fires.



- **The In-Kernel Verifier**: Guarantees that the program cannot crash the kernel, dereference uninitialized pointers, or enter infinite loops. If the verifier cannot prove safety mathematically, program loading is rejected.
- **JIT Compilation**: Bytecode is compiled directly into native CPU machine instructions, executing at near-native C speed.

BPF Map Types and Communication

BPF programs communicate with userspace and maintain state across events using **BPF Maps**—key-value storage structures allocated in kernel memory:

Map Type	Description	Optimization Target
`BPF_MAP_TYPE_HASH`	Generic associative hash table	Fast lookup by arbitrary keys (e.g. PID, IP address).
`BPF_MAP_TYPE_ARRAY`	Fixed-size pre-allocated array	Fastest possible indexed lookup; no allocation during execution.
`BPF_MAP_TYPE_PERCPU_HASH`	Hash table with private per-CPU storage	Lockless execution; eliminates cross-core cache invalidation.
`BPF_MAP_TYPE_RINGBUF`	High-throughput, multi-producer ring buffer	Asynchronous streaming of events to userspace with low overhead.

bpftrace: High-Level Tracing

`bpftrace` is the premier high-level tracing language for Linux, inspired by AWK and DTrace:

```
# Measure block I/O latency distribution (visualized as a power-of-2 histogram)
sudo bpftrace -e '
  kprobe:blk_account_io_start { @start[arg0] = nsecs; }
  kprobe:blk_account_io_done /@start[arg0]/ {
    @latency_us = hist((nsecs - @start[arg0]) / 1000);
    delete(@start[arg0]);
  }'

# Trace slow filesystem sync calls taking longer than 10 milliseconds
sudo bpftrace -e '
  kprobe:ksys_fsync { @t[tid] = nsecs; }
  kretprobe:ksys_fsync /@t[tid]/ {
    $dur_ms = (nsecs - @t[tid]) / 1000000;
    if ($dur_ms > 10) {
      printf("fsync PID %d (%s) took %d ms\n", pid, comm, $dur_ms);
```

```
    }
    delete(@t[tid]);
  }'
```

BCC Tools: Key Utilities

The BPF Compiler Collection (BCC) ships pre-built, production-tested diagnostic utilities:

```
# Monitor new process executions in real time with parent PID and command line
sudo execsnoop-bpfcc

# Trace file open operations, including latency and error codes (ENOENT, EACCES)
sudo opensnoop-bpfcc -T -e

# Summarize disk I/O latency distribution every 5 seconds
sudo biolatency-bpfcc 5 1

# Trace TCP connection establishment latency (SYN to SYN-ACK)
sudo tcpconnect-bpfcc -u

# Detect TCP packet retransmissions with source/destination addresses
sudo tcpretrans-bpfcc
```

Writing Custom BPF Programs (CO-RE and libbpf)

Modern eBPF development uses **Compile Once – Run Everywhere (CO-RE)** with `libbpf`:

- **BTF (BPF Type Format)**: Embedded kernel debug metadata that allows eBPF programs to read kernel structs without needing kernel headers installed on the target machine.
- **Relocation**: Clang and `libbpf` dynamically adjust struct field offsets at load time, ensuring a binary compiled on kernel 6.2 runs unmodified on kernel 6.8.

```
// Example CO-RE probe reading task_struct->pid safely
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_core_read.h>

SEC("kprobe/sched_switch")
int BPF_KPROBE(trace_sched_switch, bool preempt, struct task_struct *prev, struct task_struc
    pid_t prev_pid = BPF_CORE_READ(prev, pid);
    bpf_printk("Switched from PID %d\n", prev_pid);
    return 0;
}
char LICENSE[] SEC("license") = "GPL";
```

eBPF Overhead and Production Safety

While eBPF is vastly safer and faster than kernel modules or `ptrace`:

1. **High-Frequency Hooks**: Attaching probes to functions called millions of times per second (e.g., `vfs_read` or memory allocators) can introduce 3%–10% CPU overhead purely from probe trampolines. Prefer tracepoints or sample-based perf events.
2. **Map Exhaustion**: Hash maps with unbounded growth will reject insertions once full. Always size maps appropriately or use LRU maps (`BPF_MAP_TYPE_LRU_HASH`).
3. **In-Kernel Aggregation**: Never push raw events to userspace one by one. Accumulate counts, sums, and histograms in BPF maps inside kernel space, and emit only summarized results periodically.

Chapter 15: Performance Analysis Tools and Flame Graphs

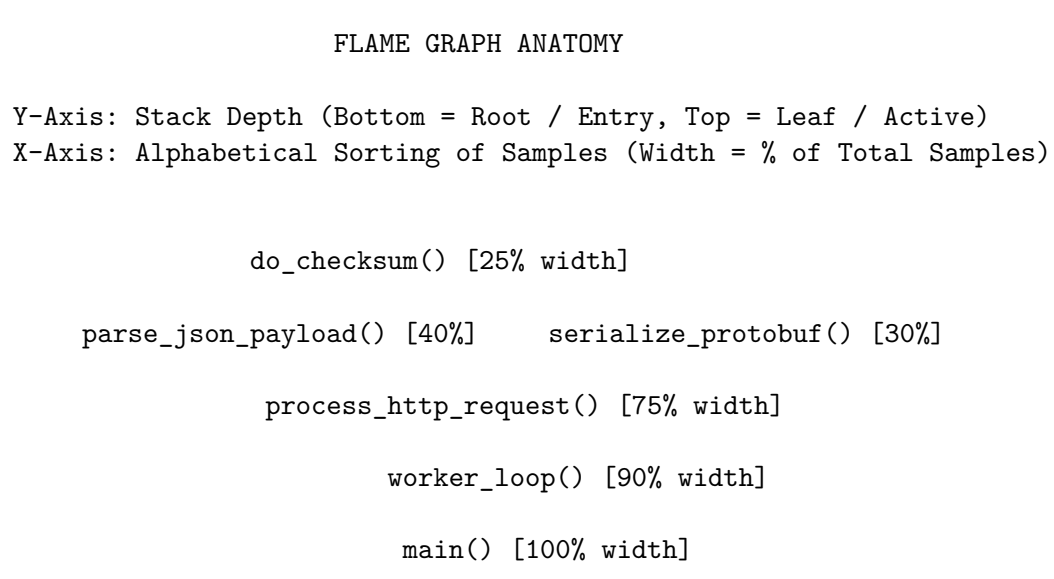
Chapter 15: Performance Analysis Tools and Flame Graphs

When profiling a complex application running millions of lines of code across thousands of threads, raw text profiler outputs (such as flat function tables) become overwhelming. Flame Graphs, invented by Brendan Gregg, convert deep call stacks into an intuitive, interactive visual representation that allows engineers to identify CPU bottlenecks and off-CPU blocking latency in seconds.

This chapter covers the construction, interpretation, and automation of on-CPU and off-CPU Flame Graphs using `perf`, eBPF, and continuous profiling pipelines.

Flame Graphs: Construction and Interpretation

A Flame Graph visualizes sampled stack traces:



Reading Rules

1. **Width Represents Sample Proportion**: The wider a frame appears on the horizontal axis, the more times that function was present in the sampled stack traces.

2. **Horizontal Axis Is Not Time**: The x-axis does not represent the passage of time; stacks are sorted alphabetically to merge identical code paths.
3. **Plateaus Are Hotspots**: A wide frame with a flat top ("plateau") indicates that the function itself was actively executing on the CPU (leaf frame), burning cycles directly rather than calling sub-functions.

perf Record and Report

Generating an On-CPU Flame Graph begins with sampling stack traces using `perf`:

```
# 1. Sample stack traces at 99 Hertz across all CPU cores for 30 seconds
# Using 99Hz instead of 100Hz avoids sampling in lockstep with periodic timers
sudo perf record -F 99 -a -g -- sleep 30

# 2. Inspect results interactively in terminal
sudo perf report -n --stdio
```

Unwinding Call Stacks: Frame Pointers vs DWARF

To reconstruct full stack traces, `perf` must unwind the stack:

- **Frame Pointers (`-fno-omit-frame-pointer`)**: Keeps the `RBP` register dedicated to stack frames. Fast, low overhead, and works seamlessly with eBPF and `perf`. Modern enterprise distros (Ubuntu 24.04+, Fedora 38+) compile packages with frame pointers enabled by default.
- **DWARF Debug Info (`--call-graph dwarf`)**: Unwinds using `.debug_frame` sections. High overhead; can record gigabytes of stack data in seconds.
- **LBR (Last Branch Record)**: Uses hardware CPU buffers for leaf-level call stacks with near-zero overhead.

Generating the Interactive SVG

```
# Script perf output and pipe to FlameGraph scripts
sudo perf script | stackcollapse-perf.pl | flamegraph.pl > cpu-profile.svg
```

Hotspot Analysis and Call Path Tracing

On-CPU vs Off-CPU Profiling

Not all performance bottlenecks burn CPU cycles:

- **On-CPU Flame Graphs**: Show where threads spend time while actively executing instructions on a core. Essential for compute-bound algorithms, JSON parsing, compression, and crypto.

- **Off-CPU Flame Graphs**: Show where threads spend time blocked and descheduled (waiting on disk I/O, network sockets, mutexes, condition variables, or kernel locks).

```
# Record off-CPU blocking time using eBPF (BCC offcputime tool)
sudo offcputime-bpfcc -df -p <PID> 10 | flamegraph.pl --color=io > offcpu.svg
```

If an application is unresponsive but CPU utilization is only 5%, an **Off-CPU Flame Graph** will immediately reveal the exact lock, socket, or file read stalling the worker threads.

Combining Tools: Integrated Workflows

A mature performance drill-down combines high-level monitoring with stack profiling:

```
                    INTEGRATED INVESTIGATION FLOW

1. System Overview (USE Method)
   sar / mpstat / vmstat    Detects CPU Saturation


2. Process Identification
   pidstat / top        Pinpoints Problematic PID


3. Stack Sampling & Flame Graphs
   perf record -p <PID>    Identifies Hot Function Path


4. Kernel/Hardware Drill-Down
   perf stat -e ...      Pinpoints PMU Cache Miss / IPC Stall
   bpftrace          Measures Function Argument Outliers
```

Sampling vs Tracing: Trade-offs

Dimension	Sampling (perf record)	Event Tracing (ftrace / kprobes)
Mechanism	Samples stack at periodic intervals (e.g., 99Hz)	Captures 100% of event entries and exits
Overhead	Fixed, deterministic (<1–2% CPU load)	Scales linearly with event frequency (can exceed 100% overhead)
Precision	Statistical approximation; can miss rare micro-bursts	Exact counts and nanosecond durations for every call

Dimension	Sampling (perf record)	Event Tracing (ftrace / kprobes)
Best Used For	Continuous profiling, CPU hotspot discovery	Rare latency outliers, lock contention, exact error paths

Automated Analysis Pipelines

In modern cloud-native fleets, manually logging into individual nodes to run `perf` is impractical. Production infrastructure deploys **Continuous Profiling Agents**:

- **Parca & Pyroscope**: Open-source, eBPF-based profiling daemons that run on every node, continuously sample CPU and memory profiles at <1% overhead, and push pprof data to central query stores.
- **Diff Flame Graphs**: Highlight architectural regressions between releases by overlaying two Flame Graphs: code paths that grew wider are colored red (regressions), while paths that shrank are colored blue (optimizations).

Chapter 16: System Monitoring and Observability

Chapter 16: System Monitoring and Observability

Long before an incident prompts deep-dive kernel tracing, continuous system monitoring must establish normal baseline behavior, identify emerging resource saturation, and fire early warning alerts. Linux provides both time-tested classic monitoring utilities and cutting-edge metrics like Pressure Stall Information (PSI).

This chapter teaches the correct interpretation of core monitoring tools, exposes misleading counters, and provides patterns for building production dashboards and alerts.

Classic Tools: sar, vmstat, iostat, mpstat, pidstat

The `sysstat` suite and core Linux utilities provide the foundation of command-line performance triage:

`vmstat 1` (System Overview)

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 3  0      0 412030  12400 892010    0    0    12    45  820 1450 45 12 40  3  0
```

- `r` (Runqueue): Runnable threads waiting for CPU time. If `r > number of CPU cores`, the system is CPU-saturated.
- `b` (Blocked): Threads sleeping in uninterruptible disk I/O (`TASK_UNINTERRUPTIBLE` / D-state).
- `si` / `so`: Swap-in and swap-out rates in KB/s. Any continuous non-zero values indicate critical memory exhaustion.
- `wa` (iowait): Time CPUs spent idle while at least one I/O request was in flight. High `wa` means the workload is I/O-bound.

`iostat -xz 1` (Extended Storage Metrics)

```
# Monitor disk utilization and queue latency, hiding idle devices (-z)
iostat -xz 1
```

- `r/s` and `w/s`: Read and write requests completed per second (IOPS).
- `r_await` and `w_await`: Average time in milliseconds that read and write requests took from creation to device completion (includes block queue time + disk service time).

- `%util`: Percentage of CPU time during which requests were in flight to the device. Note: on modern multi-queue SSDs, `%util = 100%` does not mean the drive is saturated, because multi-queue NVMe devices can handle 64 parallel queues simultaneously. Use `r_await` and queue depth instead.

`mpstat -P ALL 1` (Per-Core Breakdown)

Identifies single-core CPU saturation: - `%usr`: Time spent in userspace code. - `%sys`: Time spent executing kernel code and system calls. - `%si`: Time spent handling software interrupts (softirqs). If a single core shows 90% `%si`, network packet processing is bottlenecked on that core.

`pidstat 1` (Per-Process Attribution)

Identifies which specific process is driving system resource consumption: - `pidstat -u 1`: CPU usage by process. - `pidstat -d 1`: Disk I/O read/write rates in KB/s per process. - `pidstat -w 1`: Voluntary vs involuntary context switches per process.

Modern Monitoring: top, htop, numastat

- **`top` / `htop`**: Real-time process viewers. In `top`, press `1` to toggle per-core display; press `H` to toggle thread-level visibility. Pay close attention to process state: `R` (running), `S` (interruptible sleep), `D` (uninterruptible disk sleep), and `Z` (zombie).
- **`numastat`**: Shows per-NUMA node memory allocation hits and misses. High miss ratios indicate memory is being allocated on remote sockets, causing interconnect latency penalties.

Network Monitoring: ss, ethtool, nstat

```
# Inspect socket queue backlogs (Send-Q and Recv-Q)
ss -tulpn

# Monitor hardware NIC error counters and dropped frames
ethtool -S eth0 | grep -E "drop|miss|error|fifo"

# View Linux IP/TCP protocol engine error counters
nstat -az
```

- In `ss -tln`: For listening sockets, `Recv-Q` indicates the number of connections waiting in the accept queue (`somaxconn`), while `Send-Q` indicates the maximum queue size. If `Recv-Q == Send-Q`, the application is failing to accept connections fast enough, causing incoming client connections to time out.

Pressure Stall Information (PSI) and Modern Metrics

Traditional load averages (`/proc/loadavg`) are flawed: they blend CPU and disk D-state tasks together, make no distinction between a 4-core machine and a 128-core machine, and react with slow exponential decay.

Pressure Stall Information (PSI), introduced in modern Linux, measures the exact percentage of time tasks are stalled waiting for CPU, Memory, or I/O:

```
# View CPU pressure
cat /proc/pressure/cpu

# View Memory pressure
cat /proc/pressure/memory

# View I/O pressure
cat /proc/pressure/io
```

Example PSI output:

```
some avg10=2.45 avg60=1.12 avg300=0.50 total=4820194
full avg10=0.00 avg60=0.00 avg300=0.00 total=0
```

- `some`: The percentage of wall-clock time during which at least one task was stalled waiting for the resource while other tasks could still make progress.
- `full`: The percentage of wall-clock time during which *all* runnable tasks were stalled waiting for the resource (total system freeze / livelock).

PSI provides an unambiguous, objective saturation indicator that powers modern container resource managers and Kubernetes autoscalers.

Building Production Dashboards and Alerting

When engineering production dashboards (e.g. Prometheus + Grafana):

1. **Dashboard Structure**:
    - **Level 1 (Top Level)**: Red/Amber/Green service SLOs (p95/p99 latency, error rate, request throughput).
    - **Level 2 (Host Saturation)**: PSI metrics (`avg10` for CPU, memory, and I/O pressure), network saturation (`ss` backlog, NIC drops).
    - **Level 3 (Subsystem Detail)**: Per-core `%si`, memory PSS, block `r_await`/`w_await`, filesystem dirty page count.
2. **Alerting Rules**:
    - Alert on symptoms affecting SLOs (e.g. p99 latency > target for 3 minutes).
    - Alert on leading indicators of failure (e.g. Memory PSI `some > 20%` for 2 minutes, indicating imminent swapping and OOM-killer invocation).

- Avoid alerting on raw CPU utilization alone; a system at 95% CPU with 0% CPU PSI pressure and stable p99 latency is simply running efficiently.

Chapter 17: Benchmarking: Design and Execution

Chapter 17: Benchmarking: Design and Execution

Benchmarking is deceptively difficult. It is trivial to write a loop that executes millions of operations and prints a number. It is extraordinarily difficult to construct a benchmark that accurately represents production workload behavior, controls for background environmental noise, avoids benchmarking artifacts, and produces statistically valid findings.

This chapter details the design, methodology, isolation controls, and statistical rigor required to execute meaningful benchmarks on Linux servers.

What Makes a Valid Benchmark

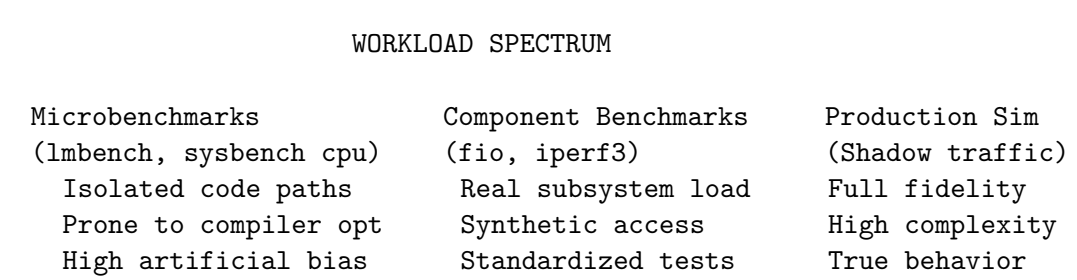
A benchmark is valid if and only if:

1. **Relevance**: The benchmark exercises the identical hardware paths, kernel subsystems, and memory access patterns that limit production performance.
2. **Repeatability**: Multiple independent runs executed on the same hardware produce statistically consistent results within narrow confidence intervals ($\pm 2\%$).
3. **Actionability**: The output allows engineers to make predictive capacity, configuration, or architectural decisions with quantifiable ROI.

If optimizing a benchmark does not improve production service metrics, the benchmark is invalid.

Designing Representative Workloads

Synthetic microbenchmarks (such as tight loops incrementing integers or reading 4KB blocks from a single file) frequently produce misleading conclusions:



To create representative workloads: - **Mirror Working Set Size**: If production uses a 128GB database, a benchmark dataset of 2GB will fit entirely inside the Linux Page Cache, giving the false impression of 200,000 IOPS while masking real storage latency. - **Model Real Concurrency & Key Distributions**: Real traffic follows Zipfian or Pareto (80/20) key distributions with bursty Poisson arrival rates, rather than uniform random access. - **Capture Mixed Operations**: Real systems execute reads, writes, transactions, and background compactions simultaneously.

Test Environment Isolation and Control

Environmental variance must be systematically eliminated:

```
# 1. Lock CPU frequency to maximum performance to eliminate governor scaling jitter
sudo cpupower frequency-set -g performance

# 2. Disable Turbo Boost / Precision Boost for strict timing consistency
echo 0 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo 2>/dev/null || \
echo 0 | sudo tee /sys/devices/system/cpu/cpufreq/boost 2>/dev/null

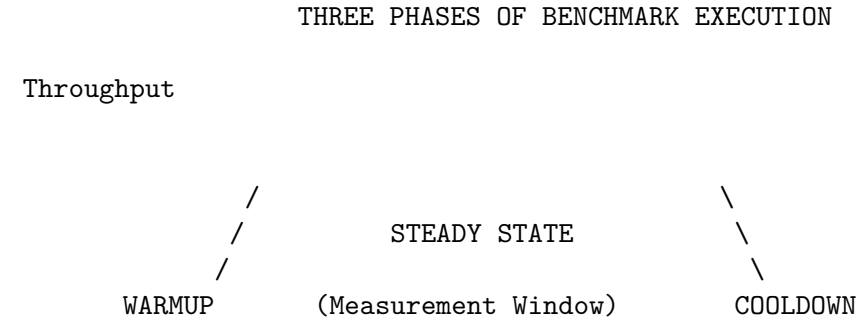
# 3. Disable ASLR (Address Space Layout Randomization) during dedicated microbenchmarking
# Note: re-enable after benchmarking
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

# 4. Flush Page Cache, Dentries, and Inodes between test iterations
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

- **Stop Background Daemons**: Disable cron jobs, package auto-updates (`unattended-upgrades`), updatedb/locate, and monitoring agents on the testbed.
- **Hardware Isolation**: Pin benchmark threads and client load generators to dedicated CPU sockets using `numactl` or `taskset`. Never run the load generator and the target server on the same CPU cores.

Warm-Up, Steady State and Cooldown

Every benchmark execution must be divided into three explicit phases:





1. **Warm-Up Phase**: Allows the system to reach thermodynamic and software steady state. Fills the Page Cache, allows the JVM JIT compiler to tier code to native assembly, warms CPU branch target buffers, and sizes connection pools. Metrics recorded during warm-up must be discarded.
2. **Steady State Phase**: The active measurement window. Must be sufficiently long (e.g. 10 to 30 minutes) to trigger background maintenance cycles such as filesystem dirty page writeback, SSD flash garbage collection, and database checkpoint flushes.
3. **Cooldown Phase**: Load is ramped down. Allows lingering asynchronous write queues to drain cleanly before taking state snapshots.

Repetition, Reproducibility and Statistical Analysis

A benchmark result must never be reported as a single number:

- Execute at least $N = 5$ to $N = 10$ full iterations from a cold reboot or clean state.
- Report the **Median**, **IQR (Interquartile Range)**, and standard deviation.
- Visualize results with cumulative distribution function (CDF) curves and latency histograms rather than simple bar charts.
- Verify that standard deviation across runs is under 2% to 3%. High variance indicates unmanaged environmental interference.

Benchmarking Mistakes: Gaming, Noise and Bias

Mistake	Root Cause	Prevention
Benchmarking the Harness	The load client (e.g., Python script) saturates its own CPU core or runs out of file descriptors before the server reaches capacity.	Monitor the load generation machine with `top` and `sar`; ensure client CPU utilization remains $< 60\%$.
Dead Code Elimination	The compiler optimizes away the benchmark loop because results are never used or printed.	Consume the result of computations via volatile variables or `asm volatile("" : : "r"(val) : "memory")`.
Coordinated Omission	Closed-loop load generators wait for previous responses before sending the next request, skipping scheduled requests during server freezes.	Use open-model load generators like `wrk2` or `k6` that schedule requests on an independent timeline.

Mistake	Root Cause	Prevention
Cold Cache Distortion	Comparing Run 1 (cold disk cache) with Run 2 (warm RAM cache) and falsely concluding a configuration tweak doubled performance.	Always enforce identical cache warm-up procedures before starting timing.

Chapter 18: Benchmarking Tools and Suites

Chapter 18: Benchmarking Tools and Suites

An engineer is only as effective as their instrumentation and benchmarking toolchain. Using the wrong tool or running a utility with flawed arguments produces distorted data that misleads engineering decisions.

This chapter provides a practical, deep-dive reference for the industry-standard benchmarking tools used to stress and profile storage, CPU, memory, networking, and application runtimes on Linux servers.

fio: Storage Benchmarking Deep Dive

Flexible I/O Tester (`fio`) is the gold standard for storage benchmarking. It simulates any arbitrary block or file access pattern, supporting multiple I/O engines (`libaio`, `io_uring`, `posix_aio`), thread counts, queue depths, and mixed read/write ratios.

Standard NVMe Random Read/Write Job File

Create a declarative job specification `nvme-benchmark.fio`:

```
[global]
ioengine=io_uring
direct=1
runtime=60
time_based=1
group_reporting=1
filename=/dev/nvme0n1
size=100%

[random-read-4k]
bs=4k
rw=randread
iodepth=64
numjobs=4
stonewall

[random-write-4k]
bs=4k
rw=randwrite
iodepth=64
```

```
numjobs=4
stonewall

[sequential-read-128k]
bs=128k
rw=read
iodepth=16
numjobs=2
stonewall
```

Execute the job:

```
sudo fio nvme-benchmark.fio --output=fio-results.json --output-format=json
```

Key output metrics to evaluate: - `iops`: Total operations per second. - `clat` (Completion Latency): Device turnaround time. Inspect the 95th, 99th, and 99.9th percentiles (`p99`). - `lat` (Total Latency): Time from submission to completion, including block layer queueing delays.

stress-ng: Comprehensive System Stress Testing

`stress-ng` stresses Linux systems across hundreds of specialized hardware and kernel stressor modules (virtual memory, schedulers, IPC, file operations, CPU vectors, matrix math):

```
# Stress CPU ALU, matrix math, and floating point on 8 cores for 60 seconds
stress-ng --cpu 8 --cpu-method matrixprod --timeout 60s --metrics-brief

# Stress virtual memory subsystem (allocating, dirtying, and unmapping 16GB RAM)
stress-ng --vm 4 --vm-bytes 4G --vm-method all --verify --timeout 60s

# Stress context switching and scheduler runqueues
stress-ng --switch 16 --timeout 30s
```

`stress-ng` reports bogo-ops/sec (operations per second), providing an immediate measure of system throughput under intense hardware and kernel saturation.

sysbench: CPU, Memory, File I/O, Database Tests

`sysbench` is a modular multi-threaded benchmark tool:

```
# 1. CPU Prime Number Benchmark (tests integer execution pipelines)
sysbench cpu --cpu-max-prime=20000 --threads=8 run

# 2. Memory Sequential and Random Throughput
sysbench memory --memory-block-size=1M --memory-total-size=100G --threads=4 run
```

```
# 3. Thread Mutex Contention Benchmark
sysbench mutex --mutex-num=1024 --mutex-locks=1000000 --threads=16 run

# 4. OLTP Database Benchmark (PostgreSQL / MySQL)
sysbench /usr/share/sysbench/oltp_read_write.lua \
  --db-driver=pgsql --pgsql-host=127.0.0.1 --pgsql-user=test --pgsql-db=test \
  --tables=10 --table-size=1000000 --threads=32 --time=120 run
```

iperf3: Network Throughput and Latency

`iperf3` measures point-to-point TCP, UDP, and SCTP bandwidth and packet loss:

```
# High-throughput multi-stream TCP test (saturating 10G/40G links)
# Server node:
iperf3 -s -p 5201

# Client node (8 parallel streams, 60-second duration, 1-second interval reports):
iperf3 -c 10.0.0.1 -p 5201 -P 8 -t 60 -i 1

# UDP test measuring jitter, latency, and packet loss at 1 Gbps rate
iperf3 -c 10.0.0.1 -u -b 1G -t 30
```

lmbench: Latency and Bandwidth Microbenchmarks

`lmbench` is the historic suite for measuring low-level OS primitives with microsecond precision: - **Null System Call Overhead**: Time to enter Ring 0 and return (`getpid()`). - **Context Switch Latency**: Time to switch between 2 to 100 competing processes. - **Memory Read/Write Bandwidth**: Bandwidth across L1, L2, L3 caches, and main DRAM. - **Pipe and IPC Latency**: Round-trip time of passing tokens through UNIX pipes.

Application-Level: wrk, ApacheBench, curl

`wrk` (High-Concurrency HTTP Benchmarking)

`wrk` uses epoll and event loops to generate massive HTTP concurrency from a single client node:

```
# Run HTTP benchmark with 12 threads and 400 concurrent connections for 30s
wrk -t12 -c400 -d30s --latency http://10.0.0.1:8080/api/v1/health
```

Using `wrk2` adds constant-throughput rate limiting (`-R <req/sec>`), eliminating coordinated omission and providing accurate p99/p99.9 tail latency percentiles.

`curl` (Detailed Connection Timing)

For individual request profiling, format curl to print exact phase durations:

```
curl -w "\nLookup: %{time_namelookup}s\nConnect: %{time_connect}s\nTLS: %{time_appconnect}s\
  -o /dev/null -s https://example.com/api
```

Phoronix Test Suite and Comparative Testing

The **Phoronix Test Suite (PTS)** is an automated open-source benchmarking and validation framework: - Automates download, compilation, execution, and metric parsing for hundreds of standard test profiles (Linux kernel compilation, computational fluid dynamics, OpenSSL encryption, PostgreSQL, 7-zip). - Generates reproducible, standardized XML/HTML reports and supports upload to OpenBenchmarking.org for cross-hardware performance comparisons.

Chapter 19: CPU and Memory Tuning for Production

Chapter 19: CPU and Memory Tuning for Production

Production performance engineering requires translating profiling discoveries into stable, deterministic host configurations. Leaving server hardware and memory management in their default out-of-the-box states introduces dynamic frequency scaling jitter, memory allocation stalls, and cross-NUMA interconnect thrashing.

This chapter provides battle-tested production recipes for CPU governors, IRQ affinity, task isolation, Huge Pages, and memory sysctl parameters, accompanied by validation tests.

CPU Governor Selection and Configuration

Default distribution power-saving governors (`powersave`, `ondemand`) introduce 10μs–100μs latency penalties whenever CPU cores ramp up clock frequency from idle P-states:

```
# Lock all cores to performance governor
sudo cpupower frequency-set -g performance

# Set Energy Performance Preference (EPP) to maximum performance on Intel/AMD
for f in /sys/devices/system/cpu/cpu*/power/energy_performance_preference; do
  echo "performance" | sudo tee "$f" > /dev/null
done

# Restrict deep C-states to prevent sleep exit latency penalties
sudo cpupower idle-set -D 2   # Disables C-states deeper than C1
```

IRQ Affinity and NUMA-Optimal Placement

Hardware devices (NICs, NVMe controllers, RAID adapters) fire interrupts to CPU cores. If `irqbalance` routes network interrupts to Core 0 while the network worker thread runs on Core 32 on a separate NUMA socket, every received packet traverses the inter-socket interconnect:

```
# Disable generic irqbalance service on dedicated low-latency hosts
sudo systemctl stop irqbalance
sudo systemctl disable irqbalance

# Manually route IRQ 84 (eth0 RX queue 0) to CPU Core 4 (bitmask: 1 << 4 = 0x10)
```

```
echo "10" | sudo tee /proc/irq/84/smp_affinity
```

Always align IRQ affinities to cores within the same NUMA node as the PCIe device:

```
# Discover NUMA node of PCIe device
cat /sys/bus/pci/devices/0000:01:00.0/numa_node
```

CPU Isolation for Critical Workloads

For deterministic microsecond execution (trading algorithms, packet processing, dedicated database engines), isolate dedicated physical cores via `/etc/default/grub`:

```
# Append isolation parameters to GRUB_CMDLINE_LINUX_DEFAULT
# Example: 64-core system, isolate cores 32-63 for dedicated worker threads
GRUB_CMDLINE_LINUX_DEFAULT="... isolcpus=32-63 nohz_full=32-63 rcu_nocbs=32-63"
```

Update GRUB and reboot:

```
sudo update-grub
```

- `isolcpus`: Excludes cores from CFS/EEVDF automatic load balancing.
- `nohz_full`: Disables the scheduler timer interrupt tick whenever a single thread runs on the core.
- `rcu_nocbs`: Moves RCU callback processing off isolated cores to housekeeping cores (cores 0–31).

Huge Pages: Configuring and Validating

Huge Pages eliminate TLB cache thrashing for memory-intensive workloads:

Allocating 1GB Static Huge Pages at Boot

In `/etc/default/grub`:

```
GRUB_CMDLINE_LINUX_DEFAULT="... default_hugepagesz=1G hugepagesz=1G hugepages=32"
```

This reserves 32GB of contiguous physical memory at boot, mapped as 32 1GB pages.

Runtime Allocation of 2MB Huge Pages

```
# Allocate 4096 2MB huge pages (8 GB total) across NUMA nodes
echo 4096 | sudo tee /proc/sys/vm/nr_hugepages

# Verify allocated and free huge pages
grep -i huge /proc/meminfo
```

Mount `hugetlbfs` for applications:

```
sudo mkdir -p /mnt/huge
sudo mount -t hugetlbfs nodev /mnt/huge
```

Memory Sysctl Parameters

Tune kernel virtual memory behavior for production stability:

```
# /etc/sysctl.d/99-memory-tuning.conf

# Reduce aggressiveness of swapping anonymous memory (default 60)
vm.swappiness = 10

# Keep 5% of memory reserved for kernel atomic allocations to prevent OOM livelock
vm.min_free_kbytes = 1048576

# Regulate dirty page writeback to avoid massive disk stalls on large RAM systems
vm.dirty_background_bytes = 268435456
vm.dirty_bytes = 1073741824

# Maintain balanced inode and dentry cache reclamation
vm.vfs_cache_pressure = 100

# Prevent overcommit memory panics for predictable database sizing
vm.overcommit_memory = 0
vm.overcommit_ratio = 50
```

Apply immediately:

```
sudo sysctl --system
```

Validating CPU and Memory Tuning

Verify that your tuning eliminated latency spikes and stabilized throughput:

```
# 1. Verify CPU frequencies are locked at maximum clock speed
watch -n 1 "grep 'cpu MHz' /proc/cpuinfo"

# 2. Check for timer tick interrupts on nohz_full cores (should be near zero)
cat /proc/interrupts | grep "Local timer interrupts"

# 3. Benchmark memory allocation and access latency before and after
sysbench memory --memory-block-size=2M --memory-total-size=50G --threads=8 run

# 4. Measure tail latency jitter with cyclictest
sudo cyclictest -m -p 90 -t 4 -n -i 1000 -l 100000
```

Under tuned conditions, `cyclictest` should report maximum latency jitter under 15μs, compared to 150μs+ on stock out-of-the-box installations.

Chapter 20: I/O and Filesystem Tuning for Production

Chapter 20: I/O and Filesystem Tuning for Production

Storage bottlenecks are among the most frequent causes of tail latency degradation in stateful systems. Without deliberate device and filesystem configuration, fast hardware SSDs spend substantial time waiting on software locks, synchronous write stalls, excessive journal commits, and uncoordinated background discards.

This chapter provides a production guide to configuring I/O schedulers, filesystem mount flags, block queue depths, dirty page thresholds, and NVMe driver tuning.

I/O Scheduler Selection by Workload

Configure the optimal block I/O scheduler per device type via udev rules:

```
# /etc/udev/rules.d/60-io-schedulers.rules

# NVMe drives: use 'none' (direct hardware submission)
ACTION=="add|change", KERNEL=="nvme[0-9]*n[0-9]*", ATTR{queue/scheduler}="none"

# SATA SSDs and fast cloud virtual disks: use 'mq-deadline'
ACTION=="add|change", KERNEL=="sd[a-z]|vd[a-z]", ATTR{queue/rotational}=="0", ATTR{queue/sch

# Mechanical hard drives (rotational): use 'bfq'
ACTION=="add|change", KERNEL=="sd[a-z]", ATTR{queue/rotational}=="1", ATTR{queue/scheduler}=
```

Apply immediately without rebooting:

```
sudo udevadm trigger --subsystem-match=block
```

Filesystem Mount Options for Performance

When mounting production filesystems for databases, message queues, and high-throughput logging:

Recommended XFS Mount Options

```
# /etc/fstab entry for high-performance XFS database volume
UUID=... /var/lib/postgresql xfs defaults,noatime,nodiratime,allocsize=64M,logbufs=8,logbsiz
```

- `noatime,nodiratime`: Eliminates access time write amplification.
- `allocsize=64M`: Sets delayed allocation pre-allocation buffer size, preventing extent fragmentation during large appends.
- `logbufs=8,logbsize=256k`: Maximizes in-memory journal buffer size before flushing transactions to disk.

Recommended ext4 Mount Options

```
# /etc/fstab entry for high-performance ext4
UUID=... /data ext4 defaults,noatime,data=ordered,delalloc,commit=30 0 2
```

- `commit=30`: Batches journal commits to disk every 30 seconds instead of the default 5 seconds, dramatically reducing `jbd2` journal write frequency.

Writeback and Dirty Page Tuning

On systems with 64GB to 512GB+ RAM, default percentage-based dirty ratios cause catastrophic multi-second write stalls:

```
# /etc/sysctl.d/99-storage-writeback.conf

# Start background asynchronous writeback at 256 MB of dirty data
vm.dirty_background_bytes = 268435456

# Force synchronous blocking writeback if dirty pages reach 1 GB
vm.dirty_bytes = 1073741824

# Expire dirty pages after 15 seconds
vm.dirty_expire_centisecs = 1500

# Wakeup writeback flusher threads every 2 seconds
vm.dirty_writeback_centisecs = 200
```

This enforces smooth, continuous background streaming of dirty pages to persistent storage, preventing massive multi-gigabyte flush bursts that overwhelm disk controller queues.

Device-Level Parameters: Queue Depth and Read-Ahead

Tune block layer device queues:

```
# Increase block layer request queue depth to prevent request starvation
echo 1024 | sudo tee /sys/block/nvme0n1/queue/nr_requests

# Tune read-ahead based on workload:
# For random I/O (OLTP databases): set read-ahead to 0 to avoid wasted reads
echo 0 | sudo tee /sys/block/nvme0n1/queue/read_ahead_kb

# For sequential streaming (Kafka, Hadoop, video): set read-ahead to 512KB
echo 512 | sudo tee /sys/block/nvme0n1/queue/read_ahead_kb
```

SSD and NVMe-Specific Tuning

1. **Disable Continuous Discard (`nodiscard`)**: Real-time TRIM on every file deletion incurs severe synchronous latency penalties. Mount with `nodiscard` and execute scheduled periodic trims via `fstrim.timer`:

   ```
   sudo systemctl enable --now fstrim.timer
   ```

2. **NVMe Interrupt Polling (`io_poll`)**: For sub-10μs database latency, enable polling queues on high-end NVMe drives:

   ```
   echo 1 | sudo tee /sys/block/nvme0n1/queue/io_poll
   ```

3. **Hardware Sector Size**: Ensure the NVMe drive is formatted with native 4096-byte LBA sectors (4Kn) rather than emulated 512-byte sectors (512e) to eliminate Read-Modify-Write penalties inside the flash controller:

   ```
   sudo nvme id-ns -H /dev/nvme0n1 | grep -i "lba format"
   ```

Validating I/O Performance Improvements

Execute before-and-after tests using `fio` and `biolatency` to verify that queue times and p99 completion latency dropped:

```
# 1. Measure disk I/O latency distribution in real time using eBPF
sudo biolatency-bpfcc -m 5 1

# 2. Execute standardized 4KB random write benchmark
fio --name=randwrite --filename=/data/test.bin --size=10G --bs=4k \
```

```
    --rw=randwrite --ioengine=io_uring --direct=1 --iodepth=32 \
    --numjobs=4 --runtime=60 --group_reporting --time_based
```

A properly tuned storage stack should show: - 0% multi-second latency outliers in `biolatency`. - Tightly clustered p99 latency in `fio` ($< 200\mu$s on NVMe). - Predictable steady-state write throughput without periodic dips.

Chapter 21: cgroups, systemd and Resource Management

Chapter 21: cgroups, systemd and Resource Management

Control Groups (cgroups) are the foundational Linux kernel mechanism for metering, allocating, limiting, and isolating system resources (CPU, Memory, I/O, PIDs, RDMA) across groups of processes. In modern Linux and container runtimes, cgroups v2 provides a unified hierarchy that coordinates resource limits cleanly with `systemd`.

This chapter explores the differences between cgroups v1 and v2, CPU CFS quota throttling, memory pressure limits, I/O bandwidth control, systemd unit isolation, and process limits (`ulimit`).

cgroups v1 vs v2: Architecture Differences

In cgroups v1, every resource controller (cpu, memory, blkio, net_cls) had its own independent hierarchy:

- A process could be in group `/web` in the CPU hierarchy, but in group `/db` in the memory hierarchy.
- This made coordinated resource management impossible: the kernel could not correlate buffered I/O writes with memory page allocation because `blkio` and `memory` were in completely separate hierarchies.
- **cgroups v2 (Unified Hierarchy)**: Standardized on a single unified process tree mounted at `/sys/fs/cgroup`. A process belongs to exactly one cgroup, and controllers operate coherently across CPU, Memory, and I/O simultaneously.

```
                    CGROUPS v2 UNIFIED TREE

/sys/fs/cgroup (Root)
  system.slice
     nginx.service (cpu.max, memory.max, io.max)
     postgresql.service
  user.slice
     user-1000.slice
  kubepods.slice (Container Pods)
```

CPU Controllers and Throttling

Under cgroups v2, CPU allocation is governed by weights and quotas:

- **Proportional Sharing (`cpu.weight`)**: Priority weight from 1 to 10000 (default 100). When CPU is saturated, cores are divided proportionally to weights. When idle, any cgroup can consume 100% of available CPU.
- **Hard Quotas (`cpu.max`)**: Restricts maximum CPU consumption over a period. Format: `$QUOTA $PERIOD` in microseconds:

```
# Limit cgroup to 2.5 CPU cores (250,000µs quota per 100,000µs period)
echo "250000 100000" | sudo tee /sys/fs/cgroup/workload/cpu.max
```

The CPU Throttling Latency Trap

When a multi-threaded application exceeds its quota within the first 20ms of a 100ms period, the kernel immediately deschedules all its threads for the remaining 80ms! This causes catastrophic p99 tail latency spikes:

```
# Check how many periods a cgroup has been throttled
grep nr_throttled /sys/fs/cgroup/system.slice/nginx.service/cpu.stat
```

Modern kernels (5.14+) include **CPU Bursting** (`cpu.max.burst`), allowing tasks to borrow unused quota from previous periods to absorb micro-bursts without triggering throttling stalls.

Memory Controllers: Limits and Pressure

cgroups v2 provides granular memory boundaries:

- **`memory.min`**: Hard memory protection. If memory drops below this threshold, the kernel will *never* reclaim memory from this cgroup.
- **`memory.low`**: Soft memory protection. Reclaim occurs only if memory cannot be reclaimed from other unprotected cgroups.
- **`memory.high`**: Throttle limit. When exceeded, the processes in the cgroup are throttled and forced into asynchronous direct reclaim to drop pages.
- **`memory.max`**: Hard maximum. If exceeded and reclaim fails, the Out-Of-Memory (OOM) killer terminates processes in the cgroup.

```
# Set soft limit to 4GB and hard maximum to 8GB
echo 4294967296 | sudo tee /sys/fs/cgroup/workload/memory.high
echo 8589934592 | sudo tee /sys/fs/cgroup/workload/memory.max
```

I/O Controllers: Throttling and Prioritization

cgroups v2 coordinates page cache writeback with block device throttling:

```
# Throttle device 259:0 (NVMe) to 5000 read IOPS and 50 MB/s write bandwidth
echo "259:0 riops=5000 wbps=52428800" | sudo tee /sys/fs/cgroup/workload/io.max
```

Because cgroups v2 unifies memory and I/O, dirty pages generated by this cgroup are attributed to its I/O budget during writeback, preventing rogue background tasks from swamping storage.

systemd Resource Controls and Service Isolation

In production, avoid interacting with `/sys/fs/cgroup` directly; configure resource limits via systemd unit files or drop-in overrides:

```
# /etc/systemd/system/api-service.service.d/override.conf
[Service]
# CPU Limits: allocate 4 cores with burst capability
CPUAccounting=true
CPUQuota=400%
CPUWeight=500

# Memory Limits: throttle at 14GB, hard OOM limit at 16GB
MemoryAccounting=true
MemoryHigh=14G
MemoryMax=16G

# I/O Limits: prioritize storage requests
IOAccounting=true
IOWeight=800
IOReadIOPSMax=/dev/nvme0n1 10000

# Tasks Limit: prevent fork bombs
TasksMax=4096
```

Reload and apply:

```
sudo systemctl daemon-reload
sudo systemctl restart api-service
```

ulimits and Resource Limits

Before cgroup limits are reached, the kernel enforces per-process resource limits (`ulimit`):

Limit	Description	Production Recommendation
`nofile`	Max open file descriptors (sockets, files)	Set to at least `65536` or `1048576` for servers.
`nproc`	Max process/thread count for user	Set to `unlimited` or generous limits for services.
`memlock`	Max memory locked into RAM (`mlock()`)	Set to `unlimited` for databases using HugePages or DPDK.

In systemd services, set these directly in unit files (`LimitNOFILE=1048576`, `LimitMEMLOCK=infinity`), bypassing legacy `/etc/security/limits.conf` limits entirely.

Chapter 22: Containers, Virtualization and Noisy Neighbors

Chapter 22: Containers, Virtualization and Noisy Neighbors

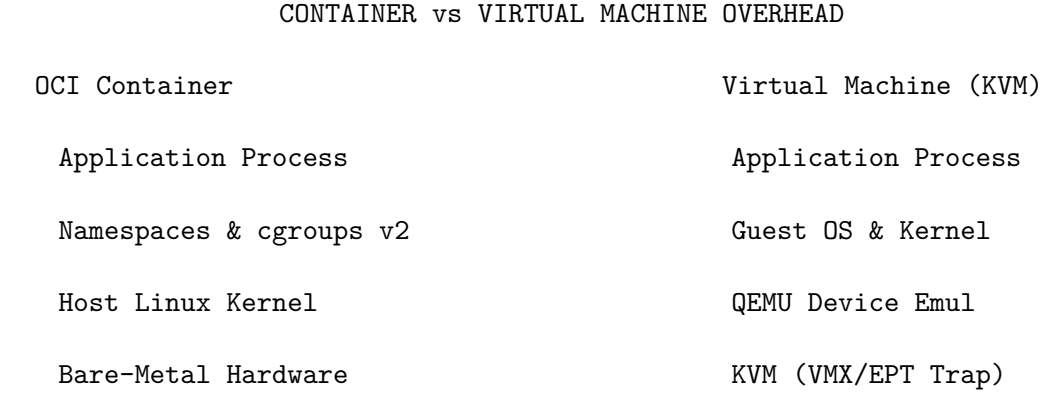
In modern infrastructure, workloads rarely run directly on dedicated bare metal. Instead, they execute inside Docker/OCI containers or Virtual Machines (VMs) managed by hypervisors like KVM. While multi-tenancy maximizes hardware utilization, it introduces isolation overhead and exposes workloads to the "noisy neighbor" problem—where an adjacent tenant monopolizes shared caches, interconnect bandwidth, or disk I/O queues.

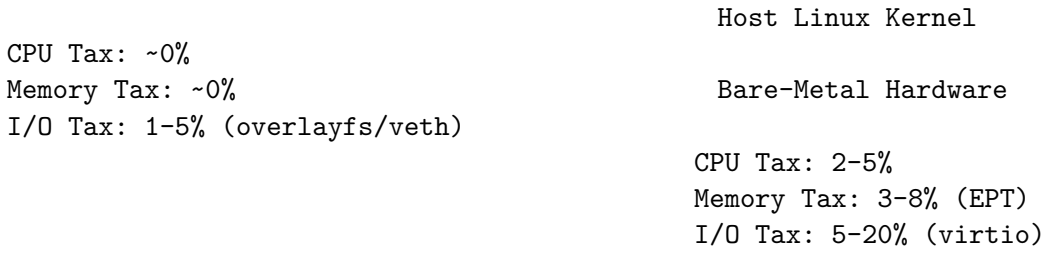
This chapter quantifies container versus virtualization overhead, analyzes noisy neighbor mechanisms, and establishes isolation best practices.

Container Performance: Isolation and Overhead

Containers are not virtual machines; they are standard Linux processes executed inside kernel **Namespaces** and governed by **cgroups**:

- **Compute & Memory Overhead**: Effectively zero. Container processes execute machine instructions directly on physical CPU cores with native hardware MMU translation. There is no hypervisor trap, no instruction emulation, and no guest kernel layer.
- **Networking Overhead**: If containers use traditional `veth` pairs and Linux bridge devices, packets traverse the kernel network stack twice, introducing 10%–20% throughput degradation and microsecond latency penalties.
- **Filesystem Overhead**: Overlay filesystems (`overlay2`) add path resolution and inode lookups across upper and lower directory layers, degrading heavy directory traversal operations.





cgroups and Namespaces in Containers

- **Namespaces** provide illusion of isolation (PID, NET, MNT, IPC, UTS, USER).
- **cgroups** enforce resource quotas.
- A critical failure in containerized environments is **runtime awareness of cgroup limits**. Older versions of runtimes (e.g., Java 8 prior to update 191) read `/proc/cpuinfo` and `/proc/meminfo` instead of cgroup limits, detecting all 128 host cores and 512GB host RAM, leading to massive thread over-allocation and instant OOM termination.

Virtualization Overhead: Hypervisors and Emulation

Virtual machines run a complete guest kernel using hardware virtualization extensions (Intel VT-x / AMD-V):

1. **VM Exits (`vmexit`)**: When the guest kernel executes privileged instructions (modifying CR3, accessing I/O ports, halting), the hardware triggers an exit to the host KVM hypervisor. Each `vmexit` costs 500ns to 2μs.
2. **Two-Dimensional Paging (EPT / NPT)**: Memory accesses require walking two sets of page tables: Guest Virtual $\rightarrow$ Guest Physical, then Guest Physical $\rightarrow$ Host Physical. A TLB miss can require up to 24 DRAM memory references, increasing memory latency by 10%–20%.
3. **vCPU Steal Time (`%steal`)**: The percentage of time a guest vCPU was ready to run but the host hypervisor was busy scheduling another guest or host task.

```
# Monitor CPU steal time in VM (st column)
sar -u 1 5
```

Detecting and Mitigating Noisy Neighbors

A noisy neighbor degrades performance without exceeding their own CPU or memory quotas by saturating unmetered shared hardware resources:

- **Last-Level Cache (LLC) Thrashing**: A streaming batch process constantly evicts latency-sensitive database cache lines from the shared L3 cache.
- **Memory Bus Saturation**: High memory throughput saturates memory controller channels, delaying memory access for adjacent cores.

- **Interconnect Saturation**: Heavy cross-NUMA traffic chokes inter-socket UPI buses.

Hardware-Level Mitigation: Intel RDT and AMD MBA

Modern enterprise CPUs support **Intel Resource Director Technology (RDT)** and AMD Memory Bandwidth Allocation (MBA): - **Cache Allocation Technology (CAT)**: Partitions the shared L3 cache into hardware bitmasks, reserving dedicated L3 cache slices for critical VMs or containers. - **Memory Bandwidth Allocation (MBA)**: Enforces memory controller bandwidth throttling on noisy batch workloads.

```
# Manage Intel RDT allocations via resctrl filesystem
sudo mount -t resctrl resctrl /sys/fs/resctrl
```

Cloud vs Bare Metal: Performance Implications

Factor	Cloud Multi-Tenant VM	Dedicated Bare-Metal Server
Tail Latency (p99/p99.9)	High jitter due to hypervisor steal, shared LLC, and virtualized network interrupts.	Deterministic microsecond execution; zero hypervisor exits.
I/O Latency Floor	~500μs–1ms (virtualized network block storage EBS/PD).	~10μs–25μs (direct PCIe Gen5 NVMe).
Hardware Control	No control over NUMA topology, CPU stepping, or firmware power states.	Full control over CPU governors, isolcpus, huge pages, and BIOS tuning.

Best Practices for Containerized Workloads

1. **Size Thread Pools to CPU Quota, Not Host Cores**: Set Go `GOMAXPROCS` or JVM active processor count to match `cpu.max` quota to avoid catastrophic thread contention and CFS throttling.
2. **Avoid Host Network Sharing Unless Required**: Use host networking (`--net=host`) for ultra-high-throughput networking services to bypass `veth` bridges.
3. **Use Dedicated Host Storage Volumes for I/O**: Mount host block devices or fast XFS directories directly into containers via bind mounts, bypassing `overlay2` storage drivers for database and logging paths.
4. **Enforce Memory Limits with Sufficient Headroom**: Set `memory.high` to trigger early graceful garbage collection before `memory.max` invokes the destructive OOM killer.

Chapter 23: Databases and High-Performance Applications

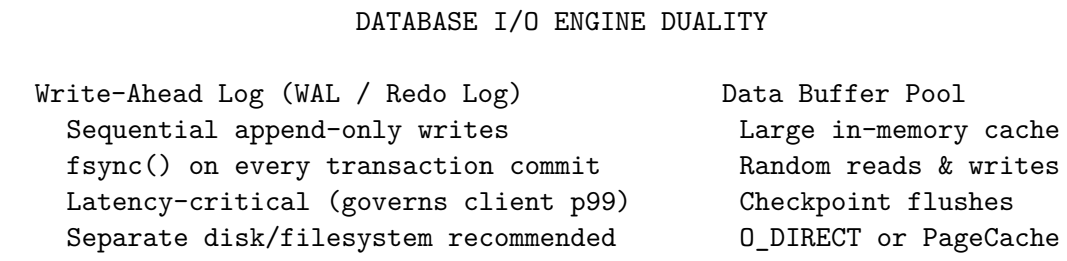
Chapter 23: Databases and High-Performance Applications

Databases and high-performance applications push the Linux operating system to its absolute limits. Relational databases (PostgreSQL, MySQL), LSM-tree key-value engines (RocksDB, Cassandra), and modern application runtimes (JVM, Go, Python) interact with memory, disk, and CPU scheduling in specialized ways. Configuring the host operating system without understanding these runtime models guarantees severe performance degradation.

This chapter details database I/O patterns, shared memory architectures, lock contention, NUMA binding, runtime optimization, and tail latency mitigation.

How Databases Use Linux: I/O and Memory Patterns

Databases typically maintain two distinct I/O paths:



1. **Write-Ahead Logging (WAL)**: Ensures ACID durability. Writes are strictly sequential appends, followed immediately by an `fsync()`. The round-trip commit latency of the WAL disk determines the maximum possible single-thread transaction throughput of the database.
2. **Buffer Pool Management**: Databases manage their own internal page caches (e.g. PostgreSQL `shared_buffers`, MySQL InnoDB Buffer Pool). If the database uses standard buffered I/O, data is cached twice: once in the database buffer pool and once in the Linux Page Cache ("double buffering"), wasting physical RAM.

Shared Memory and IPC for Databases

PostgreSQL and enterprise databases rely on shared memory to communicate across worker processes:

- **POSIX Shared Memory (`shm_open`) vs System V IPC (`shmget`)**: Modern applications use POSIX shared memory mapped via `/dev/shm` (tmpfs).
- **Huge Pages for Shared Buffers**: Configuring PostgreSQL `shared_buffers` with 2MB or 1GB Huge Pages is mandatory for multi-gigabyte buffer pools. Huge Pages prevent page table thrashing and protect the database cache from being accidentally swapped out by the kernel.

```
# postgresql.conf
shared_buffers = 32GB
huge_pages = on              # Fails to start if hugetlbfs is not available
```

Locking and Contention in Database Workloads

High-concurrency database transactions trigger lock contention in both userspace and kernel space:

- **Userspace Spinlocks vs Futexes**: Databases use lightweight spinlocks for brief buffer header updates and `futex` sleep locks for long-running row or page locks. Under extreme concurrency, threads spin on shared memory cache lines, causing CPU saturation without useful work.
- **Kernel Lock Contention**: Traced with `perf lock record` or `bpftrace`. High contention on filesystem inode mutexes or mm page table locks during concurrent checkpoints indicates need for partitioning or switching to XFS allocation groups.

NUMA and Database Placement

For databases larger than the memory capacity of a single NUMA socket:

- **Cross-NUMA Thrashing**: If PostgreSQL processes run on Node 0 while shared memory buffers reside on Node 1, every query pays an interconnect penalty.
- **Interleaving vs Node Binding**:
  - For single database instances spanning the whole machine: launch with `numactl --interleave=all` to evenly stripe buffer pages across all memory channels, maximizing aggregate memory bandwidth.
  - For multi-instance architectures: run multiple independent PostgreSQL instances, each strictly pinned to a dedicated NUMA node (`numactl --cpunodebind=0 --membind=0`).

Application Runtimes: JVM, Go, Python Considerations

Runtime	Execution Model	Performance Bottlenecks & Tuning
JVM (Java)	JIT compiled, managed heap	**GC Pauses**: Stop-the-world pauses destroy p99 latency. Use modern low-pause collectors (ZGC or Shenandoah). Set `-XX:+AlwaysPreTouch` to allocate all pages at startup and avoid runtime page faults.
Go	Native machine code, M:N scheduler	**GOMAXPROCS**: By default equals physical host cores. In containers with CPU quotas, explicitly set `GOMAXPROCS` to match the integer CPU limit (e.g. using `uber-go/automaxprocs`) to avoid goroutine scheduling stalls.
Python	Bytecode interpreter, GIL	**GIL Contention**: Multi-threaded Python cannot scale across multiple CPU cores for CPU-bound tasks. Use multi-process architectures (e.g. `gunicorn` with worker processes) or Python 3.13+ free-threaded builds (`--disable-gil`).

Tail Latency Optimization

Tail latency (p99 and p99.9) dictates user experience in distributed systems. Eliminating tail latency spikes requires removing all sources of non-deterministic jitter:

1. **Pre-Fault Application Memory**: Call `mlockall(MCL_CURRENT | MCL_FUTURE)` or runtime equivalents to pin all code and data into physical RAM, eliminating major page faults during execution.
2. **Use Epoll / `io_uring` Asynchronous Architectures**: Eliminate synchronous blocking I/O calls on request-handling worker threads.
3. **Decouple Checkpoints and Compactions**: Ensure background LSM-tree compactions (RocksDB) or database checkpoints execute on isolated background threads with lowered I/O priority (`ionice -c 3`).

Chapter 24: Distributed Systems and Cluster Performance

Chapter 24: Distributed Systems and Cluster Performance

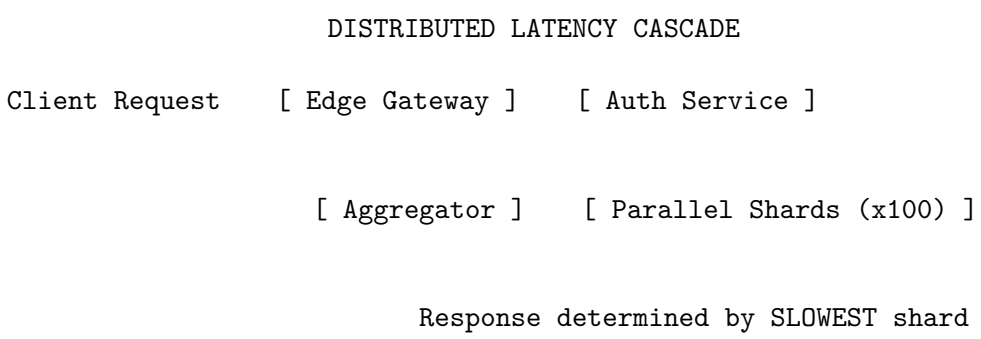
In distributed systems, individual node performance is multiplied across networks, consensus protocols, and replication boundaries. A single microsecond latency spike or packet retransmission on one node can cascade into a second-long tail latency outage across a cluster of thousands of nodes.

This chapter analyzes the systemic contributors to distributed latency, hardware clock synchronization, network partition performance penalties, consensus costs, and cluster-scale observability.

Distributed Latency: Network and System Contributions

End-to-end request latency in a microservices or distributed database cluster is composed of sequential and parallel stages:

$$\text{Total Latency} = T_{\text{client}} + T_{\text{serialization}} + T_{\text{network_hop}} + T_{\text{switch_queue}} + T_{\text{kernel_rx}} + T_{\text{app_exec}} + T_{\text{disk_fsync}} + \dots$$



- **The Fan-Out Latency Multiplier**: If a single request fans out to 100 backend database nodes in parallel, and each node has an individual 1% chance of suffering a 100ms latency pause (p99 = 100ms), the probability that the client experiences the 100ms pause is:

  $$P(\text{Client Slow}) = 1 - (1 - 0.01)^{100} = 1 - 0.366 = 63.4\%$$

  The client experiences 99th-percentile backend latency on **nearly two out of every three requests**.

Clock Synchronization and Time Skew

Distributed databases (CockroachDB, Google Spanner, YugabyteDB) and consensus algorithms rely on synchronized physical timestamps to order transactions without expensive cross-node locks:

- **NTP (Network Time Protocol)**: Classic synchronization via `chrony` or `ntpd`. Achieves sub-millisecond precision over local networks, but exhibits microsecond drift under load.
- **PTP (Precision Time Protocol / IEEE 1588)**: Uses hardware timestamping in the network interface card (NIC) PHY layer, synchronizing clocks across a local datacenter fabric to **under 100 nanoseconds**.

```
# Verify clock synchronization status and current offset using chrony
chronyc tracking
```

Clock skew exceeding transaction commit wait bounds forces distributed databases to abort transactions or stall execution until the skew window passes.

Network Partitions and Performance

When network degradation, asymmetric routing, or switch buffer drops occur:

- **TCP Retransmission Timeouts (RTO)**: Default Linux initial RTO is 1 second (`TCP_RTO_MIN` = 200ms). If a heartbeat packet is lost during network congestion, the connection stalls for 200ms to 1s before retransmitting, causing health check failures and unnecessary cluster leader failovers.
- **Flapping Detection**: Flapping cluster nodes trigger continuous state reconciliation, log replication storms, and cluster-wide throughput collapse.

Consensus Protocols and Performance Costs

Distributed state machines (Raft, Paxos) maintain replicated logs across quorums (e.g. 3 or 5 nodes):

```
Leader    Write to Local WAL & fsync()
          Replicate to Follower 1 & fsync()     Wait for Quorum ACK
          Replicate to Follower 2 & fsync()
```

- **The fsync Bottleneck**: A Raft leader cannot acknowledge a client write until a majority of quorum followers have persisted the entry to disk. The commit throughput of the entire distributed system is bounded by the slowest disk `fsync()` speed among the quorum majority.
- **Colocating Heartbeats and Data**: If Raft heartbeats share the same network socket or disk volume as heavy bulk data transfers, disk write queues delay heartbeats, triggering false election timeouts and split-brain recovery storms.

Cluster Resource Management

In Kubernetes and distributed orchestrators:

- **Topology-Aware Scheduling**: Pods that exchange gigabits of traffic should be co-located within the same availability zone, rack, or physical chassis to minimize network hop latency.
- **CPU Manager Static Policy**: Binds critical pods to dedicated physical CPU cores, eliminating CFS quota throttling and hypervisor context switching in production.

Observability at Scale

Monitoring hundreds of nodes requires distributed tracing standards:

- **OpenTelemetry (OTel)**: Injects trace IDs (`traceparent`) into RPC and HTTP headers, linking client requests through gateways, services, database queries, and kernel events into a single causal timeline.
- **Sampling Strategies**: Recording 100% of traces at 500,000 req/sec would overwhelm telemetry infrastructure. Use **Tail-Based Sampling**: collect all spans in memory temporarily, but only persist traces that returned errors or exceeded the p99 latency threshold.

Chapter 25: Production Case Studies

Chapter 25: Production Case Studies

Theoretical understanding of Linux subsystems must be proven in the crucible of production outages. When complex systems fail under high load, the symptoms often masquerade as application bugs, network outages, or hardware degradation. Only disciplined forensic analysis using kernel tracing and performance telemetry can unmask the true root causes.

This chapter presents five detailed, real-world production case studies, detailing initial symptoms, diagnostic investigations, root causes, and permanent engineering mitigations.

Case Study: CPU Saturation from Interrupt Storms

The Incident

An API gateway cluster handling 200,000 requests per second suffered an unannounced 80% drop in throughput. Client connections timed out, and average latency spiked from 2ms to over 800ms. High-level monitoring showed overall CPU utilization was only 12% across the 64-core host.

Diagnostic Investigation

1. Running `mpstat -P ALL 1` revealed that while 63 cores were sitting 98% idle, **CPU 0 was pegged at 100% utilization in `%si` (software interrupts)**.
2. Inspecting `/proc/interrupts` and `/proc/softirqs` showed millions of interrupts incrementing exclusively on CPU 0 under `NET_RX`.
3. Running `ethtool -l eth0` showed the modern 100GbE NIC had 32 hardware queues available, but `ethtool -S eth0` revealed that all packets were being directed to Queue 0.

Root Cause

A recent firmware update reset the NIC's RSS configuration back to single-queue mode, forcing every incoming packet to be handled by a single hardware interrupt line routed exclusively to CPU 0. CPU 0's softirq processing pipeline saturated, dropping packets at the kernel boundary while the remaining 63 CPU cores starved for work.

Mitigation & Resolution

1. Enabled all 32 hardware queues via `ethtool -L eth0 combined 32`.
2. Verified that MSI-X interrupts were distributed across cores via `/proc/irq/*/smp_affinity`.
3. Throughput restored immediately to 200,000 req/sec; CPU utilization distributed evenly across all 32 cores at ~15% each.

Case Study: Memory Pressure and Swapping Crisis

The Incident

A high-throughput in-memory Redis cluster hosted on 256GB bare-metal servers began experiencing periodic, catastrophic 10-second latency pauses. During these pauses, Redis ceased responding to client heartbeats, triggering false cluster failovers.

Diagnostic Investigation

1. `vmstat 1` showed periods where `si` and `so` spiked from 0 to over 50,000 KB/s.
2. `top` showed the kernel thread `kswapd0` burning 100% CPU during the freezes.
3. Reading `/proc/sys/vm/swappiness` revealed the distribution default value of `60`.
4. Reading `/proc/zoneinfo` showed that `ZONE_NORMAL` had fallen below the `min` watermark threshold, forcing synchronous direct page reclaim.

Root Cause

While Redis consumed 180GB of RAM, heavy nightly database backups dumped multi-gigabyte snapshot files to disk. The Linux Page Cache aggressively expanded to cache the backup files. Because `vm.swappiness` was set to 60, the kernel decided to reclaim memory by writing inactive anonymous Redis heap pages to swap storage, rather than dropping file-backed Page Cache pages.

Mitigation & Resolution

1. Reduced swappiness to prevent swapping anonymous memory: `sysctl -w vm.swappiness=1`.
2. Configured the backup utility to read and write with `posix_fadvise(POSIX_FADV_DONTNEED)`, instructing the kernel not to pollute the Page Cache with backup data.
3. Locked Redis heap memory using `vm.overcommit_memory=1` and `mlockall()`.

Case Study: NUMA Imbalance Causing 5x Latency

The Incident

A dual-socket AMD EPYC server (128 cores, 512GB RAM across 2 NUMA nodes) hosted a multi-threaded compute application. Under identical request load, latency on this modern server was 5x worse than on an older, smaller single-socket server.

Diagnostic Investigation

1. Profiling with `perf stat -e cycles,instructions,cache-misses` showed IPC was a dismal `0.42`, with cache miss rates exceeding 45%.
2. Running `numastat -c` revealed that `numa_miss` on Node 0 was exceeding 150,000 pages per second, and `other_node` allocations accounted for 80% of all memory touches.
3. Running `perf c2c record` pinpointed massive cross-socket cache line invalidations on shared data structures.

Root Cause

The application was launched without NUMA awareness. The main allocator thread ran on Node 0 and allocated all memory buffers from Node 0's local memory channels. However, the thread pool spawned 128 worker threads spanning Node 0 and Node 1. The 64 worker threads running on Node 1 were constantly accessing data across the inter-socket Infinity Fabric bus, saturating interconnect bandwidth and tripling memory access latency.

Mitigation & Resolution

1. Partitioned the application into two independent processes: Instance A bound to NUMA Node 0 (`numactl --cpunodebind=0 --membind=0`), and Instance B bound to NUMA Node 1 (`numactl --cpunodebind=1 --membind=1`).
2. IPC leaped from `0.42` to `2.10`, and latency dropped by 82%, outperforming the single-socket baseline.

Case Study: I/O Bottleneck on Shared Storage

The Incident

An analytics database hosted on cloud virtual machines suffered severe query timeouts during business hours. Read throughput on queries dropped from 600 MB/s to under 40 MB/s.

Diagnostic Investigation

1. `iostat -xz 1` showed disk `%util` was pinned at 100%, and `r_await` had skyrocketed from 1.2ms to over 85ms.
2. Checking cloud provider metrics revealed that the storage volume's **Burst Balance** had reached 0%.
3. Inspecting the filesystem mount flags showed `relatime` was active, and applications were updating access timestamps on millions of small metadata files.

Root Cause

The cloud block storage volume relied on burst IOPS credits. The continuous metadata updates from unoptimized access times slowly drained the credit bucket over the week. Once credits depleted, the cloud hypervisor throttled the volume from 3,000 IOPS down to its baseline limit of 100 IOPS, choking all database reads.

Mitigation & Resolution

1. Remounted volumes with `noatime,nodiratime`.
2. Switched the cloud volume to provisioned IOPS (`gp3` / `io2`) with guaranteed baseline throughput independent of burst credits.
3. Latency immediately returned to sub-2ms levels.

Case Study: Noisy Neighbor in Multi-Tenant Cloud

The Incident

A latency-critical microservice in a Kubernetes cluster experienced unpredictable p99 latency spikes (jumping from 8ms to 120ms) despite CPU utilization within the container remaining below 40% of its cgroup limit.

Diagnostic Investigation

1. Inspecting `/proc/pressure/cpu` inside the container showed `some avg10=22.4`, indicating that tasks were stalled waiting for CPU 22% of the time.
2. Checking `cat /sys/fs/cgroup/cpu.stat` revealed `nr_throttled` was rapidly incrementing.
3. Looking at the host node with `perf top` revealed that an adjacent batch container running on the same host was executing unthrottled Python processes with hundreds of threads, thrashing the CPU's shared L3 cache and exhausting CFS bandwidth.

Root Cause

The latency-critical container had fractional CPU limits (`resources.limits.cpu: "2"`), subjecting it to CFS quota throttling whenever it experienced micro-bursts within the 100ms CFS period. Simultaneously, the adjacent batch container was monopolizing the CPU core's hyperthread sibling.

Mitigation & Resolution

1. Set the Kubernetes container's QoS class to **Guaranteed** with integer CPU requests (`requests.cpu: "4"`, `limits.cpu: "4"`).
2. Enabled the Kubernetes `CPU Manager` with `static` policy, granting the container exclusive access to isolated physical CPU cores.
3. Latency jitter vanished; p99 latency remained rock-solid at 8ms.

Chapter 26: Capacity Planning and Performance Regression

Chapter 26: Capacity Planning and Performance Regression

Performance engineering is not a one-off optimization project; it is an ongoing operational lifecycle. Without continuous validation, performance naturally decays over time as new software releases introduce memory regressions, slow queries, and CPU cycle bloat. Furthermore, infrastructure teams must predict when hardware will saturate months before user traffic overwhelms capacity.

This chapter covers mathematical capacity modeling, baseline versioning, automated regression detection in CI/CD, load testing methodologies, and performance budgets.

Capacity Modeling and Forecasting

Capacity planning relies on queueing theory and mathematical models rather than simple linear extrapolation:

Little's Law

In any stable system, the average number of concurrent requests in the system (L) equals the average arrival rate (λ) multiplied by the average time spent in the system (W):

$$L = \lambda \times W$$

- If your API receives $\lambda = 5,000$ req/sec and average service time is $W = 0.020$ sec (20ms), the system holds:
  $$L = 5,000 \times 0.020 = 100 \text{ concurrent in-flight requests}$$
- If an architectural regression increases latency to 100ms (0.100s), the required concurrency spikes to:
  $$L = 5,000 \times 0.100 = 500 \text{ concurrent in-flight requests}$$

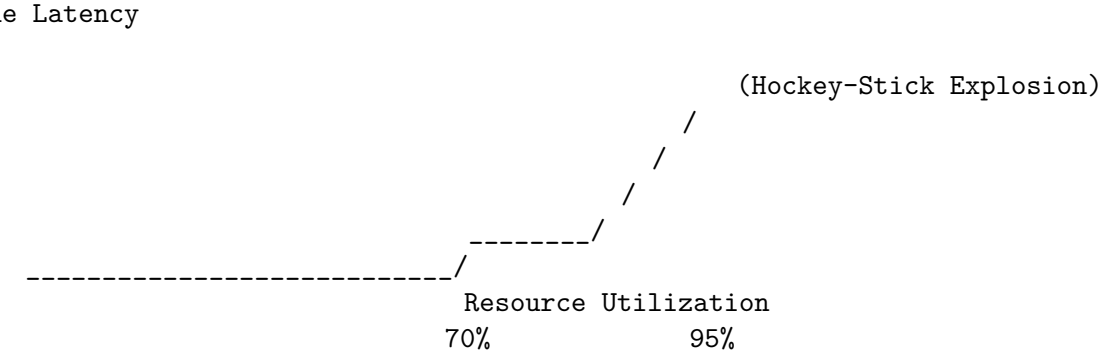
  Connection pools, thread pools, and memory structures suddenly exhaust, causing system failure even though incoming request volume did not change!

Queueing Theory and the Hockey-Stick Effect (M/M/m Models)

As resource utilization (ρ) approaches 100%, queue wait time (W_q) grows asymptotically:

$$W_q \approx \frac{\rho}{1-\rho}$$

At 50% utilization, queueing delay is negligible ($W_q \propto 1$). At 80% utilization, delay is 4×. At 95% utilization, delay is 19×. Capacity planning must ensure production utilization thresholds remain below 70%–75% to absorb natural traffic bursts without queue explosions.



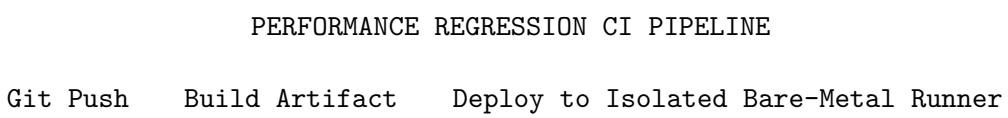
Baseline Management and Versioning

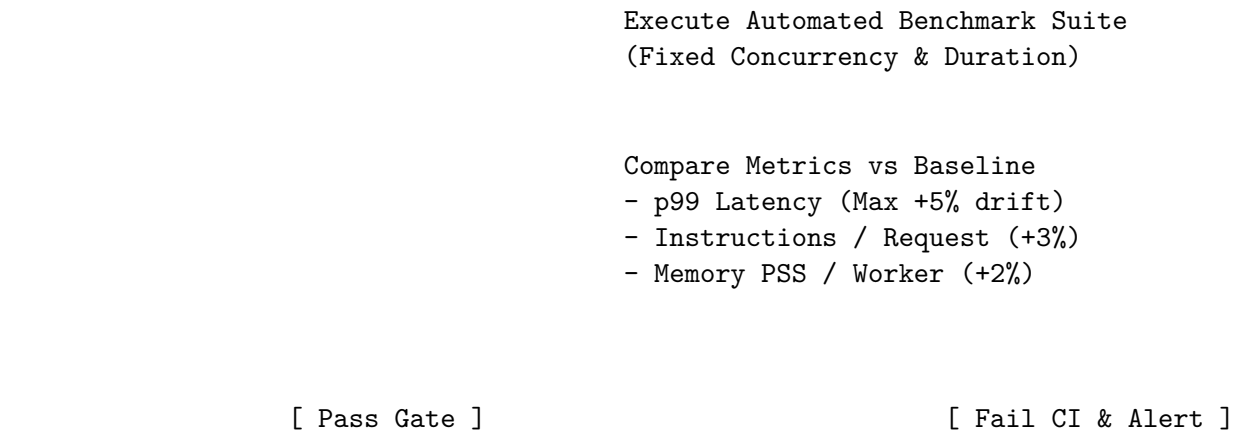
A performance baseline is an engineering artifact that must be version-controlled alongside code:

- **Performance Fingerprints**: For every major release, archive a standardized metrics package:
  - Raw JSON output of automated benchmark suites (`fio`, `wrk2`, `sysbench`).
  - Flame Graph SVGs of on-CPU and off-CPU profiles.
  - Hardware counter statistics (`perf stat` IPC, cache misses, branch misses).
  - Kernel and OS configuration dumps.
- Store baselines in Git or artifact repositories to track long-term trends across quarters.

Regression Detection: Automated and Manual

Detecting performance regressions manually in staging is ineffective. Performance verification must be integrated into automated CI/CD pipelines:





Use statistical threshold tests (e.g. Student's t-test or Mann-Whitney U test) to distinguish real regressions from runner environmental variance.

Load Testing and Stress Testing Strategies

1. **Soak Testing (Endurance)**: Run steady 70% load for 24 to 72 hours. Unmasks slow memory leaks, file descriptor leaks, connection pool exhaustion, and disk fragmentation that short tests miss.
2. **Spike Testing**: Instantly jump from 10% to 150% load to evaluate autoscaling reaction times, queue overflow behaviors, and graceful degradation mechanisms.
3. **Stress / Breakpoint Testing**: Continuously ramp traffic until the system breaks. Identifies the exact resource bottleneck (CPU, disk I/O, database locks, or network bandwidth) that defines the upper limit of the architecture.

Performance Budgets and Guardrails

Establish clear, non-negotiable performance budgets for engineering teams:

- **Latency Budget**: API endpoints must complete within 25ms at p95 and 50ms at p99.
- **Resource Budget**: A service pod must not exceed 250MB RSS memory under 1,000 concurrent connections.
- **CPU Budget**: A single transaction must not consume more than 500,000 CPU instructions.
- If a pull request breaches the budget, the CI gate blocks merge until the regression is addressed.

Continuous Performance Validation

In production, run continuous background synthetic transactions ("canary traffic") to validate real end-to-end performance under live conditions, measuring latency trends and verifying that production hosts maintain parity with benchmark baselines.

Chapter 27: Advanced Topics and Emerging Technologies

Chapter 27: Advanced Topics and Emerging Technologies

As hardware speeds approach physical silicon and optical limits, traditional kernel architectures face structural boundaries. Crossing these boundaries requires advanced kernel primitives, hardware offloads, kernel bypass networking, real-time determinism, and next-generation interconnects.

This chapter explores PREEMPT_RT real-time kernels, kernel bypass engines (DPDK, XDP, AF_XDP), architectural comparisons between x86-64 and ARM64, RDMA fabrics, advanced eBPF patterns, and emerging technologies like CXL.

Real-Time Kernels and PREEMPT_RT

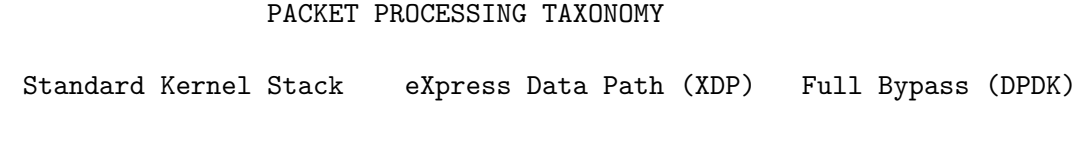
Standard Linux prioritizes overall system throughput over bounded latency. In standard kernels, non-preemptible spinlock sections, interrupt handlers, and RCU grace periods can defer task execution for milliseconds.

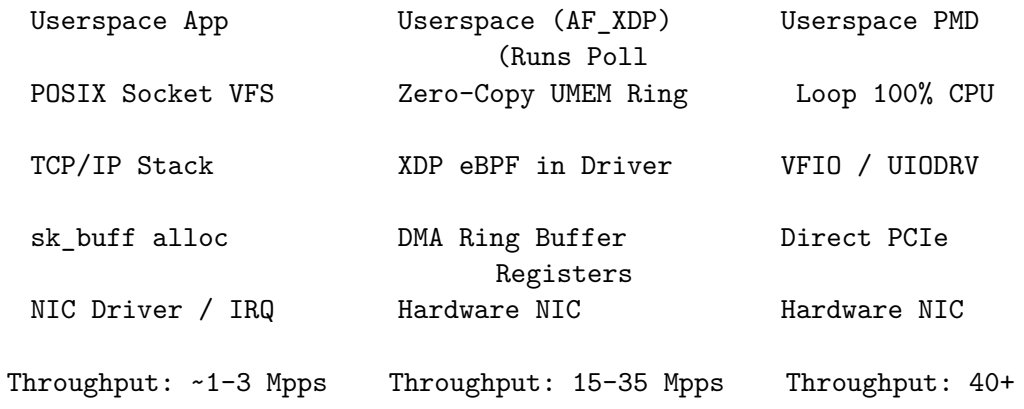
The **PREEMPT_RT** real-time patchset (officially merged into mainline Linux 6.12+) transforms Linux into a deterministic real-time operating system:

- **Preemptible Critical Sections**: Replaces standard spinlocks (`spinlock_t`) with sleepable, preemptible rt-mutexes that support **Priority Inheritance** (preventing priority inversion where a low-priority task holds a lock needed by a real-time task).
- **Threaded Interrupt Handlers**: Forces all hardware interrupt handlers to execute as preemptible kernel threads (`irq/N-device`), allowing high-priority real-time tasks to preempt incoming hardware interrupts.
- **Deterministic Latency**: Guarantees bounded worst-case scheduling latency under 15–20 microseconds on enterprise x86/ARM hardware.

Kernel Bypass: DPDK, XDP and AF_XDP

At 100Gbps–400Gbps line rates, allocating an `sk_buff`, traversing the Linux network stack, and context switching to userspace consumes more CPU time than the packet arrival interval (a 64-byte frame arrives every 5.1 nanoseconds on a 100G link).





- **DPDK (Data Plane Development Kit)**: Bypasses the Linux kernel completely using Userspace I/O (UIO) or VFIO. Userspace threads poll PCIe hardware ring buffers directly in a 100% CPU busy-loop. Massive throughput (>40 Mpps), but burns entire CPU cores continuously and loses all standard Linux network tooling (`iptables`, `ip`, `tcpdump`).
- **XDP (eXpress Data Path)**: In-kernel bypass. Executes an eBPF program directly inside the network driver's RX ring buffer *before* allocating an `sk_buff` or touching the network stack. Packets can be dropped (`XDP_DROP`), reflected (`XDP_TX`), or routed at line rate (over 30 Mpps per core).
- **AF_XDP (XSK)**: Delivers raw packets from XDP directly into a userspace zero-copy ring buffer (`UMEM`), marrying DPDK-level throughput with kernel security and driver compatibility.

x86-64 vs ARM64: Performance Differences

Enterprise computing is no longer an x86 monoculture. ARM64 processors (AWS Graviton3/4, Ampere Altra, NVIDIA Grace) dominate modern cloud data centers:

Feature	x86-64 (Intel Xeon / AMD EPYC)	ARM64 (Neoverse V2 / N2)
Memory Consistency	**Total Store Order (TSO)**: Hardware guarantees strong memory ordering; loads and stores rarely reorder.	**Weak Memory Ordering**: Stores and loads reorder aggressively in hardware; requires explicit memory barrier instructions (`dmb`, `dsb`).
Instruction Set	Complex Instruction Set (CISC), variable-length instructions (1 to 15 bytes).	Reduced Instruction Set (RISC), fixed 4-byte instructions; simplified decode pipelines.
Cache Architecture	Shared L3 cache across core complexes (CCX) or sockets.	Massive private L2 caches per core (1MB to 2MB) with distributed mesh interconnects.

Feature	x86-64 (Intel Xeon / AMD EPYC)	ARM64 (Neoverse V2 / N2)
Efficiency	High peak single-thread instruction throughput.	Superior performance-per-watt and predictable scaling across hundreds of physical cores without SMT.

RDMA and Low-Latency Networking

Remote Direct Memory Access (RDMA) enables one server to read or write memory directly to or from the physical RAM of a remote server across an InfiniBand or Ethernet (RoCEv2 - RDMA over Converged Ethernet) fabric:

- **Zero-Copy**: Data moves directly from local memory to remote memory without buffering in either host's operating system.
- **Kernel-Bypass**: Hardware NICs execute the RDMA transport engine in silicon, eliminating CPU involvement on both ends.
- **Latency**: End-to-end network transfer latency drops from 25μs (standard TCP) to **under 1.5 microseconds**. Essential for distributed AI model training clusters and NVMe-over-Fabrics (NVMe-oF).

eBPF in Production: Advanced Patterns

- **Sockops & Redirection**: eBPF programs attached to `sock_ops` can bypass TCP/IP protocol parsing for co-located containers, redirecting socket buffers directly between socket queues in memory (`bpf_msg_redirect_hash`).
- **BPF LSM (Linux Security Modules)**: Replaces static security modules with dynamically compiled, ultra-low-overhead security policies that evaluate performance and safety in a single pass.
- **Layer 4 Load Balancing (Katran)**: Uses XDP to route millions of packets per second across backend server fleets with consistent hashing.

Future Directions: Kernel and Hardware Trends

1. **Compute Express Link (CXL)**: PCIe-attached memory fabrics that pool disaggregated DRAM across servers with cache-coherent hardware semantics, turning memory into a network-attached, shared resource.
2. **io_uring Ubiquity**: Transitioning from synchronous system calls to asynchronous ring buffers for all system interfaces—storage, networking, process management, and IPC.
3. **Rust for Linux**: Integrating memory-safe Rust into the core kernel driver model, eliminating entire classes of memory corruption and concurrency bugs without sacrificing C execution speed.

Conclusion: Putting It All Together

Conclusion: Putting It All Together

We have traveled from the nanosecond mechanics of CPU pipelines and page table walks down through the block layer, network drivers, eBPF tracing engines, and cluster-scale consensus protocols. Mastery of performance engineering is not simply a technical encyclopedia of Linux sysctl knobs; it is an intellectual discipline and a cultural framework.

This concluding chapter summarizes the performance engineer's mindset, the critical judgment of knowing when *not* to optimize, and the practices required to build an enduring performance culture across engineering organizations.

The Engineer's Mindset for Performance

True performance engineers cultivate specific cognitive habits that distinguish them from reactive tuners:

```
                THE PERFORMANCE ENGINEERING MINDSET

Skepticism       Question assumptions; verify every claim
                     with hardware counters and kernel traces.

Mechanistic      Insist on understanding the physical and
Understanding        architectural reason why a change worked.

Statistical      Distrust single numbers, averages, and short
Humility             tests. Respect variance and tail percentiles.

Empirical       Never ship a configuration without proving its
Verification         benefit under controlled, reproducible load.
```

1. **Reject Cargo-Culting**: Never apply a sysctl knob, mount option, or compiler flag simply because a blog post or documentation claimed it was "best practice." Environments, kernels, and workloads differ. Prove the necessity on your hardware.
2. **Seek First-Principles Explanations**: If modifying `vm.dirty_background_bytes` reduced p99 latency from 120ms to 8ms, do not celebrate until you can draw the diagram showing how writeback flusher threads unblocked the application's synchronous `write()` calls.
3. **Trace the Whole Stack**: Never blame "the network" or "the database" or "the kernel" without evidence. Use end-to-end tracing to prove where time was actually spent.

When Not to Optimize

The greatest danger in systems engineering is spending weeks optimizing components that have negligible impact on real system performance.

Amdahl's Law

The maximum overall speedup achieved by optimizing a single component is strictly bounded by the fraction of time (f) the system spends using that component:

$$S_{\text{overall}} = \frac{1}{(1-f) + \frac{f}{S_{\text{component}}}}$$

- If a database query spends 95% of its time waiting on a disk read ($f = 0.95$) and 5% executing CPU code ($1 - f = 0.05$):
  - Optimizing the C++ CPU code to run 10× faster ($S = 10$) improves total query latency by only:

$$S_{\text{overall}} = \frac{1}{0.05 + \frac{0.95}{10}} = \frac{1}{0.05 + 0.095} = \frac{1}{0.145} \approx 6.9\% \text{ overall speedup!}$$

  - Conversely, cutting the disk I/O time in half yields an immediate 45% overall speedup.

The Costs of Premature and Over-Optimization

1. **Complexity Tax**: Specialized lockless algorithms, custom memory allocators, and kernel bypass drivers make code vastly harder to read, debug, and maintain.
2. **Fragility**: Hyper-tuned configurations tuned tightly to a specific CPU stepping or kernel version often break or regress catastrophically upon the next OS upgrade.
3. **Opportunity Cost**: Engineering time spent shaving 2 microseconds off an already-acceptable internal RPC is time not spent fixing security vulnerabilities, operational resilience, or business capabilities.

Optimize only when: - An agreed-upon Service Level Objective (SLO) is breached or approaching breach under projected growth. - Cloud or infrastructure hosting costs can be measurably reduced with high ROI. - Latency or throughput directly limits user experience or platform scale.

Building a Performance Culture

Performance is not the sole responsibility of a specialized "performance team" called in during emergencies. High-performing engineering organizations embed performance into daily workflows:

1. **Democratize Observability**: Give every engineer easy, self-service access to production dashboards, distributed traces, and Continuous Profiling Flame Graphs (Parca, Pyroscope).

2. **Automate Guardrails in CI/CD**: Ensure every pull request runs automated regression checks against established performance budgets. Catch regressions before code merges to `main`.
3. **Document and Version Baselines**: Keep historical benchmark records linked to architecture decision records (ADRs). When incidents occur, compare current telemetry directly against the archived baseline.
4. **Celebrate Simplification**: Recognize engineers who delete redundant abstractions, eliminate unnecessary round-trips, and simplify architectures, rather than only praising those who introduce complex micro-optimizations.

Performance engineering is ultimately about clarity: understanding the system so deeply that you can make it do exactly what is required—reliably, predictably, and efficiently.